Foλang Programming Language

Foλang is a general-purpose programming language designed to be **expressive, consistent, and extensible**, merging functional fluency with object-centric abstractions.

Normative Status

This document is the authoritative and normative definition of the FoLang programming language.

All FoLang syntax, semantics, type-system rules, name-resolution rules, execution behavior, and other language requirements are defined by this document.

Every addition, correction, clarification, modification, or extension to the FoLang language must be recorded in this document before it becomes part of the language.

A compiler or other language implementation may use any internal architecture, parser, intermediate representation, runtime, or backend. However, to be identified as a conforming FoLang implementation, its externally observable behavior must adhere to the requirements defined in this document.

When an implementation conflicts with this specification, the specification governs unless the discrepancy is formally accepted and incorporated as a specification revision.

Implementation-specific extensions must be clearly identified as extensions and must not be represented as standard FoLang features unless they have been incorporated into this specification.

Grammar, Semantics, and Examples

The lexical and syntactic grammar defined by this specification is the authoritative definition of the FoLang source forms accepted by the current language profile. Normative semantic rules define the meaning of those forms and any additional validity constraints that are checked after parsing.

Examples are **illustrative only**. The presence or absence of an example does not enable, disable, reserve, or otherwise modify a grammar production, token, declaration form, operator, or semantic rule. Likewise, an inventory or explanatory table does not create new source syntax unless the specification explicitly defines the corresponding grammar or identifies the spelling as an explicitly reserved future form.

For parser and grammar generation, the governing classification is therefore deterministic:

```
active lexical/syntactic grammar
    -> accepted and parsed according to the applicable grammar rules

explicitly identified reserved/future form
    -> recognized as language-owned where its reserved spelling is defined
    -> rejected with an unsupported-feature diagnostic in the current profile

all other source text
    -> ordinary lexical or syntax error when it does not match the active grammar
```

Language Evolution and Compatibility

The alpha period permits experimentation, consolidation, implementation, renaming, syntax changes, and removal. Before version 1.0, a proposed structural feature becomes part of the active language only when a specification revision explicitly incorporates it into the current lexical/syntactic grammar and defines its applicable normative semantics. A proposed or future feature that is mentioned only descriptively does not become active grammar.

Nothing is carried into version 1.0 as implicitly reserved or unsupported merely because it appeared in an example, table, inventory, or design discussion. A spelling is reserved while unsupported only when this specification explicitly identifies that spelling or form as reserved/future syntax.

At version 1.0, the **structural FoLang language surface** closes permanently. After version 1.0, no later major or minor release introduces new core grammar forms, hard/contextual keywords, declaration kinds, operator spellings or fixities, or built-in `@co.*` metadata-form names. Existing structural constructs retain the externally observable semantics defined by the 1.0 specification, except for corrections that restore already-stated intent.

The standard-package rule is narrower and separate. After version 1.0, the `co.*` **package/subpackage hierarchy** is frozen, while ordinary declarations inside those existing packages may evolve through later language-provided `.folenc` artifacts that expose the standard package contexts. Adding or updating an ordinary type, unit-level function, method, module, algorithm, data structure, or other declaration inside an existing package is package-API evolution and does not create new grammar or a new package path.

FoLang remains extensible after 1.0 through extension mechanisms already defined by the 1.0 language, including third-party libraries, user-defined annotations and decorators, macros, custom operators, native and FFI integration, dynamic-runtime facilities, backend integration points, and other explicitly defined extension mechanisms. Frontend, backend, runtime, optimizer, intermediate representation, diagnostics, compilation strategy, memory management, code generation, performance, and hardware support may evolve provided externally observable FoLang semantics remain conforming.

A post-1.0 correction may fix an implementation/specification discrepancy or remove an internal contradiction when doing so restores the already-stated intent of an existing feature. A correction does not introduce a new grammar form, keyword, declaration kind, operator grammar, metadata-form name, or previously unavailable structural capability.

Design Review Resolution Log

> **Tracking note — non-normative.** This section records design-review decisions so that settled issues are not repeatedly reopened accidentally. The normative rules remain the dedicated language sections referenced below. If this log ever conflicts with a normative section, the normative section governs and this log must be corrected.

Resolved Review Items — 2026-08-15

Review item	Resolution	Status
Library/component surface contradiction	`src/component.fol` and `components/<kind>/component.fol` use `_co.lang.component`. `@co.dap.library` is reserved for standalone projected libraries rooted at `src/component.fol`; project-local components are not libraries.	**Resolved**
Library/component/export taxonomy	Project-local component kinds are `application`, `ffi`, `system`, `dynamicvmrt`, `packaged`, and `operators`. The former `advanced` and `exports` component kinds are removed. `packaged` is an open-package exposure model using application capabilities.	**Resolved**
Package/source stale forms	`src/library.fol`, `co.lang.library`, the special `co.lang.package` declaration, and special `package.fol` logical-package renaming are removed. Canonical package identity is filesystem-derived.	**Resolved**
Component visibility	Every descendant package under `components/<kind>/` is private by default. A projected component exposes only APIs published by its `component.fol`. A `packaged` component exposes only package contexts explicitly selected by `@co.dap.export(...)`; unselected packages remain private.	**Resolved**
Packaged-component application-only visibility	Package contexts selected by `components/packaged/component.fol` are injected only into the executable application's primary `src/` package/open graph. No peer project-local component can import, qualify, reference, or otherwise consume those selected contexts. Standalone packaged-library exports remain externally consumable by their application through the loaded artifact.	**Resolved**
Project-local component dependency direction	Project-local components are executable-application implementation domains and are isolated from peer components. Source under any `components/<kind>/` domain may not import, reference, or otherwise consume another project-local component surface. `component=` is executable-application-primary-`src/` only. Components may consume standalone **projected** libraries through their published `library=` surfaces, but never standalone packaged-library package exports. Cross-standalone-projected-library dependency topology is an architectural choice; the compiler enforces API/exposure, capability, visibility, type-boundary, liveness, and cycle rules rather than a pairwise dependency matrix.	**Resolved**
Operator-component ownership and consumption	`components/operators/` may exist in an executable application project and, as the **only** permitted `components/` child for a standalone projected application library, in that library project. Its custom operator table is consumable only by the owning ordinary primary `src/` source domain and is never visible to any project-local component, including `components/packaged/`. Standalone packaged, `ffi`, `system`, and `dynamicvmrt` libraries have no `components/` tree. Exporting a packaged-component package into the application open graph does not grant it custom-operator syntax. Restricted domains may still overload FoLang-owned built-in or pre-declared operators.	**Resolved**
Dynamic multi-dispatch selection	`@co.ddap.dynamicdispatch(true)` selects from applicable overloads using the runtime argument-type tuple. Exact matches dominate; otherwise parent/supertype widening applies and the unique most-specific applicable overload is required. Reachable incomparable best candidates are compile-time ambiguities.	**Resolved**

Dynamic-dispatch ownership and library isolation	`@co.ddap.dynamicdispatch(...)` is valid only for an executable application and is a compiler error in every standalone library and every project-local component. Projected libraries/components retain statically resolved internal calls behind their surface APIs. The only integration exception is explicitly exported packaged-component or packaged-library code: after it joins the executable application's open graph, eligible call sites inherit that application's dynamic-dispatch mode.	**Resolved**
Packaged/open dynamic-dispatch participation	A packaged library/component cannot enable dynamic dispatch itself. Only package contexts explicitly exported from packaged producers join the consuming application's open semantic graph. When the executable application enables dynamic dispatch, those exported contexts and their participating overload call sites follow the application's dynamic-dispatch semantics; unexported packaged packages remain private and static.	**Resolved**
Callable identity and static overload selection	The owning context, callable name, and applicable callable/receiver category are resolved before overload selection. Within that identity, static overload resolution uses the compile-time argument-type tuple, nominal subtype-to-supertype applicability, and the unique most-specific applicable parameter signature.	**Resolved**
Overload-family return contract	Return types never distinguish ordinary named overloads. Sibling declarations in one ordinary overload family must have an identical declared return signature; neither static nor dynamic overload resolution performs implicit return widening or common-result inference. A single generic declaration may resolve a declared result generic through `mapping=` after parameter/generic resolution; this is generic instantiation, not return-type overloading.	**Resolved**
Operator overload result contract	Operator selection requires an exact normalized operand-type signature. Nominal subtype widening, numeric promotion, and `to` / `from` conversion do not make operator candidates applicable. Distinct operand signatures may declare different result types; the result type is part of the complete implementation contract but never disambiguates identical operand signatures.	**Resolved**
Generic result resolution, mapping, and augmentation	A generic return needs no `mapping=` when all generic markers required by the return signature are already resolvable from parameter-position generic information or explicit generic arguments. `mapping=` derives otherwise-unresolved generic markers from already-resolved inputs; expected/destination return context is never an inference source. Mapping augmentation is class-inheritance-only and augments the inherited generic method's effective mapping set as metadata; it is **not** a bodyless generic declaration. A bodyless generic declaration is an ordinary forward and may not carry `mapping=`. Package/free functions are not cross-package augmentation targets because package ownership gives them distinct callable identities.	**Resolved**
Dynamic-dispatch frontend universe	The frontend prepares and validates the closed dynamic-dispatch type/overload universe from application-owned types plus explicitly exported packaged types. Projected internals are excluded. Backend representation and runtime lookup strategy are implementation choices once the frontend semantic contract is emitted.	**Resolved**
Dynamic-runtime directive ownership	`@co.ddap.dynamicruntime` is valid only in a `dynamicvmrt` capability domain, whether standalone or project-local; using it elsewhere is a compiler error.	**Resolved**
Projected-surface generics	Projected `application`, `ffi`, `system`, and `dynamicvmrt` surfaces expose concrete boundary contracts only; generic declarations, generic type parameters, and `@co.dap.generic` are forbidden on the projected surface. Packaged exports are unaffected because selected declarations join the open graph rather than cross a projected boundary.	**Resolved**

Built-in collection construction	`co.core.List`, `co.core.Set`, and `co.core.Map` have exactly two current-alpha construction forms: a declared generic collection type with a no-arrow constructor on the right-hand side, or a type-deduced declaration whose constructor explicitly supplies `-> (...)` generic arguments. No third collection-constructor inference form exists.	**Resolved**
Standalone-library component topology	The full project-local component model belongs to executable applications. A standalone projected **application** library may have only `components/operators/`; every other component kind is forbidden there. Standalone packaged, `ffi`, `system`, and `dynamicvmrt` libraries must not contain `components/` at all. Reusable library dependencies are supplied through `lib/`, not through project-local components.	**Resolved**
Packaged-library consumer restriction	A standalone packaged library exposes open package contexts rather than a projected/safe API surface. Its exported packages may be loaded only into an executable application's primary open graph. Standalone libraries and project-local components may not import standalone packaged-library package contexts.	**Resolved**
Generic marker classification	Names listed by the immediately associated `@co.dap.generic(types=[...])` are generic markers in applicable type positions and are not ordinary type references. A type-position name not listed in that annotation is resolved normally through the type symbol table and is an error if unresolved. Generic-marker spelling alone does not distinguish duplicate generic signatures; ordinary parameter structure does.	**Resolved**
Generic metadata frontend policy	The frontend parses and preserves all generic annotation fields. Fields required for frontend type/call resolution are interpreted where necessary; backend- or later-stage-oriented fields are preserved in the frontend artifact and do not block Final AST generation merely because the frontend has no semantic handler for them.	**Resolved**
Runtime-loaded `dynamicvmrt` types	Runtime-created/loaded types may exist and inherit visible compiled types inside the isolated `dynamicvmrt` domain, but they do not extend the consuming application's compiled dynamic-multidispatch type/overload universe. Internal `dynamicvmrt` overload binding remains static.	**Resolved**
Capability hierarchy and dependency topology	`advanced` is removed. `application` is the normal full application language domain. `dynamicvmrt` adds the defined dynamic-runtime facilities to application capabilities. `system` and `ffi` remain isolated capability domains exposed through projected surfaces. Capability isolation does not impose a pairwise dependency matrix on **standalone projected libraries**: developers may compose them through published projected APIs and the compiler enforces those contracts rather than rejecting a dependency such as FFI-to-system merely because of its direction. Standalone packaged-library exports are different: they have no safe projected surface and may be consumed only by an executable application's open graph. Project-local components are executable-application composition domains; peer component-to-component consumption is forbidden.	**Resolved**

Open Follow-Up Items

The following details remain intentionally open and must be settled before the affected semantics are considered frozen:

1. **Explicit cast syntax in the dynamic-dispatch example.** The example currently uses `(Dog)some1`, but this document does not otherwise define a normative cast syntax. Either define the FoLang cast form or remove/replace that example.
2. **Extension target/adoption model.** The current extension section still uses fixed `fortype=...` targeting, while the newer class-adoption design using `@co.dap.oops(... uses=true ...)` has not yet been made normative. One model should be selected and the other removed.
3. **Pointer capability placement.** Pointer/fat-pointer/address forms need a direct normative scope statement distinguishing what is legal in `system`, what limited ABI forms are legal in `ffi`, and what is forbidden in application/packaged/dynamicvmrt domains.
4. **Unknown built-in `@co.*` metadata.** Appendix C currently permits syntactically valid built-in metadata with no semantic handler to be silently ignored. Whether unknown language-owned metadata should instead be a compiler error remains a policy decision.

When an open item is settled, its normative section must be updated first; this log should then move the item into the resolved table with a concise record of the adopted rule.

Design Overview

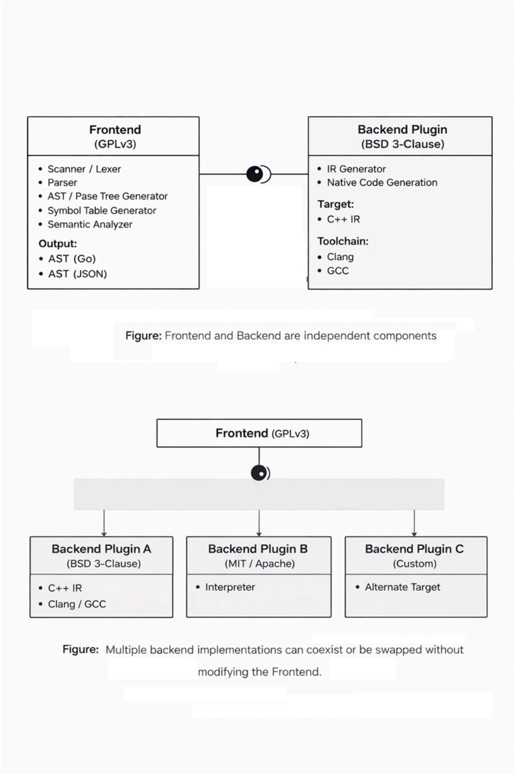


Figure: Frontend and Backend are independent components

Figure: Multiple backend implementations can coexist or be swapped without modifying the Frontend.

FoLang follows a deliberately different approach from conventional programming language designs. The system is structured to ensure **clear separation of concerns**, **license isolation**, and **extensibility through well-defined integration boundaries**.

Compiler and Backend

1. Frontend

The Frontend is responsible for source-level analysis and semantic processing of FoLang programs.

Frontend Components

- Scanner / Lexer
- Parser
- AST / Parse Tree Generator
- Symbol Table Generator
- Semantic Analyzer

Implementation

- Implemented in **Go**
- Uses Go structures internally for AST, symbol-table, and semantic processing.
- The externally consumable frontend output is serialized according to the selected backend's interchange contract.
- The frontend writes that artifact beneath the reserved project-root `build/` domain.
- The backend-supplied contract identifies the supported FoLang/plugin protocol version, HIR schema version, and wire format.
- The backend installs/provides this interchange-contract file in the same installation directory as the FoLang compiler executable.
- The frontend reads that installed backend contract to determine the interchange representation required by the selected backend.
- Canonical `build/` presence, ownership, and source-discovery rules are defined in Project Layout.

License

- **GNU General Public License v3 (GPLv3)**

---

2. Backend

The Backend is responsible for transforming validated frontend output into executable artifacts.

> **The default backend must be downloaded or built separately. It is not bundled with the frontend binary.**

Backend Components

- Intermediate Representation (IR) Generator
- Native Binary Executable Generation

Implementation

A backend may be implemented in any language. It consumes the validated frontend artifact from the project-root `build/` directory. The artifact encoding is selected by the backend-supplied interchange contract.

During backend installation, that interchange-contract file is placed in the same installation directory as the FoLang compiler executable. The frontend reads the installed contract to determine the compatible FoLang/plugin protocol version, HIR schema version, and wire format required by the selected backend. The exact artifact basename and format-specific schema/version may therefore vary by backend contract, but the artifact location is always beneath `<project-root>/build/`.

Default / Reference Backend

The default FoLang backend is also the **reference backend implementation**. Its purpose is to provide an executable, inspectable example of how the FoLang specification can be implemented and to give backend implementers a concrete behavioural baseline for conformance testing.

The written FoLang specification remains normative. The reference backend demonstrates the required externally observable semantics, but its internal algorithms, allocation strategy, memory-management choices, data structures, optimization level, and performance characteristics are not themselves language requirements unless this specification explicitly says otherwise. If an implementation defect in the reference backend conflicts with the written specification, the written specification takes precedence.

A third-party backend may use a completely different runtime architecture or memory model and may optimize semantics differently, provided that FoLang programs observe behaviour conforming to this specification. Backend implementations may validate their behaviour against the reference backend and the FoLang conformance tests where applicable.

- Backend orchestration is implemented in **Go**
- Code generation target is **C++**
- Uses **Clang** or **GCC** to generate native binaries from generated C++ IR

License

Third-party backends may use their own licensing terms and implementation choices. The default backend uses the following license. **Default backend is not part of the complete compiler binary and is separate**; it must be downloaded or built separately.

- **BSD 3-Clause License** ***

Frontend Output Contract

Installed Backend Interchange Contract

The selected backend supplies this contract to tell the frontend which FoLang/plugin protocol, HIR schema, and wire format it accepts. During backend installation, the contract file is placed in the same installation directory as the FoLang compiler executable. The frontend reads it from that installation location. The default backend uses the same mechanism.

```
{
  "protocol":   "folang-plugin/1.0",
  "hir_schema": "folang-hir/1",
  "wire":       "protobuf",
}
```

The frontend has one fixed **location** contract and a backend-selected **encoding** contract:

```
<project-root>/build/
    └── <frontend-artifact>    encoding/version selected by backend contract
```

Rules:

- the frontend/backend interchange artifact is written beneath the reserved root-level `build/` domain;
- the selected backend supplies the supported protocol version, HIR schema version, and wire format through its installed interchange-contract file;
- that contract file resides in the same installation directory as the FoLang compiler executable;
- `wire="protobuf"` in the example above means that particular backend contract requests Protocol Buffers; another supported backend contract may request a different compatible wire format/version;
- backends consume the validated frontend artifact from `build/`;
- canonical filesystem rules for `build/` are defined only in Project Layout.

Licensing Summary

Layer	Responsibility	Implementation	License
Frontend	Parsing and semantic analysis	Go	GPLv3
Backend (default)	IR processing and native code generation	Go (orchestration) + C++ (target)	BSD 3-Clause

> The copyrightable material in the FoLang Language Definition and Documentation, including its syntax, grammar, and semantic-rule descriptions, is licensed separately under CC BY 4.0. ***

3. Capability Security Model

FoLang's compiler ships with all language features compiled in but **systems and FFI features are disabled by default**. The compiler has no hardcoded keys — capability configuration happens entirely at install time. This moves authorization from source code (developer-controlled) to the compiler installation (organization-controlled).

Capability Domains and Install-Time Gates

Domain	Features	Default State
`application`	Ordinary FoLang language/application capabilities, including macros, templates, concurrency/control abstractions, `co.net`, `co.core`, `co.encoding`, `co.crypto`, etc.	✅ enabled
`dynamicvmrt`	application capabilities plus the defined dynamic-runtime / `co.meta` facilities	explicit source capability domain; toolchain policy may restrict availability
`system`	Raw pointers, pointer arithmetic, `@co.dap.native`, `co.native`, `co.sys.unsafe`, MMIO, heap allocators, low-level platform/runtime implementation	🔒 disabled by default — requires install-time configuration
`ffi`	`co.sys.ffi`, extern declarations/types, foreign calling conventions and linkage, `co.lang.void` pointers, C ABI	🔒 disabled by default — requires install-time configuration

`packaged` is an exposure model using application capabilities, not a separate privileged capability domain.

Quick Start

Hello World

```
// src/appl.fol — fixed application entry file, no annotation needed
co.out.println("Hello FoLang!");
```

Or with an alias for shorter form:

```
@co.ddap.alias(co.out, as="out")

out.println("Hello FoLang!");
```

Variables

```
// typed
name co.lang.string = "Rao";
age  co.lang.int    = 30;

// inferred from value — := errors if already declared
name := "Rao";
age  := 30;

// define, infer, and assign if not defined; otherwise assign a new value
name ?= "Kumar";
```

`co.lang.string` and `co.lang.int` are built-in data types. For more information, see Builtin Data Types.

`=`, `:=`, and `?=` are built-in operators. For more information, see Builtin Operators.

Constants and Immutability

FoLang separates two properties that are often confused. One is about *when* a value is known. The other is about *whether* it can change.

```
// compile-time constant — value known while compiling, substitutable
@co.dap.const SIZE co.lang.int = 1024;

// immutable binding — cannot be reassigned, value need not be known early
@co.dap.final startedAt co.lang.int = co.sys.now();
```

Annotation	Guarantees	Value known at compile time	Usable as an index
`@co.dap.const`	value is fixed and known while compiling	✅	✅
`@co.dap.final`	binding cannot be reassigned	❌ not required	❌

Every `@co.dap.const` is also immutable. The reverse does not hold: a `@co.dap.final` binding may be initialised from a function call, a file, or user input, so the compiler cannot substitute a literal for it.

`@co.dap.final` is the declaration-site form of the immutable object kind described in the object mutation policy. `makeImmutable` applies the same property to a value at run time.

Outside a declaration whose signature binds a dependent index parameter, only `@co.dap.const` may supply a named array size or dependent type index. A signature-bound index name is permitted symbolically even when its concrete value is supplied by the caller. See Dependent Type Index Rules.

Single Source Application File

FoLang developers can create a complete executable program in one source file. A **single-source application** is an application whose entry file contains the complete program and does not depend on user package source files.

A single-source application file and an application entry file are the same fixed structural source, `src/appl.fol`, and use the same entry-file grammar, context, and restrictions. A project is a single-source application when `src/appl.fol` contains the complete program and there are no application package directories below `src/`. This section presents the allowed constructs, so a developer can start programming without first reading the complete specification.

> For the single-source application layout, see Project Layout.

Allowed Constructs

The application file may contain:

- built-in directives that are valid for an entry file
- package and library imports
- import aliases declared with `as=`
- file-local aliases for `co.*` paths declared with `@co.ddap.alias`
- type aliases and ADTs declared with `co.lang.type`
- parameterized `co.lang.type` constructors such as `Option(T) co.lang.type = Some(T) | None()`
- new types declared with `co.lang.newtype`
- opaque types declared with `co.lang.opaquetype`
- dependent-type aliases and dependent-type usages that do not declare an ordinary or type-level function
- refinement-type declarations
- subtype declarations
- supertype declarations
- non-capturing entry-local function-pattern groups
- capturing entry-local `let` function-pattern groups
- variable declarations, initialization, assignment, and mutation
- calls to built-in methods and functions
- calls to imported package and library APIs
- ordinary expressions
- comprehensions
- ternary expressions
- conditions and loops
- containment operations such as `contains`
- iterators such as `each`
- pattern matching and destructuring

```
a co.lang.int = 10;
b co.lang.int = 20;

co.out.println( a + b);
```

> `co` is a reserved word in FoLang. For more information, see Reserved Words.

> `co` is the built-in root package in FoLang. For more information, see Builtin Packages.

> `a + b` is an expression in FoLang.

> For the expression rules, see Expressions.

Built-in and Imported Names

All `co.*` paths are always available.

A developer may use the complete built-in path:

```
co.out.println("Hello");
co.core.List.of(1, 2, 3);
```

A developer may optionally create a file-local alias:

```
@co.ddap.alias(co.out, as="out")
@co.ddap.alias(co.core.List, as="list")

out.println("Hello");
values := list.of(1, 2, 3);
```

Creating an alias does not hide the complete `co.*` name; both forms remain valid in that file.

Third-party packages, user packages, and libraries are not automatically available. They must be imported using `@co.ddap.import`. When `as=` is present, the imported API is accessed through that alias. When `as=` is omitted, the complete imported package or library path must be used.

```
@co.ddap.import(package="hr.employee", as="emp")
first := emp.EmployeeService.find(1001);

@co.ddap.import(package="finance.payroll")
second := finance.payroll.calculate(request);
```

A developer must import a package before using it.

> `@co.ddap.import` and `@co.ddap.alias` are built-in directives. For more information, see Built-in Directives.

> For more information about FoLang imports, see Import Details.

> `println` is a built-in method on the `co.out` object.

Variable Kinds

FoLang supports several variable kinds for different purposes. Their availability is context-dependent; a developer cannot use every variable kind in every location. For the contexts in which each kind is supported, see Variable Kind Support.

Simple Variable Declaration

```
someVar co.lang.int;
someString co.lang.string;
```

With Initialization

```
someBool co.lang.bool = co.const.true;
someInt co.lang.int = 42;
```

With Type Inference

```
someVal := "Hello, World!";
someNum := 3.14;   // if not defined, define and initialize; else throws error
someR ?= "Kamesh"; // if not defined, define and initialize; else reassign
```

Alpha Character and String Literals

The alpha release accepts only the basic literal subset:

```
message := "hello";
letter := 'A';
```

A string is unprefixed, remains on one source line, and cannot contain a backslash or an embedded double quote. A character literal contains exactly one non-backslash character.

Encoding prefixes, escaped characters, raw strings, and universal character names are reserved for a later release. The scanner recognizes the complete spelling and reports an unsupported-feature error instead of tokenizing it as several unrelated expressions. For example, all of these are rejected:

```
x := "quoted: \"text\"";
x := '\n';
x := u8"hello";
x := R"(raw text)";
x := '\N{LATIN CAPITAL LETTER A}';
```

Pointer Declaration

```
somePtr    co.lang.int->(*);
someDblPtr co.lang.int->(**);
someDeepPtr co.lang.int->(*****);
```

The number of consecutive `*` characters is the pointer degree. Any positive degree is permitted; `*` and `**` above are common examples, not a maximum.

Array Declaration

```
someArray       co.lang.int->([5]); // single dimension
someDblArray    co.lang.int->([2,3]); // multi dimension
someJaggedArray co.lang.int->([2][3]); // jagged
someVLArray     co.lang.int->([...]); // variable length
someZeroLA      co.lang.int->([0]); //zero length array
someZeroDimA    co.lang.int->([.]); // zero-dimensional array
```

Array Declaration with Initialization

```
someInitializedArray    co.lang.int->([3])  = [1, 2, 3];
someInitializedArray1   co.lang.int->([])   = [1, 2, 3];
someInitializedDblArray co.lang.int->([,])  = [[1, 2], [3, 4]];
```

Reference Declaration

```
someRef       co.lang.int->(&);    // reference
someLValueRef co.lang.int->(&&);   // LValue reference
someHpRef     co.lang.int->(~);    // heap allocated reference
someAddr      co.lang.int->(@);    // address
someThunk     co.lang.int->(^);    // thunk
someSlice     co.lang.int->([:]);  // slice
```

Range Declaration

```
// Typed range variable declaration
someRange co.lang.int->(..);

// Inferred range declarations
rangeI := 1 .. 10;      // [1, 10]   ExcludeStart=false, ExcludeEnd=false
rangeS := 0 <.. 5;      // (0, 5]    ExcludeStart=true,  ExcludeEnd=false
rangeL := 0 ..< 100;    // [0, 100)  ExcludeStart=false, ExcludeEnd=true
rangeB := 0 <..< 100;   // (0, 100)  ExcludeStart=true,  ExcludeEnd=true
rangeE := .. 100;      // open lower bound  (_, 100]
rangeF := 1 ..;        // open upper bound  [1, _)
```

Auto and Dynamic Variable Declaration

```
someAutoVar    co.lang.auto    = "Hello"; // type inferred from value; initialization required
someDynamicVar co.lang.dynamic;           // dynamic typing
```

Lazy

```
@co.dap.lazy
x = add(1, 2);  // evaluating/using x invokes add(1, 2); until then the right-hand function is not invoked
```

Bind Variables

`$[1-9][0-9]*`

Bind variables are available in function-chaining expressions. At each chained call, `$1`, `$2`, `$3`, ... denote the return components of the **immediately preceding function invocation**, in declared return order. If the preceding function returns one value, only `$1` is available for that step; if it returns multiple values, `$1` through `$N` correspond to those return values. `$0` is not a bind variable.

```
dosomething(a co.lang.int, b co.lang.int)->(co.lang.int)
    =>> somePack.someMethod(a)
    =>> someOthPack.someOtherMeth($1, b);

// If previous() returns (A, B, C), the next chained call may consume
// those return components as $1, $2, and $3 respectively.
previous()
    =>> consume($1, $2, $3);
```

Discard / Wildcard Variable

`_`

`_` is contextual rather than an ordinary identifier. In discard/wildcard positions it means "ignore this binding or match position." It is also used as the filename-derived primary-declaration placeholder and as an unnamed type-parameter slot in type-constructor forms such as `F(_)`. In the predicate of a `co.lang.refinementType` declaration, `_` instead denotes the candidate value of the base type being tested. See Refinement Types.

Comma and Grouping

```
// Comma
x co.lang.int = 10, y co.lang.string = "Hello", z co.lang.bool = co.const.true;

// Grouping
(x co.lang.int = 10, y co.lang.string = "Hello", z co.lang.bool = co.const.true);

// Tuple/grouping expression
pair := (x, y);
```

A comma inside a parenthesized expression or grouped declaration must introduce another expression or declarator. Therefore `(x,)`, `(x, y,)`, and `(x co.lang.int = 10,)` are invalid. Collection literals have their own rules and may permit a trailing comma where their production says so. Record patterns also require another field after every comma; `Employee{id: value,}` is invalid.

Fat Pointers

```
x co.lang.int->(*, kind="", meta={});

co.lang.int->(*, meta={});

y co.lang.int->(*, meta={len:co.lang.usize, vtab:somepkg.VTable->(*)});

z co.lang.int->(*,kind=region, meta={});
```

```
Pointer
├── base_type: T
├── kind: <FatKind>
│     ├── thin
│     ├── slice
│     ├── relative
│     ├── trait
│     ├── buffer
│     ├── view
│     ├── opaque
│     ├── custom
│     ├── mem
│     ├── nullptr
│     ├── sptr
│     ├── uptr
│     ├── ptrdiff
│     ├── usize
│     ├── ssize
│     └── (region)  ← optional syntactic sugar
└── meta:
      ├── region: heap | stack | global | numa(N) | mmio | constant | …
      ├── len, cap, vtab, bits, endian, …
```

Pointers for address manipulation

```
y co.lang.word->(repr=intptr);
z co.lang.word->(sign=unsigned, repr=uintptr);
p co.lang.word->(repr=ptrdiff);
n co.lang.word->(sign=unsigned, repr=usize);
m co.lang.word->(repr=isize);
o co.lang.void->(repr=nullptr);
```

Relative Pointers

```
z co.lang.int->(*,kind=relative, meta={});
```

Note Other than normal variable declaration rest have restrictions

Conditions, Loops and Iterators

Conditions

`then` is the one-shot conditional branch verb. Its argument may be a block or an ordinary value/expression. `otherwise(condition)` always introduces another Boolean condition; a conditionless `otherwise` form does not exist. `default(result)` is the optional terminal fallback and may likewise receive a block or value.

```
(boolean truth).then({
}).otherwise(boolean truth).then({
}).default({
});

//conditionEg1.unit.fol

_ co.lang.unit = {

    someFun()->()={

        x := co.const.true;

        x.then({

        }).default({

        });
    }

    someOtherFun()->()={

        (co.const.true).then({  //parenthesis mandatory

        }).default({

        });
    }

    someOtherFun1()->()={
        x co.lang.bool =co.const.true;
        (x).then({   // parentheses optional

        }).default({

        });

    }

    someOtherDiffFun1()->()={

        x co.lang.int=10;
        y := 30;

        (x > y).then({     // parentheses around (x > y) are mandatory

        }).otherwise(x < y).then({

        }).default({

        });
    }
}
```

Loops

`loop` is the repeated-execution verb and always has exactly one loop condition and one loop body. Unlike `then`, a `loop(...)` form cannot participate in an `otherwise(condition)` chain, and `default(...)` cannot follow a loop. When the loop condition is false, whether on the first test or after one or more iterations, the loop simply terminates.

To choose between different looping behaviours, first use a `then` / `otherwise(condition)` / `default` selection chain and place the required ordinary loop inside the selected block. The selection chain chooses a behaviour; each nested `loop(...)` then performs repetition using only its own condition.

```
(boolean truth).loop({
});


//loopsEg1.unit.fol

_ co.lang.unit = {

    someFun()->()={

        x := co.const.true;

        x.loop({
        });
    }

    someOtherFun()->()={

        (co.const.true).loop({  //parenthesis mandatory
        });
    }

    someOtherFun1()->()={
        x co.lang.bool =co.const.true;
        (x).loop({   // parentheses optional
        });

    }

    someOtherDiffFun1()->()={

        x co.lang.int=10;
        y := 30;

        (x > y).loop({     // parentheses around (x > y) are mandatory
        });
    }
}
```

```
//loopsEg2.unit.fol

_ co.lang.unit = {

    someFun()->()={

        x := co.const.true;
        v := 0;
        x.loop({
            (v == 10).then({
                this.break;
            });
            v += 1;
        });
    }

    someOtherFun()->()={
        v := 20;
        (co.const.true).loop({  //parenthesis mandatory
            (v == 30).then({
                this.continue;
            });
            v += 5;
        });
    }


}
```

Combining Conditions and Loops

A loop does not become a branch verb inside an `otherwise(condition)` chain and has no `default(...)` fallback of its own. When a program must choose between different looping behaviours, the selection is expressed separately with `then` / `otherwise(condition)` / `default`, and each selected block may contain its own ordinary single-condition loop.

```
(first condition).then({
    (first loop condition).loop({
        ...
    });
}).otherwise(second condition).then({
    (second loop condition).loop({
        ...
    });
}).default({
    ...
});
```

Ternary Operator

The ternary/value form uses the same `then` / `otherwise(condition)` / `default` selection vocabulary as block conditionals. Only the branch arguments differ: a value-producing chain supplies values or expressions instead of statement blocks.

```
s = (boolean truth).then(some var/value).default(some val/var);
s = (boolean truth).then(some var/val).otherwise(boolean truth).then(some var/val).default(some var/val);

//TernaryExample.unit.fol

_ co.lang.unit = {

    someFunction()->()={

        s := (co.const.true).then(20).default(10);
        y co.lang.int;
        z co.lang.int=20;
        y ?= (co.const.true).then(20).default(z);


    }

    someOtherFunction()->()={

        k co.lang.int=10;
        p co.lang.int =20;
        s ?= (k>10).then(30).otherwise(k<10).then(p).default(10);

    }
}
```

Parenthesizing the chain head

A conditional or ternary selection chain is a sequence of postfix method calls using `.then(...)`, `.otherwise(condition)`, and optional terminal `.default(...)`. A loop uses the same postfix-call style but is a single-condition repetition form: `.loop(...)` cannot be followed by `.otherwise(condition)` or `.default(...)`.

These calls always carry their own call parentheses, whatever the argument is. Nothing about the argument changes that.

`otherwise` always has the form `.otherwise(condition)` and therefore never acts as a terminal else marker. The terminal fallback is `.default(...)`; its argument may be a block or a value/expression according to the applicable form.

The only place a choice exists is the **head**: the subject before the first `.then(...)` or `.loop(...)` call. The head must already be a complete postfix expression that cannot absorb the following control verb.

Chain head	Parentheses	Why
`x`	optional	an identifier is already a complete postfix expression
`arr[0]`, `f()`	optional	index and call suffixes are postfix too
`co.const.true`, `myPkg.flag`	**required**	a qualified name would absorb `.then` as a further segment, giving `co.const.true.then`
`x > y`	**required**	`.then` binds tighter than `>`, so `x > y.then({...})` groups as `x > (y.then({...}))`

A qualified name needs parentheses whether it names a literal or an identifier, so `myPkg.flag` is the same case as `co.const.true`.

```
x.then({ ... });                                       // head is an identifier
(co.const.true).then({ ... });                         // head is a qualified name
(x > y).then({ ... }).otherwise(x < y).then({ ... });    // head is an expression
(k > 10).then(30).otherwise(k < 10).then(p).default(10);
```

In the last two lines the parentheses after `.otherwise` are its call parentheses, not an application of this rule.

Iterating Arrays / Lists / Maps / Ranges

`each` is itself the element-iteration operation. It does not produce a value that must subsequently be passed to `loop`, and `.loop(...)` must not follow an `each(...)` call.

The explicit-binding form takes the index/key binding, the value binding, and the per-element action directly. The action may be a block or any ordinary expression valid in that position. A callable may also be supplied directly as the sole callback argument; the callable receives the receiver’s defined iteration tuple. A lambda is a callable callback and is permitted directly in this collection-operation context.

```
arr co.lang.int->([5]) = [6,7,8,9,10];

// block action
arr.each(idx, val, {
    co.out.print(idx);
    co.out.print(" :: ");
    co.out.println(val);
});

arr.each(_, val, {
    co.out.println(val);
});

// expression action — evaluated once for each element
arr.each(_, val, co.out.println(val));

// direct callable callback — receives the receiver's iteration tuple
arr.each(printItem);

// direct lambda callback
arr.each(|idx, val| => co.out.println(val));
```

The explicit-binding form establishes its bindings separately for every iteration and evaluates its action exactly once for that element. The action is therefore the body of `each`; `each` never participates in a `then`, `otherwise`, `default`, or `loop` chain. In the single-argument callback form, the argument must resolve to a callable compatible with the receiver's iteration tuple.

Array / List / Map / Range — Contains Element

```
arr co.lang.int->([5]) = [35,57,96,81,31];
k co.lang.int = 31;
arr.contains(k).then({
    co.out.println(k);
}).default({
    co.out.println("Not Found");
});
```

Comprehensions

```


k := (1 .. 10).filter(|x| => x % 2 == 0).map(|x| => x * x);

result := for (x <- co.core.List->(co.lang.int)[1,2,3]).yield(x * 2);        // co.core.List->(co.lang.int)[2, 4, 6]
result := for (x <- co.core.Set->(co.lang.int)(1,2,3)).yield(x * 2);         // co.core.Set->(co.lang.int)(2, 4, 6)
result := for (x <- Some(5)).yield(x * 2);             // Some(10)
result := for (x <- fetchData()).yield(x.process());   // Future

ages := co.core.Map->(key=co.lang.string, val=co.lang.int){"A":30,"B":40,"c":66,"e":88};
upper := for ((name, age) <- ages).yield(name.toUpperCase, age);
```

Pattern Matching

```
x co.lang.int = 10;

x.match.case(n: n > 10 => { n = n+100; "GT" }).case( n: n < 10 => "LT").default("EQ");
x.match.case(n: n > 10 => { n = n+100; "GT" }).case( n: n < 10 => "LT").case(_=>"EQ");
x.match(co.pattern.Type).case(co.lang.int => ...).case(co.lang.float => ...);
x.match(co.pattern.Value).case(0 => ...).case(1 => ...);
x.match(co.pattern.Instance).case(xx.CAT => ...).case(xx.DOG => ...).default("Animal");
x.match(co.pattern.Object).case(xx.Ball => "Ball").case(xx.CAT => "CAT").default("Unknown");
x.match(co.pattern.Shape).case(Point{x, y} => ...).default(...);

x.match(co.pattern.Any).case(co.lang.int => ...).case(co.lang.float => ...).case(0 => ...).default( ...);

x.match(PositiveEvenMatcher).case(0 => "Neither even nor odd").case(2 => "First Even Prime").default(...);
```

A match chain contains one or more `.case(...)` arms followed by at most one terminal `.default(...)` arm. `.otherwise` is not a match arm; it always introduces another Boolean condition in conditional `then` chains. Loops do not participate in `otherwise(condition)` chains. `default` is the common terminal fallback vocabulary for Boolean selection and match/case selection. A case or default result may itself be a ternary then expression, and that nested expression may consequently contain `.default(...)`.

For value dispatch, `match().case(...).default(...)` is also FoLang's generalized analogue of a traditional `switch` / `case` / `default` construct; match cases additionally support the pattern, type, object, and custom-matcher forms defined by this specification.

> `_` is a special discard/wildcard variable. In a call it is permitted only as the first, index/key binding of the explicit receiver-qualified `.each` form, as in `items.each(_, value, { ... })`. Transparent grouping around the member callee, for example `(items.each)(_, value, { ... })`, does not change this rule. The value binding and the iteration-action argument of `each` cannot be discarded, and `contains(_)` and `containsVal(_)` are invalid because containment must compare a real value. Patterns and the filename-derived declaration-name form, type-constructor placeholder forms such as `F(_)`, and refinement-type predicates give `_` their own explicitly described meanings. In a refinement predicate, `_` denotes the candidate value being tested; it is not a general expression identifier.

> `PositiveEvenMatcher` is a custom matcher. For more information about defining custom matchers, see Custom Matcher.

Type Declarations

```
// Alias
x co.lang.type = co.lang.int;

// New
x co.lang.newtype = co.lang.int;

// Opaque
EmpIdType co.lang.opaquetype = co.lang.int;
DeptIdType co.lang.opaquetype = co.lang.int;

empId EmpIdType = 10;          // valid: base representation is accepted when constructing EmpIdType
deptId DeptIdType = 20;        // valid

empId = deptId;                // invalid: distinct opaque types are not assignment-compatible
empId2 EmpIdType = empId;      // valid: same opaque type

x co.lang.int = empId;         // invalid: an opaque value does not implicitly become its base type


// ADT (tagged union)
y co.lang.type = co.lang.int | co.lang.char;

// Proper subtypes of Employee; Employee itself is excluded.
// Employee is a class.
empSubType co.lang.subtype = somePackage.Employee;

// PermanentEmployee inherits Employee.
permanentEmp empSubType = PermanentEmployee{};      // valid: proper subtype

// ContractualEmployee inherits Employee.
contractualEmp empSubType = ContractualEmployee{};  // valid: proper subtype

dancerEmp empSubType = DashingDancer{};            // compiler error: not a subtype of Employee
baseEmp   empSubType = Employee{};                  // compiler error: Employee itself is excluded

// To accept Employee itself together with all proper subtypes:
empPlusType co.lang.type = empSubType | Employee;

// Proper supertypes of Toyota; Toyota itself is excluded.
superToyota co.lang.supertype = somePackage.Toyota;

// somePackage.Toyota extends somePackage.Car extends somePackage.FourWheeler extends somePackage.Vehicle

toyota  superToyota = somePackage.Toyota{};        // compiler error: Toyota itself is excluded
car     superToyota = somePackage.Car{};           // valid
four    superToyota = somePackage.FourWheeler{};   // valid
vehicle superToyota = somePackage.Vehicle{};       // valid
truck   superToyota = somePackage.Truck{};         // compiler error: not a supertype of Toyota

// To accept Toyota itself together with all proper supertypes:
superToyotaPlus co.lang.type = superToyota | somePackage.Toyota;


// Refinement type
positiveInt co.lang.refinementType = (co.lang.int).where(_ > 0);

percentage co.lang.refinementType =
    (co.lang.int).where(_ >= 0 && _ <= 100);

evenInt co.lang.refinementType =
    (co.lang.int).where(_ % 2 == 0);

nonEmptyString co.lang.refinementType =
    (co.lang.string).where(_.length > 0);

// Inside a refinement predicate, _ denotes the candidate value of the base type.
```

> For the normative refinement-type rules, see Refinement Types.

For `co.lang.opaquetype`, the declared base type supplies the representation accepted where opaque-type construction permits it, but the resulting opaque type has distinct type identity. Distinct opaque types are not assignment-compatible merely because they share the same base type, and an opaque value is not implicitly assignable back to its base type. Thus `x co.lang.int = empId;` is invalid when `empId` has type `EmpIdType`.

`co.lang.subtype` and `co.lang.supertype`

`co.lang.subtype` and `co.lang.supertype` define type sets for these two declaration kinds only. Their semantics do not define or replace inheritance, interface, generic, object-model, variance, or other assignability rules elsewhere in FoLang; those facilities retain their own independently defined semantics.

A declaration of the form:

```
TSub co.lang.subtype = BaseType;
```

defines a type whose admissible values have concrete types that are **proper transitive subtypes** of `BaseType`. `BaseType` itself is excluded. Direct and indirect subtypes are included, while unrelated types are excluded.

To admit `BaseType` itself together with those proper subtypes, form an explicit union:

```
TSubPlusBase co.lang.type = TSub | BaseType;
```

A declaration of the form:

```
TSuper co.lang.supertype = BaseType;
```

defines a type whose admissible values have concrete types that are **proper transitive supertypes** of `BaseType`. `BaseType` itself is excluded. Direct and indirect supertypes are included, while unrelated types are excluded.

To admit `BaseType` itself together with those proper supertypes, form an explicit union:

```
TSuperPlusBase co.lang.type = TSuper | BaseType;
```

Let Bindings

```
y co.lang.int = let({x = 10}).in({x + 1});
y co.lang.int = let({$ = 10}).in({$ + 1});  // $ refers to the value being defined

x co.lang.int = (x + 1).where(x = 10);
x co.lang.int = ($ + 1).where($ = 10);

offset := 100;

let adjust(0) = offset;
let adjust(n) = n + offset;
```

> `$` is a special identifier usable in ordinary `let` binding expressions for recursive or self-referential expressions.
>
> Ordinary `let` value-binding expressions remain available in language contexts that permit them, but they are forbidden directly in the application entry file. In the entry file, `let` is reserved exclusively for a named function-pattern group that captures at least one surrounding runtime binding. It cannot introduce an anonymous function, a general closure value, or a curried function.

Function Pattern

```

Option(T) co.lang.type= Some(T) | None();

f(Some(x)) => { this.return x + 1; }
f(None())  => { this.return 0; }

// desugars to:
f(v Option(co.lang.int))->(co.lang.int) = {
    this.return v.match()
        .case(x: Some(x) => x + 1)
        .case(_: None() => 0);
}
```

`=>` introduces a bare function-pattern clause. `=>>` is the distinct function-delegation operator, while `==>>` is the closure/curry expression; neither introduces a function pattern.

Function-pattern groups are permitted in the application entry file as restricted entry-local dispatch helpers. A bare group cannot capture surrounding runtime variables. A `let` function-pattern group must capture at least one already initialized entry-file runtime binding and is the only entry-file construct that permits such capture. Neither form permits ordinary function declarations, anonymous functions, general closure values, currying, partial application, or escape as a function value.

> For more information about `let` and function patterns, see Function Pattern.

> A single-source application file is useful for testing FoLang and becoming familiar with the language. Real-world applications normally contain more than a single source file. They use abstraction, encapsulation, inheritance, polymorphism, and other language features, and they often depend on external packages and libraries to establish clear boundaries. At minimum, such applications require the following structural features:

1. package source files under packages
2. Entry File
3. Libraries and Project Layout
4. imports

Foλang supports many features for developing enterprise applications. The following list should be read together with the language intent and FoLang Philosophy — Uniform Object Model.

Complete feature list:

1. Project Layout
2. Packages

In FoLang, file-backed primary declarations use their own `<Name>.fol` files. Package functions and non-UDT type declarations are grouped in any number of `*.unit.fol` files, while struct-associated behavior is placed in `<StructName>.comp.unit.fol`. These are all package source files. The following sections begin with the canonical project layout and package model before moving to UDTs and functions.

Project Layout

This section is the **single canonical definition of FoLang project filesystem structure**. It defines the project-root domains, their allowed direct contents, structural source files, component slots, package roots, and artifact locations. Later sections describe application, library, component, packaged/export, operator, import, and compilation semantics without redefining these filesystem rules; they link back here when structural context is needed.

Compilation Invocation

The compiler is invoked with the project root:

```
folangcc <project-root>
```

The basename of the canonical project-root directory is the logical project name and the default output basename.

Root Domains

A FoLang project uses four standardized root domains:

```
<project-root>/
├── src/          <- mandatory primary project source
├── components/   <- optional project-owned isolated/specialized source
├── lib/          <- optional compiled .folenc dependencies
└── build/        <- compiler-managed generated output
```

These names are compiler-defined filesystem domains. They are not packages and never contribute package-name components.

Root domain	Presence	Allowed direct content	Not allowed
`src/`	mandatory, non-empty	exactly one primary surface (`appl.fol` or `component.fol`) plus package directories	both primary surfaces, no primary surface, any other direct file
`components/`	executable application: optional; projected application library: optional only as `components/operators/` ; every other standalone library: forbidden	application component kinds listed below, or the single `operators/` exception for a projected application library	unknown immediate child, empty component root, or any component tree forbidden by the project kind
`lib/`	optional; non-empty when present	one or more direct `<name>.folenc` compiled artifacts	FoLang source files or non- `.folenc` project content
`build/`	compiler-managed	generated frontend/backend artifacts	source/package discovery input

If `components/` or `lib/` is unused, it is omitted rather than created empty. `build/` may be absent before compilation; the compiler creates/manages it as needed.

`src/` — Primary Project Source

`src/` contains exactly one primary project surface:

```
src/appl.fol
```

or:

```
src/component.fol
```

The two are mutually exclusive:

- `src/appl.fol` classifies the project as an executable application;
- `src/component.fol` classifies the project as a standalone distributable component project;
- both present -> compile-time project-layout error;
- neither present -> compile-time project-layout error.

No other file may occur directly in `src/`. Every other direct entry under `src/` must be a non-empty package directory containing valid FoLang package source. Ordinary project package dot paths begin below `src/`.

Every `src/component.fol` contains exactly one `_ co.lang.component = { ... }` structural declaration. The source then selects one of two mutually exclusive standalone exposure models:

1. **projected library** — `@co.dap.library` annotates the component declaration. Omitted `type` means `application`; explicit projected kinds are `application`, `dynamicvmrt`, `system`, and `ffi`;
2. **packaged library** — no `@co.dap.library` is present, and the component body contains the applicable `@co.dap.export(packages={...})` selector. Selected `src/` package contexts are the distributable open-package surface.

`system`, `ffi`, and `dynamicvmrt` standalone libraries are always projected and therefore require `@co.dap.library(type=...)`; they cannot use the packaged exposure model. Standalone libraries do not use project-local components as a library-composition mechanism: reusable dependencies belong in `lib/` and are consumed as standalone libraries. The sole structural exception is a projected **application** library, which may contain exactly one optional project-local component kind, `components/operators/`, because custom operator syntax must be registered while compiling that library's own source. A projected application library may not contain `components/application/`, `components/ffi/`, `components/system/`, `components/dynamicvmrt/`, or `components/packaged/`. Standalone packaged, `ffi`, `system`, and `dynamicvmrt` libraries must not contain a `components/` tree at all. Restricted library forms may still overload FoLang-owned built-in or pre-declared operators, but they cannot create new operator spellings. Detailed standalone semantics are defined in Libraries and Operators.

`components/` — Project-Owned Components

For an **executable application**, `components/` contains project-owned implementation domains that are parsed and compiled as part of that application. A project-local component is **not a library** and never produces an independent `.folenc` artifact. Its immediate folder determines its component kind before `component.fol` is parsed, so no file below `components/` uses `@co.dap.library` to identify itself.

The full component-kind set below is valid only for an executable application. A standalone projected application library has only the separate `components/operators/` exception described above; every other standalone library has no `components/` tree.

For an executable application, only these immediate child directories are valid:

```
components/
├── application/
├── ffi/
├── system/
├── dynamicvmrt/
├── packaged/
└── operators/
```

Every component-kind directory contains exactly one direct structural source file named `component.fol`, and every such file contains exactly one `_ co.lang.component = { ... }` declaration. No alternative direct component-surface filename is valid.

Component path	Kind supplied by folder	`component.fol` role	Descendant package directories
`components/application/`	`application`	projected ordinary-application API surface	allowed; private to component
`components/ffi/`	`ffi`	projected FFI API surface	allowed; private to component
`components/system/`	`system`	projected system API surface	allowed; private to component
`components/dynamicvmrt/`	`dynamicvmrt`	projected dynamic-runtime API surface	allowed; private to component
`components/packaged/`	`packaged`	selective application-package export surface	allowed; **all descendant packages are private by default; explicitly selected contexts are exposed only to the executable application's primary `src/` graph and never to peer components**
`components/operators/`	`operators`	project-local custom-operator declaration surface	**not allowed**

Every descendant package below a component kind that permits package directories is **private to that component by default**. The `components/` tree is not an ordinary package-import root, and physical presence below it never makes a package directly importable or otherwise accessible from the owning application/project.

For projected component kinds (`application`, `ffi`, `system`, and `dynamicvmrt`), descendant packages remain private implementation source. Only the owning primary `src/` domain may consume APIs published by that component's `component.fol` surface; a peer project-local component may not import or reference that surface. For `components/packaged/`, descendant packages are likewise private by default; `@co.dap.export(...)` in `components/packaged/component.fol` is the only mechanism that promotes selected package contexts into the executable application's primary `src/` open package graph. Those selected contexts are **application-facing only**: no other project-local component (`application`, `ffi`, `system`, `dynamicvmrt`, `operators`, or another component context) may import, qualify, reference, or otherwise consume them. Every unselected packaged-component package remains private and cannot be referenced from outside that component. The operator component permits no descendant package directories and contributes syntax metadata only to the permitted owning primary `src/` domain; no source under `components/<kind>/` receives that custom operator table.

A component can affect its owner only through its `component.fol` surface:

```
projected component API
    -> exposed through the component surface; implementation packages remain private

packaged component export selector
    -> only explicitly selected descendant packages leave component privacy
       and join the executable application's primary src/ open package graph
    -> never enters any peer component's package/import graph

operator component
    -> declared custom operator spellings enter the permitted owning operator table
```

Executable-application project-local components are **peer-isolated**. Dependency flow is one-way from the executable application's primary `src/` domain toward component surfaces:

```
executable application primary src/
    -> components/application surface
    -> components/ffi surface
    -> components/system surface
    -> components/dynamicvmrt surface
    -> selected components/packaged package contexts (application src only)

components/<kind>/
    -X-> components/<other-kind>/
```

No project-local component may import, qualify, reference, or transitively acquire another project-local component surface. A component may consume a standalone **projected** library through its published `library=` surface when the ordinary capability/type-boundary rules permit it, but it may not consume a standalone packaged library because packaged artifacts expose no safe projected API. This component restriction does not create a pairwise dependency-direction matrix between independently built standalone **projected** libraries.

The exact projected-component semantics are defined in Components, packaged selection in Packaged Component, and operator declaration rules in Operators. The operator component is additionally isolated from every executable-application project-local component: its custom spellings are available only to the executable application's ordinary primary `src/` domain, or to the projected application library's ordinary primary `src/` domain under that library's sole `components/operators/` exception; they are never available to source under another `components/<kind>/` domain.

`lib/` — Compiled Dependencies

`lib/` contains only compiled FoLang artifacts:

```
lib/
├── first.folenc
├── second.folenc
└── ...
```

A `.folenc` is not FoLang source and is never passed through the source parser. The frontend validates and deserializes its stored library/package metadata, canonical symbol tables and contexts, type hierarchy, applicable overload-family information, and applicable AST/backend information before dependent source parsing. Under the standalone-library topology rules, one artifact has one primary exposure model: a projected library surface **or** packaged/open package contexts.

A projected `.folenc` preserves its internal library boundary and may be consumed by another library/application through its published surface. A packaged `.folenc` instead contributes explicitly exported open package contexts and may be consumed only by an executable application's primary open graph; libraries and project-local components cannot import those packaged contexts.

`build/` — Generated Output

`build/` is compiler-managed generated output. It is never a package, component, dependency, or source-discovery root. The compiler may create it when absent and may replace or update generated contents according to the selected backend interchange contract.

Canonical Full Layout

The following full tree is the **executable-application** layout. Optional component kinds are simply absent when unused. Standalone libraries use the restricted layouts stated immediately after the tree.

```
<project-root>/
├── src/
│   ├── appl.fol                        <- executable application project
│   │       OR
│   ├── component.fol                   <- standalone distributable component project
│   └── <package directories>/
│
├── components/
│   ├── application/
│   │   ├── component.fol
│   │   └── <private package directories>/
│   ├── ffi/
│   │   ├── component.fol
│   │   └── <private package directories>/
│   ├── system/
│   │   ├── component.fol
│   │   └── <private package directories>/
│   ├── dynamicvmrt/
│   │   ├── component.fol
│   │   └── <private package directories>/
│   ├── packaged/
│   │   ├── component.fol
│   │   └── <export package directories>/
│   └── operators/
│       └── component.fol
│
├── lib/
│   ├── <name>.folenc
│   └── ...
│
└── build/
    └── <compiler-generated artifacts>
```

The tree above is illustrative for an executable application: it uses `src/appl.fol`, not `src/component.fol`, and optional directories must not be created empty.

Standalone-library layouts are stricter:

```
projected application library
├── src/component.fol
├── src/<package directories>/
├── components/operators/       optional; the only permitted component
├── lib/                        optional projected-library dependencies
└── build/

packaged | ffi | system | dynamicvmrt standalone library
├── src/component.fol
├── src/<package directories>/
├── lib/                        optional projected-library dependencies
└── build/
```

For the second group, `components/` is forbidden. A packaged standalone library also cannot consume another packaged standalone library; its reusable dependencies must be projected libraries available through `lib/`.

Project Identity and Output

The project-root basename supplies project/artifact identity; structural filenames do not supply the project name.

```
/projects/payroll/
└── src/appl.fol

project kind     = application
project name     = payroll
output basename  = payroll
```

```
/libraries/hrlib/
└── src/component.fol

project kind               = standalone component/library producer
project name                = hrlib
compiled artifact           = hrlib.folenc
```

For an application, the backend/platform determines the executable convention. A standalone `src/component.fol` project produces `<project-name>.folenc`.

Structural Source Filenames

Filename	Canonical structural location	Meaning	Package file?
`appl.fol`	direct child of application `src/`	application entry surface	no
`component.fol`	direct child of standalone producer `src/`	standalone projected-library or packaged-library surface	no
`component.fol`	direct child of standardized `components/<kind>/`	project-owned component surface; kind comes from folder	no
`<Name>.fol`	package directory	file-backed primary declaration	yes

`<Fragment>.unit.fol`	package directory	ordinary package unit	yes
`<StructName>.comp.unit.fol`	package directory	struct companion unit	yes

Detailed package-file grammar belongs to Package Source Files; this table defines only structural placement.

Common Source Parsing Rule

FoLang source under `src/` and `components/` uses the same source grammar and parser. Filesystem placement supplies source role and component kind for semantic validation; it does not select a separate grammar. `lib/*.folenc` loading is artifact deserialization, not source parsing.

Packages

Package Identity

For canonical filesystem ownership and reserved root-domain rules, see Project Layout.

Ordinary project packages are directories below `src/`; their dot paths are relative to `src/`:

```
/appl/src/hr/           -> package "hr"
/appl/src/hr/employee/  -> package "hr.employee"
/appl/src/auth/         -> package "auth"
```

The project root and `src/` are not packages. Structural surfaces such as `src/appl.fol`, `src/component.fol`, and `components/<kind>/component.fol` are not package source files.

All descendant packages below `components/application/`, `components/ffi/`, `components/system/`, `components/dynamicvmrt/`, and `components/packaged/` are component-private by default. Projected-component packages remain inaccessible outside their component and are represented externally only through APIs published by `component.fol`. Packaged-component packages remain equally private unless their package contexts are explicitly selected by `components/packaged/component.fol`; only those selected contexts enter the executable application's primary `src/` open package graph. They are not inserted into any peer component's package graph. The operator component creates no package namespace.

Multi-File Packages

Multiple `.fol` files in the same package directory automatically belong to that package:

```
src/hr/employee/
├── Employee.fol      -> hr.employee
├── EmpService.fol    -> hr.employee
└── EmpValidator.fol  -> hr.employee
```

Package Names and Import Aliases

FoLang package identity is derived directly from the physical directory path below the applicable package root. There is no source-level package declaration, package-metadata file, or logical package-name override.

For example:

```
/appl/src/hr/empl/Employee.fol
    -> package hr.empl
    -> declaration hr.empl.Employee
```

The canonical import therefore uses the folder-derived package path:

```
@co.ddap.import(package="hr.empl", as="emp")
```

`as=` is optional and creates only a file-local import alias. It does not rename the package, change its canonical identity, or alter package resolution. With the alias above, `Employee` may be referenced as `emp.Employee`; without an alias, the complete imported package path is used according to the ordinary import rules.

If the canonical package name must change from `hr.empl` to `hr.emp`, the directory itself must be renamed from `src/hr/empl/` to `src/hr/emp/`. The filesystem and package namespace therefore cannot diverge.

FoLang defines no `co.lang.package` declaration kind and no reserved `package.fol` metadata form. A file named `package.fol`, if otherwise legal, has no structural package meaning and is classified by the ordinary `<Name>.fol` filename rule.

UDT (User defined Data types)

FoLang provides the following user-defined data declaration kinds:

1. cstructs
2. structs
3. unions
4. enums
5. classes
6. modules
7. interfaces
8. signatures

Each ordinary file-backed primary declaration is placed in its own `<Name>.fol` file. The declaration name is never repeated in source; `_` occupies the declaration-name position and the compiler derives the name from the filename.

For canonical declaration-usage rules for structs, cstructs, unions, enums, classes, interfaces, and related primary declarations, see Unused Symbols, Liveness, and Reachability.

> For more information about UDTs, see Built In Kinds.

Struct Declaration

```
// Employee.fol
_ co.lang.struct = {
    id   co.lang.int;
    name co.lang.string;
}
```

> More about structs: `Structs in detail`.

C-Struct Declaration

`co.lang.cstruct` is a C-like value type: it is passed by value, has a simple memory layout, and is safe to cross supported ABI boundaries.

```
// Point.fol
_ co.lang.cstruct = {
    x co.lang.int;
    y co.lang.int;
}
```

```
// Rect.fol
_ co.lang.cstruct = {
    origin Point;
    width  co.lang.int;
    height co.lang.int;
}
```

Enum Declaration

`co.lang.enum` declares an enumerated UDT whose body lists the permitted named variants.

```
// Status.fol
_ co.lang.enum = {
    Active,
    Inactive
}
```

Union Declaration

```
// NumberOrText.fol
_ co.lang.union = {
    intValue co.lang.int;
    strValue co.lang.string;
}
```

Class Declaration

```
// Employee.fol
_ co.lang.class = {
    getEmployeeDetails()->(Employee) = empmodule.getEmployeeDetails;

    getEmployeeInfo()->(Employee) =>> empmodule.getEmployeeDetails();
    // delegating – internally redirecting the call to module function
}

// $1, $2, $3 ... are return components of the immediately previous chained function
//Emp.fol
_ co.lang.class = {
    dosomething(a co.lang.int, b co.lang.int)->(co.lang.int)=>>somePack.someMethod(a)=>>someOthPack.someOtherMeth($1, b);
}
```

> More about classes: `Classes in detail`.

For the canonical class usage rule, including the distinction between developer-authored methods and interface-required methods, see Unused Symbols, Liveness, and Reachability.

Interface vs Signature

```
// EmployeeContract.fol
_ co.lang.signature = {
    ...
}
```

> More about signatures: `signatures in detail`.

```
// EmployeeApi.fol
_ co.lang.interface = {
    ...
}
```

> More about interfaces: `interfaces in detail`.

Module Declaration

```
// Employee.fol
_ co.lang.struct = {
    ...
}
```

```
// EmployeeModule.fol
_ co.lang.signature = {
    ...
}
```

```
// EmployeeModImpl.fol
@co.dap.module(signature=EmployeeModule)
_ co.lang.module->(
    signature=EmployeeModule,
    matches=EmployeeModule
) = {
    ...
}
```

> More about modules: `Modules in detail`.

Extension Declarations

// EmployeeExtension.fol

```


_ co.lang.extension->(fortype=somePkg.Employee)={

    @co.dap.instance
    someFun()->()={
        co.out.println(this.someName);
    }

    @co.dap.class
    someOtherFun()->()={
        co.out.println(self.clsVariable);
    }
}
```

> An extension is a reusable collection of fully implemented functions. fortype will provide field/attribute will associate with class .

Traits

// EmployeeTrait.fol

```
_ co.lang.trait={

    @co.dap.abstract
    someFunction()->();

    somerealFun()->()={
        co.out.println("Doing real work" );
    }

}
```

> A trait is interface-like but may provide default function implementations. A trait carries no instance state.

> A class participating in the `@co.dap.oops` model may include the trait through the applicable composition relationship.

> A consuming class must implement every abstract function that remains unsatisfied.

> Virtual methods are not permitted in a trait.

Mixins

//EmployeeMixin.fol

```
_ co.lang.mixin={

    someNum co.lang.int;

    someFun1()->()={
        co.out.println(someNum);
    }

    @co.dap.abstract
    someotherFun()->();

    @co.dap.virtual
    someVirtFun()->()={
        ...
    }


}
```

> A mixin is the dedicated abstract-class-like composition form; it avoids declaring an ordinary class merely to mark it abstract.

> A mixin may contain state, abstract methods, fully implemented methods, and virtual methods where the mixin rules permit them.

> A class may incorporate a mixin through the applicable `extends=true` relationship, implement its abstract methods, and override its virtual methods.

Units

A unit is a stateless source container. A package may contain any number of ordinary unit files, and all their members are consolidated directly into the package namespace.

```
arithmetic.unit.fol
conversion.unit.fol
optional.unit.fol
```

Each ordinary unit file contains:

```
_ co.lang.unit = {
    ...
}
```

The filename is organizational only. It does not create a unit symbol or another qualification level. For package `math`, a function `abs` declared in any ordinary unit is accessed as `math.abs(...)`, not `math.Arithmetic.abs(...)`.

A companion unit uses the reserved filename form `<StructName>.comp.unit.fol`. Its members attach to the matching same-package struct:

```
Employee.fol
Employee.comp.unit.fol
```

Only `co.lang.struct` supports a companion unit. See `Units in detail` and Struct Companion Units.

Matchers

Custom Matcher

```
// PositiveEvenMatcher.fol
@co.dap.matcher
_ co.lang.matcher->(type=co.lang.int) = {
    matchCase(
        value   co.lang.int,
        pattern co.lang.untyped
    )->(co.lang.int, co.lang.MatchBindings) = {
        // user logic
        // 0 = no match, >0 = match
    }
}
```

A matcher declaration supports exactly one matched-subject type, specified by `type=`. It must declare exactly one protocol function named `matchCase`; overloading `matchCase` inside a matcher is not permitted. A matcher for another subject type must be declared as a separate `<Name>.fol` primary declaration.

At compile time, the compiler resolves the type named by `type=` and the type of the first `matchCase` parameter. The two resolved types must be equivalent. Type aliases are compared after alias resolution. The second parameter must have type `co.lang.untyped`, and the result must be exactly `(co.lang.int, co.lang.MatchBindings)`.

```
// InvalidMatcher.fol
_ co.lang.matcher->(type=co.lang.int) = {
    matchCase(
        value   co.lang.string,
        pattern co.lang.untyped
    )->(co.lang.int, co.lang.MatchBindings) = {
        ...
    }
    // compiler error: declared matcher type and first parameter type differ
}
```

The parameter names themselves are not significant; their order and resolved types define the protocol shape.

Matcher liveness is defined in Unused Symbols, Liveness, and Reachability.

Comprehensions

//comprehensionseg1.unit.fol

```
_ co.lang.unit = {
    someFun()->() = {
        k := (1 .. 10).filter(|x| => x % 2 == 0).map(|x| => x * x);

        result := for (x <- co.core.List->(co.lang.int)[1,2,3]).yield(x * 2);          // co.core.List->(co.lang.int)[2, 4, 6]
        result := for (x <- co.core.Set->(co.lang.int)(1,2,3)).yield(x * 2);           // co.core.Set->(co.lang.int)(2, 4, 6)
        result := for (x <- Some(5)).yield(x * 2);              // Some(10)
        result := for (x <- fetchData()).yield(x.process());    // Future

        ages := co.core.Map->(key=co.lang.string, val=co.lang.int){"A":30,"B":40,"c":66,"e":88};
        upper := for ((name, age) <- ages).yield(name.toUpperCase, age);
    }
}
```

Comprehension Semantics

A FoLang comprehension is a source-driven transformation. Its canonical form is:

```
result := for (pattern <- source).yield(resultExpression);
```

The core comprehension syntax defines the binding and transformation structure; the source type defines the source-specific comprehension behaviour. The syntax does not implicitly convert every source to a `List`, and an arbitrary value does not become a valid comprehension source merely because it appears to the right of `<-`. A source is valid only when it is a FoLang iterable, `Some(T)`, or `Future(T)`. No other non-iterable source category can acquire comprehension capability through extensions, ordinary package APIs, operators, or similarly named methods.

Permitted Comprehension Sources

FoLang comprehensions intentionally accept only the following source categories:

1. **Any FoLang iterable**, including standard iterable forms such as arrays, lists, sets, maps/dictionaries, and ranges.
2. **`Some(T)`**.
3. **`Future(T)`**.

This set is closed. Comprehension support is **not** an open extension mechanism. No other non-iterable source category participates in `for ... yield`.

For iterable sources, the comprehension consumes the values exposed by the source's ordinary iteration semantics. The precise traversal order, entry shape, duplicate behaviour, and result-container behaviour remain those defined for that iterable type.

`Some(T)` and `Future(T)` are the only permitted non-iterable comprehension sources. They do not become iterable; instead, `for ... yield` applies their language-defined transformation semantics.

For example:

```
for (x <- co.core.List->(co.lang.int)[1,2,3]).yield(x * 2);   // valid: iterable
for (x <- 1 .. 10).yield(x * 2);       // valid: iterable range
for ((k, v) <- valuesMap).yield(k, v);  // valid: iterable map/dictionary
for (x <- Some(5)).yield(x * 2);        // valid: permitted non-iterable source
for (x <- someFuture).yield(f(x));      // valid: permitted non-iterable source
```

A struct, class, module, or other UDT that is **not iterable** is not a valid comprehension source. Defining `map`, `each`, extensions, operators, or similarly named functions does not make such a type comprehension-capable.

```
emp Employee = ...;

for (x <- emp).yield(x.salary + 1000); // compiler error if Employee is not iterable
```

A user-defined type may therefore appear as a comprehension source only when it participates in FoLang's ordinary iterable model. It cannot define a new non-iterable comprehension meaning comparable to `Some(T)` or `Future(T)`.

The parts of the form have the following meanings:

- `source` is the value being traversed or transformed;
- `pattern` introduces the comprehension-local binding or destructuring pattern for each value produced by the source;
- `<-` associates that binding pattern with the source;
- `.yield(...)` supplies the result expression evaluated for each participating source value according to that source type's comprehension semantics.

Bindings introduced by `pattern` are local to that comprehension. They are visible in the corresponding `yield` expression and do not introduce names into the enclosing scope. A destructuring pattern, such as `(name, age)`, introduces each successful component binding in the same comprehension-local scope.

The result shape is source-defined rather than selected by a universal core-language conversion rule. The examples above establish the following current forms:

```
co.core.List->(A)   --yield B--> co.core.List->(B)
co.core.Set->(A)    --yield B--> co.core.Set->(B)
Some(A)   --yield B--> Some(B)
Future(A) --yield B--> Future(B)
```

For example:

```
result := for (x <- co.core.List->(co.lang.int)[1,2,3]).yield(x * 2);
// co.core.List->(co.lang.int)[2, 4, 6]

result := for (x <- co.core.Set->(co.lang.int)(1,2,3)).yield(x * 2);
// co.core.Set->(co.lang.int)(2, 4, 6)

result := for (x <- Some(5)).yield(x * 2);
// Some(10)

result := for (x <- fetchData()).yield(x.process());
// Future
```

The `Map` form demonstrates source destructuring and pair production:

```
ages := co.core.Map->(key=co.lang.string, val=co.lang.int){"A":30,"B":40,"c":66,"e":88};
upper := for ((name, age) <- ages).yield(name.toUpperCase, age);
```

Here `(name, age)` destructures the source entry for the current comprehension step. The result-container rules, duplicate-key behaviour, ordering, and other map-specific properties are those defined by the applicable `Map` API rather than by the core `for ... yield` grammar.

Source-specific semantics also govern cardinality, emptiness, deferred execution, failure propagation, ordering, duplicate handling, and similar behaviour. Thus the `Some` and `Future` forms preserve the source abstraction shown by their examples, while the precise empty/failure/scheduling behaviour belongs to the corresponding type's defined API semantics. The comprehension syntax itself does not introduce implicit blocking, concurrency, retries, or error suppression.

The current core `for (pattern <- source).yield(...)` form does not define an inline filter clause. Filtering is expressed separately through the source's supported operations, for example:

```
k := (1 .. 10)
    .filter(|x| => x % 2 == 0)
    .map(|x| => x * x);
```

If a later source-specific comprehension form defines filtering, its filter conditions are evaluated before its result expression as required by the general evaluation-order rules. No filter is implied by the core `for ... yield` syntax shown above.

A comprehension does not imply parallel or concurrent traversal. Execution is sequential unless the selected source semantics or an explicitly requested FoLang execution model states otherwise.

Extension Methods

Extension functions may be declared in an ordinary package unit:

```
// string_extension.unit.fol
_ co.lang.unit = {

    @co.dap.extension(fortype=co.lang.string, what=extends)
    upperCase()->(co.lang.string) = {
        this.return this.upper();
    }

    @co.dap.extension(fortype=[co.lang.string], what=overrides)
    equals(str co.lang.string)->(co.lang.bool) = {
        this.return this == str;
    }
}
```

Because ordinary unit filenames create no symbol, extension activation identifies the contributing package, not the unit file.

For the current package, omit `from`:

```
@co.ddap.use(methods=[equals, upperCase])
k.upperCase();
```

For another package, use its alias or complete package path:

```
@co.ddap.import(package="text.util", as="tu")

@co.ddap.use(from="tu", methods=[upperCase])
@co.ddap.use(from="text.util", methods=[upperCase])
```

See Activating Instance Methods for activation of typeclass instances.

> These are different from Extension Declaration, in a way that these are only applicable to existing co.* package built in types and structs.

> Extension Declaration is re usable components for classes.

> Modules, and other constructs we don't have this kind of extension capabilities.

> Benefits classes can be extended without subclassing

> Types can be enabled with new featues or enhancements without requiring library owners to provide.

> Methods overriding should not be sealed methods @co.dap.sealed

---

Reflections

The `co.meta` reflection form shown below is a dynamic-runtime facility and is valid only inside a `dynamicvmrt` capability domain; it does not grant `co.meta` access to ordinary application or packaged code.

```
@co.dap.reflection(enable=true, package="co.meta")

x co.lang.int = 10;
x.reflect().getType();   //co.lang.int
x.reflect().getValue();  //10;
x.reflect().getKind();   // value
```

Type Classes

Monads, Applicatives, Functors, Monoids and Transformers

> `@co.dap.typeclass(kind=...)` is the single annotation for all typeclass definitions. `kind` specifies the algebraic structure — `Functor`, `Applicative`, `Monad`, `Monoid`, `Transformer`, or any user-defined kind. Instances of any typeclass always use `co.lang.instance`.

Typeclass and instance liveness is defined in Unused Symbols, Liveness, and Reachability.

In a file-backed typeclass declaration, `_` is the filename-derived declaration-name placeholder and the following parenthesized clause declares the typeclass parameters. They are separate grammar components, so the canonical spelling includes a space: `_ (F(_))`, not `_(F(_))`. A parameter such as `T` denotes an ordinary type, while `F(_)` denotes a unary type constructor and `G(_, _)` denotes a binary type constructor. Otherwise-unbound type variables introduced in an operation signature, such as `A` and `B`, are implicitly universally quantified within that operation.

Typeclass signatures use their own higher-kinded/type-constructor application notation. Forms such as `F(A)`, `List(A)`, or `Set(B)` in a typeclass/instance signature express application of the applicable type constructor inside that model; they are not the ordinary concrete `co.core.List->(...)`, `co.core.Set->(...)`, or collection-construction syntax defined for built-in collection values.

Functor

```
//Functor.fol
@co.dap.typeclass(kind=Functor)
_ (F(_)) co.lang.typeclass = {
    map(value F(A), f (A)->B) -> (F(B));
}

// ListFunctor.fol
_ co.lang.instance->(for=Functor, type=List) = {
    map(value List(A), f (A)->B)->(List(B)) = {
        result = List(B){};
        value.each(_, item, { result.append(f(item)) });
        this.return result;
    }
}
```

Applicative

```
//Applicative.fol
@co.dap.typeclass(kind=Applicative)
_ (F(_)) co.lang.typeclass = {
    pure(x A) -> (F(A));
    apply(fab F(A->B), fa F(A)) -> (F(B));
}

// OptionApplicative.fol
_ co.lang.instance->(for=Applicative, type=Option) = {
    pure(x A)->(Option(A)) = { this.return Some(x); }
    apply(fab Option(A->B), fa Option(A))->(Option(B)) = {
        this.return (fab, fa)
            .match
            .case((Some(f), Some(x)) => Some(f(x)))
            .default(None());
    }
}
```

Monad

```
//Monad.fol
@co.dap.typeclass(kind=Monad)
_ (F(_)) co.lang.typeclass = {
    pure(x A) -> (F(A));
    flatMap(fa F(A), f (A)->F(B)) -> (F(B));
}

// OptionMonad.fol
_ co.lang.instance->(for=Monad, type=Option) = {
    pure(x A)->(Option(A)) = { this.return Some(x); }
    flatMap(fa Option(A), f (A)->Option(B))->(Option(B)) = {
        this.return fa.match().case(Some(x) => f(x)).default(None);
    }
}
```

Monoid

```
//Monoid.fol
@co.dap.typeclass(kind=Monoid)
_ (T) co.lang.typeclass = {
    empty() -> (T);
    combine(a T, b T) -> (T);
}

// IntMonoid.fol
_ co.lang.instance->(for=Monoid, type=co.lang.int) = {
    empty()->(co.lang.int) = { this.return 0; }
    combine(a co.lang.int, b co.lang.int)->(co.lang.int) = { this.return a + b; }
}
```

Transformer

```
//Transformer.fol
@co.dap.typeclass(kind=Transformer)
_ (F(_), G(_)) co.lang.typeclass = {
    map(value F(A), f (A)->B) -> (G(B));
}

// ListToSetTransformer.fol
_ co.lang.instance->(for=Transformer, types=[List, Set]) = {
    map(value List(A), f (A)->B)->(Set(B)) = {
        result = Set(B){};
        value.each(_, item, { result.insert(f(item)) });
        this.return result;
    }
}
```

Using an Instance

An instance is selected **by name**. There is no implicit search.

```
@co.ddap.import(package="abc.tc", as="tc")

xs List(co.lang.int) = [1, 2, 3];
double(x co.lang.int)->(co.lang.int) = { this.return x * 2; }

ys := tc.ListFunctor.map(xs, double);
```

`map` takes the container as its first argument, exactly as the typeclass declares it. The call names `ListFunctor`, so the compiler resolves it the same way it resolves any other imported declaration.

FoLang does **not** search visible packages for an instance that happens to match a type. A typeclass is a contract, an instance is a named implementation of that contract, and the caller names the one it means. Nothing is inferred, so an unresolved call reports a name the developer actually wrote rather than a failed search they never saw.

> Method syntax such as `xs.map(f)` is unrelated to typeclasses. It calls an associated function declared in a companion unit for the type. Companion units live in the type's own package, so method calls are unambiguous by construction. A companion function and a typeclass instance may both be named `map`; they are different declarations and are reached differently.

Activating Instance Methods

An instance may also be **activated**, which makes its functions callable as methods on the receiver. Activation uses the same directive that activates extensions.

```
@co.ddap.import(package="abc.tc", as="tc")
@co.ddap.use(from="tc.ListFunctor", methods=[map, reduce])

ys := xs.map(double);        // resolves to tc.ListFunctor.map(xs, double)
```

Activation is explicit and block-scoped. Importing a package does **not** activate anything in it, so adding an import can never change how an existing call resolves.

`methods` and `from`

`methods` selects the functions to activate.

For extension functions contributed by ordinary package units:

- omit `from` to select the current package
- use an imported package alias to select another package
- use the complete package path when no alias exists

```
@co.ddap.use(methods=[upperCase])               // current package
@co.ddap.use(from="tu", methods=[upperCase])   // imported package alias
@co.ddap.use(from="text.util", methods=[upperCase]) // complete package path
```

Ordinary unit filenames are not accepted by `from`, because they do not create symbols.

For a typeclass instance, `from` continues to name the instance declaration:

```
@co.ddap.use(from="tc.ListFunctor", methods=[map, reduce])
```

Listing names is optional. Omit `methods` to activate every eligible method from the selected package or instance; provide it to activate a subset. Conflict detection remains receiver-aware and block-scoped.

How a method call resolves

For `xs.map(f)`, where `xs` has type `List(A)`:

1. a class method or companion-unit function on `List`
2. an activated extension for `List`
3. an activated instance function whose typeclass declares `map` with the receiver as its first parameter
4. otherwise, an error

The first match wins. A type's own declarations therefore always take precedence over anything activated into scope, and no activation can silently replace behaviour the type already defines.

Within one scope a given method name may be activated at most once for a given receiver type. Activating `map` for `List` from two sources is an error at the second `@co.ddap.use`, which names both. The conflict is reported where the activation is written, never at a distant call site.

The parser's built-in-method classification is only a lookup candidate. If the frontend recognizes a control-chain shape before receiver-aware lookup, it must retain the complete original member/call chain. A receiver-owned, companion, extension, or activated-instance declaration that wins the order above restores/remains the ordinary method call represented by that chain. An early dedicated control-flow node is therefore only a lowering candidate and becomes final only when the built-in meaning wins.

Activation never affects generic code. A function polymorphic over a typeclass takes the instance as an ordinary parameter and calls it by name.

A Typeclass Is a Type

A typeclass may be used wherever a type is expected. Its values are its instances. This is the same relationship a signature has to a module:

```
mm EmployeeModule = EmployeeModImpl;     // signature as type, module as value
f  Functor(List)  = tc.ListFunctor;      // typeclass as type, instance as value
```

An instance is therefore an ordinary first-class value. It can be held in a variable, stored in a collection, returned from a function, and chosen at run time.

```
inst := (useCache).then(cachedFunctor).default(plainFunctor);
result := inst.map(xs, transform);
```

Nothing about an instance is special to the compiler, which is why FoLang needs no instance resolution algorithm and no coherence rules.

Writing Code Generic Over a Typeclass

Activation makes `xs.map(f)` work for a **known** container. Code that must work for **any** container cannot activate anything, because the container type is not known until the caller supplies it. Such code takes the instance as an ordinary parameter.

```
@co.dap.generic(types=[{name=F}, {name=A}, {name=B}])
mapAll(inst Functor(F), value F(A), fn (A)->B)->(F(B)) = {
    this.return inst.map(value, fn);
}
```

Parameter	What it is
`F`	the container kind — `List`, `Option`, `Tree`
`inst`	the instance, an implementation of `Functor` for `F`
`value`	the container itself, an ordinary value

The caller supplies the instance:

```
ys   := mapAll(tc.ListFunctor,   xs,  double);
opt2 := mapAll(tc.OptionFunctor, opt, double);
```

The function never learns what `F` is. It knows only that `inst` provides `map`, which is the whole contract. One definition therefore serves every container that has a `Functor` instance.

> A type is never “a Functor” in FoLang. `List` does not become a Functor; an instance implements Functor operations *for* `List`, and the list stays a plain list. The Functor-ness lives entirely in `inst`.

When a wrapper is worth writing

`mapAll` above forwards directly to `inst.map`, so it adds nothing a caller could not write themselves. A wrapper earns its place when it does more than forward — when it combines several typeclass operations, adds logic, or fixes some parameters and leaves others open.

```
@co.dap.generic(types=[{name=F}])
doubleAll(inst Functor(F), value F(co.lang.int))->(F(co.lang.int)) = {
    this.return inst.map(value, (x co.lang.int)->(co.lang.int){
        this.return x * 2;
    });
}
```

This one fixes the element type and the operation, so it says something a bare `inst.map` call does not.

Where an Instance Is Declared

An instance is declared in **the package that defines the typeclass**, or in **the package that defines the type**. That exact package, not a sub-package.

```
abc.tc.ListFunctor      for=Functor, type=List           // OK  typeclass's package
myapp.ab.TreeFunctor    for=Functor, type=myapp.ab.Tree  // OK  type's package
other.util.ListFunctor  for=Functor, type=List           // ERR neither is theirs
```

A typeclass is an ordinary declaration and may live in any package; `abc.tc` above is a user package, not a built-in one. Sub-packages are distinct packages, so an instance for `myapp.ab.Tree` belongs in `myapp.ab`, not in `myapp` or `myapp.ab.instances`. This matches the rule for companion units, which also sit in their type’s own package. Because a package spans every `.fol` file in its folder, each instance still gets its own `<InstanceName>.fol` file and uses `_ co.lang.instance` in source.

The rule is permissive on both sides on purpose. Requiring the typeclass’s package alone would make a typeclass usable only by its own author, since nobody could implement it for their own types. Requiring the type’s package alone would stop a library from shipping instances for common built-in types.

For library authors. If you define a type and want it usable with someone else’s typeclass, declare the instance in your package. If you define a typeclass, you may ship instances for types you do not own, including built-in ones — `IntMonoid` above sits beside `Monoid`, which is exactly this case. If you need an instance for a typeclass and a type you both do not own, wrap the type in a `co.lang.newtype` you do own and declare the instance for the wrapper.

This placement rule is semantic, not syntactic. A misplaced instance parses correctly and is reported during name resolution, so the diagnostic can name the typeclass, the type, and the two packages in which the instance would have been legal.

Labels and Named Blocks

```
// Label
outer:{
    // statements
}

//Blocks Anonymous block
 {
    // statements
 }

// Named Block
labelBlock co.lang.block={

}

labelBlock.expand();
```

> Blocks have their own variable scope and context. A variable declared outside a block remains accessible inside the block unless it is shadowed. A block may declare a variable with the same name and either the same or a different type; that declaration shadows the outer binding and is scoped only to the block. This model is similar to block scoping in C and C++.

> Blocks cannot appear outside functions or methods.

> Inner executable blocks are prohibited directly inside classes, structs, typeclasses, modules, and other non-function/non-method declarations; such usage is a compile-time error.

```
somefun (a co.lang.int, b co.lang.int)->(co.lang.int)={

     some_other co.lang.float =  20.1f;
     {
        some_other co.lang.char = 'c';
        co.out.println( some_other);   //prints c
     }

     co.out.println(some_other); //prints 20.1;

     {
        some_other co.lang.float = 11.1f;
        co.out.println(some_other); // prints 11.1f
     }

     co.out.println(some_other); // still prints 20.1f;

     {
        some_other = some_other + 1.1f;
        co.out.println(some_other); // prints 21.2
     }

     co.out.println(some_other); // prints 21.2; as this was changed in third block
}
```

---

imports

FoLang supports package imports, standalone projected-library imports, and executable-application project-local component imports. `component=` is deliberately **executable-application-primary-`src/` only** and never appears in standalone-library source or source under `components/<kind>/`. Standalone packaged-library exports are likewise executable-application-open-graph inputs rather than general library dependencies. Import targets must be available in the prepared compilation environment before semantic resolution consumes them.

An import declaration makes the target context available for name resolution; import liveness and zero-use validation are defined in Unused Symbols, Liveness, and Reachability.

1. Package Import

Use `package=` for ordinary/open package contexts. A package may come from:

```
the current project's src/ package tree
selected package source under components/packaged/ (executable application primary `src/` only)
a packaged/exported package context reconstructed from a loaded `lib/*.folenc` artifact (executable application primary `src/` only)
the language-provided co.* package artifact
```

Example:

```
@co.ddap.import(package="hr.employee", as="emp")

e := emp.getEmployee(1);
co.out.println(e.name);
```

Conceptual resolution:

```
package="hr.employee"
    -> exactly one matching open package context from the prepared package index
```

The package path must resolve uniquely. Multiple providers of the same package path are an ambiguity error. A `.folenc` filename never becomes an implicit package-name prefix.

Project-local descendant source under `components/application/`, `components/ffi/`, `components/system/`, or `components/dynamicvmrt/` is not an open package root and cannot be imported with `package=`. Packages explicitly selected by `components/packaged/component.fol` enter **only the executable application's primary `src/` package index**. They are deliberately absent from the package indexes used while parsing or resolving every project-local component, so no peer component can import them with `package=`, reach them by qualification, or acquire them indirectly through another component surface.

2. Component Import

Use `component=` only from an **executable application's primary `src/` domain** to consume a same-owner projected component surface:

```
@co.ddap.import(component="ffi", as="ffilib")
```

Resolution:

```
component="ffi"
    -> <project-root>/components/ffi/component.fol
    -> projected component surface context
```

The permitted `component=` values are:

```
application
ffi
system
dynamicvmrt
```

Those values are fixed component identities. Their matching `components/<kind>/` folder determines capability context. The component is not a standalone library and does not produce an independent artifact.

`component=` is an **executable-application-root-to-component composition mechanism**, not a library dependency mechanism and not a component-to-component dependency mechanism. It is invalid in every `src/component.fol` standalone library project and anywhere under `components/application/`, `components/ffi/`, `components/system/`, `components/dynamicvmrt/`, `components/packaged/`, or `components/operators/`. Full qualification, aliases, re-export tricks, or transitive resolution cannot bypass this rule.

For projected components:

- the resolved surface is the fixed `components/<kind>/component.fol`;
- the surface uses `_ co.lang.component` and no `@co.dap.library`;
- only declarations exposed through that surface are visible to the owning open source graph;
- implementation packages below the component root remain private to that component compilation domain.

`components/packaged/` is consumed through ordinary `package=` imports **only by executable application code in the primary `src/` domain** after selection because it contributes application-open package contexts rather than a projected API namespace. No project-local component may consume those selected package contexts. `components/operators/` is compilation infrastructure and is never imported.

These restrictions are enforced by **separate component/package-resolution environments**. Preparing an executable application's primary `src/` domain makes same-owner projected component surfaces addressable through `component=`; preparing any standalone library or any `components/<kind>/` domain makes **no project-local projected component surface addressable through `component=`**. The executable application's primary `src/` package index additionally receives package contexts explicitly selected by `components/packaged/`; component and library package indexes do not. A component/library therefore cannot obtain a forbidden component or packaged context by aliasing, full qualification, transitive lookup, or re-export.

This peer-component prohibition does not impose a pairwise architectural matrix on standalone **projected** libraries. Code inside an executable-application component may import an independently built projected library with `library=` when normal visibility, capability, boundary-type, liveness, and cycle rules are satisfied. The compiler does not reject such a projected-library dependency merely because its architectural direction is, for example, FFI-to-system. A standalone packaged library is different: it has no safe projected surface, so its exported package contexts are not importable by another library or by a project-local component; they may join only an executable application's primary open graph.

3. Standalone Projected Library Import

Use `library=` for the projected library surface of a prebuilt `.folenc` artifact in `lib/`:

```
@co.ddap.import(library="hrlib", as="hr")
```

Resolution:

```
library="hrlib"
    -> <project-root>/lib/hrlib.folenc
    -> reconstructed projected component/library surface context
```

The source used to build the `.folenc` is not reparsed. The artifact loader reconstructs canonical symbol/context metadata and applicable AST information before source parsing begins.

A standalone packaged `.folenc` contains the package contexts selected by its producer rather than a projected `library=` surface. Those selected contexts are consumed with ordinary `package=` imports **only by an executable application's primary `src/` open graph**. A standalone library or project-local component attempting to import a packaged-library context is a compiler error.

Import Directive Fields

Field	Required	Meaning
`package`	exactly one of `package`, `library`, or `component`	logical open-package dot path resolved from the prepared package index
`library`	exactly one of `package`, `library`, or `component`	logical standalone projected-library identity resolved from `lib/<name>.folenc`
`component`	exactly one of `package`, `library`, or `component`	standardized same-owner projected component identity: `application`, `ffi`, `system`, or `dynamicvmrt`
`as`	optional	local alias; valid FoLang identifier; when omitted, the complete imported package/library/component identity is used

Notes:

- `package`, `library`, and `component` are mutually exclusive; exactly one must be supplied;
- `as` is optional;
- dots are not allowed in `as`;
- component identities and canonical paths are defined in Project Layout;
- packaged-component and packaged-library contexts are addressed through `package=`;
- the operator component is not imported;
- a packaged `.folenc` provides open package contexts only; it has no projected `library=` surface and is consumable only by an executable application's primary open graph.

Examples:

```
@co.ddap.import(package="hr.employee", as="emp")
em emp.Employee;

@co.ddap.import(package="hr.employee")
em hr.employee.Employee;

@co.ddap.import(component="ffi", as="ffi")
ffi.someSurfaceFunction();

@co.ddap.import(library="hrlib", as="hr")
hr.someSurfaceFunction();
```

Valid `as` Values

```
as="hr"       ✅
as="v1_hr"    ✅
as="v1.hr"    ❌
as="123hr"    ❌
```

Cycles

Compiler error if any dependency cycle exists through effective package, component-surface, or standalone-library dependencies. A written but unused import is not an effective edge and is handled by the unused-import rules.

Symbol Resolution

Resolution Order

For symbols without a prefix:

1. current package symbol table
2. parent package chain upward
3. error if not found

For symbols with an alias prefix:

1. current package imported-context map
2. parent package chain imported-context maps
3. error if not found

For imports declared **without** `as`:

1. use the full imported package path directly
2. resolve it through imported-context lookup using that full path
3. error if not found

Example Context Graph

```
pp1 (grandparent package)
    ├── SymbolTable
    ├── ImportedContexts
    └── ChildCtxs

p1 (parent package)
    ├── Parent -> &pp1
    ├── SymbolTable
    ├── ImportedContexts
    └── ChildCtxs

c1 (current package)
    ├── Parent -> &p1
    ├── SymbolTable
    ├── ImportedContexts
    └── ChildCtxs
```

Resolution Examples

```
lookup "OwnType"
    -> current symbol table

lookup "ParentType"
    -> parent chain

lookup "hr.Employee"
    -> imported-context lookup for alias "hr"

lookup "hr.employee.Employee"
    -> imported-context lookup for full imported package path when no alias exists

lookup "unknown.Type"
    -> imported-context lookup fails -> compiler error
```

Package in detail

Package identity and multi-file membership are defined in Packages. Canonical project, component, library, and artifact placement is defined only in Project Layout. This section continues with package visibility and access semantics.

Package Access Rules

Four access levels control visibility across package boundaries:

Annotation	Same Package	Parent	Sub-Package	Unrelated	Entry / Surface
`@co.dap.public`	✅	✅	✅	✅	✅
`@co.dap.package`	✅	✅	✅	❌	❌
`@co.dap.protected`	✅	❌	✅	❌	❌
`@co.dap.private`	✅	❌	❌	❌	❌

Meaning:

- `@co.dap.public` -> visible everywhere
- `@co.dap.package` -> visible across the package family
- `@co.dap.protected` -> visible downward into subpackages only
- `@co.dap.private` -> visible only inside the declaring package

Example:

```
// hr/employee-access.unit.fol — package "hr"

_ co.lang.unit = {

    @co.dap.public
    getEmployee(id co.lang.int)->(Employee) = { ... }     // anyone can call

    @co.dap.package
    validateId(id co.lang.int)->(co.lang.bool) = { ... }  // hr package family

    @co.dap.protected
    baseQuery()->(co.lang.string) = { ... }               // visible to subpackages

    @co.dap.private
    normalizeId(id co.lang.int)->(co.lang.int) = { ... }  // private declaration
}
```

`co.*` Paths and Aliases

`co.*` Is Always Available

The FoLang distribution provides the standard `co.*` package tree through a language-supplied `.folenc` artifact that exposes the standard package contexts. The compiler/toolchain loads that artifact automatically and makes the `co` root implicitly available, so source code never imports standard `co.*` packages through `@co.ddap.import`.

Implicit availability changes package discovery only. Once loaded, declarations supplied by `co.*` packages use ordinary package, symbol, type, member, visibility, extension, and overload resolution in the same way as declarations reconstructed from exported package contexts in another `.folenc` artifact.

```
co.out.println("hello");
co.in.readln();
x co.lang.int = 42;
```

Being **in scope** does not automatically mean **permitted in every context**.

A package rule, capability domain, projected boundary, or other compiler restriction may still forbid use of a particular `co.*` facility. In such cases the name is still known to the compiler, but its use is rejected at compile time.

`@co.ddap.alias`

Use aliases only to shorten `co.*` paths.

```
@co.ddap.alias(co.out, as="out")
@co.ddap.alias(co.core.list, as="list")
@co.ddap.alias(co.encoding, as="enc")

out.println("hello");
list.of(1, 2, 3);
enc.json.serialize(data);

// full form still works alongside
co.out.println("world");
```

Rules:

- target must be a `co.*` path
- alias must be a valid identifier
- alias scope is file-local
- aliasing user packages is not allowed; use `as=` in `@co.ddap.import`
- duplicate aliases in the same file are compiler errors
- when no alias is declared, the complete `co.*` path is used
- declaring an alias does not disable or hide the complete `co.*` path ***

Package Source Files

A package folder may contain three ordinary source-file categories. The fixed project surfaces `src/appl.fol` and `src/component.fol` are not ordinary package-source files and are invalid inside a package. Likewise, every direct `components/<kind>/component.fol` surface is a structural component file and is invalid inside an ordinary package. The compiler classifies ordinary package files from filenames before parsing, using the longest recognized suffix first:

```
<Name>.comp.unit.fol  -> companion unit
<Fragment>.unit.fol   -> ordinary package unit
<Name>.fol            -> file-backed primary declaration
```

A filename ending in `.comp.unit.fol` is recognized before the ordinary `.unit.fol` suffix and is never classified as an ordinary unit file. Structural project/component filenames are validated by Project Layout before ordinary package-file classification.

File-Backed Primary Declarations

A primary declaration file has the form `<Name>.fol` and contains exactly one primary top-level declaration. The source must use `_` in the declaration-name position:

```
// Employee.fol
_ co.lang.struct = {
    id   co.lang.int;
    name co.lang.string;
}
```

The compiler derives the declaration name `Employee` from the filename. An explicit primary name is invalid:

```
// Employee.fol
Employee co.lang.struct = { ... }
// compiler error: file-backed primary declarations must use `_`
```

This rule applies to ordinary file-backed declarations that use a `<name> co.lang.<kind>` primary form, including classes, structs, cstructs, enums, unions, interfaces, signatures, modules, instances, matchers, objects, and other ordinary package-owned declaration kinds. `co.lang.component` is not an ordinary `<Name>.fol` primary. It is valid only as the single structural declaration in `src/component.fol` or `components/<kind>/component.fol`, as defined in Project Layout, Libraries, and Components.

The following declaration forms are stated exceptions and keep an explicit name in the head, because filename derivation cannot express what they need:

Form	Why
surface `struct` / `cstruct` declarations inside a projected `_ co.lang.component`	one projected component surface may carry several boundary declarations
`co.lang.data` algebraic data type	the head names the variants
parameterized `co.lang.type`	a filename cannot carry `(T)`
type declarations in `src/appl.fol`	the entry file is not filename-backed as an ordinary package-owned declaration

File-level directives, imports, aliases, annotations, and decorators may appear before the primary declaration. They do not count as additional primary declarations.

FoLang permits the following top-level declaration kinds across ordinary package source files and their reserved special source forms:

1. struct
2. cstruct
3. class
4. module
5. signature
6. interface
7. enum
8. union
9. typeclass
10. instance
11. matcher
12. annotation or object declaration
13. component declaration in its reserved structural surface
14. unit declaration

Ordinary package-source placement is:

- filename-backed primary declarations use `<Name>.fol`;
- ordinary units use `<Fragment>.unit.fol`;
- companion units use `<StructName>.comp.unit.fol`.

`src/appl.fol`, `src/component.fol`, and `components/<kind>/component.fol` are structural project/component surfaces rather than ordinary package-source forms. Their canonical locations and allowed structural forms are defined once in Project Layout.

A package source file is invalid when it contains multiple unrelated primary declarations or places a unit declaration outside a recognized unit filename. Structural-surface placement errors are validated according to Project Layout.

Package-level functions and non-UDT type declarations belong inside ordinary unit files.

Filename Canonicalization

Filename-derived declaration identity is independent of filesystem case sensitivity. The compiler normalizes and case-folds the filename stem to construct the duplicate-detection and lookup key:

```
canonical file key = caseFold(normalize(filename stem))
```

The language-level declaration is then represented using FoLang's canonical declaration spelling. Therefore:

```
employee.fol
Employee.fol
EMPLOYEE.fol
```

all denote the canonical declaration name `Employee`. They conflict rather than declaring separate types, even on a case-sensitive filesystem.

The same rule applies to companion owners:

```
employee.comp.unit.fol -> companion of Employee
```

This rule is specific to filename-derived primary declarations and companion-owner identity; it does not make every FoLang identifier case-insensitive. The compiler must not rely on operating-system filename comparison.

The filename stem must form a valid FoLang declaration identifier after normalization. Case variants and canonically equivalent spellings produce the same package-index key.

Ordinary Package Units

An ordinary package-unit file has the form `<Fragment>.unit.fol`:

```
// arithmetic.unit.fol
_ co.lang.unit = {
    abs(value co.lang.int)->(co.lang.int) = {
        ...
    }
}
```

The fragment name is organizational only. It does not create a public `Arithmetic` symbol. All ordinary unit members in the package are merged into one package namespace:

```
math/
├── arithmetic.unit.fol
├── comparison.unit.fol
└── optional.unit.fol
```

```
math.abs(-10);
math.max(10, 20);
value math.Option(co.lang.int);
```

The following qualification is invalid:

```
math.Arithmetic.abs(-10); // compiler error
```

Any number of ordinary unit files may exist in one package. During package indexing, the compiler shallow-parses their declarations, merges their members with filename-derived primary declarations, and reports duplicate names or duplicate function signatures according to FoLang overload rules.

Companion Unit Files

A companion unit has the form `<StructName>.comp.unit.fol` and must contain `_ co.lang.unit`. The canonical owner name comes from the filename, not from the source body.

```
Employee.fol
Employee.comp.unit.fol
```

The compiler can detect the following before parsing either body:

- duplicate primary declarations after filename canonicalization
- duplicate companion files for the same canonical owner
- an orphan companion file for which no same-package primary declaration exists

After the owner header is known, the compiler also verifies that the owner is a `co.lang.struct`. Other primary declaration kinds cannot own companion units.

Package Indexing Levels

```
Level 1: filename and package indexing
    classify primary, ordinary-unit, and companion-unit files
    canonicalize primary names and companion owners
    detect duplicate primary files
    detect duplicate or orphan companion files
    shallow-parse ordinary units
    merge ordinary-unit members into the package namespace
    report package-namespace conflicts

Level 2: companion validation
    parse companion declarations
    validate explicit receivers against the filename-derived owner
    merge companion members into the owner's companion namespace
    report duplicate companion members and receiver mismatches

Level 3: full semantic resolution
    resolve declaration bodies, expressions, calls, patterns, and types
```

This organization lets an imported source package build most of its symbol index cheaply before full parsing. Compiled packages may store the same index as metadata so imports need no source parsing.

Application Entry File

For the canonical application project structure and root-domain rules, see Project Layout. The application-specific structural slice is simply:

```
src/
├── appl.fol
└── <package directories>/
```

The application entry file is the fixed **special executable source form** `src/appl.fol`. It is not a package, does not create an importable namespace, and is not subject to the ordinary package-file rule requiring exactly one primary declaration.

The compiler creates a dedicated **entry-file context** for it:

```
ApplicationEntryContext
├── file directives, imports, and aliases
├── entry-local non-structural type declarations
├── non-capturing function-pattern groups
├── capturing `let` function-pattern groups
└── executable statements and expressions
```

Everything declared directly in this context is private to the entry file. Entry-local symbols cannot be imported, exported, or referenced by package or library source files.

Allowed Entry-File Constructs

The application entry file uses exactly the same grammar and allowed-construct rules described in Single Source Application File. The earlier section provides the developer-facing overview; this section defines the entry file's formal context, privacy, and dependency direction.

Entry-Local Function Patterns

Function-pattern groups are allowed as a special entry-file construct even though ordinary function declarations are forbidden. FoLang provides two entry-file forms with the same pattern-dispatch model but different capture semantics.

Entry-File Dependency Direction

The entry file may depend on packages and libraries, but packages and libraries may never depend on the entry context:

```
application entry file
    ↓ uses
packages and libraries
```

This allows the entry file to coordinate application startup while preserving package and library independence.

Packaged Component

For canonical component placement, allowed descendant directories, and the `component.fol` structural form, see Project Layout.

Purpose

The packaged component is a component-private package domain with **selective application-package publication**. Every descendant package begins private to `components/packaged/`. Its `component.fol` may explicitly select package contexts that are permitted to leave that private domain and participate directly in the executable application's primary `src/` open package/type graph. This publication is one-way toward the application: selected contexts are never made visible to any peer project-local component. It introduces no projected API adapter, snapshot boundary, or separate type identity for packages that are exported.

Its structural position is:

```
components/packaged/
├── component.fol
└── <package directories>/
```

The filesystem constraints are defined only in Project Layout. This section defines selection and open-package semantics.

Packaged Selector Surface

`components/packaged/component.fol` contains exactly one `_ co.lang.component` structural declaration. It does **not** use `@co.dap.library` because a project-local component is not a library. Inside this special component body, `@co.dap.export(...)` is a component-surface metadata entry applying to the containing `_ co.lang.component`; it does not annotate a following declaration:

```
// components/packaged/component.fol
_ co.lang.component = {
    @co.dap.export(
        packages={
            hr.employee: {recurse=true},
            text.format: {recurse=true}
        }
    )
}
```

The `packages` map names package contexts relative to the `components/packaged/` component root. Selection changes **package-context accessibility**, not the visibility of declarations inside the selected package:

- every descendant package is private to the packaged component before selection;
- `recurse=false` exports exactly the named package context;
- `recurse=true` exports the named package context and every descendant package context;
- every selected package path must exist;
- duplicate or overlapping selections normalize to one selected package context;
- selected package names are preserved exactly;
- an unselected package remains component-private even when another package in the same subtree is exported, unless that package is covered by an applicable recursive selection;
- an unselected package cannot be imported, reached through a fully qualified package name, or otherwise referenced from outside `components/packaged/`;
- a selected package is visible only from the executable application's primary `src/` domain; every other project-local component is treated as outside the publication target and cannot import, qualify, reference, or otherwise consume it.

Exporting a package context does not widen declaration visibility. A private declaration remains private, a protected declaration remains protected, and ordinary subtype, extension, typeclass, ownership, overload, and visibility rules continue to be determined by the declarations themselves.

Open Package Semantics

Only a package context explicitly selected by the packaged surface participates as an ordinary open package context **for the executable application's primary `src/` domain**. Unselected packaged-component packages remain outside this graph, and peer components never receive selected packaged contexts:

```
src package
selected components/packaged package
packaged package context reconstructed from lib/*.folenc
language-provided co.* package context
        |
        v
ordinary package / symbol / member / overload resolution
```

Executable application source under the primary `src/` domain imports a selected package through the ordinary package form:

```
@co.ddap.import(package="hr.employee", as="emp")
```

It never imports the `packaged` component itself. Source under any `components/<kind>/` domain cannot perform this import; its package-resolution environment does not contain the selected packaged-component contexts.

Packaging into `.folenc`

A project-local packaged component does not produce an independent artifact.

For an executable application project, only package contexts explicitly selected by `components/packaged/component.fol` become project-owned open source contexts in the application's **primary `src/` compilation graph**. The frontend does not add those contexts to any project-local component's package index. Unselected descendant packages remain component-private implementation source.

A standalone projected **application** library cannot contain `components/packaged/` or any other project-local component except its optional `components/operators/`. It therefore produces a purely projected `.folenc` surface. Reusable implementation dependencies for that library come from `lib/`, not from project-local components.

Standalone `system`, `ffi`, and `dynamicvmrt` libraries likewise have no `components/` tree and remain isolated projected domains.

A standalone packaged library selects `src/` package contexts from its own `src/component.fol`; see Libraries. It has no `components/` tree at all, cannot create project-local custom operator spellings, and may only overload built-in or language-predeclared operators according to the ordinary operator rules.

Application-Wide Semantic Participation

Packaged/open source is intentionally integrated rather than isolated **only after explicit export selection and only into the executable application's primary `src/` graph**. Once a selected package context enters that application graph, its accessible types, inheritance relations, overload families, and applicable AST participate in application-wide semantics. The same context is not injected into any peer component's semantic graph. Unselected packaged-component packages remain private and do not enter the application-wide dispatch/type universe. Therefore application-wide semantic directives that operate on the open application graph, including `@co.ddap.dynamicdispatch(true)`, apply to participating declarations and overload call sites from exported packaged contexts. See Dynamic Multi Dispatch.

Interaction with Operators

Packaging does not export or consume a project-local parser operator table. A packaged library or packaged component may overload built-in or language-predeclared operators according to the ordinary operator ownership rules, but it cannot create a new operator spelling. In an executable application project, packages selected from `components/packaged/` also **cannot use custom operator spellings declared by the owning application's `components/operators/`**, even after those packages are exported into the application's open package graph. Custom operator syntax is available only to the owning application's ordinary `src/` source domain.

Custom operator creation and application-only consumption are governed exclusively by Operators.

Libraries

A standalone distributable producer uses the canonical structure defined in Project Layout. Its fixed primary surface is `src/component.fol`.

Standalone Structure

```
<project-root>/
└── src/
    ├── component.fol
    └── <package directories>/
```

`components/`, `lib/`, and `build/`, when applicable, follow Project Layout.

Common Structural Container

Every standalone `src/component.fol` contains exactly one:

```
_ co.lang.component = {
    ...
}
```

`co.lang.component` is the structural surface/container. It does not by itself say whether the produced artifact exposes a projected library API or open package contexts. That exposure model is determined as follows.

Projected Library Form

A projected standalone library annotates the component declaration with `@co.dap.library`:

```
@co.dap.library
_ co.lang.component = {
    // projected application APIs
}
```

Omitting `type` means `application`. The explicit equivalent is:

```
@co.dap.library(type=application)
_ co.lang.component = {
    // projected application APIs
}
```

Other legal projected kinds are:

```
@co.dap.library(type=dynamicvmrt)
@co.dap.library(type=system)
@co.dap.library(type=ffi)
```

Each annotation applies to the single `_ co.lang.component` declaration in `src/component.fol`. The resulting `<project-name>.folenc` exposes that projected surface through `@co.ddap.import(library="...")`. `@co.dap.library` is valid for this standalone `src/component.fol` projected form only; it is invalid on ordinary package-owned declarations and on every project-local `components/<kind>/component.fol` surface.

`system`, `ffi`, and `dynamicvmrt` are always projected. Their internal packages cannot be exposed directly as packaged/open packages; consumers reach them only through the declared projected surface. A projected `application` library may contain **only** the optional `components/operators/` exception; it may not contain any other project-local component kind. Standalone `ffi`, `system`, and `dynamicvmrt` libraries have no `components/` tree. They cannot create new operator spellings, though they may provide legal overloads for FoLang-owned built-in or pre-declared operators.

Packaged Library Form

A packaged standalone library has no `@co.dap.library` annotation. Its component body contains the packaged export selector. In this structural context `@co.dap.export(...)` applies to the containing `_ co.lang.component` and is not waiting for a following declaration: The package-selector form of `@co.dap.export(...)` is valid only in packaged `src/component.fol` and `components/packaged/component.fol` structural contexts.

```
// src/component.fol
_ co.lang.component = {
    @co.dap.export(
        packages={
            hr.employee: {recurse=true},
            text.format: {recurse=true}
        }
    )
}
```

All `src/` packages of a packaged standalone library are producer-private until selected by the `@co.dap.export(...)` entry in `src/component.fol`. Selected package paths are relative to `src/`; only those selected package contexts become intentional distributable roots of the resulting `<project-name>.folenc`. Because this form has no projected/safe API surface, those contexts are consumable through `@co.ddap.import(package="...")` **only by an executable application's primary open graph**. Standalone libraries and project-local components may not import them. Every unselected `src/` package remains producer-private and is retained only as internal implementation when required by reachable exported code.

A packaged library uses ordinary application-language capabilities. If it needs FFI, system, or dynamic-runtime functionality, it must consume those capabilities through their projected component/library surfaces rather than acquiring those capabilities itself.

A packaged standalone library has **no `components/` tree**. Consequently it cannot declare new project-local operator spellings. It may still provide overload implementations for built-in and language-predeclared operators whose syntax is already owned by FoLang, subject to the normal operator ownership/signature rules.

Form Exclusivity

For `src/component.fol`:

```
@co.dap.library present
    -> projected standalone library

@co.dap.library absent
+ @co.dap.export(...) in the component body
    -> packaged standalone library
```

A standalone `src/component.fol` that establishes neither form is invalid. `@co.dap.library` and the standalone packaged-library selector are mutually exclusive for the primary exposure model.

Packaged/Open Integration

A packaged library is intentionally open to an **executable application's** type and overload graph only through package contexts explicitly selected by its export selector. It is not a general library-to-library dependency form because it has no projected/safe surface API. Metadata required by those exported contexts—including hierarchy information, overload families, and applicable AST/HIR—is retained in `.folenc` so that the consuming application can merge and validate the exported open graph under application-wide semantic directives. Unselected producer packages remain private implementation contexts and are not published into the consumer's package graph. This includes dynamic multiple dispatch when enabled by the application.

A projected library is different: its implementation remains behind its declared surface and the consumer does not reinterpret internal projected-library calls merely because the consumer enables an application-wide directive.

Components

For the authoritative list of component kinds, filesystem locations, direct `component.fol` form, and descendant-package constraints, see Project Layout.

Every project-local component surface has the structural form:

```
// components/<kind>/component.fol
_ co.lang.component = {
    ...
}
```

The ordinary project-local component model is executable-application-owned source compiled as part of that application. A component is **not** a standalone library, uses no `@co.dap.library`, and produces no independent `.folenc` artifact. The immediate `components/<kind>/` folder supplies the kind and capability context. The sole standalone-library exception is `components/operators/` in a projected application library; that exception contributes parser metadata only and is not an importable component API.

All descendant packages of every component kind that permits package directories are **private to that component by default**. `components/` is not an ordinary package-import root. The only outward gateway is the owning `component.fol` surface:

```
projected component
    -> publishes only its declared surface APIs
    -> descendant packages remain private

packaged component
    -> publishes only package contexts selected by @co.dap.export(...)
    -> publication target is the executable application's primary src/ graph only
    -> no peer component can consume selected package contexts
    -> unselected descendant packages remain private

operator component
    -> publishes operator syntax metadata to its permitted owning domain
    -> has no descendant packages
```

Projected Components

The projected component kinds are `application`, `ffi`, `system`, and `dynamicvmrt`.

```
// components/ffi/component.fol
_ co.lang.component = {
    // projected APIs exposed to the owning project
}
```

The owning primary `src/` domain sees only declarations published through a projected component surface. Every descendant implementation package remains private to that component compilation context and cannot be imported, qualified, or referenced directly through `package=` or any physical component path. **Peer components do not see the surface at all.** A project-local component is therefore a leaf dependency domain with respect to other project-local components.

Projected components are imported by component identity only from the executable application's primary `src/` domain:

```
@co.ddap.import(component="ffi", as="ffilib")
```

The same directive inside any `components/<kind>/` source is a compiler error; peer component surfaces are not importable from a component.

The packaged component exposes only package contexts explicitly selected by its `@co.dap.export(...)` surface entry rather than a projected component namespace; those selected contexts are application-facing only and are not visible from any other project-local component. All unselected descendant packages remain private. See Packaged Component. The operator component contributes project-local parser/operator metadata rather than an importable API; see Operators.

Component Boundary Model

For projected component and projected standalone-library surfaces, the kind controls boundary representation and transfer semantics:

Kind	Boundary data	Transfer semantics
`application`	`co.lang.struct`	automatic deep snapshot
`dynamicvmrt`	`co.lang.struct`	automatic deep snapshot
`system`	`co.lang.cstruct`	system ABI value
`ffi`	`co.lang.cstruct`	C ABI value

`co.lang.struct` is a FoLang semantic data contract; `co.lang.cstruct` is a physical ABI-compatible value contract. Application and dynamic-runtime projected surfaces therefore use managed `struct` contracts transferred as deep snapshots, while system and FFI projected surfaces use restricted `cstruct` contracts.

Projected Library and Component Surface Rules

The following rules apply to projected `_ co.lang.component` surfaces: a standalone `src/component.fol` annotated with `@co.dap.library`, or a projected project-local component under `components/application/`, `components/ffi/`, `components/system/`, or `components/dynamicvmrt/`. They do not apply to packaged/export selector surfaces or to the operator component. Structural placement is defined in Project Layout.

Allowed Projected Surface Declarations

This subsection applies to every projected `_ co.lang.component` surface covered by this section. The canonical source locations for those surfaces are defined in Project Layout.

A projected component/library surface may contain only:

- file- or library-level imports needed by its adapter implementations
- `co.lang.struct` boundary declarations for `application` and `dynamicvmrt`
- `co.lang.cstruct` boundary declarations for `system` and `ffi`
- public free-function API declarations with boundary-adapter definitions

The following declaration kinds are forbidden directly in every projected `co.lang.component` surface:

- classes
- modules
- interfaces
- signatures
- units and companion units

- associated functions
- operator functions
- enums, unions, and other ADTs
- type aliases, newtypes, and opaque types
- objects, instances, type classes, and dependent types
- generic declarations, generic type parameters, `@co.dap.generic`, and generic public surface signatures
- macros, templates, annotations, and decorators
- global variables, pointers, references, addresses, or mutable surface state

Surface `struct` and `cstruct` declarations are data contracts only. They cannot have companion units, associated functions, operators, methods, or other behavior on the surface.

Declarations directly inside a projected `co.lang.component` body are exported by default. Imports and implementation details are never exported.

Public Signature Type Closure

Every public surface function signature must be closed over the projected surface's exported boundary type set. Generic parameters are not boundary types: projected surface functions are concrete API contracts and cannot declare or expose generics. Packaged/exported source is different because selected package declarations join the open graph rather than crossing a projected surface.

For `application` and `dynamicvmrt` surfaces, parameters and results may use only:

- approved built-in types
- `co.lang.struct` types declared in the same projected surface

For `system` and `ffi` surfaces, parameters and results may use only:

- ABI-safe built-in types
- `co.lang.cstruct` types declared in the same projected surface

The same closure rule applies recursively to fields of surface boundary types:

- a surface `struct` field may use an approved built-in type or another surface-declared `struct`
- a surface `cstruct` field may use an ABI-safe built-in type or another surface-declared `cstruct`
- an internal package type may never appear in a public function signature or surface boundary-type field
- pointers, references, and addresses may never cross any projected public surface

A built-in type is not automatically surface-safe merely because it belongs to `co.lang`. An approved surface built-in must be concrete, fully resolved, and transferable under the applicable projected kind's boundary semantics.

The following categories are forbidden in public surface fields and signatures:

- inference-only types such as `co.lang.auto` and `co.lang.infer`
- dynamically typed or unconstrained carriers such as `co.lang.dynamic`, `co.lang.any`, `co.lang.typed`, and `co.lang.untyped`
- function, closure, delegate, loader, AST, reflection, or runtime implementation values
- pointer, reference, address, thunk, and implementation-handle types
- any type whose reachable representation contains a forbidden type

For application-family surfaces, managed built-ins such as `co.lang.string` are permitted when the compiler defines deep-snapshot reconstruction for them. For system and FFI surfaces, only built-ins with a defined ABI representation are permitted; for example, `co.lang.string` is not directly cstruct-compatible.

Valid:

```
// Employee.fol
_ co.lang.struct = {
    id      co.lang.int;
    name    co.lang.string;
    address Address;
}

// Address.fol
_ co.lang.struct = {
    city co.lang.string;
}
```

Invalid:

```
getEmployee(id co.lang.int)->(emp.internal.Employee);
// compiler error: an internal type escapes through the public signature
```

Boundary-Adapter Functions

A public surface function may contain a definition, but its definition is restricted to boundary adaptation.

A boundary-adapter function may:

- read its input parameters and their fields
- declare temporary local values used only for conversion
- construct internal input values
- perform deterministic field-to-field or representation conversion
- invoke exactly one internal implementation operation

- receive that operation's result
- construct and return an allowed surface `struct`, surface `cstruct`, or built-in value
- use a direct delegation expression when no conversion is required

A boundary-adapter function may not:

- implement business rules or domain calculations
- perform business validation or authorization decisions
- orchestrate multiple implementation operations
- call another surface function
- perform persistence, networking, file I/O, retries, caching, or transactions
- start concurrency, scheduling, asynchronous work, processes, or continuations
- contain loops, recursion, workflow branching, or externally observable state mutation
- expose an internal value directly without converting it to an allowed boundary type

The compiler enforces the restricted adapter statement set. The restriction is structural: arbitrary executable logic is not accepted merely because it is placed in a surface file.

A direct delegate is the simplest adapter form:

```
health()->(co.lang.bool)
    =>> health.internal.Service.health();
```

A converting adapter may map between the public contract and an internal model.

Standalone Projected Application-Library Surface Example

```
// src/component.fol — standalone application library
@co.dap.library
_ co.lang.component = {

    @co.ddap.import(package="emp", as="emp")

    Employee co.lang.struct = {
        name co.lang.string;
        id   co.lang.int;
    }

    getEmployee(empId co.lang.int)->(Employee) = {
        internalEmployee := emp.EmployeeService.getEmployee(empId);

        this.return Employee{
            name: internalEmployee.name,
            id: internalEmployee.id
        };
    }
}
```

The surface owns conversion from the internal `emp.Employee` representation to the public `hrlib.Employee` contract. The `emp` package owns validation, business rules, persistence, and workflow.

The consumer sees the equivalent API contract:

```
// Employee.fol
_ co.lang.struct = {
    name co.lang.string;
    id   co.lang.int;
}

getEmployee(empId co.lang.int)->(Employee);
```

The consumer does not see the body of `getEmployee` or the `emp` package.

Standalone Projected System-Library Surface Example

The following example is a separately authored library project. Project-local `components/system/component.fol` does not repeat the kind annotation because its fixed path already establishes `system`.

```
// src/component.fol — standalone system library
@co.dap.library(type=system)
_ co.lang.component = {

    @co.ddap.import(package="driver.internal", as="impl")

    Point co.lang.cstruct = {
        x co.lang.int;
        y co.lang.int;
    }

    getOrigin()->(Point) = {
        internalPoint := impl.DriverService.getOrigin();

        this.return Point{
            x: internalPoint.x,
            y: internalPoint.y
        };
    }
}
```

Pointers, addresses, native bindings, and hardware state belong in `driver.internal`. They are forbidden in the public surface and cannot appear in the exported signature.

Boundary Transfer Semantics

For `application` and `dynamicvmrt` projected surfaces:

```
consumer boundary value
    ↓ automatic deep snapshot of the complete reachable value graph
library-local built-in or surface struct
    ↓ surface conversion
internal implementation value
    ↓ surface conversion
library-local built-in or surface struct
    ↓ automatic deep snapshot
consumer-local boundary value
```

The snapshot rule applies to both surface structs and approved built-in arguments/results. The library never receives the consumer's live object identity, and the consumer never receives a live internal-library object.

For projected `system` surfaces, `cstruct` values cross according to the selected backend/platform system ABI.

For projected `ffi` surfaces, `cstruct` values cross according to the declared C ABI, including its layout, alignment, and calling-convention requirements.

Surface-to-Internal Dependency Direction

The source-level dependency is one-way:

```
projected surface
    ↓ imports and invokes
internal packages
```

Internal packages:

- do not import the projected surface
- do not use surface-declared `struct` or `cstruct` types
- define their own implementation and domain types
- return internal values to the surface adapter
- contain all business logic, validation, workflow, I/O, and state management

This prevents a surface/internal compilation cycle and keeps the public contract independent from the internal domain model.

An internal implementation symbol may be visible to its own projected surface without becoming part of the consumer API. Library encapsulation takes precedence over ordinary package visibility: consumers cannot resolve internal package symbols even when those symbols are callable from the surface during library compilation.

Consumer API Projection

The compiler does not expose the complete surface compilation context to a consumer. It creates a projected imported symbol table.

The projected API contains:

- the projected library/component identity and kind
- complete public surface `struct` or `cstruct` definitions, including field names and permitted field types
- public function names
- public function parameter names and types
- public function result types
- calling-convention, effect, error, and linkage metadata where applicable

The projected API does not contain:

- function bodies or implementation AST/HIR
- imports used by the surface

- local conversion variables
- delegate or implementation targets
- internal package paths or symbols
- private compiler-generated helpers
- business classes, modules, units, or internal data types

Therefore, a function definition may exist in the surface source and compiled artifact while the consumer's symbol table contains only its signature.

The backend may retain implementation bodies for code generation, linking, or whole-program optimization. Backend possession of implementation code does not make that code semantically visible to the consumer.

Project-local projected components follow the same boundary rule. Availability of their source code inside the owning project does not make their private implementation packages semantically visible outside the component surface.

Library Compilation Order

A library uses the same import-usage and symbol-usage validator as an application. It is processed in these logical stages:

1. parse the surface header, boundary data declarations, and public function signatures;
2. parse/prepare internal packages without depending on surface types;
3. register internal imports as provisional context references rather than copying symbol tables;
4. type-check and link surface boundary-adapter bodies against internal package symbols, activating an import only when a symbol from it is actually used;
5. complete semantic resolution and mark exact usage-checkable symbols live;
6. prune and report zero-use imports;
7. reject unused usage-checkable symbols and unreachable internal source/packages;
8. emit the compiled implementation and projected consumer API.

The declarations intentionally exported by a projected surface (`src/component.fol` or a projected `components/<kind>/component.fol`) are semantic roots of the applicable library/component internal compilation graph. This order permits the surface to call internal packages while preventing internal packages from depending on the surface. The different producer, project-local component, and `.folenc` consumer usage rules are defined in Unused Symbols, Liveness, and Reachability.

Projected Capability Kinds

Filesystem placement and component slots are defined in Project Layout. This section defines capability semantics only.

FoLang has four projected capability kinds:

```
application
dynamicvmrt
system
ffi
```

Macros, templates, concurrency, asynchronous execution, continuations, scheduling, transformation facilities, and other ordinary non-privileged language features belong to the application capability domain unless another section explicitly restricts them.

A packaged library/component is not a fifth privileged capability kind. It uses ordinary application capabilities and differs by **exposure model**: only packages explicitly selected by `@co.dap.export(...)` join the open package/type graph; every unselected package remains private to the packaged producer/component.

`application`

`application` is the ordinary FoLang programming capability domain. It permits normal language features including classes, modules, interfaces, signatures, units, structs, generics, macros, templates, concurrency, async/parallel abstractions, continuations, scheduling, transformations, and other standard application-level facilities.

It does not directly acquire `system`, `ffi`, or `dynamicvmrt` privileged capabilities. Application code may consume those domains only through their projected surfaces.

Public projected boundary:

- surface `struct` contracts;
- approved built-in types;
- automatic deep-snapshot transfer.

`dynamicvmrt`

`dynamicvmrt` includes application capabilities plus the dynamic-runtime/metaprogramming capabilities defined for `co.meta`, including runtime reflection, instrumentation, dynamic loading, patching, eval-based facilities, and related dynamic-runtime operations.

It does not acquire `system` or `ffi` capabilities. Those domains remain accessible only through projected surfaces.

Public projected boundary:

- surface `struct` contracts;
- approved built-in types;
- automatic deep-snapshot transfer.

`system`

`system` is an isolated low-level capability domain for raw pointers, references, addresses, word-level types, native operations, platform/runtime implementation, MMIO, low-level process/thread primitives, and other explicitly system-authorized facilities.

It does not inherit application, dynamic-runtime, or FFI capabilities merely because those features exist elsewhere in FoLang. Cross-domain use occurs only through projected surfaces.

Public projected boundary:

- surface `cstruct` contracts;
- ABI-safe built-in types;
- system ABI value transfer.

`ffi`

`ffi` is an isolated foreign-interface capability domain for extern declarations, foreign symbol bindings, pointers/addresses needed for supported ABI work, C-compatible types/cstructs, calling conventions, and linkage metadata.

It does not inherit application, dynamic-runtime, or system capabilities. Cross-domain use occurs only through projected surfaces.

Public projected boundary:

- surface `cstruct` contracts;
- C-ABI-safe built-in types;
- C ABI value transfer.

`packaged`

A packaged library or packaged component uses application capabilities but has no projected API boundary for the packages it explicitly exports. For a project-local `components/packaged/`, only explicitly selected package contexts participate directly in the executable application's primary `src/` open package/type/overload graph; they are not visible to peer components. A standalone packaged library likewise contributes its explicitly exported package contexts **only to an executable consuming application's primary open graph** through the loaded artifact. It cannot be imported into another standalone library or project-local component. Unselected packages remain private implementation contexts.

A packaged domain cannot directly acquire FFI, system, or dynamic-runtime capabilities. Packaged library code and packaged-component code may call independently built **projected libraries** through their published `library=` surfaces when the applicable boundary rules permit it; they may not reach peer project-local components. The executable application remains the composition root for its own project-local component surfaces.

Dependency Direction

FoLang capability domains are isolated rather than arranged in the former linear capability ladder.

```
application
    -> ordinary language capabilities

packaged
    -> ordinary application capabilities + open package exposure

dynamicvmrt
    -> application capabilities + dynamic-runtime capabilities

system
    -> isolated system/native capability domain

ffi
    -> isolated foreign-ABI capability domain
```

The absence of direct capability inheritance does **not** prevent independently built **standalone projected libraries** from calling other standalone projected libraries through their published surface APIs. Calling a projected library API does not grant the caller the callee's internal capability set or expose the callee's private types.

FoLang deliberately does **not** encode a pairwise architectural dependency matrix for standalone projected `application`, `ffi`, `system`, or `dynamicvmrt` libraries. For example, an FFI library depending on a system library through that system library's published API may be architecturally questionable in a particular project, but it is not rejected merely because of the FFI-to-system direction. The compiler instead enforces the actual language boundary: import/exposure legality, public API types, capability use, visibility, liveness, cycles, and other applicable semantic constraints. Standalone packaged libraries are excluded from this rule: their exported contexts may be consumed only by an executable application and never by another library or component.

Project-local components are different. They are not independently consumable libraries; they are implementation components owned by one project. Their dependency direction is fixed:

```
owning primary src/ -> project-local component     allowed
project-local component -> peer component          compiler error
```

No `components/application/`, `components/ffi/`, `components/system/`, `components/dynamicvmrt/`, `components/packaged/`, or `components/operators/` source may consume another project-local component. `components/packaged/` publishes selected packages only to the executable application's primary `src/` graph, and `components/operators/` publishes custom syntax only to the executable application's ordinary primary `src/` domain. A projected application library may also own `components/operators/` as its sole component exception, with syntax visible only to that library's ordinary primary `src/` source.

Examples:

```
application src -> components/ffi surface              allowed
application src -> components/system surface           allowed
components/ffi -> components/system                     compiler error
components/system -> components/ffi                     compiler error
components/application -> components/dynamicvmrt        compiler error
components/packaged -> components/application           compiler error

standalone ffi library -> standalone projected system library API allowed if API/boundary rules pass
standalone system library -> standalone projected ffi library API allowed if API/boundary rules pass
standalone library/component -> standalone packaged-library packages compiler error
caller -> use callee-private privileged capability      forbidden

system / ffi remain isolated implementation domains
application or packaged code reaches them only through their projected surfaces
```

Dependency cycles through effective imports remain compile-time errors.

Capability Isolation

`system`, `ffi`, and `dynamicvmrt` are mutually distinct capability domains. `dynamicvmrt` additionally includes ordinary application capabilities; `system` and `ffi` do not. No source file gains another domain's privileged capability merely by importing its projected surface.

Types crossing a projected boundary must satisfy the boundary rules for that domain. Internal implementation types remain private and cannot be imported directly.

Packaged/Open Code

Packaged/open package contexts are application-domain code from the **executable application consumer's** perspective. Their actual declarations and applicable AST are merged into that application's open graph rather than hidden behind a projected boundary. They are not legal imports into standalone libraries or project-local components. Consequently, once admitted to the executable application graph, they inherit application-wide semantic rules, including dynamic multiple dispatch when enabled.

Units in detail

A `co.lang.unit` is a stateless file-level declaration container. It is not instantiable and does not create an object, runtime scope, or public unit namespace.

FoLang has two unit-file forms:

1. **ordinary package unit** — `<Fragment>.unit.fol`
2. **struct companion unit** — `<StructName>.comp.unit.fol`

Both forms use `_ co.lang.unit`; explicit unit names are invalid.

A unit is an organizational container rather than a public symbol namespace. Visibility annotations apply to the declarations contributed by the unit; they do not apply to `_ co.lang.unit` itself.

Ordinary Package Units

Example package layout:

```
math/
├── arithmetic.unit.fol
├── comparison.unit.fol
└── optional.unit.fol
```

```
// arithmetic.unit.fol
_ co.lang.unit = {
    abs(value co.lang.int)->(co.lang.int) = { ... }

    max(
        a co.lang.int,
        b co.lang.int
    )->(co.lang.int) = { ... }
}
```

```
// optional.unit.fol
_ co.lang.unit = {
    Option(T) co.lang.type =
        Some(T) | None();

    isSome(value Option(co.lang.int))->(co.lang.bool) = {
        ...
    }
}
```

All declarations are contributed directly to the `math` package namespace:

```
math.abs(-10);
math.max(10, 20);
value math.Option(co.lang.int);
```

The unit filenames never appear in qualified names:

```
math.Arithmetic.abs(-10); // compiler error
math.Optional.Option(co.lang.int); // compiler error
```

Within the same package, the functions and types may be referenced without the package prefix according to ordinary package name-resolution rules.

An ordinary unit may contain:

- receiverless functions
- `co.lang.type` aliases and ADT/type-constructor declarations
- newtype and opaque-type declarations
- subtype and supertype declarations
- macros and template declarations
- other non-instantiable type declarations explicitly permitted by their sections
- public, package, protected, or private members

An ordinary unit may not contain:

- fields or mutable unit state
- executable loose statements
- classes, structs, cstructs, enums, unions, modules, interfaces, or signatures
- lifecycle declarations for the unit
- a declaration that creates an independent nested primary namespace

A unit file may group closely related declarations or merely organize a convenient set of package-level functions and types. Semantic cohesion is recommended but not enforced.

For canonical unit and per-symbol usage rules, see Unused Symbols, Liveness, and Reachability.

Package-Namespace Consolidation

All ordinary unit files in one package are shallow-parsed and consolidated into one package symbol table. Their order and filenames have no semantic effect.

```
package members =
    filename-derived primary declarations
    + declarations from every ordinary unit file
```

A duplicate is reported as soon as the package index is built. Conflicts include:

- the same type or value name contributed by two ordinary units
- an ordinary-unit member colliding with a filename-derived primary declaration
- duplicate function signatures not permitted by overload rules
- declarations that normalize to the same canonical package symbol

Diagnostics should identify both source files.

Companion Units

A companion unit is governed by the filename and owner-validation rules in Struct Companion Units. Its members do not merge directly into the package namespace; they attach to the owner struct's companion namespace.

Companion-unit functions follow the canonical strict per-symbol usage rule in Unused Symbols, Liveness, and Reachability.

Units have no fields, identity, instances, inheritance, polymorphic dispatch, or independent construction.

CStructs

`co.lang.cstruct` is a C-like value type — passed by value, simple memory layout, safe to cross zone boundaries. Unlike `co.lang.struct` which is passed by reference, `co.lang.cstruct` is always copied on pass.

```
// Point.fol
_ co.lang.cstruct = {
    x co.lang.int;
    y co.lang.int;
}

// Rect.fol
_ co.lang.cstruct = {
    origin Point;
    width  co.lang.int;
    height co.lang.int;
}
```

cstruct Rules

```
always passed by value — never by reference
simple memory layout — no metadata
can contain only simple types and other cstructs
cannot contain co.lang.struct             ❌  has metadata
cannot contain co.lang.string             ❌  heap allocated
cannot contain co.lang.dynamic            ❌  runtime type info
cannot contain classes                    ❌  vtable, metadata
cannot contain modules                    ❌
cannot contain any heap allocated type    ❌
cannot have methods
cannot have associated functions
cannot embed co.lang.struct
safe to cross direct ABI and zone boundaries

allowed field types:
    co.lang.int, co.lang.uint, co.lang.float  ✅  primitives
    co.lang.bool, co.lang.char, co.lang.byte  ✅  primitives
    co.lang.int->([N])                         ✅  fixed size arrays
    co.lang.cstruct                            ✅  other cstructs
```

Packed cstruct — no padding, exact memory layout

Used for hardware registers, binary protocols, exact memory mapped formats:

```
// Register.fol
@co.dap.packed
_ co.lang.cstruct = {
    flags  co.lang.uint8;
    status co.lang.uint8;
    data   co.lang.uint16;
}
```

SIMD cstruct — aligned for vector operations

Used for math, graphics, signal processing:

```
// Vec4.fol
@co.dap.simd(align=16)
_ co.lang.cstruct = {
    x co.lang.float;
    y co.lang.float;
    z co.lang.float;
    w co.lang.float;
}
```

Both together

```
// AVXVec.fol
@co.dap.packed
@co.dap.simd(align=32)
_ co.lang.cstruct = {
    data co.lang.float;
}
```

> `@co.dap.packed` and `@co.dap.simd` are specialisations of `co.lang.cstruct` — same rules, same zone boundary safety. They are not separate types.

Structs

```
// myStruct.fol
_ co.lang.struct={
    field1 co.lang.int;
    field2 co.lang.string;
    field3 co.lang.bool;
}
```

Struct Rules

```
structs cannot extend other structs
structs cannot contain methods directly
structs can compose other structs
structs cannot have default values to fields/members
structs can embed other structs (Go lang like)
structs can have a same-package companion file named `<StructName>.comp.unit.fol`
```

The struct declaration remains pure data. Associated behaviour is declared separately in `<StructName>.comp.unit.fol`.

Struct Embedding

Embedding promotes fields of an embedded struct directly into the outer struct — they act as the outer struct's own fields at construction and access sites. This is distinct from composition where the embedded struct is a named field.

```
// E.fol
_ co.lang.struct = {
    id   co.lang.int;
    name co.lang.string;
}

// ✅ No conflict — id and name promoted as B's own fields
// B.fol
_ co.lang.struct = {
    age co.lang.float;
    E;                    // embedded — id and name promoted
}

b := B{ age: 25.0, id: 1, name: "Rao" };   // all fields at same level
b.id    // direct — no b.E.id needed
b.name  // direct — no b.E.name needed
b.age   // direct
```

```
// ❌ Compiler error — name conflict between B.name and E.name
// B.fol
_ co.lang.struct = {
    name co.lang.string;   // conflicts with E.name
    E;
    age  co.lang.float;
}
// Fix 1 — rename B's conflicting field
// Fix 2 — use explicit composition instead: e E;
```

```
// Explicit composition — no promotion, always qualified access
// B.fol
_ co.lang.struct = {
    name co.lang.string;
    e    E;               // named field — no conflict, no promotion
    age  co.lang.float;
}

b.name ;   // B's own name
b.e.id ;   // E's id — always explicit
b.e.name;  // E's name — always explicit
```

Embedding Rules

Situation	Behavior
Embedded field, no conflict	Promoted — acts as the outer struct's own field
Embedded field, name conflict with outer	❌ Compiler error — rename or use composition
Multiple embeds, no conflict between them	All fields promoted
Multiple embeds, conflict between embedded structs	❌ Compiler error
Explicit composition (`e E`)	No promotion — always accessed via `b.e.field`

> FoLang does **not** silently shadow conflicting fields. Any name conflict is a compiler error — the programmer must make a conscious decision to rename or switch to explicit composition.

Struct Declaration Relationships

A struct cannot physically declare another struct, enum, class, module, function, signature, interface, or other named declaration inside its body. A struct body remains a pure data declaration containing fields and permitted embeddings only.

Use one of the following instead:

- **composition** through a named field
- **embedding** where the declaration kind's embedding rules permit it
- a separately declared target-local declaration restricted with `@co.dap.local`

```
// EmployeeAddress.fol
@co.dap.local(for=hr.employee.Employee)
_ co.lang.struct = {
    street co.lang.string;
    city   co.lang.string;
}
```

```
// Employee.fol
_ co.lang.struct = {
    id      co.lang.int;
    name    co.lang.string;
    address EmployeeAddress; // composition
}
```

The following physical nesting is invalid:

```
// Employee.fol
_ co.lang.struct = {
    Address co.lang.struct = { // ❌ nested declaration
        city co.lang.string;
    }

    address Address;
}
```

```
structs cannot declare inner structs     ❌  compiler error — only through @co.dap.local
structs can declare inner enums/ADTs     ❌  compiler error — only through @co.dap.local or @co.dap.nested
structs cannot declare inner classes     ❌  compiler error — struct is pure data
structs cannot declare inner modules     ❌  compiler error — struct is pure data
```

`@co.dap.local` controls declaration visibility only. It does not automatically compose or embed the annotated declaration into the target struct.

Struct Companion Units

A struct companion unit is declared in a file named `<StructName>.comp.unit.fol`. The matching struct remains in `<StructName>.fol`, and both files must belong to the same package.

```
// Vector.fol
_ co.lang.struct = {
    x co.lang.float;
    y co.lang.float;
}
```

```
// Vector.comp.unit.fol
_ co.lang.unit = {
    distance(
        left  Vector,
        right Vector
    )->(co.lang.float) = {
        ...
    }

    zero()->(Vector) = {
        this.return Vector{x: 0.0, y: 0.0};
    }

    (value Vector) magnitude()->(co.lang.float) = {
        this.return co.math.sqrt(
            value.x * value.x + value.y * value.y
        );
    }

    (Vector) create(
        x co.lang.float,
        y co.lang.float
    )->(Vector) = {
        this.return Vector{x: x, y: y};
    }
}
```

The filename establishes ownership. No companion member needs an ordinary parameter merely to prove association.

Call forms:

```
d      := Vector.distance(first, second);
origin := Vector.zero();
length := value.magnitude();
point  := Vector.create(10.0, 20.0);
```

A companion unit may declare:

- receiverless companion functions, including zero-parameter functions and factories
- instance-associated functions with an explicit value receiver
- type-associated functions with an explicit type receiver
- operator functions permitted for the owner struct
- non-UDT type declarations associated with the owner when their own sections permit them

```
receiverless non-operator companion function
    -> has no explicit receiver
    -> ordinary parameters may have any legal types
    -> no owner-typed parameter is required merely for association

instance-associated function
    -> explicit receiver is a value of the matching struct

type-associated function
    -> explicit receiver is the matching struct type itself
```

These are the general companion rules. Operator functions have the additional operand requirement defined in the Operator Functions subsection.

Companion declarations do not make the struct a class and do not add object identity, inheritance, virtual dispatch, lifecycle methods, or unit-level state.

Filename and Owner Validation

The compiler classifies companion files before parsing:

```
Vector.comp.unit.fol -> canonical owner Vector
```

At package-indexing level, the compiler verifies:

- a canonical primary declaration named `Vector` exists in the same package
- no second companion file resolves to the same canonical owner
- the companion is not orphaned

After the owner declaration header is available, the compiler verifies that the owner is a `co.lang.struct`. A class, cstruct, enum, union, module, interface, signature, or imported declaration with the same short name cannot become the owner.

Receiver Validation

The companion filename determines the required receiver root. Any explicit receiver must match that owner.

```
// Employee.comp.unit.fol
_ co.lang.unit = {
    create(id co.lang.int)->(Employee) = { ... } // valid: receiverless

    (emp Employee) isValid()->(co.lang.bool) = { ... } // valid

    (Employee) empty()->(Employee) = { ... } // valid

    (dept Department) isValid()->(co.lang.bool) = { ... }
    // compiler error: receiver Department does not match companion owner Employee

    (Department) create()->(Department) = { ... }
    // compiler error: type receiver Department does not match companion owner Employee
}
```

Ordinary parameters may have any legal types. They are not used to establish companion ownership.

For a generic struct owner, receiver validation compares the canonical root declaration and generic arity. For example, a receiver based on `Box->(T)` may belong to `Box.comp.unit.fol`; `Box->(T, E)` or an unrelated root does not.

Companion Namespace and Conflicts

Companion members are resolved through the owner:

```
employee := Employee.create(1);
valid    := employee.isValid();
```

The complete companion namespace is formed from all legal owner-associated declarations and the single companion file. Duplicate names and normalized duplicate signatures are compile-time errors.

The following are invalid:

```
Employee.comp.unit.fol without Employee.fol
Employee.comp.unit.fol and employee.comp.unit.fol in the same package
Employee.comp.unit.fol when Employee.fol declares a class or cstruct
an explicit receiver whose canonical root is not Employee
```

Operator Functions

Operator functions associated with a struct belong in its companion unit. Ownership comes from the companion filename; receiver validation follows the same rule as other companion functions.

```
// Employee.comp.unit.fol
_ co.lang.unit = {
    @co.dap.operator(symbol="+")
    (emp Employee) add(other Employee)->(Employee) = {
        ...
    }

    @co.dap.operator(symbol="==")
    equals(
        left  Employee,
        right Employee
    )->(co.lang.bool) = {
        ...
    }

    @co.dap.operator(symbol=">")
    (Employee) greater(
        left  Employee,
        right Employee
    )->(co.lang.bool) = {
        ...
    }
}
```

> Companion ownership always comes from `<StructName>.comp.unit.fol`. An operator function has an additional operand rule: a value receiver already supplies the owner instance, but a receiverless operator function or a type-receiver operator function must declare the companion owner type as its first ordinary parameter. This allows operator lowering to pass the actual owner instance to the function. The compiler compares the resolved first-parameter type with the companion owner type at compile time. This requirement establishes the operator operand, not companion ownership. An operator declaration that does not operate on the owner struct is invalid.

Unions

`co.lang.union` declares an untagged ADT. Its body lists the alternative members defined by the union.

```
 // myUnion.fol
 _ co.lang.union={
    intValue co.lang.int;
    strValue co.lang.string;
}
```

Enums

```
// myEnum.fol
_ co.lang.enum={
    Variant1,
    Variant2,
    Variant3
}
```

An enum value's constant expression may use a registered custom operator at any declared precedence. Runtime assignment is forbidden everywhere in that constant-expression subtree, including inside grouping, arguments, and collection or object elements. Whether the resulting expression can actually be evaluated at compile time—including whether a custom operator is foldable— is checked after parsing.

classes

```
// Employee.fol
_ co.lang.class = {
    getEmployeeDetails()->(Employee) = empmodule.getEmployeeDetails;
    // assigning module function to class's method

    getEmployeeInfo()->(Employee) =>> empmodule.getEmployeeDetails();
    // delegating – internally redirecting the call to module function
}

// $1, $2, $3 ... are return components of the immediately previous chained function
// Emp.fol
_ co.lang.class = {
    dosomething(a co.lang.int, b co.lang.int)->(co.lang.int)=>>somePack.someMethod(a)=>>someOthPack.someOtherMeth($1, b);
}
```

Classes with Operator methods

```
// Employee.fol
_ co.lang.class = {
    @co.dap.operator(symbol="+")
    add(other Employee)->(Employee) = {
       // implicit instance method: operands are this and other
    }

    @co.dap.operator(symbol="==")
    @co.dap.static
    equals(
        left  Employee,
        right Employee
    )->(co.lang.bool) = {
        // static operator implementation
    }

    @co.dap.operator(symbol=">")
    @co.dap.class
    greater(
        left  Employee,
        right Employee
    )->(co.lang.bool) = {
        // class-associated operator implementation
    }

}
```

An unannotated operator function in a class is an implicit instance method; the hidden `this` value contributes the first operand. `@co.dap.instance` is the explicit spelling of the same category. `@co.dap.static` and `@co.dap.class` do not contribute an implicit operand, so their declared parameters are the complete operator operand list. Operator signature normalization includes these method categories so equivalent declarations are diagnosed as duplicates.

Class Declaration Relationships

A class cannot physically contain named class, struct, enum, module, function, signature, interface, or other declaration definitions as nested declarations. Class bodies contain fields, lifecycle declarations, and methods permitted by the class model.

Types and helper declarations used only by one class are declared in their ordinary legal source locations and restricted to that class with `@co.dap.local`:

```
// EmployeeAddress.fol
@co.dap.local(for=hr.employee.Employee)
_ co.lang.struct = {
    street co.lang.string;
    city   co.lang.string;
}
```

```
// EmployeeStatus.fol
@co.dap.local(for=hr.employee.Employee)
_ co.lang.enum = {
    Active,
    Inactive,
    Pending
}
```

```
// Employee.fol
_ co.lang.class = {
    address EmployeeAddress;
    status  EmployeeStatus;

    getAddress()->(EmployeeAddress) = {
        this.return this.address;
    }
}
```

The local declarations are visible while compiling `hr.employee.Employee`, but they do not become nested names such as `Employee.EmployeeAddress` or `Employee.EmployeeStatus`.

The following is invalid:

```
// Employee.fol
_ co.lang.class = {
    Address co.lang.struct = { // ❌ physical nested declaration
        city co.lang.string;
    }
}
```

Ordinary visibility annotations do not widen a target-local declaration beyond the declaration or closed target set named by `@co.dap.local` or `@co.dap.nested`.

Method Types

```
// Employee.fol
_ co.lang.class = {

    @co.dap.static
    getEmployee()->(Employee) ={}

    @co.dap.instance
    getEmployee()->(Employee)={}

    @co.dap.class
    getEmployee()->(Employee) ={}

    @co.dap.object
    getEmployee()->(Employee)={}
}

@co.dap.oops(
    A: { inherit:true, virtual:true },
    B: { implements:true }, //interfaces
    C: { inherits:true, abstract=true },
    D: { inherits:true }, // inherits are classes abstract classes
    E: { uses:true },
    F: { composes:true },  //concreate classes
    G: { extends:true }, //mixins
    H: { with:true }, //traits
    I: { associate:true }, //concreate classes
)
// test.fol
_ co.lang.class = {
    getTest(id co.lang.int)->(test) ={}
}
```

The @@new and @@init Methods

`@@new` and `@@init` are lifecycle methods — compiler-owned, not user-definable outside the class. `@@` signals they are restricted lifecycle symbols, not regular methods.

Lifecycle names are valid only as class members. A unit (including a struct companion unit), module, interface, signature, local block, or package declaration cannot declare a lifecycle-named function.

```
// Employee.fol
@co.dap.generic(types=[{name=T}, {name=R}])
_ co.lang.class = {

    id T;
    name R;

    // @@new is provided by default even if not overridden.
    // Override only when you genuinely need to change allocation behavior.

    @co.dap.class
    @co.dap.private
    @@new()->(co.lang.uninit) = { self.return co.const.none; }

    @co.dap.class
    @co.dap.public
    @@new(a co.lang.typevalue, b co.lang.typevalue)->(co.lang.uninit) = {
        // Manual type aliasing — @co.dap.generic handles this automatically
        // Override @@new only when you need custom allocation logic
        T co.lang.type = a;
        R co.lang.type = b;

        // self keyword is allowed only in class methods
        self.parent.new();

        // uninit instance method internally calls new and init
        self.return co.lang.uninit.instance(Employee, self);
    }

    @co.dap.override
    @co.dap.constructor(access=private)
    @@init() = {}

    @co.dap.override
    @co.dap.constructor(access=public)
    @@init(id T, name R) = {
        this.parent.init();
        this.id   = id;
        this.name = name;
    }

    getEmployee(id T)->(Employee) = {}
}
```

Anonymous Classes/Types

```
emp := co.lang.class{};

empObj := emp.init();

empobj1 := co.lang.class{
    name co.lang.string;
}.init();
```

Interfaces

```
// IEmployee.fol
_ co.lang.interface = {
    storeEmployee(emp Employee)->(Employee);
}
```

Signatures

```
// Employee is an ordinary package-level declaration.
// MEmployee.fol
_ co.lang.signature = {
    storeEmployee(emp Employee)->(Employee);
}
```

Structurally they look similar — both are lists of contracts. The difference is **who implements them and how**.

	`co.lang.signature`	`co.lang.interface`
Implemented by	module via `matches=`	class via `implements=[]`
Number of implementations	Any number of distinct modules may match one signature	Any number of classes may implement one interface
Runtime cardinality of one implementation declaration	One module object per loaded module identity	Any number of independently constructed class objects
State model	One shared state for all references to the same module	Separate per-instance state for each class object
May specify required values	✅	❌ — methods only
May reference package-level types	✅	✅
May declare associated or fixed/manifest type components	✅	❌
May require generic type constructors	✅	❌
May declare physical nested/local types	❌	❌
May use `@co.dap.local`	❌	❌
Instantiation involved	Module is declared once, not constructed	Class objects are constructed
Reference use	Compatible module references may be used through the signature type without creating another module	Interface references may refer to any implementing object instance
OOP dispatch	❌	✅ virtual/dynamic
Contract style	module values, functions, and associated/fixed type components	behavioral methods on object instances
Practical analogy	singleton component contract	object-instance behavioral contract
Origin	ML/OCaml-inspired modules	Java/C#/Go interfaces

- A `signature` is a **module contract** over values, functions, existing package-level types, associated-type requirements, and fixed/manifest type components. A type-component specification is a contract slot, not a physical nested type definition. Multiple modules may match one signature, but each module declaration denotes one module object with shared module state.
- An `interface` is a **behavioral contract** tied to class dispatch and polymorphism. It cannot declare associated or fixed module type components or own nested type definitions. A class implementing an interface may create any number of independent runtime objects.
- The approximation `module + signature ≈ singleton object + interface` is useful for understanding cardinality and shared state, but a module is a language-level component rather than a class-based singleton pattern.

Modules

A module is an ML/OCaml-style abstraction governed by an optional signature. A module may use package-level types, satisfy associated-type requirements declared by its signature, and use fixed/manifest type components established by that signature. It does not physically own or nest arbitrary type declarations. A module should not be introduced merely to prevent functions from appearing loose in a file; use `co.lang.unit` for that simpler structural purpose.

```
// Employee.fol — ordinary package-level type
_ co.lang.struct = {
    Id   co.lang.int;
    Name co.lang.string;
}

// EmployeeModule.fol
_ co.lang.signature = {
    getEmployee(id co.lang.int)->(Employee);
}

// EmployeeModImpl.fol
@co.dap.module(signature=EmployeeModule)
_ co.lang.module->(signature=EmployeeModule, matches=EmployeeModule) = {

    getEmployee(id co.lang.int)->(Employee) = {
        this.return Employee{
            Id: 10,
            Name: "Rao"
        };
    }
}

mm EmployeeModule = EmployeeModImpl;
v Employee = mm.getEmployee(10);
```

Usage and Liveness

Module, signature, interface, typeclass, instance, matcher, data-shape, and annotation/object liveness is defined centrally in Unused Symbols, Liveness, and Reachability.

Module Cardinality and Singleton Analogy

A module declaration defines exactly one named module object for that loaded module identity. It is not an instantiable blueprint and does not create a new module object each time its name is referenced.

```
first  EmployeeModule = EmployeeModImpl;
second EmployeeModule = EmployeeModImpl;
```

`first`, `second`, and `EmployeeModImpl` refer to the same module object. These bindings copy or retain the module reference; they do not clone or instantiate the module. Consequently, values declared directly by the module represent one shared module state for all references to that module.

A signature does not itself create a module object. It is a contract, and any number of separately declared modules may conform to the same signature:

```
DatabaseBackend signature
├── PostgreSQLBackend module   -> one named module object
├── MySQLBackend module        -> one named module object
└── TestDatabaseBackend module -> one named module object
```

Each conforming module declaration contributes its own single module object and its own module state. The signature does not restrict the number of distinct conforming module declarations.

A useful analogy is:

```
signature         ≈ interface contract
conforming module ≈ singleton object implementing that contract
class             ≈ instantiable object type
```

The analogy is intentionally limited. A FoLang module is a compiler-recognized named component, not a class made singleton through a private constructor, static field, or runtime pattern. It cannot be repeatedly constructed. Because module references are first-class in FoLang, they may be bound and used through a compatible signature type, but every reference to the same module declaration still denotes the same module object.

Modules are also broader than ordinary interface implementations. A matching module may provide module values and functions, bind associated types required by its signature, and supply generic associated-type constructors where required. Fixed/manifest type components are established by the signature itself and are used by the module without being rebound. An interface constrains object behaviour; it does not provide the same module type-component abstraction.

By contrast, a class declaration defines an instantiable type. Every class construction creates a distinct object with independent identity, state, and lifetime:

```
PostgreSQLBackend module
└── one shared module object and module state

PostgreSQLConnection class
├── connection1 -> independent object and state
├── connection2 -> independent object and state
└── connection3 -> independent object and state
```

> **Formal mental model:** A FoLang module is a single named implementation component that may conform to a signature. It is comparable to a singleton object implementing an interface, but it is not instantiated from a class. Multiple distinct modules may conform to the same signature, while each module declaration denotes one module object. Unlike an ordinary singleton-interface implementation, a module may also satisfy associated-type requirements, including generic associated-type constructors, and use fixed/manifest type components declared by its signature.

> **Module instantiation** A FoLang class or struct declaration introduces an instantiable type but does not create an instance. A FoLang module declaration introduces one named module component directly into its package. The module name acts as the binding for that component, so no separate construction expression is required. The module's runtime state is initialized once according to the language's module-initialization rules.

> A module declaration is a singleton component declaration and binding, rather than merely an object definition.

Module Signature Contents

A `co.lang.signature` is a declarative contract for a module. It may specify required module values and functions, associated-type requirements, and fixed/manifest type components. A signature does not allocate storage, initialize variables, execute statements, or provide function bodies.

A signature may contain:

- value specifications
- function signatures
- references to already existing accessible types
- associated-type specifications
- fixed or manifest type-component specifications
- generic associated-type-constructor specifications

A signature may not contain:

- value initializers
- executable statements
- function bodies
- concrete class, struct, enum, module, interface, or signature definitions
- arbitrary nested or target-local declarations

Associated and fixed/manifest type-component specifications are part of module conformance. They are contract slots, not Java-, C++-, or C#-style inner types, and they do not participate in `@co.dap.local`.

Value Specifications

A declaration such as:

```
// Counter.fol
_ co.lang.signature = {
    count co.lang.int;
    increment(amount co.lang.int)->();
}
```

requires a matching module to provide a value named `count` of type `co.lang.int` and a compatible `increment` function. The signature does not initialize `count` and does not define `co.lang.int`; the built-in type already exists.

```
// CounterImpl.fol
_ co.lang.module->(
    signature=Counter,
    matches=Counter
) = {
    count co.lang.int = 0;

    increment(amount co.lang.int)->() = {
        count.value = count + amount;
    }
}
```

The same rule applies when a value or function specification uses an existing accessible package type:

```
// EmployeeRepository.fol
_ co.lang.signature = {
    current hr.employee.Employee;
    find(id co.lang.int)->(hr.employee.Employee);
}
```

The matching module must provide `current` and `find`. It does not redefine `hr.employee.Employee`.

Associated Type Components

An `associatedType` component declares that every matching module must supply a concrete type binding for that component. When the signature declaration has no initializer, it is an abstract associated-type requirement:

```
// Repository.fol
_ co.lang.signature = {
    Entity co.lang.associatedType;

    current Entity;
    find(id co.lang.int)->(Entity);
}
```

`Entity co.lang.associatedType;` does not define the representation of `Entity`. It declares an associated-type requirement named `Entity`. Every matching module must bind that requirement to a compatible existing type:

```
// EmployeeRepositoryImpl.fol
_ co.lang.module->(
    signature=Repository,
    matches=Repository
) = {
    Entity co.lang.associatedType = hr.employee.Employee;

    current Entity = ...;
    find(id co.lang.int)->(Entity) = { ... }
}
```

Within a matching module, `Entity co.lang.associatedType = ...` is an **associated-type binding**, not an arbitrary nested type declaration. Its name must correspond to an `associatedType` component declared by the matched signature. A module cannot use this form to introduce unrelated module-local types.

An associated-type requirement differs from `forward` and `extern` declarations:

```
associated type requirement
    -> each matching module supplies its own compatible type binding

forward declaration
    -> one specific declaration is completed later

extern declaration
    -> one specific declaration is defined in another linkage or compilation unit
```

Fixed or Manifest Type Components

A signature may fix a type component to an already known type:

```
// IntegerRepository.fol
_ co.lang.signature = {
    Id co.lang.type = co.lang.int;

    find(id Id)->(co.lang.bool);
}
```

Here `Id` is predetermined as `co.lang.int`. This is a fixed/manifest type component rather than an associated-type requirement: the signature itself establishes the type equality, so a matching module uses `Id` but does not bind or redefine it.

```
Entity co.lang.associatedType;      -> associated; matching module supplies the binding
Id co.lang.type = co.lang.int;      -> fixed/manifest; signature supplies the type equality
```

Abstract Generic Type Constructors

A signature may require a generic type constructor without defining its representation:

```
// StackSignature.fol
_ co.lang.signature = {
    Stack(T) co.lang.associatedType;

    empty(T)->(Stack(T));
    push(value T, stack Stack(T))->(Stack(T));
    pop(stack Stack(T))->(T, Stack(T));
}
```

`Stack(T) co.lang.associatedType;` declares a **generic associated-type component**, also described as an abstract type constructor of arity one. The signature specifies that `Stack` accepts one type argument, but it does not define the concrete type constructor represented by `Stack(T)`.

A matching module must provide a compatible type-constructor binding with the same name, arity, and declared constraints:

```
// ListStackModule.fol
_ co.lang.module->(
    signature=StackSignature,
    matches=StackSignature
) = {
    Stack(T) co.lang.associatedType = co.core.list(T);

    empty(T)->(Stack(T)) = { ... }
    push(value T, stack Stack(T))->(Stack(T)) = { ... }
    pop(stack Stack(T))->(T, Stack(T)) = { ... }
}
```

Another matching module may choose another representation:

```
// ArrayStackModule.fol
_ co.lang.module->(
    signature=StackSignature,
    matches=StackSignature
) = {
    Stack(T) co.lang.associatedType = collections.ArrayStack(T);
    ...
}
```

Therefore:

```
StackSignature
    -> requires a generic type constructor Stack(T)

ListStackModule
    -> binds Stack(T) to co.core.list(T)

ArrayStackModule
    -> binds Stack(T) to collections.ArrayStack(T)
```

An associated-type binding does not permit physical type nesting. When an implementation needs a new concrete struct, class, or enum representation, that declaration remains an ordinary package-owned declaration and may be restricted to the implementing module with `@co.dap.local`; the module then binds the associated-type component to that declaration.

Signature Conformance Rules for Type Components

For every type component in a matched signature:

- each unbound `co.lang.associatedType` component must receive exactly one compatible binding from the matching module
- a fixed/manifest component must retain the type equality declared by the signature and must not be rebound by the module
- a generic associated-type binding must preserve generic arity, parameter kinds, bounds, variance, and other declared constraints
- component names must be unique within the signature
- associated-type bindings cannot contain executable code
- an associated-type binding that does not correspond to a component declared by the matched signature is a compiler error
- types referenced by value and function specifications must resolve after applying the module's associated-type bindings and fixed/manifest type equalities

Module Declaration Relationships

A module cannot physically declare nested structs, enums, classes, modules, signatures, interfaces, or other arbitrary named declarations. It references ordinary package-level declarations through its functions and signature. The only type-like declarations permitted directly in a matching module are `co.lang.associatedType` bindings that satisfy associated-type components declared by its matched signature; such bindings do not create independent nested declarations.

A declaration intended only for one module may be restricted with `@co.dap.local`:

```
// EmployeeModuleConfig.fol
@co.dap.local(for=hr.employee.EmployeeModImpl)
_ co.lang.struct = {
    timeout co.lang.int;
    retries co.lang.int;
}
```

```
// EmployeeModImpl.fol
_ co.lang.module = {
    connect(cfg EmployeeModuleConfig)->(co.lang.bool) = {
        ...
    }
}
```

The following remains invalid:

```
// EmployeeModImpl.fol
_ co.lang.module = {
    Config co.lang.struct = { // ❌ physical nested declaration
        timeout co.lang.int;
    }
}
```

A target-local declaration does not automatically become a module member name and is not projected through the module's signature. It becomes part of the signature view only when an associated-type component is explicitly bound to it or when a signature value/function specification references it through an allowed type component.

Structs vs Classes vs Modules vs Units vs Packages

	Struct	CStruct	Class	Module	Unit	Package
Purpose	Pure data shape	C-like value type	Behaviour + data	Signature-backed ML-style abstraction	Stateless package-fragment or struct-companion container	Folder-based grouping
Fields	✅	✅ simple only	✅ per instance	❌	❌	❌
Module-level values	❌	❌	❌	✅ when declared directly or required by a signature	❌	❌
Functions / methods	Companion functions through `<StructName>.comp.unit.fol`; explicit receivers must match the struct	❌	✅ methods	✅ module functions	✅ package functions in ordinary units; companion functions in companion units	❌

Lifecycle (`@@new` / `@@init`)	❌	❌	✅	❌	❌	❌
`this` / `self`	❌	❌	✅	❌	❌	❌
Value/literal construction	✅	✅	❌	❌	❌	❌
Lifecycle instantiation (`new` / `init`)	❌	❌	✅ multiple objects	❌ — one module object per declaration	❌	❌
Runtime state cardinality	Per bound struct object	Per value	Per class object	One shared state for the module declaration	—	—
First class	✅	✅	✅	✅ module reference; referencing does not instantiate	❌	❌
Pass by	Reference	Value	Reference	Reference to the same module object	—	—
Contract	—	—	`interface` via `implements=[]`	`signature` via `matches=`	none	—
OOP / inheritance	❌	❌	✅	❌	❌	❌
Physically nested independent named type/container declarations	❌	❌	❌	❌	❌	N/A — packages contain separate source declarations
May be an `@co.dap.local` target	✅	❌	✅	✅	❌	❌
Pattern matching	✅	✅	✅	❌	❌	❌
Direct ABI / zone boundary safe	❌ — library boundaries require snapshots	✅	❌	❌	❌	❌
Associated functions	✅ through `<StructName>.comp.unit.fol`	❌	—	—	✅ only in a struct companion unit	❌
Embedding	✅	❌	—	—	❌	❌
Declared with	`co.lang.struct`	`co.lang.cstruct`	`co.lang.class`	`co.lang.module`	`co.lang.unit`	folder path
C++ backend analogy	struct without methods	plain C struct	class	struct/class abstraction	package namespace fragment or static companion scope	namespace
Closest mental model	Rust struct	C struct	Java/C# class	singleton implementation component with ML-style type members	source fragment merged into a package, or a filename-bound struct companion	filesystem namespace

Mental model:

```
reach for struct   → pure data; use `<StructName>.comp.unit.fol` for associated behaviour
reach for cstruct  → physical ABI-compatible value data crossing direct zone or native boundaries
reach for class    → behaviour, lifecycle, multiple instances
reach for module   → one named implementation component with shared state, governed by an optional signature and capable of satisfying
associated-type requirements
reach for unit     → package fragment (`*.unit.fol`) or struct companion (`*.comp.unit.fol`)
reach for package  → folder-based grouping only, not a value
```

> **Declaration scoping rule:** FoLang does not permit physical nesting of independent file-backed primary declarations. Classes, structs, cstructs, enums, unions, modules, interfaces, signatures, instances, matchers, and other package-owned primary declarations remain in their own `<Name>.fol` files. Ordinary and companion unit files are explicit package containers: they may contain functions and the non-UDT type declarations permitted by the unit rules, but they may not contain independent primary declarations such as classes, structs, enums, modules, interfaces, or signatures. Ordinary local functions and anonymous expressions remain the other explicit nesting exceptions. Supported package-owned declarations may restrict visibility to exact same-package targets with `@co.dap.local`; the annotation changes visibility, not physical ownership. Signature type components and matching-module `co.lang.associatedType` bindings are contract slots rather than arbitrary nested package declarations. ***

Local and/or Nested types and functions

FoLang does not provide Java-, C++-, or C#-style physical nesting of independent named type and container declarations. Such declarations remain in their ordinary legal source locations. Ordinary local functions and anonymous expressions are explicit exceptions governed by the rules below.

Physical Nesting Rules

Prohibited Independent Named Declarations

Independent file-backed primary declarations cannot be physically declared inside another class, struct, cstruct, enum, union, module, unit, interface, signature, function, or executable block. This includes:

- classes, structs, cstructs, enums, and unions;
- modules, interfaces, signatures, and additional units;
- instances, matchers and other file-backed primary declarations.

Non-UDT type declarations are the deliberate unit exception. Type aliases, `co.lang.type` ADTs and type constructors, newtypes, opaque types, refinement types, subtypes, and supertypes may be declared directly inside an ordinary unit, and inside a companion unit where their own rules permit association with the owner. Macros, templates, and decorators follow their own declaration rules. These declarations are not permitted loose at package-file scope or physically inside classes, structs, modules, functions, or executable blocks unless another section explicitly grants that context.

File-backed primary declarations retain package-owned identity and follow their normal `<Name>.fol` placement rules. An association or visibility annotation such as `@co.dap.local`, `@co.dap.nested`, or `@co.dap.inner` does not physically move a separately declared declaration inside its target.

Local-Function Exception

A function body may contain an ordinary named local or inner function where the grammar permits a local-function declaration:

```
outer()->() = {
    value co.lang.int = 10;

    inner()->() = {
        co.out.println(value);
    }

    inner();
}
```

The local function has block-local declaration identity. Its free runtime names are resolved from its lexical declaration context, and its lifetime and escape behavior follow the ordinary inner-function rules. It is not a package member and cannot be independently imported or exported.

This exception permits local functions only. It does not permit a named class, struct, enum, module, unit, interface, signature, or another named type/container declaration inside a function body.

Anonymous-Expression Exception

The physical-nesting restriction does not apply to anonymous constructs that are expressions or type expressions rather than independent named declarations. They may be nested wherever their specific grammar category is permitted.

These include:

- anonymous function expressions;
- lambdas and callback blocks;
- anonymous class or anonymous type expressions;
- permitted anonymous polymorphic type expressions introduced by `forall`;
- ordinary nested block, object-construction, map, collection, and other value-producing expressions.

```
process()->() = {
    operation := (value co.lang.int)->(co.lang.int) {
        this.return value * 2;
    };

    worker := co.lang.class {
        run(value co.lang.int)->(co.lang.int)={
            this.return operation(value);
        }
    }.init();
}

transformer co.lang.type = forall(T).(T)->(T);
```

An anonymous construct has no independently addressable package-owned declaration identity. Its scope, capture, lifetime, type, and escape behavior are determined by the rules for that specific construct. Syntactic containment of an anonymous expression does not create a Java-, C++-, or C#-style named nested declaration and does not violate the one-primary-declaration-per-package-file rule.

Relationship to Association Annotations

The exceptions above are distinct from FoLang's association annotations:

- an ordinary local function is physically declared inside an enclosing function and is lexically scoped;
- an anonymous construct is an expression without independent declaration identity;
- `@co.dap.local`, `@co.dap.nested`, and `@co.dap.inner` apply to separately declared declarations that retain their normal source identity.

The remainder of this section defines those separately declared association forms.

```
@co.dap.local(for=<declaration-reference>)
```

or:

```
@co.dap.local(
    for=[
        <declaration-reference>,
        <declaration-reference>
    ]
)
```

The annotated declaration is called a **target-local declaration**. The declarations named by `for` form its **local target set**.

Supported Declaration and Target Kinds

`@co.dap.local` may be applied only to these declaration kinds:

- `co.lang.class`
- `co.lang.struct`
- `co.lang.enum`
- `co.lang.module`
- a named function declared in a context that normally permits functions

Every entry in the local target set must resolve to exactly one:

- class
- struct
- enum
- module
- function overload

A target list may contain any combination of supported target kinds.

Signatures and interfaces do not support target-local or physically nested declarations. A `co.lang.signature` or `co.lang.interface` cannot be annotated with `@co.dap.local` and cannot appear in a local target set. A signature may declare abstract, fixed, or generic **type-component specifications** as part of its module contract; these are contract requirements rather than local or nested type definitions. Interfaces do not declare signature type components.

Other declaration kinds are not target-local unless a later section explicitly permits them.

Target-Set Syntax

The `for` argument accepts either:

1. one declaration reference; or
2. a non-empty list of declaration references.

The scalar form is equivalent to a singleton target list:

```
@co.dap.local(for=hr.employee.Employee)
```

```
@co.dap.local(for=[hr.employee.Employee])
```

The scalar form is canonical when only one target is required. Use the list form when two or more declarations require access:

```
@co.dap.local(
    for=[
        hr.employee.Employee,
        hr.employee.EmployeeService,
        hr.employee.EmployeeValidator
    ]
)
// EmployeeState.fol
_ co.lang.enum = {
    Active,
    Inactive
}
```

Rules:

- the target list must contain at least one declaration reference
- each reference must resolve independently and unambiguously
- duplicate references to the same resolved declaration are a compiler error
- list order does not affect visibility or declaration identity
- the target list is a closed set; access is not granted to declarations omitted from it
- repeated `@co.dap.local` annotations are not used to accumulate targets; use one annotation with one complete target list

Invalid:

```
// State.fol
@co.dap.local(for=[])
_ co.lang.struct = { ... }
// ❌ empty target list
```

```
@co.dap.local(
    for=[
        hr.employee.Employee,
        hr.employee.Employee
    ]
)
// State.fol
_ co.lang.struct = { ... }
// ❌ duplicate resolved target
```

Declaration-Reference Syntax

Each `for` entry is a compiler-resolved declaration reference, not a string, runtime expression, function call, or `co.meta.symbol` value.

For a non-function target, use only its complete qualified declaration name:

```
@co.dap.local(for=hr.employee.Employee)
@co.dap.local(for=hr.employee.EmployeeStatus)
@co.dap.local(for=hr.employee.EmployeeRules)
```

For a function target, use its complete qualified callable reference because FoLang permits overloads. The callable identity and canonical parameter signature select the overload; the written return tail records and validates the selected declaration's return contract but does not participate in overload selection:

```
@co.dap.local(
    for=hr.employee.Employee.calculate(co.lang.float)->()
)
```

Function references in a target list follow the same rule:

```
@co.dap.local(
    for=[
        hr.employee.Employee.calculate(co.lang.float)->(),
        hr.employee.Employee.calculate(co.lang.int)->(),
        hr.employee.Employee.validate()->(co.lang.bool)
    ]
)
// CalculationState.fol
_ co.lang.struct = { ... }
```

Parameter names are not part of the reference:

```
// ✅ canonical
hr.employee.Employee.calculate(co.lang.float)->()

// ❌ parameter names are not declaration identity
hr.employee.Employee.calculate(amount co.lang.float)->()
```

The qualified callable identity and parameter types must select exactly one overload. The return tail must then match that already-selected overload's declared return signature; it is validation metadata, not an overload discriminator. An abbreviated function name, unresolved target, parameter-ambiguous overload, or mismatching return tail is a compiler error.

```
@co.dap.local(for=hr.employee.Employee.calculate)
// ❌ ambiguous when calculate is overloaded
```

Each listed overload is an independent target. Listing one overload does not grant access to sibling overloads with the same function name.

Same-Package Requirement

The target-local declaration and **every declaration in its local target set** must belong to the same exact package. The compiler compares their complete folder-derived package identities; matching only a parent package, package family, import alias, library, or source root is not sufficient.

For a function target, the target package is the package that owns the function's enclosing class, struct companion unit, module, unit, or other legal function container. A target-local function is checked in the same way: the package containing its legal function-owning declaration must match the package of every listed target.

The invariant is:

```
package(target-local declaration)
    == package(target 1)
    == package(target 2)
    == ...
```

```
// hr/employee/Employee.fol
_ co.lang.class = { ... }

// hr/employee/EmployeeService.fol
_ co.lang.class = { ... }

// hr/employee/EmployeeState.fol
@co.dap.local(
    for=[
        hr.employee.Employee,
        hr.employee.EmployeeService
    ]
)
// EmployeeState.fol
_ co.lang.enum = { Active, Inactive }
// ✅ all declarations belong to package hr.employee
```

A declaration in another package cannot participate in the local target set, even when ordinary package visibility, imports, aliases, parent/subpackage relationships, or library membership would otherwise make the declaration resolvable.

```
// EmployeeState is declared in package hr.internal
@co.dap.local(
    for=[
        hr.employee.Employee,
        hr.internal.EmployeeService
    ]
)
// EmployeeState.fol
_ co.lang.enum = { Active, Inactive }
// ❌ local declaration and every target must have the same package
```

```
// CalculationState is declared in package hr.employee.internal
@co.dap.local(
    for=hr.employee.Employee.calculate(co.lang.float)->()
)
// CalculationState.fol
_ co.lang.struct = { ... }
// ❌ subpackages are distinct packages; both sides must be exactly hr.employee
```

The same-package rule prevents `@co.dap.local` from becoming a cross-package friendship or visibility-bypass mechanism. A local declaration is a selectively shared implementation detail inside one package, not an imported helper attached from elsewhere.

Visibility Domain

A target-local declaration may be resolved only from:

1. each exact declaration in its local target set; and
2. lexical scopes contained within each listed target's implementation.

For targets `A`, `B`, and `C`, the visibility domain is the union of their implementation scopes:

```
visibility(local declaration)
    = scope(A) ∪ scope(B) ∪ scope(C)
```

It is not an intersection, and code does not need to be simultaneously inside every target.

Consequences:

- a declaration local to a class is available to that class body and its methods
- a declaration local to a module is available to that module body and its functions
- a declaration local to a struct is available while resolving that struct declaration
- a declaration local to an enum is available while resolving that enum declaration
- a declaration local to a function is available only in the body of the exact targeted overload and its nested statement scopes
- when several targets are listed, each listed target receives access independently
- sibling declarations and sibling overloads not listed cannot resolve it
- subclasses, companion units, extensions, callees, callers, and related declarations do not gain access automatically
- visibility is not inherited, transitive, or propagated through calls, composition, embedding, imports, or other local declarations
- it cannot be imported, exported, or projected through a library surface

```
@co.dap.local(
    for=[
        hr.employee.Employee,
        hr.employee.EmployeeService
    ]
)
// EmployeeState.fol
_ co.lang.enum = {
    Active,
    Inactive
}

// Employee.fol
_ co.lang.class = {
    state EmployeeState; // ✅ listed target
}

// EmployeeService.fol
_ co.lang.class = {
    state EmployeeState; // ✅ listed target
}

// Payroll.fol
_ co.lang.class = {
    state EmployeeState; // ❌ not listed
}
```

Interaction with Ordinary Visibility

`@co.dap.local` establishes the final visibility boundary. Ordinary visibility annotations such as `@co.dap.public`, `@co.dap.package`, `@co.dap.protected`, or `@co.dap.private` cannot widen the declaration beyond its closed local target set.

```
@co.dap.public
@co.dap.local(
    for=[
        hr.employee.Employee,
        hr.employee.EmployeeService
    ]
)
// EmployeeState.fol
_ co.lang.enum = { Active, Inactive }
```

`EmployeeState` is still visible only to the listed targets. The ordinary visibility annotation is redundant for external resolution and may produce a compiler warning, but it never overrides `@co.dap.local`.

When most or all declarations in a package require access, ordinary package visibility should be used instead of maintaining a large target list.

Source Placement and Identity

A target-local declaration remains in the source position normally required for its declaration kind:

- a class, struct, enum, or module remains a normal primary declaration and follows the one-primary-declaration-per-package-file rule
- a function remains inside a unit, class, module, or another declaration context that normally permits functions
- `@co.dap.local` does not permit a free function to appear loose at package-file scope

The declaration retains a stable package-owned compiler identity. Its package identity must be exactly the same as the package identity of every target. It does not become physically nested and is not addressed as `Target.LocalName`.

```
hr.employee.EmployeeState
    local-for [
        hr.employee.Employee,
        hr.employee.EmployeeService
    ]
```

The normalized local target set is declaration metadata. Target-list order does not create a different declaration identity.

No Implicit Relationship or Privilege

`@co.dap.local` changes visibility only. It does not imply:

- composition
- embedding
- inheritance
- lifecycle ownership
- memory ownership
- friendship or privileged private-member access
- automatic membership in any listed target
- shared runtime state among the targets

Composition must still be written explicitly through a field or another supported relation. Embedding remains a separate facility: only struct and enum declarations are eligible for embedding, and each use must follow the embedding rules defined for that declaration kind.

Escape Restrictions

A target-local type must not leak outside its complete visibility domain.

It cannot appear in:

- a public, package, or protected API visible outside the local target set
- a library surface
- the parameter or return types of a function to which it is local
- a field or method signature that exposes it beyond the local target set
- an exported generic specialization or type alias

A value whose static type is target-local must be converted to an externally visible type before leaving the union of the listed targets' visibility domains.

Usage

```
@co.dap.public
@co.dap.local(
    for=[
        hr.employee.Employee,
        hr.employee.EmployeeService
    ]
)
// EmployeeState.fol
_ co.lang.enum = { Active, Inactive }

// Employee.fol
_ co.lang.struct={

    state EmployeeState;
}
```

Invalid Physical Nesting

```
// Employee.fol
_ co.lang.class = {
    Address co.lang.struct = { ... } // ❌
}

// EmployeeModule.fol
_ co.lang.module = {
    Config co.lang.struct = { ... } // ❌
}

process()->() = {
    State co.lang.enum = { Ready, Done } // ❌ named type declaration
}

// EmployeeContract.fol
_ co.lang.signature = {
    Employee co.lang.struct; // ❌
}

// EmployeeApi.fol
_ co.lang.interface = {
    Result co.lang.struct; // ❌
}
```

The `process` example rejects the named enum declaration; it does not reject ordinary local functions or anonymous expressions. Those are permitted only under the explicit exceptions in Physical Nesting Rules.

Use separately declared package-owned declarations, composition, embedding where allowed, and `@co.dap.local` when a closed set of declarations requires selective access.

There is another annotation `@co.dap.nested` which is similar to local but captures target state.

@co.dap.nested(target=hr.emp.Employee)

Instead of `for` we use `target` and `target` is always a single fully qualified type/function.

`@co.dap.nested` provides nested/inner-style target-state capture while the declaration itself remains physically separate, as defined above.

Comparison Table

Annotation	attribute	multiple targets	capture the target state
@co.dap.local	for	✅ as list for single target can mention without list syntax	❌
@co.dap.nested	target	❌	✅

`@co.dap.inner` Declarations

`@co.dap.inner` is an association annotation. It does **not** create a physically nested type, method, or function. The annotated declaration remains in its ordinary legal source location, retains its package-owned identity, and must satisfy the same exact-package rule that applies to `@co.dap.local` and `@co.dap.nested`.

Unlike `@co.dap.local` and `@co.dap.nested`, `@co.dap.inner` does not contain a `for` or `target` attribute. Its target relationship is established at the use site by explicitly embedding or attaching the declaration to a legal target. An `@co.dap.inner` declaration cannot be used as a standalone declaration.

Usage

```
@co.dap.public
// EmployeeState.fol
@co.dap.inner
_ co.lang.enum = {
    Active,
    Inactive
}

// Employee.fol
_ co.lang.struct = {
    EmployeeState;
    state EmployeeState;
}
```

The use of `EmployeeState;` establishes the inner association for this target. The annotation itself does not name the target.

Scope of `@co.dap.inner` Executable Declarations

For an `@co.dap.inner` function or method, parameters and local declarations resolve normally. A free runtime name is resolved from the **lexical context of the active call or attachment site**, not from the declaration site of the separately declared `@co.dap.inner` function.

The lookup order is:

```
1. parameters and local declarations of the @co.dap.inner function or method
2. the innermost lexical scope at the active call or attachment site
3. enclosing lexical scopes of that call or attachment site
4. statically resolved types, annotations, imports, and built-in co.* names
5. compiler error
```

This model is called **call-site lexical-context resolution**.

It is distinct from an ordinary inner function. An ordinary inner function is physically declared inside another function and resolves free runtime names from its enclosing lexical **declaration** context. An `@co.dap.inner` function is declared separately and therefore obtains its runtime context from the lexical scope in which it is actively attached or called.

It is also distinct from `@co.dap.dynamicscope`. Call-site lexical-context resolution does not search arbitrary runtime caller frames beyond the lexical scope chain of the active call site, and it does not use mixed-scope fallback. At every statically known use or call site, the compiler validates that each required runtime binding exists, is definitely initialized, has a compatible type, and permits the requested access or mutation.

For `@co.dap.inner` types and other non-executable declarations, ordinary static name and type resolution continues to apply. Call-site lexical-context resolution concerns only free runtime names used by executable function or method bodies.

An implementation may lower an `@co.dap.inner` declaration by copying, inlining, specialization, or another equivalent mechanism. Such lowering is not observable language semantics; the association and scope rules above are normative.

Exception

> `@co.dap.inner`, `@co.dap.local`, and `@co.dap.nested` cannot be used with `co.lang.cstruct`.

Statements

A statement is a complete executable or declarative instruction. It may contain one or more expressions and may change program state, control execution, introduce declarations, or produce observable effects.

Common statement categories in `Folang`

1. Declaration Statement
2. Initialization Statement
3. Expression Statement
4. Conditional Statement
5. Loop Statement etc,.

Expressions

An expression is a construct that evaluates to a value, produces an effect, or both, and may be contained within a larger expression or statement.

Expression Evaluation Order

FoLang defines a deterministic evaluation order for expressions.

Expression structure is determined first by:

1. explicit grouping with parentheses;
2. operator precedence;
3. operator associativity.

After the expression structure has been determined, operands and subexpressions are evaluated from left to right in their source-code order, except where a construct explicitly defines conditional, lazy, asynchronous, concurrent, or otherwise specialized evaluation behavior.

Ordinary Expressions

For an ordinary expression:

```
result = first() + second();
```

the evaluation order is:

1. evaluate `first()`;
2. evaluate `second()`;
3. apply the `+` operator;
4. assign the resulting value to `result`.

Precedence and Evaluation Order

Left-to-right evaluation does not override operator precedence.

For example:

```
result = first() + second() * third();
```

operator precedence determines the expression structure as:

```
result = first() + (second() * third());
```

The function calls are evaluated in source order:

1. evaluate `first()`;
2. evaluate `second()`;
3. evaluate `third()`;
4. multiply the results of `second()` and `third()`;
5. add the multiplication result to the result of `first()`;
6. assign the final result to `result`.

Therefore, precedence determines how values are combined, while evaluation order determines when each operand or subexpression is evaluated.

Function Calls

For a function call, the callable expression is evaluated first, followed by the arguments from left to right.

```
result = calculate(first(), second(), third());
```

The evaluation order is:

1. evaluate the callable expression `calculate`;
2. evaluate `first()`;
3. evaluate `second()`;
4. evaluate `third()`;
5. invoke `calculate` with the resulting argument values;
6. assign the returned value to `result`.

Name resolution performed at compile time does not constitute runtime evaluation.

Method Calls

For a method call, the receiver expression is evaluated first, followed by the arguments from left to right.

```
result = getEmployee().calculate(first(), second());
```

The evaluation order is:

1. evaluate `getEmployee()`;
2. resolve the resulting object as the method receiver;
3. evaluate `first()`;
4. evaluate `second()`;
5. invoke `calculate`;
6. assign the returned value to `result`.

The receiver expression must be evaluated exactly once.

Member Access

For member access, the expression producing the containing object is evaluated before the member is accessed.

```
value = getEmployee().address.city;
```

The evaluation order is:

1. evaluate `getEmployee()`;
2. access `address` on the resulting object;
3. access `city` on the resulting address;
4. assign the resulting value to `value`.

Each intermediate receiver is evaluated only once.

Indexing

For an indexing expression, the indexed object is evaluated first, followed by index expressions from left to right.

```
value = getMatrix()[row()][column()];
```

The evaluation order is:

1. evaluate `getMatrix()`;
2. evaluate `row()`;
3. perform the first indexing operation;
4. evaluate `column()`;
5. perform the second indexing operation;
6. assign the resulting value to `value`.

Assignment

For assignment, FoLang evaluates the assignment target first, then evaluates the right-hand-side expression, and finally performs the assignment.

```
array[index()] = calculate();
```

The evaluation order is:

1. evaluate `array`;
2. evaluate `index()`;
3. determine the destination location;
4. evaluate `calculate()`;
5. assign the resulting value to the destination.

Determining the assignment target does not read the previous value stored at that location unless the assignment operation explicitly requires it.

Simple Variable Assignment

For assignment to a simple variable:

```
result = first() + second();
```

the evaluation order is:

1. determine the binding represented by `result`;
2. evaluate `first()`;
3. evaluate `second()`;
4. apply the `+` operator;
5. assign the resulting value to `result`.

Compound Assignment

A compound assignment evaluates its target exactly once.

```
array[index()] += calculate();
```

The evaluation order is:

1. evaluate `array`;
2. evaluate `index()`;
3. determine the destination location;
4. read the current value from that location;
5. evaluate `calculate()`;
6. apply the `+` operator;
7. assign the resulting value to the same destination.

The expression above must not behave as though it were expanded into a form that evaluates `array` or `index()` more than once.

Multiple Assignment

When an assignment contains multiple right-hand-side expressions, those expressions are evaluated completely from left to right before values are assigned to their targets.

```
a, b = first(), second();
```

The evaluation order is:

1. determine the target for `a`;
2. determine the target for `b`;
3. evaluate `first()`;
4. evaluate `second()`;
5. assign the first resulting value to `a`;
6. assign the second resulting value to `b`.

Right-hand-side evaluation must complete before any target receives its new value.

This permits value exchange without an explicit temporary variable:

```
a, b = b, a;
```

Operator Expressions

Built-in and user-defined operators follow the same operand-evaluation rules.

For a binary operator:

```
left() + right();
```

FoLang evaluates `left()` before `right()`.

For a prefix unary operator:

```
-operand();
```

FoLang evaluates `operand()` before applying the operator.

For a postfix unary operator:

```
operand()!;
```

FoLang evaluates `operand()` before applying the operator.

Operator overloading must not change the specified evaluation order of operands.

Short-Circuit Boolean Operations

Short-circuit Boolean operators evaluate the left operand first.

The right operand is evaluated only when it is required to determine the result.

For logical AND:

```
left() && right();
```

the evaluation order is:

1. evaluate `left()`;
2. when the result is false, return false without evaluating `right()`;
3. otherwise, evaluate `right()` and use its Boolean result.

For logical OR:

```
left() || right();
```

the evaluation order is:

1. evaluate `left()`;
2. when the result is true, return true without evaluating `right()`;
3. otherwise, evaluate `right()` and use its Boolean result.

A skipped operand produces no value, mutation, side effect, or error.

Conditional Expressions

A conditional expression evaluates its condition first and then evaluates exactly one selected result expression.

```
result = condition()
    .then(whenTrue())
    .default(whenFalse());
```

The evaluation order is:

1. evaluate `condition()`;
2. when the condition is true, evaluate `whenTrue()` only;
3. otherwise, evaluate `whenFalse()` only;
4. assign the selected result to `result`.

The unselected expression must not be evaluated.

Conditions and Branches

For a condition-and-branch construct, conditions are evaluated sequentially from left to right.

```
firstCondition().then({
    firstBranch();
}).otherwise(secondCondition()).then({
    secondBranch();
}).default({
    finalBranch();
});
```

The evaluation order is:

1. evaluate `firstCondition()`;
2. when true, execute `firstBranch()` and skip the remaining conditions and branches;
3. otherwise, evaluate `secondCondition()`;
4. when true, execute `secondBranch()` and skip the final branch;
5. otherwise, execute `finalBranch()`.

Only the selected branch is evaluated. `otherwise(condition)` is considered only after every earlier condition fails. If all conditional branches fail, `default(...)`, when present, supplies the terminal branch.

Pattern Matching

A pattern-matching subject is evaluated exactly once.

Cases are examined in source order unless a particular matcher explicitly defines another ordering rule.

```
getValue().match
    .case(firstPattern => firstResult())
    .case(secondPattern => secondResult())
    .default(defaultResult());
```

The evaluation order is:

1. evaluate `getValue()` exactly once;
2. test `firstPattern`;
3. when it matches, evaluate `firstResult()` and stop;
4. otherwise, test `secondPattern`;
5. when it matches, evaluate `secondResult()` and stop;
6. otherwise, evaluate `defaultResult()`.

Results belonging to unmatched cases are not evaluated.

Pattern guards are evaluated only after their corresponding structural pattern has matched.

Collection Literals

Elements of an array, list, tuple, set, map, or other collection literal are evaluated from left to right as they appear in the source.

```
values = [first(), second(), third()];
```

The evaluation order is:

1. evaluate `first()`;
2. evaluate `second()`;
3. evaluate `third()`;
4. construct the collection from the resulting values.

For a map entry, the key expression is evaluated before its corresponding value expression.

```
values = {
    firstKey(): firstValue(),
    secondKey(): secondValue()
};
```

The evaluation order is:

1. evaluate `firstKey()`;
2. evaluate `firstValue()`;
3. evaluate `secondKey()`;
4. evaluate `secondValue()`;
5. construct the map.

Range Expressions

For a bounded range, the lower-bound expression is evaluated before the upper-bound expression.

```
range = lower() .. upper();
```

The evaluation order is:

1. evaluate `lower()`;
2. evaluate `upper()`;
3. construct the range.

An omitted bound is not evaluated and does not produce an implicit function call or side effect.

Comprehensions and Iteration

Comprehension binding, result-shape, and source-specific behaviour are defined in Comprehension Semantics. This section defines only their interaction with the general evaluation-order rules.

Within each iteration:

- source expressions are evaluated in the order required by the applicable comprehension or iteration form;
- when an explicitly defined filter is present, its condition is evaluated before the corresponding result expression;
- a result expression is evaluated only for a source value that participates according to the source's defined semantics and any explicit filter;
- individual operand evaluation remains left to right.

The core `for (pattern <- source).yield(...)` form does not itself introduce an inline filter. The language does not implicitly evaluate comprehension iterations concurrently unless concurrency is explicitly requested or the selected source semantics explicitly define another execution model.

Lazy Expressions

An expression declared lazy is not evaluated when the lazy binding is created.

Its evaluation occurs only when demanded according to the rules of the corresponding lazy construct.

Once evaluation begins, the expression itself follows the normal FoLang evaluation-order rules unless the lazy construct specifies otherwise.

The specification for each lazy construct must also state whether its result is:

- evaluated once and cached;
- evaluated again for every demand;
- safe for concurrent demand;
- permitted to propagate errors more than once.

Asynchronous and Concurrent Expressions

Left-to-right evaluation determines the order in which asynchronous or concurrent operations are created, invoked, or submitted.

It does not necessarily determine the order in which independently executing operations complete.

```
submit(first());
submit(second());
```

FoLang evaluates and submits `first()` before `second()`. However, completion order depends on the execution-model rules unless explicit ordering is requested.

Concurrent execution never arises implicitly from an ordinary expression. It must be introduced by an explicit FoLang concurrency, parallelism, asynchronous-execution, scheduling, or execution-model construct.

Errors During Evaluation

When evaluation of a subexpression produces an error that prevents further evaluation, later subexpressions are not evaluated.

```
result = first() + failing() + third();
```

When `failing()` terminates evaluation with an error:

1. `first()` has already been evaluated;
2. `failing()` produces the error;
3. `third()` is not evaluated;
4. no final value is assigned to `result`.

The applicable error-handling rules determine whether the error is propagated, matched, converted, recovered from, or terminates execution.

Side Effects

All observable side effects occur according to the defined evaluation order.

Observable effects include:

- object mutation;
- variable rebinding;
- input and output;
- file-system operations;
- network operations;
- synchronization;
- exception or error generation;
- interaction with foreign code;
- interaction with the runtime environment.

An implementation must not reorder operations when doing so could change any observable effect.

Compiler Optimizations

A compiler may optimize, combine, eliminate, or internally reorder operations only when the transformation preserves all behavior required by this specification.

In particular, an optimization must not change:

- the resulting value;
- the order or presence of observable side effects;
- the identity or aliasing behavior of objects;
- the timing or presence of specified errors;
- the selected conditional or pattern-matching branch;
- the number of times an effectful expression is evaluated;
- synchronization or concurrency guarantees.

Pure internal operations may be reordered when no conforming FoLang program can observe the difference.

Implementation Requirement

Every conforming FoLang implementation must preserve the evaluation order defined in this section.

An implementation may use any parser, intermediate representation, optimizer, runtime, or backend, but those implementation choices must not alter the externally observable behavior required by these rules.

Operators

FoLang distinguishes a **symbolic spelling** from an **operator**. A sequence of symbol characters is not automatically an operator merely because it resembles one. Its syntactic role is determined by the grammar context in which the whole spelling occurs.

For example:

```
a co.lang.int->(**);
```

In this declaration, `->` is the structural type-derivation marker and `**` is pointer-degree metadata inside the derivation. Neither occurrence is parsed as an expression operator. The same `**` spelling in `left ** right` is parsed as the registered power operator because it occurs in an operator-expression position.

FoLang uses one uniform implementation model for every expression operator. The difference between built-in, pre-declared, and project-local custom operators is only who registers the symbol and its parse properties.

```
built-in operator
    symbol and parse properties registered by the language
    one or more implementations may already exist

pre-declared operator glyph
    symbol and parse properties registered by the language
    no implementation is required to exist

project-local custom operator
    symbol and parse properties registered once in components/operators/component.fol
    no implementation is contained in the operator declaration

all three categories
    implementations are ordinary mode=overload operator functions
    duplicate normalized operand signatures are errors
```

Symbolic Runs, Classification, and Boundaries

After comments, literals, and closed scanner-known composite spellings such as `@@new` and `@@init` are recognized, the lexer consumes each remaining complete contiguous run of symbol characters as one symbolic token candidate. It does not backtrack or split an unrecognized run into shorter valid tokens. Whitespace, comments, and delimiters such as `(`, `)`, `[`, `]`, `{`, and `}` end a symbolic run. Comment openers are recognized before ordinary symbolic-run scanning.

The complete run is then classified by its grammar context as one of the following:

1. a fixed structural or declaration spelling such as `->`, `=>`, `:=`, or `?=`;
2. a contextual metadata spelling, such as a contiguous run of one or more `*` characters inside `->(...)` to express pointer degree;
3. a registered expression operator; or
4. an unrecognized symbolic token, which is a compilation error.

There is no fallback splitting. Therefore, when `++` is not registered:

```
++ a;       // error: unrecognized symbolic token `++`
a ++ b;     // error: unrecognized symbolic token `++`
a++b;       // error: the contiguous run is still the single candidate `++`

+ +a;       // valid: two `+` operators separated by whitespace
+(+a);      // valid: `(` creates the token and operand boundary
a + +b;    // valid: binary `+` followed by unary `+`
```

A registered expression operator whose spelling contains more than one symbol character must have an explicit boundary on every operand-facing side. A boundary may be whitespace, a comment, or an applicable delimiter. Boundary presence is checked from the original source before whitespace and comments are discarded. The rule is based on the operator's fixity:

- an infix multi-symbol operator requires a boundary before and after it;
- a prefix multi-symbol operator requires a boundary after it;
- a postfix multi-symbol operator requires a boundary before it.

Suppose `+-` is registered as an infix operator:

```
a +- b;       // valid
(a)+-(b);     // valid: the parentheses provide both operand boundaries
a+-b;         // error: missing boundaries
a +-b;        // error: missing the right boundary
a+- b;        // error: missing the left boundary
a + -b;       // valid: separate `+` and unary `-` operators
a + (-b);     // valid: the parenthesis separates the operators
```

This boundary requirement applies uniformly to built-in, pre-declared, and custom multi-symbol expression operators. It does not apply merely because a structural token or metadata spelling contains multiple symbols. Thus `co.lang.int->(**)` remains valid without spaces around `->` or inside the pointer metadata.

The statement-level definition spellings `:=` and `?=` are not expression operators, but they deliberately use the analogous two-sided boundary rule. Write `name := value` and `name ?= value`; compact forms such as `name:=value` and `name?=value` are invalid.

Operator `mode=override` and `mode=extends` are unsupported. Ordinary class method overriding through `@co.dap.override` remains a separate class feature.

Operator Implementations

An operator implementation is a normal function with operator metadata. It cannot be declared loose at package scope. Every implementation must be in one of these legal function-owning locations:

Operand owner	Required implementation location
built-in type	an `@co.dap.extension` function inside a unit
`co.lang.struct`	the struct's same-package companion unit
`co.lang.class`	an operator method declared by the class
module, enum, union, interface, signature, `co.lang.cstruct`	unsupported

An implementation uses `mode=overload`, or omits `mode` because omission is shorthand for `mode=overload`. A one-character symbol may use a character literal; a multi-character symbol must use a string literal:

```
@co.dap.operator(symbol='+', mode=overload)
add(left Employee, right Employee)->(Employee) = {
    ...
}

@co.dap.operator(symbol="+-", mode=overload)
combine(left Employee, right Employee)->(Employee) = {
    ...
}
```

FoLang does not attempt to prove that an overload follows the conventional meaning of its symbol. A developer may overload `+` with any implementation. Using one symbol consistently for one recognizable concept is recommended for readability, but it is not a compiler-enforced semantic restriction.

For a receiverless struct-companion operator function, the first declared operand must have the matching struct type. A matching struct type in a later operand is insufficient. For a unary operator, the sole operand must have the matching struct type.

A matching instance receiver contributes the first operand. A matching type receiver establishes ownership but contributes no operand; its ordinary parameter list is the complete operator signature. In a class, an ordinary or `@co.dap.instance` operator method has an implicit `this` first operand, while `@co.dap.static` and `@co.dap.class` operator methods use only their declared parameters and require the first declared operand to have the enclosing class type.

After receiver normalization, the operand count must match the registered arity of the operator. Each distinct normalized operand signature is an overload. A second implementation with the same normalized operand signature is a conflict and is rejected even when its declared result type differs. This rule applies equally to built-in operators, pre-declared glyphs, and project-local custom operators.

The operator result type is part of the complete implementation signature and type contract, and **different normalized operand signatures may have different result types**. The result type does not participate in selecting between two operator implementations with the same normalized operand signature. Therefore:

```
// valid: distinct operand signatures may have distinct result types
+(co.lang.int, co.lang.int)       -> (co.lang.int)
+(co.lang.float, co.lang.float)   -> (co.lang.float)
+(Employee, Employee)             -> (Employee)

// invalid: the same normalized operands cannot be overloaded by result type
+(co.lang.int, co.lang.int)       -> (co.lang.int)
+(co.lang.int, co.lang.int)       -> (co.lang.float) // compiler error
```

This operator rule is separate from the ordinary named-callable rule that sibling overloads share one declared return signature.

Operator Type Resolution, Conversion, and Overflow

Operator overload resolution requires an **exact normalized operand-type match**. It does not perform nominal subtype-to-supertype widening, C-/C++-/Java-style numeric promotion, or implicit casting/conversion while selecting an operator implementation. A subtype therefore does not inherit a supertype's operator implementation merely because the supertype signature would be applicable under ordinary function-overload rules. The subtype must have an explicit matching operator implementation when that operation is intended to be valid for the subtype.

This exactness is deliberate: an operator implementation for a supertype may rely on invariants or semantics that are not valid for every subtype. Operator overloading therefore remains separate from ordinary named-callable overload widening. An overflow condition likewise never causes the compiler or runtime to widen an operand or select another numeric operator overload.

An operator implementation has its own declared result type. After that operator has been selected, an enclosing declaration, assignment, argument, or return position may require the produced value to satisfy another target type. That is handled by FoLang's ordinary conversion model, not by operator promotion.

The language-provided simple types, such as `co.lang.int` and `co.lang.float`, provide standard `to` and `from` conversion methods through their ordinary package APIs. When the required target type has an applicable conversion and the value is representable by that target, FoLang uses that conversion to satisfy the target context. The operator overload that produced the value is not changed by that conversion.

User-defined types may participate in the same model by supplying supported `to` and/or `from` conversions through the extension mechanisms defined by FoLang. A conversion provided by an extension is an ordinary type conversion; it does not create automatic promotion between the operands of an operator expression.

Consequently:

```
exact normalized operand types
    -> exact operator-overload resolution
    -> operator result type
    -> optional target-context conversion through to/from
```

Overflow never implies promotion. If a declared result/target context provides an applicable conversion to a type capable of representing the produced value, that conversion handles the value at the target boundary. Otherwise the selected operator and result type retain responsibility for their own overflow, division-by-zero, range, and other value-level arithmetic behavior.

For the standard language-provided simple types, arithmetic, ordering, equality, logical, and bitwise operators follow their conventional meanings. `!=` is the logical negation of `==`; `&&` and `||` are short-circuit Boolean operators; and `&`, `|`, and `^` are the ordinary bitwise operators for operand types that support those operations. Exact exceptional-value behavior belongs to the selected type/operator implementation rather than to parser precedence or operator-token classification.

Language-Owned Operators

Language-owned operators already have a registered symbol, fixity, precedence, associativity, and arity. They must not be redeclared in the project operator source. They receive additional implementations through `mode=overload`.

This category includes:

1. ordinary built-in operators such as `+`, `-`, `*`, and `==`;
2. pre-declared operator glyphs whose parse properties are fixed by the language but which may initially have no implementation.

Pre-Declared Operator Glyphs

The current alpha profile pre-declares exactly two mathematical operator glyphs: `∪` and `∩`. Their parse properties are fixed by the language and are listed in C.9: both are binary infix operators with precedence `500` and left associativity. These glyphs are language-owned and therefore cannot be redeclared with `co.lang.operator`.

Unlike an explicitly reserved future/unsupported operator spelling, `∪` and `∩` are enabled expression operators in the current alpha profile. The lexer recognizes them as registered operator tokens and the parser applies their language-defined fixity, precedence, associativity, and arity. Their concrete behavior is supplied through ordinary `mode=overload` operator implementations using the same ownership, signature-normalization, and overload-resolution rules as other language-owned operators.

A use of `∪` or `∩` therefore parses as an operator expression even when no applicable implementation exists. If overload resolution finds no matching implementation for the operand types, compilation fails during operator resolution rather than during lexing or parsing.

See C.10 for the normative current-alpha rule.

Hard-reserved spellings such as `::=`, `->>`, `<->`, backtick, backslash, `#`, and comment openers are different: they are not overloadable or declarable unless a later language revision explicitly assigns them operator semantics.

Project-Local Custom Operator Source

A custom operator is a symbol that is neither language-owned nor hard-reserved. Its symbol and parse properties are registered only in the fixed operator component surface:

```
<project-root>/components/operators/component.fol
```

The operator component's canonical placement and no-subdirectory constraint are defined in Project Layout. If the operator component is absent, the owning compilation introduces no custom operator spellings.

`components/operators/component.fol` uses the common structural component declaration and no library annotation:

```
// components/operators/component.fol
_ co.lang.component = {

    ⊗ co.lang.operator = {
        fixity: co.operator.fixity.infix,
        precedence: 60,
        associativity: co.operator.associativity.left,
        arity: co.operator.arity.binary,
        commutative: co.const.false,
        idempotent: co.const.false,
        identity: co.const.none,
        foldable: co.const.false,
        vectorizable: co.const.false,
        distributes_over: [],
        desugar: "intrinsic:tensor_product"
    };

    +- co.lang.operator = {
        fixity: co.operator.fixity.infix,
        precedence: 60,
        associativity: co.operator.associativity.left,
        arity: co.operator.arity.binary
    };
}
```

The body may contain only `co.lang.operator` declarations. Imports, functions, ordinary types, variables, executable expressions, implementation packages, and nested component/library declarations are forbidden.

`co.lang.operator` is accepted only while parsing `components/operators/component.fol` in the folder-derived operator component context. It cannot appear in package source, `src/appl.fol`, `src/component.fol`, or another component kind.

Where Custom Operator Creation Is Allowed

A project-local `components/operators/` domain is permitted only for:

1. an executable application project rooted at `src/appl.fol`; or
2. a standalone **projected application library** rooted at `src/component.fol` with `@co.dap.library` (or `@co.dap.library(type=application)`).

It is forbidden for:

- standalone packaged libraries — this prohibition is absolute even though packaged code may overload language-owned operators;
- standalone `ffi`, `system`, or `dynamicvmrt` libraries;
- peer project-local components such as `components/ffi/`, `components/system/`, `components/dynamicvmrt/`, `components/application/`, or `components/packaged/` as independent operator domains;
- loaded `lib/*.folenc` dependencies.

For an executable application, the resulting `ProjectOperatorTable` applies **only to the executable application's own `src/` source domain**. It is not made visible while parsing or resolving any project-local component, including `components/packaged/`. Exporting a packaged-component package into the application's open package graph does not retroactively grant that package access to application-defined custom operator spellings.

Consequently, no project-local component may consume, reference, implement, or use a custom operator spelling declared by `components/operators/`. This restriction applies uniformly to `components/application/`, `components/ffi/`, `components/system/`, `components/dynamicvmrt/`, and `components/packaged/`. Those components may still use and legally overload FoLang-owned built-in or pre-declared operators because those spellings belong to the language rather than to the application's `ProjectOperatorTable`.

For a projected application library producer, the table likewise applies only while compiling that producer's own primary `src/` package domain. It is not visible to any project-local component owned by that producer. Custom operator syntax is fully resolved/lowered before `.folenc` emission and does not become importable syntax for consumers.

Operator declaration attributes

Every custom declaration must contain each required parse attribute exactly once. Optional metadata may appear at most once. Duplicate and unknown attributes are errors.

Attribute	Required	Accepted value	Meaning
`fixity`	yes	`infix`, `prefix`, `postfix`, or a reserved future fixity name (`circumfix`, `postcircumfix`, `precircumfix`,etc)	parser placement
`precedence`	yes	decimal integer from `0` through `100`	binding strength
`associativity`	yes	`left`, `right`, or `none`	grouping for equal precedence
`arity`	yes	`unary`, `binary`, `ternary`, or a positive decimal integer	normalized operand count
`commutative`	no	`co.const.true` or `co.const.false`	semantic/optimization metadata
`idempotent`	no	`co.const.true` or `co.const.false`	semantic/optimization metadata
`identity`	no	a FoLang literal, including `co.const.none`	identity-value metadata
`foldable`	no	`co.const.true` or `co.const.false`	compile-time folding metadata
`vectorizable`	no	`co.const.true` or `co.const.false`	vectorization metadata

`distributes_over`	no	a list of quoted operator symbols	distributivity metadata
`desugar`	no	a string literal	intrinsic or lowering hook metadata

In the alpha profile, only `infix`, `prefix`, and `postfix` are implemented. Consequently, an alpha `prefix` or `postfix` declaration must be unary and an alpha `infix` declaration must be binary. Other fixity names remain reserved for future delimiter and slot grammars and are rejected by the alpha conformance profile.

The four required parse attributes construct the tokenizer and precedence table. Optional semantic/optimization attributes do not change tokenization or parsing.

Declaration and implementation are separate

The operator component registers only the symbol and parse/semantic metadata. It contains no implementation. Implementations use the same `mode=overload` form as built-in and pre-declared operators:

```
// vector/Vector.comp.unit.fol
_ co.lang.unit = {
    @co.dap.operator(symbol='⊗', mode=overload)
    tensorProduct(left Vector, right Vector)->(Vector) = {
        ...
    }
}
```

A custom symbol has exactly one declaration in the operator component, but may have any number of distinct implementation overloads in legal owners. Duplicate normalized operand signatures are errors.

Built-In / Pre-Declared Overloading Does Not Require Operator Creation

`ffi`, `system`, `dynamicvmrt`, packaged code, and other domains that are forbidden from creating new operator spellings may still provide legal overload implementations for **language-owned** built-in or pre-declared operators such as `+`, `==`, `∪`, or `∩`, subject to the ordinary operator ownership and signature rules. They do not need `components/operators/` because the parser already knows those spellings.

Creating syntax and implementing already-known syntax are therefore separate capabilities:

```
new operator spelling / precedence / fixity
    -> components/operators/component.fol
    -> only application project or projected application-library producer

mode=overload for language-owned operator
    -> legal owner declaration
    -> no custom-operator component required
```

Symbol registration and exact recognition

When parsing the operator component surface, the scanner reads an operator declaration name as one maximal contiguous symbol run. After the permitted operator table is built, ordinary source scanning in that compilation domain uses the same whole-run rule. A registered custom symbol is recognized only when the complete run matches that symbol; an unknown run is rejected without being split into shorter operators.

Multi-symbol operator uses must also satisfy the operand-boundary rule defined in Symbolic Runs, Classification, and Boundaries.

A custom symbol:

- must not be language-owned;
- must not be a hard-reserved spelling;
- must not contain `//` or `/*`;
- must have exactly one declaration in the permitted operator component.

Operators Do Not Cross Projected or Packaged Artifact Boundaries

A `.folenc` never contributes an importable parser operator table. Projected surfaces export named signatures and boundary contracts, not custom syntax. Packaged/open `.folenc` contexts may carry implementations of already language-known operators, but loading them cannot register a new operator spelling or parse properties in the consumer.

A consuming executable application that wants a custom spelling must register that spelling in its own permitted `components/operators/component.fol` and provide a legal local implementation/delegation. A projected application library may likewise register custom syntax only for its own ordinary primary `src/` compilation through its sole permitted `components/operators/` exception; that parser table is never exported in `.folenc`.

Frontend Preparation, Parsing Order, Imports, and Reachability

FoLang uses one common source parser, but compilation preparation occurs in a defined order so that artifact metadata, component surfaces, application directives, and operator syntax are available before dependent semantic resolution.

Preparing or loading a component does **not** by itself make all of its internal symbols visible elsewhere. A projected project-local component surface can be imported only by an executable application's primary `src/` domain; peer components and standalone libraries cannot import it. Packaged-component selection/import is likewise executable-application-facing only. Standalone projected-library imports follow the independent library exposure rules, while standalone packaged-library contexts are application-open-graph-only.

Preparation and Parsing Order

```
1. project discovery and structural validation
       ↓
2. identify primary src surface
       src/appl.fol OR src/component.fol
       ↓
3. deserialize lib/*.folenc
       reconstruct projected surfaces when present
       reconstruct packaged/open package contexts when present
       reconstruct type hierarchy + overload-family metadata
       reconstruct applicable AST/backend information
       ↓
4. establish primary-source header semantics early
       application: collect application-wide directives such as
                    @co.ddap.dynamicdispatch(...)
       standalone src/component.fol:
                    classify projected @co.dap.library form
                    OR packaged @co.dap.export form
       ↓
5. when permitted by project kind, process components/operators/component.fol
       executable application OR projected application library only
       common parser + componentKind=operators
       build ProjectOperatorTable
       ↓
6. for executable applications only, parse remaining project-local components
       application / ffi / system / dynamicvmrt / packaged
       common parser + folder-derived componentKind
       do not expose the owning ProjectOperatorTable to component source
       build canonical contexts/symbol tables/ASTs
   for standalone libraries, reject every other components/<kind>/ tree
       ↓
7. establish prepared dependency/component environment
       project-local component contexts remain peer-isolated
       standalone library contexts remain available according to their exposure rules
       ↓
8. parse the complete primary src domain
       src/appl.fol + src packages
       OR
       src/component.fol + src packages
       ↓
9. resolve imports by canonical context reference
       ↓
10. for executable applications, merge selected packaged/open package contexts into the application open graph
       ↓
11. semantic/name/type/operator resolution to fixed point
       including dynamic-multidispatch validation when enabled
       ↓
12. unused-symbol/liveness/reachability/capability/source-context validation
       ↓
13. merge applicable AST material -> Final AST
       ↓
14. serialize frontend artifact beneath build/
       ↓
15. backend
```

`.folenc` handling is pre-parse artifact loading; `.folenc` source is never reparsed.

Executable-application project-local components are source parsing. Every `component.fol` and applicable descendant source file uses the common FoLang grammar with its filesystem-derived source context. While compiling such a context, peer project-local component identities are intentionally absent from `component=` resolution; encountering a peer `component=` import is a semantic error rather than a missing-file fallback. Standalone library projects reject project-local component kinds except for the projected application library's `components/operators/` exception. Standalone projected-library dependencies are resolved independently through `lib/`; packaged-library contexts are application-open-graph-only inputs.

The operator component is processed before the permitted ordinary primary `src/` source that may use custom operator spellings: executable application source, or projected application-library source under its sole operator-component exception. Its `ProjectOperatorTable` is withheld from every executable-application project-local component parse, including `components/packaged/`. The application directive preamble is established early enough that `@co.ddap.dynamicdispatch(true)` can affect later overload-family validation and packaged/open AST integration.

For standalone `src/component.fol`:

```
@co.dap.library [type omitted/application/dynamicvmrt/system/ffi]
+ _ co.lang.component
    -> projected standalone library

no @co.dap.library
+ _ co.lang.component containing @co.dap.export(...)
    -> packaged standalone library
```

For project-local components, the folder supplies the kind and every `component.fol` uses `_ co.lang.component`; no project-local component uses `@co.dap.library`.

One canonical symbol table; imports store references

The frontend keeps one canonical context/symbol-table representation for each prepared package, packaged package, projected component surface, or standalone library surface. An import does not copy or merge the target symbol table into the importer.

When a symbol is successfully resolved through a provisional imported context, the compiler marks the exact SymbolId used, marks the import used, activates the dependency edge, and makes the target context reachable.

Packaged/open contexts differ from projected surfaces: only explicitly exported packaged contexts admitted by an **executable application** participate directly in that application's open package/type/overload graph. Unselected packaged contexts remain private to their producer/component, and packaged contexts are not admitted into standalone-library or project-local-component graphs. Projected component/library implementation contexts remain isolated behind their surface.

Unused Symbols, Liveness, and Reachability

This section is the **single canonical definition** of FoLang unused-symbol, declaration liveness, import liveness, project reachability, project-owned component/packaged-component usage, library-project producer usage, and packaged `.folenc` consumer usage. Other sections define syntax and mechanics and link here rather than restating these rules.

Core Meaning

FoLang distinguishes availability from semantic participation:

```
parsed / loaded / deserialized
    != used
    != live
    != reachable
```

- **used** — semantic resolution actually consumes the declaration or operation, or a language-defined contract/protocol rule establishes its required liveness;
- **live** — the declaration or project entity satisfies the usage condition in the matrix below;
- **reachable** — the package, component domain, internal package, or packaged artifact is connected to the applicable project root through effective dependencies;
- **unused** — a usage-checkable project-owned declaration remains non-live after semantic resolution reaches a fixed point;
- **orphan** — physically participating project content remains unreachable after the effective dependency graph is constructed.

Parsing a declaration, importing a package/library, loading a `.folenc`, preparing a symbol table, naming a declaration without exercising the required semantic use, or merely making something visible does not by itself establish liveness.

Governing Principle

FoLang checks **behavior, not shape**.

A function or method has to be written, understood, tested, and maintained. An orphan one is cost with no return, so callables are usage-checked. A type or declaration is lighter, but someone must still know it exists, so it is checked as a whole. A field, variant, or member is a slot in a data shape: it carries no logic to test or reason about, so it is not checked at all.

Everything below follows from that. It is also why a contract-required member is exempt: it is not orphan behavior, because the contract needs it and the developer could not remove it. And it is why a function-local variable is exempt: a local is not behavior anyone can call, test, or depend on — it is scratch inside behavior that is already checked.

FoLang applies three member-level rules:

```
developer-chosen behavior
    -> independently usage-checked

contract/protocol-required behavior
    -> live by conformance once the implementation itself is live

data/object shape
    -> declaration/type is usage-checked;
       fields, members, or variants are not checked independently
```

A callable the developer could remove without breaking a declared contract is independently usage-checked. A callable that must exist because a contract or protocol requires it does not need an artificial separate call merely to avoid an unused-symbol error.

Complete Usage and Liveness Matrix

Entity / declaration	What makes it live / valid	What is independently checked	Unused result
class	class/type participates in the live semantic graph	every developer-authored method not required by an implemented interface	unused class or unused freely-authored method = compile-time error; fields are not checked
module	module participates as a live module value/declaration	every developer-authored function/method not required by its signature	unused module or unused freely-authored member = compile-time error; state values are not checked
struct	struct type is semantically used	declaration/type only	unused struct = error; fields are not independently checked
cstruct	cstruct type is semantically used	declaration/type only	unused cstruct = error; fields are not independently checked
union	union type or member is semantically used	declaration/type only	unused union = error
enum	enum type or variant is semantically used	declaration/type only	unused enum = error
user-defined object	declaration is semantically referenced	declaration as a whole	unused object = error
user-defined annotation object	annotation is actually applied	declaration as a whole	never-applied annotation object = error

matcher	matcher is selected in a live `.match(...)` chain	declaration as a whole	unused matcher = error
interface	at least one live class implements it	declaration/contract as a whole	interface with no live implementing class = error
signature	at least one live module conforms/matches it	declaration/contract as a whole	signature with no live conforming module = error
typeclass	at least one live instance implements it	declaration/contract as a whole	typeclass with no live instance = error
typeclass instance	at least one operation is exercised through that instance	instance as a whole	unused instance = error
ordinary unit	its contributed symbols are used	every usage-checkable contributed symbol	every unused contributed symbol = error
struct companion unit	its contributed companion functions are used	every user-declared companion function	unused companion function = error
ordinary import	at least one symbol is consumed through the imported context	import edge	zero-use import = error and no effective dependency edge
ordinary `src/` package	reachable from `src/appl.fol` or standalone `src/component.fol` roots through effective use edges	ordinary project-owned declarations	unreachable package/unused declaration = error
project-local projected component surface	exposed APIs are consumed by the owning graph	every exposed surface API	unconsumed projected API = error
project-local projected component internal package	reachable from its component surface implementation	ordinary declarations	unreachable/unused internal implementation = error
`components/packaged/component.fol`	selector is structurally valid	selector validity and package-publication boundary	invalid/missing selection = error; unselected packages remain component-private
selected packaged-component package in application	explicit selection permits it to enter the open graph; actual package/symbol use determines ordinary liveness	ordinary package/declaration rules	ordinary unused/reachability error
selected packaged-component package in projected application-library producer	selection makes it an intentional artifact package root	selected exports are producer roots; implementation dependencies remain strict	invalid selection/unreachable required implementation = error
`components/operators/component.fol`	structurally valid and permitted for owning application or projected application-library producer	operator-specific declaration rules	handled by operator rules
projected standalone `src/component.fol`	intentional projected API root	projected surface declarations are producer roots; internals remain strict	unused implementation source = error
packaged standalone `src/component.fol`	valid export selector establishes intentional package roots	selected `src/` package contexts are producer roots	invalid selection/unreachable required implementation = error
loaded `lib/<name>.folenc`	at least one projected or packaged/exported symbol is used	consumer artifact/import liveness	zero used exports = error; unused sibling exports valid
private projected implementation inside `.folenc`	producer already validated it	not consumer-revalidated	no consumer-side unused-symbol analysis
application project	graph rooted at `src/appl.fol` reaches participating project-owned/open source	all applicable rows	orphan project-owned source/entity = error
projected standalone library project	graph rooted at projected `src/component.fol` surface reaches producer implementation; a projected application library may additionally own only its optional operator metadata component	all applicable rows	orphan producer source/entity = error
packaged standalone library project	graph rooted at package contexts selected by `src/component.fol` export selector reaches required producer source	all applicable rows	orphan producer source/entity = error

Contract and Protocol Relationships

Interfaces, signatures, typeclasses, and matchers are contracts/protocols rather than collections of unrelated callable utilities:

```
interface  -> live implementing class
signature  -> live conforming/matching module
typeclass  -> live implementing instance
matcher    -> actual matcher selection in .match(...)
```

Their required members are not independently usage-checked.

For a class or module implementing a contract:

```
contract-required member
    -> live by conformance

additional developer-authored member
    -> independently usage-checked
```

Conformance therefore does not excuse helper/debug APIs that the developer freely added.

Typeclass instance

A typeclass instance becomes live only when at least one operation is actually exercised through that specific instance:

```
OptionMonad instance
├── pure()       required by Monad
└── flatMap()    required by Monad

no operation exercised through OptionMonad
    -> OptionMonad UNUSED

OptionMonad.flatMap(...) resolves successfully
    -> OptionMonad LIVE
    -> pure() valid by conformance
    -> flatMap() valid by conformance
```

An exercised operation includes:

- a qualified operation call through the instance;
- an activated method call that resolves to that instance through `@co.ddap.use`;
- live generic code that is supplied that instance and actually invokes a typeclass operation through it.

Merely importing, naming, binding, storing, passing around, or activating the instance without an operation being exercised does not by itself satisfy the instance's usage requirement.

Direct Usage Declarations

The following are live by their own semantic use rather than by implementation of a separate contract:

```
struct / cstruct / union / enum
    -> type use

object
    -> object declaration/reference use

annotation object
    -> annotation application

matcher
    -> matcher selection in .match(...)
```

For data-shape declarations, fields/members/variants are not independently checked. For matchers, `matchCase` is protocol-required and therefore not independently checked.

Language-owned annotations, directives, and pragmas such as `@co.dap.*`, `@co.ddap.*`, and `@co.pdap.*` are compiler/language facilities rather than project-owned declaration symbols and are outside project unused-symbol validation.

Strict Per-Symbol Declarations

Ordinary units, struct companion units, freely-authored class methods, and freely-authored module functions/methods are strict because each symbol was independently chosen by the developer.

```
utility.unit.fol
├── Id alias            USED
├── Result type         USED
├── parse()             USED
└── format()            UNUSED -> compile-time error
```

```
Employee struct                   USED
Employee.comp.unit.fol
├── calculateTax()                USED
├── serialize()                   USED
└── debugDump()                   UNUSED -> compile-time error
```

Using one unit symbol does not use its siblings. Using a struct does not use its companion functions. Using one companion function does not use its siblings.

Imports, Components, and Packaged Artifacts

All dependency-bearing entities follow the same general rule:

```
written / discovered / loaded
    != live dependency

actual semantic contribution
    -> effective dependency edge
```

A zero-use import is pruned and reported as unused.

Project-local projected components are strict because the same project owns both their surface and implementation: every exposed API must be consumed, and every internal implementation package/declaration must be reachable from that surface.

Project-local packaged components contribute **only explicitly selected** package contexts, and those contexts are published only to the executable application's primary `src/` graph. Every unselected descendant package remains component-private and cannot be imported or referenced from outside the component. No peer project-local component receives selected packaged contexts in its package index. In an executable application, selected package-owned declarations follow ordinary application package reachability. A projected application-library producer has no packaged component; its only permitted project-local component is `components/operators/`. Packaged/open artifact roots therefore arise only from standalone packaged libraries or from `components/packaged/` inside executable applications.

A loaded `.folenc` is consumer-relaxed within its legal consumer domain: at least one projected surface symbol or, for an executable application consuming a packaged artifact, one packaged/exported symbol must be consumed for the physical dependency to be live. Unused sibling exported symbols do not become consumer-side errors. Packaged `.folenc` contexts are never legal consumer inputs for standalone libraries or project-local components.

Closed-Project Reachability

For an executable application:

```
root = src/appl.fol
open graph additionally includes:
    ordinary src/ packages
    selected components/packaged packages
    packaged/open contexts loaded from lib/*.folenc when imported
```

For a projected standalone library producer:

```
root = projected src/component.fol surface
+
permitted intentional packaged-component roots, if any
```

For a packaged standalone library producer:

```
roots = src/ package contexts selected by
        src/component.fol @co.dap.export(...)
```

After semantic/name/type/operator/dynamic-dispatch resolution reaches a fixed point, the compiler validates zero-use imports, declaration/member usage, package reachability, projected component implementation reachability, packaged selection validity, dependency liveness, and orphan project-owned entities.

Bootstrap Order

The normative high-level bootstrap sequence is the preparation order defined in Frontend Preparation, Parsing Order, Imports, and Reachability. In particular:

```
project discovery
    -> lib/*.folenc deserialization
    -> early primary-surface/directive classification
    -> permitted operator component
    -> remaining components
    -> complete src parsing
    -> import/open-graph integration
    -> semantic/type/dispatch fixed point
    -> liveness/capability validation
    -> Final AST
    -> build artifact
    -> backend
```

All FoLang source uses the common parser. `src/component.fol` and `components/<kind>/component.fol` use the same `_ co.lang.component` structural declaration but receive different semantics from their source context and metadata. `.folenc` loading remains artifact preparation rather than source parsing.

Forward / Extern Declarations

Variables extern declaration

// someUnit.unit.fol

```
_ co.lang.unit = {
    @co.dap.declare(extern)
    someBool co.lang.bool;
}
```

Functions forward declaration

// someOtherUnit.unit.fol

```
_ co.lang.unit = {

    @co.dap.declare(forward)
    getEmployee(id co.lang.int)->(somepack.Employee);

    // or — @co.dap.declare is optional for functions
    getEmployee(id co.lang.int)->(somepack.Employee);

}
```

A bodyless `@co.dap.generic` function or method declaration is an ordinary forward declaration under the rules above. A bodyless generic declaration **must not carry** `mapping=`; doing so is a compiler error. Generic mapping augmentation is metadata that extends the effective mapping set of an inherited generic method as defined in Generic Mapping, Result Resolution, and Class-Inheritance Augmentation; it is not represented by a bodyless callable declaration.

Types external declaration

```
// Employee.fol
_ co.lang.class = {

    @co.dap.declare(extern)
    Dept co.lang.struct;

}
```

> For functions and types `@co.dap.declare` is optional. For variables it is required.

Functions

FoLang does not allow free-flowing package functions. Package functions must be declared inside an ordinary `<Fragment>.unit.fol` file. Their public identity is the package member name, not the unit filename.

Normal

```
// general.unit.fol
_ co.lang.unit = {

    fun1(k co.lang.int, b co.lang.char)->(co.lang.int, co.lang.char) = {
        // function body
    }
}
```

A FoLang function may return multiple values.

Default Parameters

// somefununit.unit.fol

```

_ co.lang.unit = {
    fun1(k co.lang.int, b co.lang.char = 'A')->(co.lang.int, co.lang.char)={
    }
}

usage:
  fun1(10, 'B'); // k = 10 and b = 'B'
  fun1(10);      // k = 10 and b = 'A', the declared default value

```

> When an argument for a default parameter is not supplied, the parameter assumes its declared default value.

Variadic Functions

Curried functions are not allowed to be variadic, and vice versa.

//someCurried.unit.fol

```
_ co.lang.unit = {
    fun1 (k co.lang.int, ...b co.lang.char)->(co.lang.int, co.lang.char)={
    }
}
```

Optional Parameters

//someOptional.unit.fol

```
_ co.lang.unit = {
    fun1(k? co.lang.int)->()={
        k.omitted.then({

        }).default({

        });

    }
}
```

> When an optional argument is not supplied, the parameter is in the language-defined omitted/unprovided state and its `omitted` flag is `true`. This state does **not** assign `co.const.none` (or any other sentinel value) to the parameter. `k.omitted` tests whether a value was supplied; developers must not assume that reading or printing an omitted `k` produces `co.const.none`.

Named Parameters

// someNamedParam.unit.fol

```
_ co.lang.unit = {
    fun1(~k co.lang.int, ~v co.lang.int)->()={

    }
}

Usage:
  fun1(v=10,k=20); // valid
  fun1(10,20); //valid here k =10 and v=20
```

> Named-parameter declaration is all-or-none within a function: a function cannot mix `~`-named parameters with ordinary parameters. When a call uses argument names, their order does not matter. A function whose parameters are all declared with the named form may still be called positionally where otherwise valid, as shown above.

Named Returns

//someNamedResults.unit.fol

```
_ co.lang.unit = {

    doManythings(a co.lang.int, b co.lang.int->(&, meta={type=out}))->(r co.lang.int, e co.lang.exception)={}
    doSomething(input co.lang.int)->(a co.lang.int, b co.lang.bool) = {
        this.return 20, co.const.true;
    }
}

Usage:
  // Before the call, a and b do not yet exist; using either name here is a compiler error.

  doSomething(10);

  // Immediately after the call, the named return values are available at the call site.
  // If a or b already exists with the matching type, that existing binding is used.
  // If either name already exists with an incompatible type, it is a compiler error.

  co.out.println(a);  // 20
  co.out.println(b);   // prints true (boolean)
```

> Named returns create call-site bindings using the declared return names when compatible bindings do not already exist.

Function Delegates

// someFunDelegate.unit.fol

```
_ co.lang.unit = {
    myFunc(s co.lang.int, t co.lang.int)->(co.lang.int, co.lang.int) = {
        this.return 10, 10;
    }

    mySecondFun(s co.lang.int, t co.lang.int)->(co.lang.int, co.lang.int) = {
        this.return 20, 20;
    }
}
```

Usage inside a legal executable function or method body:

```
@co.dap.delegate someDelegate co.lang.delegate =
    (a co.lang.int, b co.lang.int)->(co.lang.int, co.lang.int);

someDelegate = myFunc;
someDelegate(10, 20); // invokes the currently registered function

someDelegate = myFunc;
someDelegate += mySecondFun;
someDelegate(10, 20); // invokes the registered delegate functions
```

> Delegates are primarily used when a developer needs to register multiple compatible functions. When a call should simply redirect or chain into another function, use function chaining as shown below.

Multicast Delegate Invocation

When a delegate has multiple registered compatible functions, invocation proceeds sequentially through the registered functions in delegate invocation order. A function added with `+=` is invoked after the functions already registered before it.

The value returned by the delegate invocation is always the result of the **last invoked function**. Results produced by earlier registered functions do not become the result of the delegate invocation. For a multi-return delegate, the complete set of return components produced by the last invoked function is returned.

For example, if `myFunc` returns `(10, 10)` and `mySecondFun` returns `(20, 20)`, then a call made after:

```
someDelegate = myFunc;
someDelegate += mySecondFun;
```

invokes `myFunc` and then `mySecondFun`, and the delegate call returns `(20, 20)`.

Function Chaining

// somefunctionChaining.unit.fol

```
_ co.lang.unit = {
    fetchEmployee(empId co.lang.string)->(Employee)=>>empMod.getEmployee(this, empId);

    dosomething(a co.lang.int, b co.lang.int)->(co.lang.int)
        =>> somePack.someMethod(a)
        =>> someOthPack.someOtherMeth($1, b);
}
```

For a chained invocation, bind variables refer only to the return values of the immediately preceding function invocation. Return components are numbered from one in declared return order: `$1` is the first return value, `$2` the second, and so on. Therefore a preceding multi-return function exposes all of its return components to the next chaining step as `$1 ... $N`. A single-return function exposes only `$1`.

Anonymous Functions

// someAnonymousFun.unit.fol

```
_ co.lang.unit = {
    add co.lang.function = (a co.lang.int, b co.lang.int) -> (co.lang.int){
        this.return a + b;
    };

    res co.lang.function = (a co.lang.int, b co.lang.int) -> (co.lang.int){
        this.return a * b;
    }(10, 20);
}
```

> Why is there no equals sign after the function signature? It is deliberately omitted because the function signature acts as the type and the body acts as the literal value. The declaration is therefore a function-object initialization, analogous to initializing any other UDT from an object literal. For more information about functions, see Functions in Detail.

Functions in detail

Inline

```
// math_functions.unit.fol
_ co.lang.unit = {
    @co.dap.inline
    add(a co.lang.int, b co.lang.int)->(co.lang.int) ={
        this.return a + b;
    }
}
```

Anonymous Functions

// someOtherAnonymousfun.unit.fol

```
_ co.lang.unit = {
    add co.lang.function = (a co.lang.int, b co.lang.int) -> (co.lang.int) {
        this.return a + b;
    };

    res co.lang.function = (a co.lang.int, b co.lang.int) -> (co.lang.int) {
        this.return a * b;
    }(10, 20);
}
```

Lambda

Only allowed as an inline callback argument to receiver-qualified collection operations (e.g. `each`, `map`, `filter`, `reduce`, `forEach`, `sortBy`, `groupBy`). Transparent grouping around the member callee is permitted, so `(nums.map)(|x| => x*x)` is equivalent to the ordinary call spelling. A bare function call such as `map(|x| => x)` is not a collection-method context. Using `|...|` anywhere else is a syntax/lint error.

// somelanbda.unit.fol

```
// Callback literal syntax, shown only in its valid collection-call context
_ co.lang.unit = {

    nums.map(|x| => x*x);
    words.filter(|s| => s.len() > 3);
    pairs.reduce(|acc, e| => acc + e, 0);
    dict.map(|k, v| => v * 10);
    list.sortBy(|a, b| => a.score - b.score);

}
```

The lambda must be a direct argument of the allowed collection call. That call may itself be nested, for example `consume(nums.map(|x| => x*x))`; the enclosing call does not make the lambda an argument of `consume`.

Inner Function

//someInnerFun.unit.fol

```
_ co.lang.unit = {
    myfun(a co.lang.int, b co.lang.int)->(co.lang.int)={
        p co.lang.int = 10;
        someother()->()={
            co.out.println(p);
        }
        someother();
        p = 20;
        someother();
    }
}
```

The function `someother` is an ordinary inner function because it is physically declared inside `myfun`. Its free runtime names are resolved from the lexical declaration context of `myfun`; therefore `p` denotes the binding declared in `myfun` regardless of where `someother` is called within its permitted lifetime.

`@co.dap.local`, `@co.dap.nested`, and `@co.dap.inner` apply to separately declared functions and do not change the lexical-scope rule for an ordinary inner function. They provide visibility, target-state capture, or call-site association without requiring the function body to be physically placed inside the target declaration.

The permitted local-function form above is useful when the inner function is small and belongs naturally to one enclosing function. This is the named local-function exception described in Physical Nesting Rules.

Curried

// someOtherCurried.unit.fol

```
_ co.lang.unit = {
    add(first co.lang.int)(second co.lang.int)->(co.lang.int)={
        this.return first + second;

    }
}
```

Closure

//someClosure.unit.fol

```
_ co.lang.unit = {
        adder() -> ((co.lang.int) -> co.lang.int) ={
            sum co.lang.int = 0;
            this.return  (x co.lang.int) -> (co.lang.int){
                sum += x;
                this.return sum;
            };
        }
}
```

Functions Taking and Returning Functions

Syntax 1 — Inline signature

//someInlineSignature.unit.fol

```
_ co.lang.unit = {
    someFunction (r (co.lang.int, co.lang.int)->(co.lang.int))->((co.lang.int)->(co.lang.int))={}
}
```

Syntax 2 — Named type alias

//sommeNamedTypeAliases.unit.fol

```
_ co.lang.unit = {
    someFArg co.lang.type = (co.lang.int, co.lang.int)->(co.lang.int);
    someFRet co.lang.type = (co.lang.int)->(co.lang.int);

    someFunction (r someFArg)->(someFRet)={}
}
```

Syntax 3 — Function objects

//someFunObject.unit.fol

```
_ co.lang.unit = {
    someFArg co.lang.function = (a co.lang.int, b co.lang.int) -> (co.lang.int){
        this.return a + b;
    };

    someFRet co.lang.function = (a co.lang.int) -> (co.lang.int){
        this.return a * 2;
    };
}
```

Other Ways to Declare Closures, Function Objects, Function Types, and Curried Functions

//someAdditionaleg.unit.fol

```
_ co.lang.unit = {
    myobj co.lang.function = (a co.lang.int, b co.lang.int)->(co.lang.int){
        this.return a + b;
    };

    add (a co.lang.int, b co.lang.int)->(co.lang.int)={ this.return a + b; }
    oObj co.lang.function = add;

    funtype co.lang.type = (a co.lang.int, b co.lang.int)->(co.lang.int);

    closure=(factor co.lang.int, val co.lang.int) ==>> factory * val;

    curry = (factor co.lang.int) (x co.lang.int) ==>> x * factor;
}
```

Associated Functions

For a user-defined struct, associated functions must be declared inside the same-package companion unit whose name matches the struct. For more information, see Associated Functions in a Companion Unit.

Some Restrictions on Special Functions

Non-overloadable Function Forms

The following function forms cannot be overloaded:

1. functions with named parameters;
2. functions with optional parameters;
3. functions with default parameters;
4. variadic functions;
5. curried functions;
6. functions that use inline function-signature syntax directly in a parameter or return position;
7. functions that use a function type as a parameter or return type;
8. multi-return functions;
9. named-return functions;
10. functions having a pointer, address, reference, thunk, or slice in any parameter or return position.

These categories are signature-level restrictions. A declaration that falls into more than one category remains non-overloadable for the same purpose; the categories do not create separate overload families.

Callable Identity and Static Overload Resolution

FoLang first resolves the **canonical callable identity** before comparing overload parameter signatures. For an ordinary named callable, that identity includes:

1. the canonical owning context;
2. the callable name; and
3. the applicable callable/receiver category.

For class methods, categories such as `instance`, `class`, `static`, and `object` are therefore distinct identity dimensions before parameter overload resolution. For struct companion units, receiverless, instance-associated value-receiver, and type-associated type-receiver functions likewise retain their defined receiver category as part of callable identity. A declaration in a different owner or callable/receiver category is not a sibling overload merely because its visible name and ordinary parameter list are the same.

Within one canonical callable identity, ordinary **static overload resolution** uses the compile-time argument-type tuple. Generic candidates first perform only the parameter-position/explicit generic deduction needed to establish their candidate parameter signatures; return context and generic `mapping=` rows are not candidate-selection inputs. For static tuple `(S1, S2, ..., Sn)`, a candidate with parameter tuple `(P1, P2, ..., Pn)` is applicable when every `Si` is the same as or a nominal subtype of the corresponding `Pi`. Exact matches are therefore maximally specific; when an exact signature is absent, one or more positions may widen through the nominal supertype hierarchy. Arbitrary `to` / `from` conversions and the expected return type are not used to manufacture an overload candidate.

Candidate signature `A` is more specific than candidate signature `B` when every parameter type in `A` is the same as or a subtype of the corresponding parameter type in `B`, and at least one parameter type in `A` is a proper subtype of the corresponding parameter type in `B`. Static overload resolution requires one **unique most-specific applicable** candidate. No applicable candidate is a compiler error; multiple incomparable most-specific candidates are an ambiguity error. Declaration order and left-parameter priority do not resolve ambiguity.

```
static argument-type tuple
    -> applicable parameter signatures
    -> exact match when present
    -> otherwise nominal parent/supertype widening
    -> unique most-specific applicable overload
```

Application dynamic multiple dispatch uses the same applicability and specificity relation; its only fundamental selection difference is that the tuple comes from runtime argument types rather than compile-time argument types.

Overload-Family Parameter and Return Rules

Within one canonical ordinary named-callable identity, overload selection is determined by the applicable **parameter signature**. A return type or return signature never participates in overload selection and cannot distinguish two sibling declarations. Therefore declarations that differ only in return type are invalid rather than separate overloads.

```
select(x Animal)->(Animal) = { ... }
select(x Animal)->(Dog)    = { ... } // ❌ invalid: differs only by return type
```

Every sibling declaration in the same ordinary overload family must declare an **identical return signature**. Parameter signatures may vary according to the ordinary overload rules, but the declared result contract of the family is invariant.

```
collide(a Animal, b Animal)->(co.lang.bool) = { ... }
collide(a Dog,    b Cat)->(co.lang.bool)    = { ... } // ✅ same return signature

transform(a Animal)->(Animal) = { ... }
transform(a Dog)->(Dog)        = { ... } // ❌ same family, different return signature
```

This rule applies equally to ordinary static overload resolution, static multi-parameter overload resolution, and application dynamic multiple dispatch. FoLang does not implicitly widen a selected overload's return type, infer a common result type across sibling overloads, or use the expected destination type to choose an overload. The caller therefore has one stable result contract after overload selection.

Operator implementations follow their separate normalized-operand rule: distinct operand signatures may have different result types, while identical operand signatures cannot be distinguished by result type. Operator candidate selection itself is exact by normalized operand types: nominal subtype widening, numeric promotion, and `to` / `from` conversion do not make an operator candidate applicable. A **single generic declaration** also does not violate the ordinary return-family rule merely because a declared result generic resolves to different concrete types for different valid generic instantiations. In that case the declaration still has one return-signature structure (for example `->(T)`), and `T` is resolved only after parameter/generic resolution according to the generic rules, including `mapping=` where present.

Existing Callback and Execution-Model Restrictions

Curried functions, functions with named, optional, variadic, or default parameters, functions that take or return functions or function types, and dynamically scoped or mixed-scoped functions retain the existing restrictions that they cannot be used as ordinary callbacks and cannot participate in Execution Models and Control Abstractions.

Scoping Rules for Functions

All functions in FoLang have a defined scope — the set of variables a function can access.

Default — Lexical / Static Scope

All of the following are **always** lexically scoped — this cannot be changed:

```
methods
inner methods
free functions
inner functions
target-local named functions
closures
lambdas
Generic Functions
Anonymous functions
Curried functions
```

Lexical scope means a function resolves names from its **declaration site**, not from the scope of its caller. Unit-level variables are forbidden, so a unit-scoped free function receives runtime values through parameters or introduces them locally. An ordinary inner function captures from the enclosing lexical declaration context. Calling that inner function does not replace its captured context with the caller's runtime scope.

```
// scope_example.unit.fol
_ co.lang.unit = {

    foo()->() = {
        x co.lang.int = 10;

        printX()->() = {
            co.out.println(x);  // ✅ x from the enclosing lexical scope
        }

        printX();
    }
}
```

`@co.dap.inner` — Call-Site Lexical Context

`@co.dap.inner` is not the syntax for an ordinary inner function. It annotates a separately declared function or method that becomes associated with a target at an explicit use or attachment site.

Its free runtime names use **call-site lexical-context resolution**:

```
1. @co.dap.inner parameters and local declarations
2. the innermost lexical scope at the active call or attachment site
3. enclosing lexical scopes of that call or attachment site
4. statically resolved types, annotations, imports, and co.* names
5. compiler error
```

The compiler validates these requirements at every statically known use or call site. This is a fixed property of `@co.dap.inner`; it is not selected through `@co.dap.lexicalscope`, `@co.dap.dynamicscope`, or `@co.dap.mixedscope`.

This differs from ordinary lexical inner functions, whose free runtime names are fixed by their declaration site, and from dynamically scoped associated functions, which may search outward through the runtime caller activation chain.

Associated Functions — Additional Scope Options

Only associated functions support non-lexical scoping via annotations:

The examples below are members of the same-package `Employee` companion unit.

`@co.dap.lexicalscope` — default, explicit declaration

```
// Employee.comp.unit.fol
_ co.lang.unit = {
    @co.dap.lexicalscope
    (emp Employee) process()->() = {
        co.out.println(emp.name);   // ✅ declaration scope
    }
}
```

`@co.dap.dynamicscope` — accesses caller's scope

```
// Employee.comp.unit.fol
_ co.lang.unit = {
    @co.dap.dynamicscope
    (emp Employee) process()->() = {
        co.out.println(name);   // name comes from caller's scope
    }
}
```

`@co.dap.mixedscope` — accesses both scopes

```
// Employee.comp.unit.fol
_ co.lang.unit = {
    // caller scope takes priority — shadows declaration scope on conflict
    @co.dap.mixedscope
    (emp Employee) process()->() = {
        co.out.println(name);   // caller scope takes priority
        co.out.println(emp.id); // falls back to declaration scope
    }
}
```

Callback Scope Inside Dynamically Scoped Associated Functions

A callback block or lambda does not independently select lexical, dynamic, or mixed scope. The associated function that executes the callback determines how the callback's free runtime names are resolved.

Callback parameters and declarations made inside the callback always belong to the callback's local scope. Names that are not callback parameters or callback-local declarations follow the executing associated function's scope policy. //someOperation.unit.fol

```
_ co.lang.unit = {
    someFun ()-> ()={
        nums.reduce(|acc, e| => {
            total.value += e;
            acc + e;
        }, 0);
    }
}
```

For this example:

```
acc, e              -> callback parameters
temporary locals    -> callback-local context
total               -> reduce's dynamic caller context
co.* and imports    -> normal built-in/import resolution
```

The callback syntax remains uniform. `reduce` is dynamically scoped, so unresolved runtime names in the callback are resolved through the active caller-context chain. A lexically scoped associated function would instead resolve free runtime names through its declaration context.

Why Dynamic Scope Exists — `.then`, `.loop`, `.each`, and Collections

FoLang's control-flow model is built on dynamically scoped associated functions. The executing function supplies the scope policy, so blocks do not require separate capturing and non-capturing forms.

//someScopeEg1.unit.fol

```
_ co.lang.unit = {
    someFun()->()={
        x     co.lang.int = 10;
        total co.lang.int = 0;
        arr   co.lang.int->([5]) = [1, 2, 3, 4, 5];

        // .then reads and modifies the caller's x
        (x > 5).then({
            x.value = 20;
            co.out.println(x);
        });

        // .loop modifies the caller's x
        (x > 0).loop({
            x.value -= 1;
        });

        // .each(..., action) performs the iteration and modifies the caller's total
        arr.each(_, val, {
            total.value += val;
        });

        // .filter, .map, and .reduce use the same dynamic caller context
        nums.filter(|x| => x > 10);
        nums.reduce(|acc, e| => {
            total.value += e;
            acc + e
        }, 0);
    }
}
```

The compiler does not create a separate capture description for each control block. It resolves names according to the scope mode of the associated function executing that block.

FoLang Control Flow Uses Dynamic Scope

```
no if/else keywords    -> .then / .otherwise / .default  — dynamic scope
no for/while keywords  -> .loop              — dynamic scope
no foreach keywords    -> .each(..., action) — dynamic scope
no in keyword          -> .contains          — dynamic scope
no map/filter keywords -> .map / .filter     — dynamic scope

all control flow is implemented as associated functions
control associated functions are dynamically scoped
caller variables are accessible and mutable under the dynamic lookup rules
block syntax requires no separate capture mode
```

Dynamic and Mixed Lookup Order

For `@co.dap.dynamicscope`, runtime variable lookup follows this order:

```
1. function/callback parameters and local declarations
2. immediate caller activation context
3. caller activation contexts outward through the dynamic call chain
4. built-in `co.*` names and imported declarations visible to the source file
5. compiler error if no compatible declaration is available
```

For `@co.dap.mixedscope`, lookup follows this order:

```
1. function/callback parameters and local declarations
2. immediate caller activation context
3. caller activation contexts outward through the dynamic call chain
4. lexical declaration context
5. built-in `co.*` names and imported declarations visible to the source file
6. compiler error if no compatible declaration is available
```

Local declarations shadow caller declarations. Caller declarations shadow declarations from the lexical declaration context in mixed scope.

Types, annotations, imports, and other compile-time declarations continue to use ordinary static resolution. Dynamic lookup applies to runtime bindings, not to the identity of types or imported APIs.

Call-Site Validation of Dynamic Requirements

A dynamically or mixed-scoped associated function may reference a caller-provided runtime name that is not declared in its lexical context: //someDynScope1.unit.fol

```
_ co.lang.unit = {
        @co.dap.dynamicscope
        (Employee) addToTotal(value co.lang.int)->() = {
            total.value += value;
        }
}
```

At every statically known call site, the compiler validates that the active caller-context chain provides a definitely initialized binding named `total` with a compatible type and permitted mutability. A call is rejected when the required binding cannot be proven to exist.

Because non-lexically scoped functions are non-first-class and non-escaping, their call sites remain statically identifiable. This avoids runtime string-based name lookup and allows the compiler to represent dynamic requirements using context and symbol-table references.

Scope Rules Summary

Function Type	Lexical	Dynamic	Mixed
methods	✅ declaration-site only	❌	❌
ordinary inner methods	✅ declaration-site only	❌	❌
free functions	✅ declaration-site only	❌	❌
ordinary inner functions	✅ enclosing declaration context only	❌	❌
target-local named functions	✅ declaration-site only	❌	❌
`@co.dap.inner` functions/methods	call-site lexical context	❌ no unrestricted caller-chain lookup	❌
anonymous functions	✅ declaration-site only	❌	❌
closures	✅ declaration-site capture	❌	❌
lambdas/callback blocks	determined by executing associated function	determined by executing associated function	determined by executing associated function
associated functions	✅ default	✅ opt-in	✅ opt-in
`.then` / `.loop` / `.each` / `.contains`	❌	✅ built-in	❌
`.map` / `.filter` / `.reduce`	❌	✅ built-in	❌

Additional Restrictions

- dynamically or mixed-scoped functions cannot be returned
- dynamically or mixed-scoped functions cannot be stored as values

- dynamically or mixed-scoped functions cannot be passed as ordinary callbacks
- dynamic or mixed caller contexts cannot cross thread or process boundaries
- dynamic or mixed execution cannot continue after the caller frame ends
- dynamic or mixed-scoped functions cannot participate in Execution Models and Control Abstractions
- dynamically or mixed-scoped functions cannot be curried
- named, optional, variadic, and default-parameter forms follow the same non-escaping and call-site-validation rules

Types

In ordinary package source, `co.lang.type`, type aliases, newtypes, opaque types, refinement types, subtypes, supertypes, and parameterized `co.lang.type` constructors must be declared inside an ordinary `*.unit.fol` file. They are contributed directly to the package namespace. Entry files, signatures, modules satisfying signature type components, and dedicated projected component surfaces follow their own explicitly stated rules.

Examples in this section that show only a type declaration are fragments from inside a legal unit or other legal enclosing declaration.

The three axes — each adds one new power:

Axis 1: Polymorphism (terms depend on types)

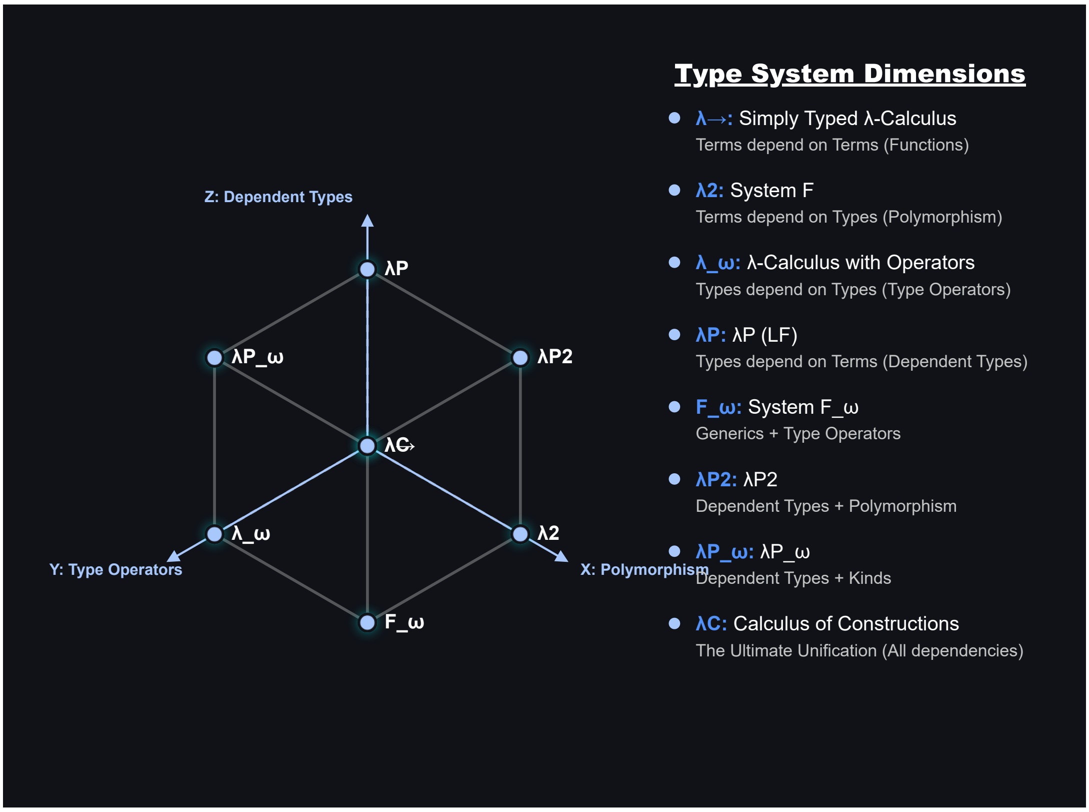
```
// "Give me a type, I'll give you a value"

// Without: write separate functions
//sometypes1.unit.fol
_ co.lang.unit = {

    identityInt(x int) → int={}
    identityStr(x string) → string={}

    // With: one function works for all types
    identity(T)(x T) → T={}

}
// This is generics / parametric polymorphism
// System F, Java generics, your @co.dap.generic
```

Axis 2: Type operators (types depend on types)

```
// "Give me a type, I'll give you a type"
//sometypes2.unit.fol
_ co.lang.unit = {
    List(Int)      → List ={}            // List of ints type → type
    Map(String, Int) → Map  ={}          // type → type → type
    Option(T)      → Some(T) | None;     // type constructor
}
// This is kinds / higher-kinded types
// Your folang: Option(T) co.lang.type = Some(T) | None()
```

Axis 3: Dependent types (types depend on values)

```
// "Give me a value, I'll give you a type"
//sometypes3.unit.fol
_ co.lang.unit = {
    Vector(3)      → array of exactly 3 elements
    Matrix(2, 3)   → 2x3 matrix
}
// The type is indexed by a value. The index may be a compile-time constant or a
// symbolic value parameter bound by the enclosing declaration signature.
// Value predicates such as "x > 0" belong to refinement types, not dependent indexing.
```



Lambda Cube

Refinement Types

A refinement type restricts the admissible values of an existing base type by a Boolean predicate. It does not introduce a separate binder name merely so the predicate can refer to the value being tested. The contextual token `_` denotes that **candidate value** inside the refinement predicate.

The canonical declaration form is:

```
// types.unit.fol
_ co.lang.unit = {
    positiveInt co.lang.refinementType =
        (co.lang.int).where(_ > 0);
}
```

The declaration above means that `positiveInt` admits values of `co.lang.int` that satisfy `_ > 0`. The `_` placeholder has the base type (`co.lang.int` in this example), and every occurrence of `_` in the same refinement predicate refers to the same candidate value.

```
// types.unit.fol
_ co.lang.unit = {
    percentage co.lang.refinementType =
        (co.lang.int).where(_ >= 0 && _ <= 100);

    evenInt co.lang.refinementType =
        (co.lang.int).where(_ % 2 == 0);

    nonEmptyString co.lang.refinementType =
        (co.lang.string).where(_.length > 0);
}
```

The predicate supplied to `.where(...)` must resolve to `co.lang.bool`. The candidate placeholder is available only while resolving that refinement predicate. It does not introduce a variable into the enclosing unit, does not escape the declaration, and cannot be rebound or assigned. Member access and ordinary expressions may use it according to the operations supported by the base type.

`_` remains contextual. In pattern and discard positions it retains its wildcard/discard meaning, and in filename-derived primary declarations it retains its declaration-name-placeholder meaning. Only the predicate belonging to a `co.lang.refinementType` declaration gives `_` the candidate-value meaning.

For a statically known candidate, the compiler can evaluate the refinement predicate directly. A statically known value that makes the predicate false cannot inhabit the refinement type:

```
// types.unit.fol
_ co.lang.unit = {
    positiveInt co.lang.refinementType =
        (co.lang.int).where(_ > 0);

    good positiveInt = 10;   // valid: 10 > 0
    bad  positiveInt = -5;   // compile-time error: -5 does not satisfy the predicate
}
```

Refinement validation is normatively a **run-time operation**. Whenever a value enters a `co.lang.refinementType` through assignment, argument passing, return, conversion, initialization, or another admissible value-transfer operation, the refinement predicate is validated at run time. If the predicate evaluates to `co.const.true`, the value is admitted; if it evaluates to `co.const.false`, refinement validation fails.

The compiler may additionally evaluate the predicate when the candidate value is statically known. This permits an invalid statically known candidate to be rejected earlier, as in the `bad` declaration above. Such compile-time analysis is an early diagnostic and does not change the language's validation model: refinement validity is defined by the run-time predicate check rather than by a requirement that the predicate be statically provable.

Refinement, Dependent, and Associated Types

These three mechanisms solve different problems:

Form	What determines or constrains the type	FoLang role
`T co.lang.refinementType = (Base).where(predicate)`	a predicate restricts which values of `Base` are admitted	value-set restriction
`Vector(n)` / a function returning `co.lang.dependentType`	a value participates in the resulting type identity or shape	value-indexed type
`Entity co.lang.associatedType;`	a signature requires a matching module to supply a compatible type binding	module contract type component

A refinement predicate therefore does not make its candidate value an index of the type in the dependent-type sense. Likewise, an associated type is not a predicate-restricted value set; it is a type component selected by a matching module.

A `co.lang.refinementType` declaration is also distinct from `co.lang.subtype` and `co.lang.supertype`. Refinement adds a value predicate to a base type. It does not by itself define the inheritance, variance, or assignability rules of the separate subtype/supertype declaration kinds.

Dependent Types

Type-Level Functions — Functions That Return Types

A function that accepts a type or value and returns a type is a **type-level function**. When its result depends on a value argument, it defines or selects a dependent type. This is distinct from a parameterized `co.lang.type` constructor such as `Option(T)`, whose declaration directly defines a family of types.

// sometypes4.unit.fol

```
_ co.lang.unit = {
        // Vector — value-indexed type-level function
        // takes  → co.lang.int (size)
        // returns → co.lang.dependentType (a type)
        Vector(n co.lang.int)->(co.lang.dependentType) =
            co.lang.int->([n]);

        // calling Vector(3) returns a TYPE at compile time
        // that type is co.lang.int->([3])
        v3 Vector(3) = [1, 2, 3];    // type is Vector(3)
        v4 Vector(4) = [1, 2, 3, 4]; // type is Vector(4) — different type!

        // Vector(3) ≠ Vector(4) — completely different types
        // size is part of the type — compiler knows at compile time
}
```

More About Type

```
Name(T) co.lang.data = variants;
    → concrete ADT type-constructor definition
    → right-hand-side definition is mandatory

Name(T) co.lang.associatedType;
    → generic associated-type requirement
    → permitted only inside a signature

Name(T) co.lang.associatedType = ExistingType(T);
    → associated type-constructor binding
    → permitted directly in a matching module for the corresponding signature component

Name(T) co.lang.type = ExistingType(T);
    → concrete type alias in an ordinary type context, or a fixed/manifest type component in a signature
```

A Type-Level Function Returns a Type

```
Vector        → type-level function
Vector(3)     → function call → returns type co.lang.int->([3])
Vector(4)     → function call → returns type co.lang.int->([4])

just like:
    add(1, 2)  →  returns a value  (3)
    Vector(3)  →  returns a type   (int[3])
```

Compiler Enforced Size Safety

//sometypes5.unit.fol

```
_ co.lang.unit = {
    // dot product — only valid for same size vectors
    // compiler enforces this via dependent types
    dotProduct(a Vector(n), b Vector(n))->(co.lang.int) = {
        // n is same for both — compiler verified
    }

    v3 Vector(3) = [1, 2, 3];
    v4 Vector(4) = [1, 2, 3, 4];

    dotProduct(v3, v3);   // ✅ same type Vector(3)
    dotProduct(v3, v4);   // ❌ compiler error — Vector(3) ≠ Vector(4)
}
```

Matrix — Two-Parameter Type-Level Function

//somematrix.unit.fol

```
_ co.lang.unit = {
        // Matrix — takes rows and cols, returns dependent type
        Matrix(r co.lang.int, c co.lang.int)->(co.lang.dependentType) =
            co.lang.int->([r, c]);

        m34 Matrix(3, 4) = [[1,2,3,4],[5,6,7,8],[9,10,11,12]];
        m45 Matrix(4, 5) = ...;

        // matrix multiply — cols of A must equal rows of B
        // n must match — compiler verified
        multiply(a Matrix(r, n), b Matrix(n, c))->(Matrix(r, c)) = {
            // compiler ensures dimensions are compatible
        }

        multiply(m34, m45);   // ✅ Matrix(3,4) × Matrix(4,5) = Matrix(3,5)
        multiply(m34, m34);   // ❌ compiler error — 4 ≠ 3
}
```

Stack — Value and Type Parameter

//somestack.unit.fol

```
_ co.lang.unit = {
    // Stack — takes size and element type
    Stack(n co.lang.int, T co.lang.type)->(co.lang.dependentType) =
        T->([n]);

    s Stack(10, co.lang.int)    = ...;  // stack of max 10 ints
    t Stack(5,  co.lang.string) = ...;  // stack of max 5 strings
}
```

Type Is Value + Kind Combined

```
Vector(3):
    kind  = Vector    (what it is)
    value = 3         (how many)
    type  = Vector(3) (both together — the dependent type)

Vector(3) ≠ Vector(4)   ←  different types entirely
Vector(3) = Vector(3)   ←  same type
```

Type Constructors and Type-Level Functions

```
// option.unit.fol
_ co.lang.unit = {
    // Parameterized type declaration: Option is a type constructor.
    Option(T) co.lang.type =
        Some(T) | None();

    // Value-indexed type-level function: Vector computes a dependent type.
    Vector(n co.lang.int)->(co.lang.dependentType) =
        co.lang.int->([n]);
}
```

`Option` and `Vector` both operate at the type level, but they are different declaration categories:

```
Option(T) co.lang.type
    -> type-constructor declaration
    -> substitution produces Option(T)

Vector(n)->(co.lang.dependentType)
    -> type-level function
    -> computation produces a type
```

Simple Dependent Type

//someiden.unit.fol

```
_ co.lang.unit = {
    identity(x co.lang.int)->(x.type) ={ this.return x; }
}
```

Compile-Time and Runtime Values in Type-Related Computation

FoLang distinguishes value-indexed dependent types, compile-time type computation, runtime type descriptors, and runtime values whose concrete types may differ. These mechanisms are related, but they are not interchangeable.

1. Value-Indexed Dependent Types

A dependent type may contain a value as part of its type identity. The value index may be a compile-time constant, a symbolic type-level value, or a runtime function parameter whose relationship to the result is tracked by the compiler. //someIndex.unit.fol

```
_ co.lang.unit = {
    readVector(n co.lang.int)->(Vector(n)) = {
        ...
    }
}
```

The compiler does not need to know the concrete value of `n` while compiling `readVector`. It records the relationship:

```
input index = n
result type = Vector(n)
```

Likewise: // sometypes6.unit.fol

```
_ co.lang.unit = {
    dotProduct(
        left  Vector(n),
        right Vector(n)
    )->(co.lang.int) = {
        ...
    }
}
```

requires both vectors to have the same value index. Dependent typing means that a type contains or is constrained by a value; it does not mean that an arbitrary runtime branch can silently change the static type of an already compiled variable.

---

2. Compile-Time Type Computation

A function may compute and return a type when it is guaranteed to execute during compilation. // someFuncomp.unit.fol

```
_ co.lang.unit = {
    @co.dap.comptime
    @co.dap.eager
    chooseType(value co.lang.int)->(co.lang.type) = {
        (value < 100)
            .then(co.lang.string)
            .default(co.lang.bool);
    }
}
```

The arguments must be compile-time evaluable when the result is used in a static type position: //someTest1.unit.fol

```
_ co.lang.unit = {
    someFun()->()={
        Selected co.lang.type = chooseType(10);
        value Selected = "Hello";
    }
}
```

Invalid: //someTest2.unit.fol

```
_ co.lang.unit = {
    someFun()->()={
        input := co.in.readInt();
        Selected co.lang.type = chooseType(input);
        // compiler error: `input` is not compile-time evaluable
    }
}
```

Conceptually:

```
compile-time value
    -> compiler executes the type function
    -> one concrete static type is available before ordinary type checking completes
```

---

3. Built-in compile-time type computation

A function may compute and return a type when it is guaranteed to execute during compilation.

> `decltype` is a built-in method.

The arguments must be compile-time evaluable when the result is used in a static type position: // someFun5.unit.fol

```
    _ co.lang.unit = {
        someFun()->()={
            someIntVar co.lang.int ;
            someVar co.hokrlt.type.decltype(someIntVar) = 200;
        }
    }
```

---

4. Runtime Type Descriptors

An ordinary function returning `co.lang.type` produces a runtime type descriptor when it is not executed at compile time. //someruntype2.unit.fol

```
_ co.lang.unit = {
    selectType(value co.lang.int)->(co.lang.type) = {
        (value < 100)
            .then(co.lang.string)
            .default(co.lang.bool);
    }


    selectedType co.lang.type = selectType(input);
}
```

Here, `selectedType` is a runtime object that describes a type. It may be used for reflection, runtime validation, dynamic loading, metadata inspection, or dynamic object creation where those capabilities are permitted.

A runtime type descriptor cannot ordinarily be used as the static type of a declaration: //someruntype3.unit.fol

```
_ co.lang.unit = {
    someFun()->()={
        value selectType(input);

        // compiler error: runtime type descriptor used in a static type position
    }
}
```

Conceptually:

```
runtime value
    -> runtime type-selection function
    -> co.lang.type descriptor object
```

A value that represents a type is not automatically a compile-time-resolved static type. ***

5. `@co.dap.typefromvalue`

`@co.dap.typefromvalue` derives a type from the type of a returned compile-time value. In the initial FoLang specification, it is permitted only for compile-time evaluation. //sometypeval.unit.fol

```
_ co.lang.unit = {
    @co.dap.comptime
    @co.dap.typefromvalue
    inferType(value co.lang.int)->(co.lang.type) = {
        (value < 100)
            .then("Hello")
            .default(co.const.true);
    }
}
```

The compiler derives:

```
"Hello"       -> co.lang.string
co.const.true -> co.lang.bool
```

Every argument must be compile-time evaluable when the result is used in a type position. Runtime value-based type selection must instead use an explicit runtime representation.

---

6. Runtime Values with Different Concrete Types

When a runtime branch may produce unrelated concrete value types, the function must return one stable outer type. FoLang may use a tagged value, an ADT, `co.lang.dynamic` where permitted, or another explicitly packaged representation.

Using `co.lang.tag`: //someruntype1.unit.fol

```
_ co.lang.unit = {
    selectValue(value co.lang.int)->(co.lang.tag) = {
        (value < 100)
            .then(co.lang.tag(co.lang.string, "Hello"))
            .default(
                co.lang.tag(co.lang.bool, co.const.true)
            );
    }
}
```

The outer static type is always `co.lang.tag`:

```
co.lang.tag
├── runtime type descriptor
└── value compatible with that descriptor
```

An ADT provides a more strongly typed alternative:

```
SelectedValue co.lang.data =
      StringValue(co.lang.string)
    | BoolValue(co.lang.bool);

selectValue(value co.lang.int)->(SelectedValue) = {
    (value < 100)
        .then(StringValue("Hello"))
        .default(BoolValue(co.const.true));
}
```

Summary

```
Vector(n)
    -> value-indexed dependent type

@co.dap.comptime function returning co.lang.type
    -> compile-time type computation

ordinary runtime function returning co.lang.type
    -> runtime type descriptor

runtime branch returning unrelated value types
    -> tag, ADT, dynamic value, or another explicit package
```

A runtime type descriptor is a value that represents a type. It must not be confused with a statically resolved dependent type.

Parameterized Type Declarations and Type-Level Functions

Two declaration families produce types from parameters. The spelling depends on whether the declaration directly defines a type family or computes a type through a function body. //someEg7.unit.fol

```
_ co.lang.unit = {
    // all parameters are types -> parameterized co.lang.type declaration
    Option(T) co.lang.type = Some(T) | None();
    someAlias(F) co.lang.type = Functor(F);

    // a value parameter is present -> type-level function syntax
    Vector(n co.lang.int)->(co.lang.dependentType) = co.lang.int->([n]);
    Stack(n co.lang.int, T co.lang.type)->(co.lang.dependentType) = T->([n]);
}
```

A parameterized `co.lang.type` declaration is a type constructor. Its type parameters appear directly in the declaration head and it does not use `@co.dap.generic`.

A function that accepts values or type values and returns `co.lang.dependentType` is a type-level function. `Stack` demonstrates why the function form exists: it can mix value parameters and type-valued parameters and compute the resulting type.

`co.lang.dependentType` is both a type-producing return kind and a direct type-declaration kind. A type-level function uses it when a value parameter determines the produced type. A direct declaration may use it when no parameter list is required:

```
LengthBound co.lang.dependentType = co.lang.int;
```

The kind is also usable in a declarator. If a function returns `co.lang.dependentType`, a binding receiving that result may therefore be declared `co.lang.dependentType`.

A type-level function has exactly one unnamed type-producing result. That result may be a union using `|`, but comma-separated multiple results are invalid: //somebadeg1.unit.fol

```
_ co.lang.unit = {
    Choice(n co.lang.int)->(co.lang.dependentType | co.lang.type) = co.lang.int;
    Bad(n co.lang.int)->(co.lang.dependentType, co.lang.type) = co.lang.int; // invalid
}
```

Parameterized aliases are transparent

An alias declared with `co.lang.type` names the same type, not a new one. //someParameg1.unit.fol

```
_ co.lang.unit = {
    someAlias(F) co.lang.type = Functor(F);

    someFun()->()={
        someAlias(List);    // the same type as Functor(List)
        someAlias(Option);  // the same type as Functor(Option)
    }
}
```

Because the alias adds no identity, it creates no separate instance slot: an instance cannot be declared for `someAlias` as distinct from one for `Functor`. Declaring one would be a duplicate.

Named parameters also allow reordering and partial application, which a positional placeholder could not express. //someTypeClz1.unit.fol

```
_ co.lang.unit = {
    Pair(F, G) co.lang.type = Transformer(F, G);
    Flip(F, G) co.lang.type = Transformer(G, F);
    Fixed(F)   co.lang.type = Transformer(F, Set);
}
```

Use `co.lang.newtype` instead when a **distinct** identity is wanted, such as the wrapper described in Where an Instance Is Declared.

Dependent Type Index Rules

An **index** is an argument to a dependent type, such as the `n` in `Vector(n)`, or a dimension in an array derivation, such as the `n` in `co.lang.int->([n])`. Both positions obey the same rules.

An index is a literal or a name

An index is an integer literal or a name. Arithmetic, function calls, indexing and every other operator are rejected. // someIdxEG1.unit.fol

```
_ co.lang.unit = {
    someFun()->()={
        @co.dap.const SIZE co.lang.int = 1024;

        v Vector(3);                  // ✅ literal
        v Vector(SIZE);               // ✅ @co.dap.const name
        buf co.lang.int->([SIZE]);    // ✅ same rule for array sizes

        v Vector(n + 1);              // ❌ arithmetic is not permitted in an index
        v Vector(computeSize());      // ❌ a call is not permitted in an index
        buf co.lang.int->([n * 2]);   // ❌ same rule for array sizes
    }
}
```

This restriction applies only to the **size** of an array, never to element access. Indexing an array is an ordinary expression and arithmetic is fine.

```
buf co.lang.int->([SIZE]);      // size — restricted
buf[i + 1] = 42;                // access — unrestricted
buf[compute(x)] = 7;            // access — unrestricted
```

What a named index may resolve to

A name used as an index resolves in exactly one of two ways.

A parameter bound by the enclosing signature. A type constructor or a function signature introduces the name, and every use of it inside that signature and its body refers to the bound parameter. The name is not a constant; it stands for whatever value the caller supplies. //someEG2.unit.fol

```
_ co.lang.unit = {
    // n is introduced here, and bound for the whole declaration
    Vector(n co.lang.int)->(co.lang.dependentType) = co.lang.int->([n]);

    // n is introduced by this signature, and both parameters must share it
    dotProduct(a Vector(n), b Vector(n))->(co.lang.int) = {
        // ...
    }

    // n is introduced as a value parameter and reused in the return type
    readVector(n co.lang.int)->(Vector(n)) = {
        // ...
    }
}
```

A `@co.dap.const` compile-time constant. Outside a signature that binds it, a name has nothing to bind to, so it must be a constant the compiler can substitute.

```
@co.dap.const SIZE co.lang.int = 1024;
buf co.lang.int->([SIZE]);      // ✅ SIZE substitutes to 1024
v Vector(SIZE);                 // ✅ same rule for dependent types
```

Nothing else qualifies. `@co.dap.final` marks an immutable binding, and an immutable value need not be known while compiling, so it cannot be substituted.

```
@co.dap.final n co.lang.int = readInput();
bad Vector(n);                  // ❌ immutable, but not known at compile time

m co.lang.int = 10;
alsoBad Vector(m);              // ❌ an ordinary variable is not an index
```

So in a plain variable declaration, where no signature is binding anything, the only legal names are `@co.dap.const` constants.

An index is non-negative

Zero is permitted; a negative index is not.

```
empty co.lang.int->([0]);       // ✅ zero-length array

buf co.lang.int->([-1]);        // ❌ rejected while parsing
v Vector(-1);                   // ❌ rejected while parsing

@co.dap.const OFFSET co.lang.int = -1;
buf co.lang.int->([OFFSET]);    // ❌ rejected after substitution
```

A negative literal cannot be written at all, because no prefix operator is reachable in an index position. A negative constant is rejected when the compiler substitutes it. Both are compile-time errors.

When two dependent types are equal

Two dependent types are equal when their constructors are the same and their indices are pairwise equal. An index comparison has exactly three cases.

Index form	Compared by
integer literal	value
`@co.dap.const` name	substituted literal value
parameter	name identity

```
Vector(3)    vs Vector(3)       // equal
Vector(n)    vs Vector(n)       // equal
Vector(n)    vs Vector(m)       // NOT equal — rejected
Vector(SIZE) vs Vector(1024)    // equal when @co.dap.const SIZE = 1024
```

Rejecting `Vector(n)` against `Vector(m)` is the point of the feature. It is what lets the compiler catch a size mismatch without the developer writing a single check.

What FoLang deliberately does not do

FoLang does not decide index equality up to arithmetic. `Vector(n+1)` and `Vector(1+n)` are not merely unequal — they cannot be written.

Accepting them would require symbolic reasoning, and there is no partial version of it. Once `n+1 == 1+n` is accepted, the next reasonable request is `2*n == n+n`, and the type checker becomes a theorem prover by accretion. That is the complexity FoLang is built to avoid.

The cost is narrow. Length-arithmetic signatures such as `concat(Vector(n), Vector(m)) -> Vector(n+m)` are out of scope; return a dynamically sized type and check at run time instead. Everything that needs only same-parameter identity still works, and that covers the common cases.

```
multiply(a Matrix(r, n), b Matrix(n, c)) -> (Matrix(r, c))
dotProduct(a Vector(n), b Vector(n))     -> (co.lang.int)
zip(a Vector(n), b Vector(n))            -> (Vector(n))
```

Matrix multiplication, the usual demonstration of dependent types, needs only that the shared `n` matches.

Dependent types are checked, never inferred

Every dependent type appears in a written signature. FoLang never infers one.

This is what keeps checking decidable without a constraint solver, and it is why FoLang does not adopt Hindley-Milner style whole-program inference. Inferring a dependent type would mean inferring the index **value**, not merely the type, which is the step that makes checking undecidable in general.

Indexer

Indexer functions for a struct are associated functions and must be declared inside `<StructName>.comp.unit.fol`.

```
// MyList.fol
_ co.lang.struct ={
    eles co.lang.int->([...]);
}

// MyList.comp.unit.fol
_ co.lang.unit = {

    @co.dap.indexer(symbol="[]")
    (g MyList) get(index co.lang.int)->(co.lang.int) ={
        this.return g.eles[index];
    }

    @co.dap.indexer(symbol="[]=")
    (g MyList) set(index co.lang.int, value co.lang.int)->() ={
        g.eles[index] = value;
    }
}

lst MyList;
co.out.println(lst[0]);
lst[1] = 22;
```

Generics

```
@co.dap.generic(
    at=callsite,
    types=[
        {name=U, variance=invariant, bound=Number, inference=param},
        {name=T, variance=invariant, bound=Number}
    ],
    mapping=[
        {U=co.lang.int,   T=co.lang.int},
        {U=co.lang.float, T=co.lang.float}
    ],
    impredicative=false,
    resolution=compiletime
)
add(a U, b U)->(T) = { this.return a + b; }
```

Generic annotation fields:

Attribute	Values
types	list of generic-parameter records such as `[{name=T}, {name=R}]`
requires	
mapping	compile-time relationships among declared generic parameters; valid only for compile-time generic resolution
resolution	`runtime`, `compiletime`
reified	`true` or `false`
at	`usesite` or `callsite`
specializable	`true` or `false`
impredicative	`true` or `false`

types attributes |Attribute | Values| |—|—| |name|| |constraints|| |upper-bound|| |lower-bound|| | bound | not needed when `upper-bound` and/or `lower-bound` is specified | |default|| |variance| `covariant`, `invariant`, `contravariant` | |nullable|| |inference| `param`, `arg`, `var` | |capabilities|| | isAKind | an interface, class, or trait; e.g. `SomeInterface` | |typekind| `type`, `class`, `function`, `struct`, `typeconstructor` | |inclusive||

> The entries above are fields of each generic-parameter record inside the `types=[...]` list.

> Example: `@co.dap.generic(types=[{name=T, variance=..., bound=...}])`

> These are not independent attributes; they describe each declared generic type entry.

Generic Marker Classification

Names listed by the immediately associated `@co.dap.generic(types=[...])` are **generic markers** for that declaration. In applicable type positions the parser/frontend records those names as generic-marker references; they are not looked up as ordinary type symbols.

A type-position identifier that is **not** listed in that declaration's `types=[...]` list is an ordinary type reference. It is resolved through the normal symbol/type environment and is a compiler error when no such type exists.

```
@co.dap.generic(types=[{name=U}])
f(x U, y T)->(U) = { ... }
```

Here `U` is a generic marker. `T` is not declared as a marker by the annotation, so `T` must resolve as an actual type symbol. If no type named `T` is visible, the declaration is invalid.

By contrast:

```
@co.dap.generic(types=[{name=U}, {name=T}])
f(x U, y T)->(U) = { ... }
```

both `U` and `T` are generic markers.

Generic-marker spelling does not manufacture a distinct generic signature. These two declarations have the same generic parameter structure and the same parameter signature, so they conflict as duplicates in one callable identity:

```
@co.dap.generic(types=[{name=T}])
f(x T)->(T) = { ... }

@co.dap.generic(types=[{name=U}])
f(x U)->(U) = { ... } // compiler error: duplicate generic signature
```

The compiler compares declared generic-marker roles/positions rather than treating `T` and `U` as ordinary type names. This normalization applies **only to names already classified as generic markers**. An undeclared type-position name is never converted into a generic marker merely because another declaration uses a marker with a similar role.

For example, if an actual type named `T` exists, these parameter structures are distinct:

```
@co.dap.generic(types=[{name=U}])
f(x U, y T)->(U) = { ... } // second parameter is actual type T

@co.dap.generic(types=[{name=U}])
f(x U, y U)->(U) = { ... } // both parameters use generic marker U
```

Return types still do not participate in ordinary overload selection; the example uses the parameter signature deliberately because that is where the structural distinction matters.

Frontend Handling of Generic Metadata

The frontend must parse and preserve the complete `@co.dap.generic` metadata application. Fields needed to establish frontend syntax, symbol identity, generic-marker classification, type resolution, callable selection, or a concrete result contract are interpreted where required. In particular, `types=` establishes generic markers and `mapping=` participates in the compile-time result-resolution rules defined below.

Other generic fields and attributes may be backend- or later-stage-oriented. The frontend records and serializes them in the Final AST/backend interchange and does **not** fail frontend generation merely because it has no semantic handler for such a field. A later compiler/backend stage may interpret, validate, specialize, reify, or reject those preserved values according to the applicable feature contract. Malformed metadata syntax remains a parser error.

When `mapping=` is present, the mapping relation itself must be resolved by the frontend wherever it is necessary to produce a concrete callable/result contract. A backend-oriented field such as `resolution`, `reified`, `at`, `specializable`, or `impredicative` does not by itself block frontend artifact generation merely because the frontend does not otherwise act on it.

Generic Mapping, Result Resolution, and Class-Inheritance Augmentation

`mapping=` defines a finite compile-time relationship among the generic parameters declared by the same `@co.dap.generic` annotation. It is **not required merely because a generic parameter appears in the return signature**. If every generic needed by the return signature is already resolvable from callable parameter-position generic information (or from explicit generic arguments), the return signature uses those already-resolved generic values directly and `mapping=` is unnecessary. `mapping=` is required only when a generic needed by the result contract would otherwise remain unresolved and is intended to be derived from already-resolved generic inputs.

```
@co.dap.generic(
    types=[
        {name=U, inference=param},
        {name=T}
    ],
    mapping=[
        {U=co.lang.int,   T=co.lang.int},
        {U=co.lang.float, T=co.lang.float}
    ]
)
f(x U, y U)->(T) = {
    ...
}
```

For `f(10, 20)`, ordinary parameter analysis first establishes `U=co.lang.int`; the mapping then establishes `T=co.lang.int`. For arguments statically typed `co.lang.float`, the second row establishes `T=co.lang.float`. The return type did not select the callable and the expected destination type was not consulted.

When `mapping=` Is Not Required

A generic result needs no mapping when every generic variable occurring in the return signature is already determined from the callable's parameter types or explicit generic arguments. The generic need not be the complete type of one parameter; occurrence inside a parameterized/container type is sufficient when ordinary generic inference can recover it.

```
@co.dap.generic(types=[{name=T}])
identity(x T)->(T) = {
    this.return x;
}

@co.dap.generic(types=[{name=T}])
first(xs co.core.List->(T))->(T) = {
    ...
}

@co.dap.generic(types=[{name=K}, {name=V}])
lookup(m co.core.Map->(key=K, val=V), key K)->(V) = {
    ...
}

@co.dap.generic(types=[{name=T}])
wrap(x T)->(co.core.List->(T)) = {
    ...
}
```

For these declarations, parameter-position inference establishes all generic values needed by the result contract before result typing begins. For example, `co.core.List->(co.lang.int)` establishes `T=co.lang.int`, so `first(...)` has result type `co.lang.int` without a mapping row.

By contrast, this declaration is incomplete without another explicit resolution source for `T`:

```
@co.dap.generic(types=[{name=U}, {name=T}])
f(x U, y U)->(T) = {
    ...
}
```

A call such as `f(10, 20)` can establish `U=co.lang.int`, but nothing in the parameter signature establishes `T`. FoLang does not infer `T` from the assignment target, expected destination type, or another return context. Therefore `T` must be supplied explicitly, resolved by an applicable `mapping=` row, or resolved by another mechanism explicitly defined by this specification.

The generic-result rule is therefore:

```
infer generic values from parameter positions and explicit generic arguments
    -> if every generic needed by the return signature is resolved
           mapping is unnecessary
    -> otherwise
           resolve the remaining generic through mapping= or another explicit mechanism
    -> expected/destination return type never participates
```

Generic candidate preparation and resolution proceed without consulting return context:

```
resolve canonical callable identity and candidate declarations
    -> for each generic candidate, infer generic values available from
       ordinary call-parameter positions and explicit generic arguments
    -> instantiate enough of each candidate's parameter signature to test
       ordinary static overload applicability/specificity
    -> select the unique parameter overload
    -> apply mapping rows to the selected generic's already-resolved inputs
    -> resolve any remaining mapped generic parameters
    -> complete generic instantiation
    -> type-check the concrete return/result contract
```

A `mapping=` row is therefore never used to make a generic function win ordinary overload selection. It resolves mapped generic parameters only after the callable has been selected from parameter information.

A generic parameter that occurs only in the return signature is **never inferred from the expected destination or return context**. It must be resolved by an explicit generic argument, by an applicable `mapping=` row, or by another generic-resolution mechanism explicitly defined by this specification. Result-context generic inference is not part of FoLang.

When `mapping=` is present, its rows define the permitted mapped relationships for the generic parameters they mention. Mapping matches use canonical type identity; overload-style subtype widening is not performed while choosing a mapping row. Once the determining generic values are known, the mapping must yield one consistent assignment for every unresolved mapped generic. No applicable row, multiple conflicting applicable rows, a cyclic unresolved dependency, or a mapping that leaves a required generic unresolved is a compile-time error.

Mapping does not create sibling function overloads. The declaration above remains one generic callable with one body and one declared return-signature structure `->(T)` even though different valid instantiations may make `T` concrete as different types. This is generic instantiation, not return-type overloading.

Generic Mapping Augmentation Through Class Inheritance

Generic mapping augmentation is intentionally restricted to **class inheritance**. Package-level/free functions do not need cross-package augmentation because package ownership is part of callable identity: two same-named functions in different packages are different callables rather than one overload family. An application therefore cannot reopen another package merely to add `mapping=` rows to a package function.

When a class inherits a visible generic method, additional mapping entries may be associated with that **already inherited generic method** in the derived-class context. The thing being augmented is the inherited method's effective **mapping set**. The augmentation does not declare another callable, does not provide another body, and is not a bodyless/forward generic method declaration.

A bodyless generic method remains an ordinary forward declaration and **cannot contain `mapping=`**. Mapping metadata intended as an inheritance augmentation must resolve to exactly one inherited generic method and must match that method's callable/receiver category, declared generic-marker structure, ordinary parameter-signature structure, and declared return-signature structure. If no such inherited generic exists, the mapping contribution is a compiler error.

This can apply to classes defined by the application itself or classes from package contexts explicitly exported by a packaged component or standalone packaged library and therefore present in the executable application's open graph. It does not penetrate projected `application`, `ffi`, `system`, or `dynamicvmrt` boundaries because their internal classes and methods remain hidden behind surface APIs.

Conceptually:

```
BaseProcessor.convert
    generic markers: U, T
    implementation body: owned by BaseProcessor
    mapping set:
        {U=co.lang.int, T=co.lang.int}

DerivedProcessor inherits BaseProcessor.convert
    augmentation mapping contribution:
        {U=abc.Employee, T=abc.SuperEmployee}

DerivedProcessor effective inherited convert mapping set:
        {U=co.lang.int,   T=co.lang.int}
        {U=abc.Employee, T=abc.SuperEmployee}
```

The compiler merges inherited and derived-context mapping entries as a set:

```
identical row + identical row
    -> one logical row

same already-resolved input assignment -> same derived assignment
    -> duplicate; one logical row

same already-resolved input assignment -> different derived assignment
    -> conflict; compiler error

different input assignment
    -> additional valid mapping row
```

The augmentation affects the effective inherited generic in that derived-class context; it does not rewrite the base declaration globally and does not alter sibling derived classes. Generic mapping remains a compile-time frontend mechanism where it is required to establish the concrete callable/result contract, and it is not re-evaluated from runtime argument types by `@co.ddap.dynamicdispatch(true)`.

Generic Functions — Parameters and Return Values

Rank-1: Outer function is generic; parameter uses the same type variable

`T` is fixed at the call site before the function parameter is used. The passed function is already monomorphic inside the body.

Syntax 1 — Inline signature //somGen1.unit.fol

```
_ co.lang.unit = {
    @co.dap.generic(types=[{name=T}])
    someFunction(f (T, T)->(T), a T)->(T) = {}
}
```

Syntax 2 — Named type alias //somGen2.unit.fol

```
_ co.lang.unit = {
    @co.dap.generic(types=[{ name=T}])
    someFArg co.lang.type = (T, T)->(T);

    @co.dap.generic(types=[{ name=T}])
    someFunction(f someFArg, a T, b T)->(T) = {}
}
```

Rank-2: The function parameter is itself polymorphic (higher-rank)

The passed function stays generic **inside the callee**. The callee decides what `T` is. Uses existing `forall`.

Syntax 1 — Inline signature //someGen3.unit.fol

```
_ co.lang.unit = {
    someFunction(f forall(T).(T, T)->(T))->(co.lang.int) = {
        this.return f(1, 2);
    }
}
```

Syntax 2 — Named type alias //someGen4.unit.fol

```
_ co.lang.unit = {
    someFArg co.lang.type = forall(T).(T, T)->(T);

    someFunction(f someFArg)->(co.lang.int) = {}
}
```

//someGen5.unit.fol

```
_ co.lang.unit = {
    // Correct — Syntax 2 with co.lang.type
    someFArg co.lang.type = forall(T).(T, T)->(T);

    someFunction(f someFArg)->(co.lang.int) = {}
}
```

Returning Generic Functions

Rank-1 return //someGen6.unit.fol

```
_ co.lang.unit = {
    @co.dap.generic(types=[{name=T}])
    makeAdder(a T)->((T)->(T)) = {
        this.return (b T)->(T){ this.return a + b; };
    }
}
```

Rank-2 return — returning a polymorphic function //somGen7.unit.fol

```
_ co.lang.unit = {
    makeIdentity()->( forall(T).(T)->(T) ) = {
        this.return forall(T).(x T)->(T){ this.return x; };
    }
}
```

Rank-3: A Parameter is Itself a Rank-2 Function

Rank-3 works naturally in FoLang via `forall` nesting. No new constructs needed.

Syntax 1 — Inline //someGen8.unit.fol

```
_ co.lang.unit = {
    // f takes a Rank-2 function as its argument — that is Rank-3
    applyRank2(
        f (forall(T).(T, T)->(T)) -> (co.lang.int)
    ) -> (co.lang.int) = {
        this.return f(1, 1);
    }
}
```

Syntax 2 — Named type aliases (cleaner) //someGen9.unit.fol

```
_ co.lang.unit = {
    rank2FnType  co.lang.type = forall(T).(T, T)->(T);
    rank3ArgType co.lang.type = (rank2FnType) -> (co.lang.int);

    applyRank2(f rank3ArgType) -> (co.lang.int) = {
        this.return f(1, 1);
    }
}
```

Rank-3 return //somGen10.unit.fol

```
_ co.lang.unit = {
    makeRank2Consumer() -> ((forall(T).(T)->(T)) -> (co.lang.int)) = {
        this.return (f forall(T).(T)->(T)) -> (co.lang.int){
            this.return f(42);
        };
    }
}
```

Impredicativity — Instantiating `T` with a `forall` Type

Impredicativity is when a type variable `T` in a generic is itself instantiated with a `forall` type. Example of what this means: //somGen11.unit.fol

```
_ co.lang.unit = {

    @co.dap.generic(types=[{name=T}])
    box(x T) -> (Box(T)) = {}

    someFun()->()={
        // Impredicative call — T being set to forall(U).(U)->(U)
        result := box(forall(U).(U)->(U));   // ❌ not legal without explicit opt-in
    }
}
```

Most type systems reject this by default. FoLang takes an opt-in approach.

Initial alpha release Workaround — Option C: Wrapping with `co.lang.type`

Not true impredicativity but solves 90% of practical cases: //somGen12.unit.fol

```
_ co.lang.unit = {
    polyId co.lang.type = forall(U).(U)->(U);

    // box takes co.lang.type — no impredicative unification needed
    box(x co.lang.type) -> (Box(co.lang.type)) = {}

    someFun()->()={
        result := box(polyId);   // ✅ works — x is co.lang.type, not a forall type
    }
}
```

1.0 release — Option A: `impredicative=true` in `@co.dap.generic`

The frontend accepts and preserves this metadata field. When true impredicative instantiation is supported by the selected backend/later compilation stage, this field provides the explicit opt-in; current-alpha frontend artifact generation does not fail merely because that later-stage feature is unavailable: //somGen13.unit.fol

```
_ co.lang.unit = {

    @co.dap.generic(
        types=[{name=T,variance=invariant}],
        impredicative=true
    )
    box(x T) -> (Box(T)) = {}

    polyId co.lang.type = forall(U).(U)->(U);

    someFun()->()={
        result := box(polyId);   // ✅ legal — impredicative=true explicitly opts in
    }
}
```

Generic Function Rank Support Matrix

Scenario	Allow?	Notes
Rank-1 generic param (Syntax 1, 2, 3)	✅ Yes	Natural extension, no new concepts
Rank-1 generic return (Syntax 1, 2, 3)	✅ Yes	Same as above
Rank-2 param via `forall` (Syntax 1, 2)	✅ Yes	`co.lang.type` naturally holds polymorphic types; no new kind needed
Rank-2 param via Syntax 3 `co.lang.function`	❌ Compiler error	Function objects are concrete values; use `co.lang.type = forall(T).(T)->(T)` instead
Rank-2 return via `forall` (Syntax 1, 2)	✅ Yes	Same reasoning as param
Rank-3 via `forall` nesting (Syntax 1, 2)	✅ Yes	No new constructs — `forall` nesting works naturally
Rank-3 return	✅ Yes	Same reasoning as Rank-3 param
Rank-3 via Syntax 3 `co.lang.function`	❌ Compiler error	Same rule as Rank-2; function objects are concrete
Impredicative — workaround (Option C)	initial alpha release ✅ Yes	Wrap `forall` type in `co.lang.type`; solves 90% of real cases

Impredicative — true opt-in (Option A)	🔜 1.0	`impredicative=true` in `@co.dap.generic`; explicit opt-in

`@co.dap.generic(types=[...])` declares generic markers that belong to a named struct, class, function, or method declaration and carries that declaration's generic metadata. This is separate from `forall(...)`, which binds names only inside an anonymous polymorphic **type expression** used for higher-rank parameter/return types or `co.lang.type` aliases. It is also separate from parameterized `co.lang.type` constructor heads such as `Option(T)`. See forall and Generic Declarations and Type Constructors.

```
// LinkedList.fol
@co.dap.generic(types=[{name=T}])
_ co.lang.struct={
    value T;
    next  LinkedList;
    prev  LinkedList;
}

k := LinkedList.new(co.lang.int); // when we call new it returns an object of type co.lang.uninit

or

k LinkedList->(T=co.lang.int);


actualList := k.init(); // this is what create a fully formed object of type class



// Employee.fol
@co.dap.generic(types=[{name=T},{name= R}])
_ co.lang.class = {
    id   T;
    name R;

    @co.dap.override
    @co.dap.constructor(access=private)
    @@init() = {}

    @co.dap.override
    @co.dap.constructor(access=public)
    @@init(id T, name R) = {
        this.parent.init();
        this.id   = id;
        this.name = name;
    }

    getEmployee(id T)->(Employee)={}
}

a := Employee.new(co.lang.int, co.lang.string);
b := a.init(1, "Rao");


Ordinary generic construction does not require overriding `@@new` or `@@init`. These lifecycle methods are used only when generic construction
requires behavior different from the default construction model.

Normal conditions to create/instantiate object of class we just call init which internally call new
In specific cases as above we need to do two calls or use call chain like below


c := Employee.new(co.lang.int,co.lang.string).init(1,"Rao");

This new and init methods will be available without you overriding/overloading new and init


// Employee.fol
@co.dap.generic(types=[{name=T},{name= R}])
_ co.lang.class = {
    id T;
    name R;

}

c := Employee.new(co.lang.int,co.lang.string).init(1,"Rao");


This works folang automaticall provides all type parameters new and all field init implementations.

or

c Employee->(T=co.lang.int, R=co.lang.string);
```

Generics Inheritances and Types

```
This is in conceptual stage not supported.

A) Abstract vs concrete type members
B) Path-dependent types
    1. Type-level projection
    2. Path-dependent In folang how it would be
```

forall

What `forall` Is — and Is Not

`forall` is **not** a general-purpose generic declaration keyword and is **not globally hard-reserved**. It is a **contextual keyword** that introduces an anonymous polymorphic type expression only when the parser is in a type-expression position and recognizes the complete `forall(...) . ...` form. It is used specifically where a polymorphic type must appear inline, including Rank-2 and Rank-3 parameter and return positions and `co.lang.type` aliases.

Outside that contextual polymorphic-type form, the spelling `forall` is an ordinary identifier and follows the normal declaration and name-resolution rules for the position in which it occurs. Recognizing `forall` contextually therefore does not consume the spelling globally.

Named generic structs, classes, functions, and methods use `@co.dap.generic` as their sole generic-parameter declaration mechanism. `forall` is not a declaration mechanism. A declaration-head form that attempts to use `forall(T)` as a generic declaration prefix is invalid because declaration grammar does not define such a prefix; the error does not arise from `forall` being globally reserved.

Where `forall` Is Allowed — Type Expression Form Only

The contextual form is `forall(T).` followed by an anonymous type body. The parser recognizes `forall` specially only when it begins this polymorphic type-expression form. The `.` after the binder list is the syntactic signal that the binder is followed by an anonymous type body rather than being an ordinary identifier/call spelling.

Pattern:

```
forall(T).  <anonymous type body>
```

Contextual-recognition rule:

```
type-expression context
        +
identifier spelling "forall"
        +
valid binder list `( ... )`
        +
`.`
        ↓
polymorphic forall type expression
```

For example, `forall(T).(T)->(T)` is a polymorphic type expression. By contrast, an occurrence of the identifier `forall` that does not satisfy this contextual form remains an ordinary identifier subject to the grammar of its surrounding position.

```
// co.lang.type alias — naming a polymorphic type for reuse
someFArg co.lang.type = forall(T).(T, T)->(T);

// Rank-2 inline parameter — callee decides what T is
someFunction(f forall(T).(T)->(T)) -> (co.lang.int) = {}

// Rank-2 return type — returning a polymorphic function
makeIdentity() -> (forall(T).(T)->(T)) = {}

// Rank-3 inline parameter — f takes a Rank-2 function
applyRank2(f (forall(T).(T, T)->(T)) -> (co.lang.int)) -> (co.lang.int) = {}
```

Where `forall` Is Banned — Use `@co.dap.generic` Instead

```
// ❌ compiler error — invalid generic declaration-head form; use @co.dap.generic
forall(T) identity(x T)->(T) = {}

// ✅ correct
@co.dap.generic(types=[{name=T,variance=invariant}])
identity(x T)->(T) = {}
```

```
// ❌ compiler error
// LinkedList.fol
forall(T)
_(T) co.lang.struct = { value T; next LinkedList; }

// ✅ correct
// LinkedList.fol
@co.dap.generic(types=[{name=T}])
_ co.lang.struct = { value T; next LinkedList; }
```

```
// ❌ compiler error — invalid generic declaration-head form; Rank-1 generics belong to @co.dap.generic
forall(T) someFunction(f (T,T)->(T), a T)->(T) = {}

// ✅ correct
@co.dap.generic(types=[{name=T,variance=invariant}])
someFunction(f (T,T)->(T), a T)->(T) = {}
```

Quick Reference

Form	Status	Context
`forall(T) name ...`	❌ Compiler error	Not a defined declaration-head generic form — use `@co.dap.generic` instead
`forall(T).(T)->(T)`	✅ Allowed	Type level only — Rank-2/3 param, return, `co.lang.type` alias

The rule in one sentence: `forall(T).` contextually forms an anonymous polymorphic type expression in a type-expression position; `forall` is never a declaration keyword or a file-backed declaration-name mechanism, and outside that contextual form the spelling remains an ordinary identifier.

> Generic declarations are supported only for structs, classes, functions, and methods. Their type parameters are introduced exclusively by `@co.dap.generic`.

The following declaration-head generic forms are invalid:

```
// Cache.fol
_(T) co.lang.module = {}            // compiler error
// operations.unit.fol
_(F(_)) co.lang.unit = {}           // compiler error
Callback(T) co.lang.delegate = (T)->(T); // compiler error
```

A parameterized `co.lang.type` declaration is the separate type-constructor form and does not use `@co.dap.generic`:

```
// option.unit.fol
_ co.lang.unit = {
    Option(T) co.lang.type =
        Some(T) | None();
}
```

Generic structs and classes remain file-backed primary declarations. Their names come from filenames:

```
// LinkedList.fol
@co.dap.generic(types=[{name=T}])
_ co.lang.struct = {
    value T;
    next  LinkedList;
    prev  LinkedList;
}

myIntList LinkedList = LinkedList.withTypes(co.lang.int);
```

```
// Employee.fol
@co.dap.generic(types=[{name=T}, {name=R}])
_ co.lang.class = {
    id   T;
    name R;
}

emp Employee = Employee.new(co.lang.int, co.lang.string).init;
```

Generic functions use the same annotation but are declared inside a legal function-owning context such as an ordinary unit, class, or companion unit:
//sommGen1.unit.fol

```
_ co.lang.unit = {

    @co.dap.generic(types=[{name=T}, {name=R}])
    add(a T, b T)->(R) = {
        ...
    }
    someFun()->()={
        add_int_int := add.withTypes(co.lang.int,co.lang.int);

        //    or

        add_int_int co.lang.function =  add.withTypes(co.lang.int,co.lang.int);

        k := add_int_int(12,10);
    }
}
```

Specialization

`@co.dap.specialize` to specialize generics for specific types upfront //sommGen2.unit.fol

```
_ co.lang.unit = {

    @co.dap.generic(
        types=[
            {name=T}
        ],
        requires=[
            co.lang.Add(left=T, right=T, result=T)
        ]
    )
    add(a T, b T)->(T) = {
        this.return a + b;
    }
}
```

for the above generic want to specialize for `co.lang.int` //sommGen5.unit.fol

```
_ co.lang.unit = {
    @co.dap.specialize(
        target=add,
        types=[
            {name=T, type=co.lang.int}
        ]
    )
    addInt(a co.lang.int, b co.lang.int)->(co.lang.int) = {
        this.return co.intrinsic.intAdd(a, b);
    }
}
```

`folang` provides partial specialization below is the example for partial specializationn //sommGen7.unit.fol

```
_ co.lang.unit = {
    @co.dap.generic(
        types=[
            {name=T},
            {name=R}
        ]
    )
    transform(value T)->(R) = {
        ...
    }
}
```

//sommGen8.unit.fol

```
_ co.lang.unit = {
    @co.dap.specialize(
        target=transform,
        types=[
            {name=T, type=co.lang.string},
            {name=R}
        ]
    )
    transformString(value co.lang.string)->(R) = {
        ...
    }
}
```

***fields of specialize**

Attribute	Values
target	the generic fully qualified name includes package name if omitted it is current package
types	resoultion types
priority	
strategy	intrinsic

Generic Declarations and Type Constructors

FoLang distinguishes annotation-based generic declarations from parameterized `co.lang.type` constructors.

Generic Structs, Classes, Functions, and Methods

`@co.dap.generic` is the sole mechanism for declaring generic structs, classes, functions, and methods. Generic parameters for these declaration kinds must not appear in the declaration head.

```
// Box.fol
@co.dap.generic(types=[{name=T}])
_ co.lang.struct = {
    value T;
}
```

```
// conversion.unit.fol
_ co.lang.unit = {
    @co.dap.generic(types=[{name=T}, {name=R}])
    convert(value T)->(R) = {
        ...
    }
}
```

These forms are invalid:

```
// Box.fol
_(T) co.lang.struct = { ... }        // compiler error
// Container.fol
_(T) co.lang.class = { ... }         // compiler error
```

`co.lang.type` Constructors

A `co.lang.type` constructor does not use `@co.dap.generic`. Its type parameters appear directly in the type declaration head, and the declaration must be inside an ordinary unit or another explicitly legal type-declaration context.

```
// option.unit.fol
_ co.lang.unit = {
    Option(T) co.lang.type =
        Some(T) | None();
}
```

`Option` denotes a unary type constructor:

```
Option : Type -> Type
```

Applying it produces a type: //applyingEg1.unit.fol

```
_ co.lang.unit = {
    someFun()->()={
        value Option(co.lang.int);
        employeeOption Option(Employee);
    }
}
```

`Some(T)` and `None()` are value constructors declared by the `Option` definition itself. They do not require separate function implementations.

`@co.dap.generic` is invalid on `co.lang.type`, and declaration-head type parameters are invalid on structs, classes, functions, methods, signatures, interfaces, modules, enums, unions, cstructs, units, and other declaration kinds unless a later specification version explicitly adds support.

No Type-Constructor or Type-Function Annotation

FoLang requires neither `@co.dap.typeconstructor` nor `@co.dap.typefunction`.

```
Option(T) co.lang.type = ...
    -> recognized syntactically as a type-constructor declaration

ElementType(container co.lang.type)->(co.lang.type) = ...
    -> recognized syntactically as a type-level function
```

The declaration form already determines the category unambiguously.

Templates

Typed

```
// myttypedtemplate.unit.fol

_ co.lang.unit = {
    @co.dap.template
    add(a co.lang.int, b co.lang.int)->(co.lang.int) ={
        this.return a + b;
    }
}
```

Untyped

```
// MyTemplate.unit.fol

_ co.lang.unit = {

    @co.dap.template
    add(a, b)->(co.lang.untyped) ={
        this.return a + b;
    }
}
```

Annotations and Decorators

```
// Annotation — static object, can carry data


// myAnnotation.fol

_ co.lang.object->(for=annotation) = {
    value   co.lang.string;
    enabled co.lang.bool;
}

// Decorator — function, transforms target, returns
// Decorator.unit.fol

_ co.lang.unit = {

    @co.dap.decorator
    myDecorator(target co.lang.function)->(co.lang.function) = { }
}

User-defined directives and pragmas cannot be declared. They are language-internal metadata categories. FoLang provides declaration constructs
for user-defined annotations and decorators only.
```

User-defined annotation/object liveness is defined in Unused Symbols, Liveness, and Reachability.

Built-in Metadata Parsing

Built-in directives, annotations, pragmas, and decorators under `@co.*` use one generic metadata application grammar:

```
annotation = "@", qualified-name,
             [ "(", [ annotation-argument-list ], ")" ] ;
```

The parser must collect and preserve the complete metadata application, including every supplied positional argument, named argument, field, attribute, and argument expression. Examples do not constrain parser acceptance or which fields are collected.

If no semantic implementation handles a collected built-in metadata form, the form and its collected arguments remain syntactically valid and have no semantic effect. If a semantic handler exists, semantic resolution validates the supported field schema, values, defaults, target restrictions, and other form-specific rules. A malformed metadata argument list remains a syntax error under the generic metadata grammar.

User-defined metadata outside `co.*` is limited to annotations and decorators and follows ordinary declaration, import, and name-resolution rules. FoLang provides no user declaration construct for directives or pragmas.

Macros

```
// a. Basic macro
//macro1.unit.fol

_ co.lang.unit = {

    @co.dap.macro
    say()->()={ this.return co.macro.quote({ println("Line 1") println("Line 2") }); }

    // b. Escape assign
    @co.dap.macro
    yes_esc_assign()->(co.lang.untyped)={
        this.return co.macro.quote({
            co.macro.esc(y) = 42;
            co.out.println("Inside macro: y = ", y);
        });
    }

}

// c. Debug macro with gensym

// macro2.unit.fol

_ co.lang.unit = {
    @co.dap.macro
    debug(expr)->(co.lang.untyped)={
        tmp := co.macro.gensym(co.lang.var, "tmp");
        this.return co.macro.quote({
            tmp = co.macro.esc(expr);
            co.out.println("Result: ", tmp);
            tmp;
        });
    }

    // d. if/else condition macro
    @co.dap.macro(
        group = {items:["if","else"], chain:true},
        sugarform={forms:["if expr block"]},
        bind={vars:["x"]},
        isolate={vars:["temp", "index"]},
        gensym={prefix:"tmp_"},
        hygienic=true,
        argtransform={param:"body", wrap:"lambda", whentype:"block"},
        desugar={exprs:["if($cond) { $block }" => "if($cond,$block)"]},
        mode="inject"
    )
    if(condition expr, body block)->()={}

    blockormacro co.lang.kind = block | macro

    @co.dap.macro(
        group={items:["if","else"], chain:true},
        sugarform={forms:["else block","else if"]},
        chainswith={macro:"if", position:"immediate", required:true},
        argtransform={param:"body", wrap:"lambda", whentype:"block"},
        standalone=false,
        desugar={exprs:[
            "else if($cond) { $block }" => "else(if($cond, $block))",
            "else { $elseblock }" => "else($elseblock)"
        ]},
    )
    else(body blockormacro)->()={}
}
```

Other macro utilities: 1. `@co.dap.compose(using=["base_if", "blockify"])` 2. `@co.dap.guard(expr="is_bool_expr(expr)")` 3. Quasiquote macros use `co.macro.quote` and `co.macro.unquote`

Collections

```
x co.core.List->(co.lang.string) = co.core.List["A","B","C"];

y co.core.Set->(co.lang.int) = co.core.Set(1,2,3);

map co.core.Map->(key=co.lang.string, val=co.lang.int) = co.core.Map{"A":1, "B":2, "C":3};

arr co.core.Array->(dims=2,type=co.lang.int, sizes=[2,4]);

matr co.core.Matrix->(rows=2,cols=4,type=co.lang.float);

//variable with type deduction

y := co.core.Set->(co.lang.int)(1,2,3);

x := co.core.List->(co.lang.string)["A","B","C"];

map := co.core.Map->(key=co.lang.string, val=co.lang.int){"A": 1, "B": 2, "C": 3};
```

Dynamic Multi Dispatch

FoLang distinguishes **static overload dispatch** from **dynamic multiple dispatch**. Ordinary overload resolution is static by default. `@co.ddap.dynamicdispatch(true)` is an application-wide directive that changes eligible overload-family selection in the application's **open compilation graph** to use runtime argument types.

This feature is distinct from ordinary class/interface virtual dispatch. Virtual dispatch selects a receiver implementation from the runtime receiver type; dynamic multiple dispatch selects among an overload family using the runtime type tuple of all participating arguments. Operator expressions are not reclassified into this named-callable dynamic-dispatch model: operator implementations continue to use the exact normalized operand-type resolution defined in Operators, and `@co.ddap.dynamicdispatch` does not introduce subtype widening for operator selection.

Dynamic multiple dispatch does not alter FoLang's ordinary overload-family signature contract: eligible instantiated overloads are distinguished only by applicable parameter signatures, return types never participate in selection, and sibling overloads in the family must have the identical declared return signature. Generic mappings have already been resolved at compile time and are not re-evaluated from runtime types. Dynamic selection may change the implementation that executes, but it cannot change the call's compile-time result contract.

Example Type Hierarchy

```
// Animal.fol
_ co.lang.class = {
}
```

```
// Dog.fol
@co.dap.oops(
    Animal: {inherits:true}
)
_ co.lang.class = {
}
```

```
// Cat.fol
@co.dap.oops(
    Animal: {inherits:true}
)
_ co.lang.class = {
}
```

```
// Human.fol
@co.dap.oops(
    Animal: {inherits:true}
)
_ co.lang.class = {
}
```

```
// collisions.unit.fol
_ co.lang.unit = {
    collide(a Animal, b Animal)->(co.lang.bool) = {
        ...
    }

    collide(a Dog, b Cat)->(co.lang.bool) = {
        ...
    }

    collide(a Dog, b Human)->(co.lang.bool) = {
        ...
    }

    collide(a Human, b Cat)->(co.lang.bool) = {
        ...
    }

    checkOverload()->() = {
        some1 Animal = Dog{}.init();
        some2 Animal = Cat{}.init();
        some3 Animal = Human{}.init();

        collide(some1, some2);
        collide(some1, some3);
        collide(some3, some2);
    }
}
```

Without dynamic dispatch, the static argument tuple at each of the three calls is `(Animal, Animal)`, so static overload resolution selects `collide(Animal, Animal)` even though the runtime objects may be `Dog`, `Cat`, or `Human`.

An explicit cast changes the static type presented to ordinary overload resolution:

```
collide((Dog)some1, some2);
```

The static tuple is then `(Dog, Animal)`. If there is no exact `collide(Dog, Animal)`, ordinary applicability/widening rules search supertypes and may select `collide(Animal, Animal)` as the unique most-specific applicable overload.

Enabling Dynamic Multiple Dispatch

`@co.ddap.dynamicdispatch(...)` is an **executable-application-only** semantic directive. It may be declared only for the application rooted at `src/appl.fol`. Using it in `src/component.fol`, in any standalone library, or in any `components/<kind>/component.fol` is a compiler error. Components are not libraries, but they are equally forbidden from enabling this application-wide semantic mode.

```
// src/appl.fol
@co.ddap.dynamicdispatch(true)

// imports and application code
```

No standalone library or project-local component may independently enable dynamic multiple dispatch. **Projected** libraries/components retain ordinary static overload bindings internally because their implementation remains hidden behind surface APIs. Packaged producers also compile without independently enabling dynamic dispatch, but their explicitly exported package contexts are different: when those contexts are later merged into an executable application's open graph, eligible packaged call sites inherit that application's dynamic-dispatch mode. This restriction is independent of ordinary class/interface virtual receiver dispatch, which retains its separately defined semantics.

When the executable application enables dynamic multiple dispatch, an eligible call in the application's open semantic graph uses the actual runtime type tuple instead of the static tuple. For example, if the runtime tuple is `(Dog, Cat)`, `collide(Dog, Cat)` is selected when that overload exists.

Applicability and Widening

Dynamic multiple dispatch uses the **same applicability, nominal widening, specificity, and ambiguity rules as ordinary static overload resolution**. The difference is only the source of the selection tuple: static overload resolution uses compile-time argument types, while dynamic multiple dispatch uses actual runtime argument types.

For a runtime tuple `(R1, R2, ..., Rn)`, an overload with parameter tuple `(P1, P2, ..., Pn)` is **applicable** when every runtime type `Ri` is the same as or a subtype of `Pi` according to FoLang's applicable nominal subtype relation.

An exact runtime-type match is therefore the most specific possible match. When no exact overload exists, dispatch widens one or more positions through the type hierarchy and considers applicable parent-type signatures.

Define signature `A` as more specific than signature `B` when:

1. every parameter type in `A` is the same as or a subtype of the corresponding parameter type in `B`; and
2. at least one parameter type in `A` is a proper subtype of the corresponding parameter type in `B`.

Dynamic dispatch selects the **unique most-specific applicable overload**. As with static overload resolution, return types, arbitrary conversions, declaration order, and left-parameter priority do not participate in candidate selection.

Compile-Time Ambiguity Detection

Consider:

```
f(a Dog,    b Animal)->(...) = { ... }
f(a Animal, b Cat)->(...)    = { ... }
f(a Animal, b Animal)->(...) = { ... }
```

For runtime tuple `(Dog, Cat)`:

```
f(Dog, Animal)  applicable
f(Animal, Cat)  applicable
f(Animal, Animal) applicable
```

The first two are each more specific than `f(Animal, Animal)`, but neither is more specific than the other. The tuple is therefore ambiguous.

FoLang does not use left-parameter priority, declaration order, or a runtime guess to resolve this case. When `@co.ddap.dynamicdispatch(true)` is enabled, the frontend constructs the application's closed dynamic-dispatch type universe from the application-owned types plus types in package contexts explicitly exported from packaged components and packaged `.folenc` artifacts. Projected-library/component internal types are excluded because those domains remain black boxes.

For each dynamically dispatched call, the frontend uses that prepared open type hierarchy and the applicable overload family to validate that every runtime-capable type combination admitted by the call's static parameter bounds has one unique most-specific overload. Multiple incomparable best candidates are a compiler ambiguity error. If no applicable overload exists for an admitted runtime type combination, compilation likewise fails unless the applicable language rule for that call form defines another explicit fallback.

The frontend is responsible for establishing and validating this semantic type/overload set. It does not prescribe the runtime lookup representation.

Binary Generation

After the frontend has validated the dynamic-dispatch universe and emitted the required semantic information in the Final AST/backend interchange, the backend may implement dispatch with tables, type-ID matrices, decision trees, inline caches, direct-call specializations, or another equivalent representation.

```
statically proven concrete tuple
    -> direct call is permitted

runtime-polymorphic tuple
    -> precomputed dynamic dispatch selection
```

The implementation strategy is not observable language semantics provided the selected overload is the one required by these rules.

Open Application Graph

The application-wide directive applies to code participating directly in the application's open semantic graph:

```
ordinary src/ application packages
+
packages explicitly selected by components/packaged/component.fol
+
explicitly exported packaged/open package contexts imported from lib/*.folenc
```

Exported packaged code is intentionally not protected by a projected API boundary. Only explicitly exported packaged package contexts join the application graph; unexported packaged packages remain private to their producer/component. The exported contexts' classes, type hierarchy, overload families, and applicable AST/HIR participate in the application graph. Consequently, when the application enables dynamic multiple dispatch, eligible overload calls **inside participating exported packaged code** are also resolved under dynamic-dispatch semantics.

A packaged library authored under assumptions of static overload dispatch can therefore change behavior or become ambiguous when consumed by a dynamic-dispatch application. This is an intentional consequence of choosing packaged/open integration rather than a projected boundary.

A packaged `.folenc` must retain enough canonical type hierarchy, overload-family, call-site, symbol/context, and applicable AST/HIR information for the final consuming application to perform this integration and validation.

Library and Component Dispatch Boundaries

The following rules apply uniformly to standalone libraries and project-local components:

1. no library or component may declare `@co.ddap.dynamicdispatch(...)`; doing so is a compiler error;
2. projected-library/component implementation call sites retain static overload bindings; packaged producer source cannot independently enable dynamic dispatch, but explicitly exported packaged call sites may be re-resolved under the consuming executable application's dynamic-dispatch mode after open-graph integration;
3. only the executable application may enable dynamic multiple dispatch;
4. an application's dynamic-dispatch mode does not penetrate projected boundaries; projected `application`, `ffi`, `system`, and `dynamicvmrt` implementations remain black boxes whose internal overload call sites retain their statically compiled bindings;
5. packaged producers are the exposure-model exception only for package contexts explicitly selected by `@co.dap.export(...)`: those exported contexts join the consuming application's open semantic graph and therefore follow the consuming application's dynamic-dispatch mode, while the packaged producer itself still cannot enable dynamic dispatch;
6. `components/operators/` is parser/operator infrastructure rather than a library and does not create a dynamic-dispatch domain; and
7. the same isolation/exposure rules apply whether the producer is standalone or project-local.

In particular, `dynamicvmrt` remains an isolated projected domain. Runtime-created or runtime-loaded types may inherit compiled types that are legitimately visible **inside that `dynamicvmrt` domain**, but those private runtime types do not become new members of the consuming application's compiled dynamic-multidispatch type universe and do not extend application overload families implicitly. Internal `dynamicvmrt` function overload binding remains static; the runtime executes the target established by those static overload rules, apart from any separately defined ordinary OOP virtual receiver dispatch.

Projected standalone libraries and projected project-local components (`application`, `ffi`, `system`, `dynamicvmrt`) therefore remain isolated behind their surface APIs. Their private packages, private types, internal overload families, and internal call sites are not reinterpreted by the owning/consuming application's `@co.ddap.dynamicdispatch(true)`.

Execution Models and Control Abstractions

Foλang executes ordinary code **sequentially by default**. A normal function or method declaration therefore requires no execution-model decorator merely to be called sequentially.

The built-in decorator `@co.dap.executionmodel(...)` is used only when a declaration requires non-default execution semantics that must remain observable across conforming FoLang implementations. The language exposes an execution-model choice only when that choice changes required FoLang behaviour. A distinction that changes only a backend's internal implementation strategy is not a separate FoLang execution model.

This gives the following separation:

```
source-level execution semantics
    -> @co.dap.executionmodel(...)

concurrent / parallel / asynchronous submission and communication
    -> co.cpca

continuation / CPS control operations
    -> co.control

backend/runtime implementation
    -> threads, pools, virtual/green threads, event loops, processes,
       work-stealing schedulers, or another conforming mechanism
```

A backend may choose any internal mechanism that preserves the semantics required by the declaration. For example, a logical FoLang task may be implemented using an operating-system thread, a thread pool, a virtual thread, a fiber-like runtime mechanism, an event loop, or another conforming scheduler. Such implementation choices do not become FoLang source-level execution kinds merely because a backend uses them.

Execution-model declarations use ordinary function/method declaration syntax and expose a callable signature, but the frontend represents them with a specialized execution-model declaration node rather than reducing them to an ordinary function node. The specialized node preserves the execution semantics explicitly for semantic analysis and backend interchange. This AST representation does not remove the declaration's callable function/method interface at the language level.

The same principle applies to other function-shaped special declarations such as templates, macros, and native functions: shared surface syntax does not require the frontend to represent every declaration with the same AST node.

Default Sequential Execution

Sequential execution is the default. FoLang therefore does **not** define `@co.dap.executionmodel(type=sequential)` as a standard execution-model form; an ordinary undecorated function or method already has sequential semantics.

```
calculate(a co.lang.int)->(co.lang.int) = {
    this.return a + 1;
}

value := calculate(10);
```

The call proceeds as an ordinary FoLang function call according to the normal call, evaluation-order, error, and return-value rules.

Concurrent Execution

`concurrent` describes work whose execution may overlap with other work. The standard source-level kinds are deliberately small:

- `task` — a logical schedulable unit of work. This is the preferred general concurrent abstraction and leaves the backend substantial scheduling freedom.
- `thread` — requests thread-semantic execution when thread identity, affinity, thread-local behaviour, or another specification-defined thread property is intentionally observable.

Backend-specific names such as `goroutine` or `greenlet` are not FoLang execution kinds. `subroutine` is ordinary callable code, while `generator` and `coroutine` primarily describe suspension/resumption behaviour rather than an independent concurrent execution carrier. `fiber` likewise is not exposed merely because a backend may use a fiber-like mechanism internally. Such abstractions may exist as ordinary package/runtime facilities when separately defined, but they are not synonyms for `concurrent.kind`.

```
// concurrent.unit.fol
_ co.lang.unit = {
    @co.dap.executionmodel(type=concurrent, kind=task)
    someConcurrent(a co.lang.int)->(co.lang.int,co.lang.error) = {
        ...
    }
}

co.cpca.submit(someConcurrent, params=[10], results=[val, errors]);
(errors.isEmpty).then(
    co.out.println(val)
).default(
    co.out.println(errors)
);
```

A declaration may request thread semantics explicitly when those semantics are required:

```
_ co.lang.unit = {
    @co.dap.executionmodel(type=concurrent, kind=thread)
    threadBoundWork(a co.lang.int)->(co.lang.int,co.lang.error) = {
        ...
    }
}

co.cpca.submit(threadBoundWork, params=[10], results=[val, errors]);
```

Scheduling is a separate dimension from execution kind. Where supported by the selected kind, an optional scheduling policy may be declared:

```
@co.dap.executionmodel(
    type=concurrent,
    kind=task,
    scheduling=cooperative
)
```

The standard scheduling values are `cooperative` and `preemptive`. When the field is omitted, the backend/runtime may choose any conforming scheduling policy. A structural policy such as `fork-join`, when requested, is likewise separate from both the execution kind and the scheduling policy; omitting it provides no fork-join guarantee.

Parallel Execution

`parallel` states that the declaration has parallel-execution semantics. FoLang does not currently require a `parallel.kind` merely to expose an implementation mechanism:

```
@co.dap.executionmodel(type=parallel)
parallelWork(data SomeData)->(Result) = {
    ...
}
```

`process`, `fork`, `spawn`, and `exec` are not parallel execution kinds. A process is an execution/isolation container, while `fork`, `spawn`, and `exec` are process/runtime operations. They remain ordinary facilities of the applicable `co.cpca`, `co.os`, or system API rather than values of `@co.dap.executionmodel(type=parallel, kind=...)`.

A conforming backend may implement parallel semantics with threads, worker pools, processes, accelerators, or another mechanism when the program cannot observe a forbidden difference.

Asynchronous Execution

`async` describes execution whose completion is not required to block the submitting context. Completion representation is separate from the execution model:

```
@co.dap.executionmodel(type=async, completion=future)
loadData(request Request)->(Data, co.lang.error) = {
    ...
}
```

The standard completion forms are:

- `future` — the caller receives or is associated with a future completion value;
- `callback` — completion is delivered through a compatible callback mechanism.

`promise` is not a separate execution-model value. If a producer-side completion object is required by a particular runtime/API, it is an implementation or package abstraction rather than another async execution kind. `await` is a consumption or control operation supplied by `co.cpca`; it is not an execution-model kind.

An actor is a concurrency abstraction with state/message semantics, not an async completion kind. Channels are communication abstractions, events are notification abstractions, and distributed scaling is a deployment/runtime policy. These may be provided through `co.cpca`, but none is a top-level `@co.dap.executionmodel` type merely because it participates in concurrent or asynchronous programs.

Continuations and Control

Continuation semantics are deliberately separated from concurrent, parallel, and asynchronous execution facilities. A declaration may use:

- `kind=full` — full/undelimited continuation semantics;
- `kind=delimited` — continuation capture is bounded by a delimiter.

Continuation and CPS operations are supplied by `co.control`, not `co.cpca`.

```
@co.dap.executionmodel(type=continuation, kind=delimited, control="shift-reset")
continuationWork(...)->(...) = {
    ...
}
```

For delimited continuations the currently documented control families are `shift-reset`, `prompt-control`, and `spawn-yield`. `csp` is not a continuation kind; Communicating Sequential Processes belong to the concurrency/communication domain.

`spawn-yield` has a specific continuation meaning in FoLang and must not be confused with an ordinary process/task spawn operation or with a scheduler-only yield. In the `spawn-yield` control family, `spawn` establishes an **implicit delimited-control boundary**. When execution reaches `yield`, the continuation from that suspension point back to the spawned control boundary is preserved internally by the control runtime. The suspended execution can later resume from the yield point.

Conceptually:

```
spawn
  └── establishes implicit continuation delimiter
        └── A
            └── B
                └── yield
                      └── preserve/suspend continuation internally
                          up to the spawned control boundary
                              ↓
                           resume
                              ↓
                      continue after yield
```

Unlike `shift-reset` and `prompt-control`, `spawn-yield` does not require the captured continuation to be exposed directly to application code as an explicit continuation value such as `k`. The continuation may remain an internal control-runtime object. The observable contract is suspension at `yield`, preservation of the delimited execution state, and later resumption from that suspension point according to the applicable `co.control` operation.

This distinction is important:

```
ordinary co.cpca / process spawn
    -> creates or submits concurrent execution

ordinary scheduler yield
    -> voluntarily gives execution opportunity to another schedulable unit

co.control spawn-yield
    -> establishes an implicit delimited continuation boundary and
       suspends/resumes execution by preserving the continuation internally
```

Accordingly, the word `spawn` may occur in more than one conceptual domain, but the package and operation determine its semantics. Concurrent/process creation remains under the applicable `co.cpca`, `co.os`, or system API. Continuation-oriented `spawn-yield` operations belong to `co.control`.

Continuation kind	Control family	Delimiter model	Continuation exposure	Meaning
`full`	supplied by the applicable `co.control` full-continuation API	no delimited boundary	according to the full-continuation API	captures the continuation according to the full-continuation contract
`delimited`	`shift-reset`	explicit nearest/applicable `reset` boundary	exposed according to Shift/Reset semantics	captures only the continuation bounded by `reset`
`delimited`	`prompt-control`	explicit/applicable prompt boundary	exposed according to Prompt/Control semantics	captures only the continuation bounded by the applicable prompt
`delimited`	`spawn-yield`	implicit spawned control boundary	internal by default	suspends at `yield`, preserves the bounded continuation internally, and later resumes from the yield point

Parameter table for delimited continuations

Operator Pair	Delimiter Targeting	Captures Delimiter into k?	Keeps Delimiter on Stack During $expr$?	Primary Modern Use Case / Paradigm
Shift / Reset	Nameless (Closest)	**Yes** (Implicitly)	**Yes**	Standard algebraic effects, functional stream processing
Prompt / Control	Nameless (Closest)	**No**	**Yes**	Dynamic context modification, legacy Scheme systems
Spawn / Yield	Implicit (Fiber Bound)	**Internal**	**No**	Cooperative multitasking, async/await engines, generators

Example declaration:

```
@co.dap.executionmodel(
    type=continuation,
    kind=delimited,
    control="spawn-yield"
)
resumableWork(...)->(...) = {
    ...
}
```

The decorator identifies the required continuation family; the actual suspension, yield, resume, and continuation-control operations are supplied by `co.control`.

Canonical Submission and Placement

`co.cpca.submit(...)` is the canonical submission operation for declarations whose execution is governed by `co.cpca`. The execution-model decorator states the required semantics; the submission call requests execution of that declaration.

A specialized operation such as `submitToPool`, `submitThread`, or `submitToEventLoop` is justified only when the standard `co.cpca` API explicitly defines an additional observable placement/resource constraint. Such a function must not exist merely as another spelling of `submit(...)`. If no additional semantic guarantee is defined, `submit(...)` is used and the backend/runtime is free to select the underlying mechanism.

Application Pool Resource Pragmas

Application pragmas may configure execution-resource limits without changing the execution semantics of the submitted declaration:

```
@co.pdap.threadpool(min=10,max=40,increment=10,queue=1000)
@co.pdap.schedularpool(min=2,max=10,increment=2,queue=5000)
```

Both runtime resource domains exist even when the corresponding pragma is omitted. `@co.pdap.threadpool` configures the application-wide resource domain used for ordinary concurrent/thread-oriented submissions. `@co.pdap.schedularpool` configures a separate application-wide resource domain used by `co.cpca.schedule(...)` and the applicable delayed, periodic, calendar-based, cron-like, or otherwise scheduled execution facilities. A scheduled submission is therefore serviced by the schedular resource domain rather than consuming the ordinary thread-pool resource domain.

The pragmas allocate and bound resources; they do not themselves define when a particular operation runs. The timing, delay, recurrence, calendar, or cron semantics of scheduled work are supplied by the applicable `co.cpca` scheduling operation.

The fields have the following resource meaning:

- `min` is the runtime's configured minimum worker capacity for that resource domain;
- `max` is a **hard upper bound** on workers/resources in that domain;
- `increment` is the amount by which the runtime may grow worker capacity while still remaining at or below `max`;
- `queue` is a **hard upper bound** on pending work accepted by that resource domain.

For the schedular resource domain, pending work includes accepted scheduled work that has not yet completed, including delayed work awaiting its trigger and runnable work waiting for execution. A recurring schedule should be represented as a recurring registration rather than by eagerly materializing every future occurrence merely to populate the pending-work queue.

When a pragma is present, neither `max` nor `queue` may be exceeded. If the permitted worker capacity is fully occupied but the queue still has capacity, the submission is accepted and queued. If worker capacity has reached `max` **and** the pending-work queue has reached `queue`, FoLang rejects the new submission immediately.

A saturated pool must not:

- create workers or equivalent runtime resources beyond `max`;
- enlarge or bypass the configured `queue` bound;
- block the submitting execution context waiting for queue capacity;
- silently redirect the work to an unbounded fallback pool.

Instead, the applicable `co.cpca` submission or scheduling operation reports a **pool-capacity rejection notification/error** through its defined result/error contract. The application developer is responsible for handling that rejection, for example by reporting overload, retrying according to application policy, dropping non-critical work, degrading functionality, or taking another application-specific action. FoLang enforces the resource boundary; it does not choose the business-level overload response.

Pool rejection is an operational capacity signal. Persistent or repeated rejection normally indicates that the configured capacity, workload characteristics, downstream latency, retry behaviour, or application work-generation rate should be investigated. Applications are expected to establish appropriate `max` and `queue` limits through capacity planning, workload analysis, load testing, and production observation rather than relying on unlimited growth.

The pragmas are applicable to applications only; they are not imposed by libraries, components, packages, or exports. A library may document resource assumptions or recommendations, but it cannot force application pool limits.

When either pragma is absent, the corresponding FoLang runtime pool still exists and operates using runtime-managed defaults. In that case FoLang defines no application hard upper bound for that pool's worker growth or pending-work capacity. This default preserves ease of use but can permit resource exhaustion under excessive or erroneous workloads. Production applications that require bounded execution resources should therefore configure explicit limits.

These application-level limits reduce the risk that one execution category exhausts resources needed by another part of the application, but they do not replace host, process, container, cgroup, service-manager, or operating-system resource controls used to isolate the machine from a misbehaving application.

Execution-Model Summary

Type	Source-level kind/model	Optional semantic dimension	Package responsible for operations
default sequential	—	—	ordinary function/method call
`concurrent`	`task`, `thread`	`scheduling=cooperative|preemptive`; structural policies such as `fork-join` when separately defined	`co.cpca`
`parallel`	—	parallel-specific policy only when separately specified	`co.cpca`
`async`	—	`completion=future|callback`	`co.cpca`
`continuation`	`full`, `delimited`	delimited `control="shift-reset"|"prompt-control"|"spawn-yield"`	`co.control`

The following are intentionally **not** execution-model types or kinds:

```
backend/runtime mechanisms:
    goroutine, greenlet, virtual thread, event-loop implementation,
    work-stealing implementation

ordinary process/runtime operations:
    process, fork, spawn, exec

completion/control operations:
    await

communication/notification abstractions:
    channel, event

higher-level concurrency abstractions:
    actor, CSP

control/suspension abstractions unless separately specified:
    coroutine, generator, fiber, yield/resume

deployment/runtime policy:
    distributed scaling, scale up/down/out/in
```

The governing rule is:

> **Expose an execution-model choice in `@co.dap.executionmodel` only when the choice changes externally observable FoLang semantics. If two choices differ only in how a backend implements the same required behaviour, they are not separate FoLang execution-model choices.**

Native Code (system capability)

`@co.dap.native` marks a function or method declaration as a **native implementation declaration**. It does not grant native capability merely because the annotation is present. The annotation is valid only inside a `system` library/component domain when the installation permits the system capability.

A system library may use the `co.native` package to express low-level implementation such as assembly or machine-level operations through facilities including `co.native.asm` and `co.native.inline`. This is the FoLang facility analogous to embedding an assembly block in a low-level systems language.

The frontend preserves a native declaration as a specialized declaration node and carries its required semantics through the backend interchange contract. The reference backend demonstrates one implementation of that contract. A conforming third-party backend is not required to reproduce the reference backend's internal native-code lowering, instruction representation, allocation strategy, or code-generation mechanism, but the externally observable behavior required by the specification must be preserved.

`ffi` is a separate foreign-function-interface domain. It is used to describe and invoke foreign functions through extern declarations, C-ABI-compatible types, calling conventions, symbol linkage, pointer/address forms, and any permitted FoLang-side marshalling or invocation code. An `ffi` library does **not** gain `@co.dap.native` or direct FoLang assembly/machine-code implementation capability merely because FFI itself permits low-level ABI types.

Native Functions

// native.unit.fol

```
_ co.lang.unit = {
    @co.dap.native
    nativeMethod(a co.lang.int, b co.lang.int)->(co.lang.int) ={
        // native/system implementation
    }
}
```

Dynamic Runtime (dynamicvmrt capability)

The `@co.ddap.dynamicruntime` directive enables full access to the `co.meta` package. It is valid **only inside a `dynamicvmrt` capability domain**: a standalone `@co.dap.library(type=dynamicvmrt)` project or the project-local `components/dynamicvmrt/` component. Using `@co.ddap.dynamicruntime` in an executable application, packaged code, an application projected library/component, an `ffi` domain, a `system` domain, or any other source context is a compiler error.

Within a valid `dynamicvmrt` domain, the directive enables dynamic class and type loading, monkey patching, runtime reflection, instrumentation, eval-based code execution, and other defined dynamic-runtime/metaprogramming capabilities through `co.meta`. These capabilities remain inside that projected dynamic-runtime boundary and do not automatically escape into ordinary application, packaged, or other library/component code.

1. runtime type creation from strings, streams, files, and other supported dynamic-runtime inputs
2. complete reflection and introspection
3. Runtime code modification add/remove/update methods etc.

> When a library is marked `dynamicvmrt` and enables `@co.ddap.dynamicruntime`, the final binary includes the runtime support required to create and manage dynamic types, methods, and objects.

> Dynamically created objects may interact with compiled types according to the dynamic-runtime boundary rules; ordinary compiled code does not gain unrestricted reverse access to dynamic-runtime facilities.

> Through surface API's they connect to runtime and through runtime apis invoke the dynamic type and the results will be returned are compiled types only.

> Runtime can directly provide handle to compiled types to dynamic code running inside it.

> Loaders are the dynamic-runtime containers used to manage these runtime-created objects and types.

> Folang provides BasicLoader but you can extend this as follows

//MySpecLoader.fol

```
@co.dap.extends(co.meta.BaseLoader)
_ co.lang.loader={


}
```

> The basic loader provides the operations required to create, update, delete, and manage runtime-created objects.

> Loaders form a hierarchy. When a referenced runtime type is not found in the current loader realm, lookup proceeds through the base-loader chain and finally to the compiled-type environment.

Variable Kinds Support

Kind	Where
Normal	All
Pointers	Library of type `system` and `ffi`
Arrays	All
References Heap, Lvalue, Rvalue	Library of type `system` and `ffi`
Addresses	Library of type `system` and `ffi`
Thunks	application-domain code and any other context whose capability rules explicitly permit thunks
Ranges	All
Slices	All

Builtin Data Types

Type	Kind
`co.lang.string`	
`co.lang.int`	
`co.lang.bit`	
`co.lang.double`	
`co.lang.float`	
`co.lang.long`	
`co.lang.byte`	
`co.lang.char`	
`co.lang.any`	
`co.lang.dynamic`	
`co.lang.auto`	
`co.lang.bool`	
`co.lang.void`	
`co.lang.value`	value types stores values when take snapshot
`co.lang.typed`	
`co.lang.untyped`	
`co.lang.word`	
`co.lang.MatchBindings`	
`co.lang.tag`	
`co.lang.typevalue`	
`co.lang.uninit`	

`co.lang.error`	
`co.lang.literal`	literal representation for simple and compound literal objects
`co.lang.operator`	declaration kind valid only in the `components/operators/component.fol` component context; parsed by the common FoLang parser and invalid in all other source contexts

A name appearing in this registry is not necessarily an enabled source-language feature. A built-in kind is usable only when this specification defines its declaration syntax and semantics. An undocumented or explicitly reserved kind remains unavailable and must produce an unsupported-feature diagnostic when used.

Built-in Directives

Kind		
`PRAGMA`	"@co.pdap.threadpool","@co.pdap.schedularpool"	
`DIRECTIVE`	"@co.ddap.import", "@co.ddap.dynamicruntime", "@co.ddap.use", "@co.ddap.alias","@co.ddap.dynamicdispatch","@co.ddap.overload"	`@co.ddap.overload` is different from `@co.dap.overload` it has takes whether `paramtypes` or `paramandreturntypes` as attributevalue of `strategy`
`ANNOTATION`	"@co.dap.template", "@co.dap.macro","@co.dap.operator", "@co.dap.annotation", "@co.dap.library", "@co.dap.module", "@co.dap.native", "@co.dap.class", "@co.dap.static","@co.dap.instance", "@co.dap.object", "@co.dap.inline","@co.dap.ctfe", "@co.dap.friend", "@co.dap.sealed", "@co.dap.extension","@co.dap.override", "@co.dap.virtual", "@co.dap.abstract", "@co.dap.delegate", "@co.dap.dynamicscope","@co.dap.lexicalscope","@co.dap.staticscope","@co.dap.mixedscope", "@co.dap.typeclass","@co.dap.matcher", "@co.dap.constructor", "@co.dap.oops","@co.dap.extends","@co.dap.hokrlt", "@co.dap.indexer", "@co.dap.generic", "@co.dap.comptime", "@co.dap.typefromvalue", "@co.dap.local", "@co.dap.private","@co.dap.public","@co.dap.package","@co.dap.protected","@co.dap.internal","@co.dap.export","@co.dap.eager", "@co.dap.lazy", "@co.dap.packed", "@co.dap.declare","@co.dap.simd", "@co.dap.reflection", "@co.dap.mop","@co.dap.nested","@co.dap.inner","@co.dap.final","@co.dap.const","@co.dap.decorator","@co.dap.specialize"	//mop => meta object programming
`DECORATOR`	"@co.dap.before", "@co.dap.after","@co.dap.around", "@co.dap.onErrExcept", "@co.dap.InvokeAlways","@co.dap.HandleEffect", "@co.dap.defer","@co.dap.callable", "@co.dap.executionmodel"	

Builtin Kinds

Kind	Purpose
`co.lang.type`	
`co.lang.struct`	
`co.lang.cstruct`	
`co.lang.class`	
`co.lang.interface`	all abstract methods
`co.lang.union`	
`co.lang.object`	
`co.lang.instance`	
`co.lang.matcher`	
`co.lang.loader`	
`co.lang.trait`	interfaces with default implementations
`co.lang.mixin`	abstract classes alias
`co.lang.extension`	reusable implemented functions that can be composed with supported classes without inheritance
`co.lang.delegate`	
`co.lang.typeclass`	
`co.lang.module`	
`co.lang.unit`	stateless file-level container; ordinary units merge into the package namespace and `*.comp.unit.fol` attaches to a struct
`co.lang.block`	
`co.lang.signature`	
`co.lang.function`	
`co.lang.newtype`	
`co.lang.opaquetype`	

`co.lang.subtype`	
`co.lang.supertype`	
`co.lang.dependentType`	result kind of a value-indexed type-level function
`co.lang.refinementType`	base type restricted by a Boolean predicate over the candidate value
`co.lang.associatedType`	signature associated-type requirement or matching-module associated-type binding
`co.lang.data`	
`co.lang.enum`	
`co.lang.component`	structural surface/container valid only in `src/component.fol` and standardized `components/<kind>/component.fol`; source context determines projected, packaged, or operator semantics

Builtin Collections

Name	Purpose
`co.core.List`	
`co.core.Set`	
`co.core.Map`	
`co.core.Tree`	
`co.core.Trie`	
`co.core.Array`	
`co.core.Tuple`	

Builtin Operators

Arithmetic operators

`+`, `-`, `*`, `/`, `%`, `**`

Logical operators

`&&`, `||`, `!`

Bitwise operators

`&`, `|`, `^`

Comparison operators

`==`, `!=`, `<`, `>`, `<=`, `>=`

Standard operator examples

```
left  co.lang.int = 10;
right co.lang.int = 3;

notEqual co.lang.bool = left != right;  // true

bitsAnd co.lang.int = 6 & 3;            // 2
bitsOr  co.lang.int = 6 | 3;            // 7
bitsXor co.lang.int = 6 ^ 3;            // 5

mulAssign co.lang.int = 6;
mulAssign *= 3;                          // 18

divAssign co.lang.int = 18;
divAssign /= 3;                          // 6

modAssign co.lang.int = 17;
modAssign %= 5;                          // 2

powAssign co.lang.int = 2;
powAssign **= 3;                         // 8

andAssign co.lang.int = 6;
andAssign &= 3;                          // 2

xorAssign co.lang.int = 6;
xorAssign ^= 3;                          // 5

orAssign co.lang.int = 6;
orAssign |= 3;                           // 7
```

For a compound assignment `lhs op= rhs`, FoLang resolves the corresponding binary operator `op`, evaluates the left-hand location only once, and stores the resulting value back through that same location. The ordinary target-type conversion rules apply to the stored result.

Other operator and language-token spellings

`@`, `#`, `!`, `~`, `$`, `^`, `(`, `)`, `_`, `` ` ``, `?`, `{`, `[`, `]`, `}`, `\`, `:`, `;`, `"`, `'`, `=`, `.`, `?=`, `:=`, `::=`, `,`, `..`, `...`, `<..`, `..<`, `<..<`, `==>>`, `=>>`, `=>`, `->`, `<-`, `->>`, `<->`, `@@`, `+=`, `-=`, `*=`, `/=`, `%=`, `**=`, `&=`, `^=`, `|=`

Contiguous symbolic spellings that are absent from this inventory and from the active custom-operator table are unrecognized symbolic tokens; the lexer does not split them into shorter operators.

This inventory includes punctuation and reserved token spellings. Presence in this list does not make a spelling declarable as a `co.lang.operator`, nor usable with `mode=overload`, `mode=implements`, `mode=extends`, `mode=inherits` or `mode=override`.

Pre-Declared Operator Glyphs

`∪`, `∩`

Every glyph in this list is language-owned and already has fixed parse properties. It cannot be redeclared with `co.lang.operator`, but it can receive implementations through `mode=overload` in a class, struct companion unit, or built-in extension package contribution. Until a matching implementation is visible, use of the glyph fails during resolution.

// union_inter_eg.unit.fol

```
_ co.lang.unit = {

    @co.dap.generic(types=[{name=T}])
    @co.dap.operator(symbol='∪', mode=overload)
    @co.dap.extension(fortype=co.core.Set->(T), what=extends)
    union(other co.core.Set->(T))->(co.core.Set->(T)) = {
        ....
    }

    @co.dap.generic(types=[{name=T}])
    @co.dap.operator(symbol='∩', mode=overload)
    @co.dap.extension(fortype=co.core.Set->(T), what=extends)
    intersection(other co.core.Set->(T))->(co.core.Set->(T)) = {
        ....
    }
}

// Uses of the pre-declared glyphs after matching overloads are visible.
v := co.core.Set->(co.lang.int)(1, 2, 3);
p := co.core.Set->(co.lang.int)(4, 5, 2);
w := co.core.Set->(co.lang.int)(7, 8);

u := v ∪ p;
i := v ∩ p;
combined := v ∩ p ∪ w;  // same precedence and left associativity: (v ∩ p) ∪ w

z := p + v ∪ w;         // parses as p + (v ∪ w)
x := p * v ∪ w;         // parses as (p * v) ∪ w
```

Generic operator candidates do not weaken exact operator matching. The frontend may first infer generic markers from the operand type structure, instantiate the generic operand signature, and then require exact normalized operand-type equality:

```
Set<int>, Set<int>
    -> infer T = int for candidate Set<T>, Set<T>
    -> instantiate candidate as Set<int>, Set<int>
    -> exact match

Set<int>, Set<float>
    -> no single consistent T
    -> candidate not applicable
```

Nominal subtype widening, numeric promotion, and `to` / `from` conversion remain forbidden during operator candidate selection.

> for UDT like classes and companion unit there is no @co.dap.extension as a developer can directly implement into the new type upfront

See Pre-Declared Operator Glyphs.

Reserved words

`co`, `let`, `this`, `for`, and `fo` are hard-reserved words. `self` and `forall` are contextual keywords.

`self` has its language-defined meaning in every method declared by a `co.lang.class`, including the lifecycle methods `@@new` and `@@init`; outside that context it has no special class-method meaning. `forall` has its language-defined meaning only when it begins the polymorphic type-expression form `forall(...).<type-body>` in a type-expression position; outside that contextual form it is an ordinary identifier.

`fo` is permanently reserved. The language originally reserved both `co` and `fo` during its naming evolution; FoLang retains `fo` as a language-owned word rather than allowing it to become an ordinary identifier. This preserves the established lexer/parser classification and prevents programs from using `fo` as a variable or declaration name.

Difference between `this` and `self`

- `this` refers to the applicable instance/object receiver.

- `self` is available throughout class methods, including `@@new` and `@@init`.
- `self` is contextual rather than globally hard-reserved; outside a class-method context it has no special class-method meaning.
- `static` has no special shortcut; use the applicable variable or class name explicitly.
- Where the class-member rules permit access, both `self` and `this` may be used to reach class/member state according to their receiver context.

Special methods

Method	Responsibility
@@new	Lifecycle method for class
@@init	Lifecycle method for class

Builtin Packages

`co` — root (reserved word)

`co` is the root of the standard package tree supplied with FoLang. The tree is distributed as a separate language-provided `.folenc` artifact exposing the standard package contexts and is loaded automatically by the compiler/toolchain. The developer therefore receives implicit access to the `co` root without an import, while the declarations inside the artifact retain ordinary exported-package semantics.

The table below describes the current standard package hierarchy and API responsibilities. After version 1.0, the package/subpackage paths in this hierarchy are fixed, but the declarations contained inside an existing package are not frozen. Later standard-package artifact versions may add or update ordinary types, unit-level functions, methods, data structures, algorithms, modules, and other declarations without creating new FoLang grammar or a new package path.

Availability of an ordinary declaration inside `co.*` is determined by the standard `.folenc` package artifact for the applicable FoLang version, not by whether that declaration has an example in this document. After the artifact is loaded, standard-package declarations participate in the ordinary package, symbol, type, member, visibility, extension, and overload-resolution model. Package API evolution cannot manufacture new special syntax; any new grammar-level form requires an explicit specification revision to the core language grammar.

Sub-package	Responsibility
`co.lang`	All data types and kinds
`co.sys`	file, concurrent, parallel, goto, invoke, bind, call, apply, settimeout, setinterval, scheduler, cron, event
`co.os`	signal, cmd, execute, run, env, getenv, setenv, sleep, exit, cwd, chdir, fork, wait, pipe, dup, dup2, close, readfd, writefd ,random
`co.meta`	ast, instrument, transform, augment, reflect, introspect, patch, inject, create, runtime(eval,proto, prototype,etc), realm
`co.core`	List, Set, Map, Tree, Trie, Sort, Search, Array, Pointer, Ref, Address, Ptr, Matrix, Word
`co.native`	load, register, asm, inline, emit, ffi, spawnon[gpu,cpu,npu,apu,fpga,asic,tpu,mki,mcu],arch[x86,x86-64,risc,arm,vliw]
`co.in`	read, readln
`co.out`	println, print
`co.regex`	stex, pattern, match, search
`co.crypto`	rsa, aes, hash, md5, rand, uuid, ssl, tls
`co.dap`	built-in decorators, annotations
`co.ddap`	built-in directives
`co.pdap`	built-in pragmas
`co.net`	tcp, udp, http
`co.const`	`true`, `false`, `none`
`co.encoding`	base64Encode, base64Decode, json, yml, bson
`co.utils`	makeImmutable, makeShared, copyOnWrite, toSnapshot — object behaviour policies
`co.dynamic`	dynamic capabilities
`co.runtime`	
`co.compiletime`	
`co.macro`	
`co.pattern`	
`co.control`	continuation, CPS, full/delimited continuation control, shift/reset, prompt/control, and continuation-oriented control abstractions
`co.cpca`	concurrent/parallel/async submission, task/thread execution facilities, future/callback completion, await, pools, channels, events, actors, process/distributed facilities, scheduling, fiber/coroutine facilities, defer, lazy, and related execution APIs
`co.hokrlt`	
`co.operator`	

FoLang Philosophy — Uniform Object Model

Core Principle

Everything in **FoLang** is an object. That is the reason for the name **FO** — **Functional Objects**.

FoLang gives the programmer one uniform object model across all types instead of forcing them to switch between separate type families for:

- ordinary values
- immutable values
- shared/concurrent values
- copy-on-write values
- literal values

The programmer writes against a single conceptual model and opts into the required object behaviour only when needed.

Uniform Object Principle

In FoLang, **everything is an object**.

Unless explicitly stated otherwise, the reference and mutation rules in this section apply to **managed FoLang objects**. `co.lang.cstruct` remains an object in the language model, but it is an explicitly value-semantic ABI representation: assignment and parameter passing copy its value rather than a managed object reference.

This includes:

- scalar objects
- collection objects
- user-defined objects
- ADT objects
- function objects

Functions are objects in the same sense as all other values.
This is true for both **named functions** and **anonymous functions**.

FoLang does not introduce a separate semantic model for functions, scalars, UDTs, or ADTs.
They all follow the same core object principles:

- assignment of managed objects copies references
- assignment of `co.lang.cstruct` copies its value
- `==` compares values
- mutation is applied through the object
- behaviour policies such as immutability, shared, copy-on-write, and literal conversion apply uniformly where meaningful

So in FoLang, the programmer does not need one mental model for data objects and another for function values.
The language treats them under one consistent object model.

---

1. Default Object Model

Every value in FoLang is an object and is **mutable by default**.

This includes:

```
co.lang.int, co.lang.float, co.lang.string   → built-in scalar types
user-defined types (structs, classes, ADTs)  → user-defined objects
co.core.List, co.core.Map, co.core.Array     → built-in collections
```

Example:

```
positive_int co.lang.int = 10;
```

`positive_int` denotes an object.
Its current value is `10`.
By default, it is mutable.

---

2. Assignment, Aliasing, and Mutation

FoLang distinguishes three related ideas:

- **assignment**
- **aliasing**
- **mutation**

2.1 Assignment copies references

When one object variable is assigned to another, FoLang copies the **reference**, not the internal contents.

```
b := a;
```

After this, `a` and `b` refer to the same object.

2.2 Mutation changes object contents

If two names refer to the same object, mutating through one name is visible through the other.

```
// Employee.fol
_ co.lang.class = {
    Name co.lang.string
}

a Employee = { Name = "Kamesh" };
b := a;
c Employee = { Name = "Kamesh" };

a == b;   // true
a == c;   // true
b == c;   // true

b.Name = "Ramesh";   // changes a.Name too
c.Name = "Ramesh";   // does not change a.Name
```

Why:

- `a` and `b` are the **same object**
- `c` is a **different object** with the same earlier value

2.3 `==` means value equality

In FoLang, `==` compares **values**, not object identity.

That rule is uniform across all object kinds, including built-in types and user-defined types.

So:

```
a == b;
```

means:

- compare contents deeply
- return `true` when the values match
- even if `a` and `b` are not the same object

2.4 Mutation accessors

The accessor used for mutation depends on the object type:

```
scalar (int, float, bool, string, char, byte)   → .value
struct field                                    → .fieldname
class member                                    → .membername
map entry                                       → .key
array / list element                            → .index
```

Examples:

```
count.value = 30;
emp.name = "Rao";
marks.math = 95;
nums[0] = 42;
```

2.5 Function-call behaviour

Function parameters introduce a **new local binding**.

So:

- rebinding a parameter does **not** affect the caller
- mutating the object through the parameter **does** affect the caller if both still refer to the same object

```
checkPositive(a co.lang.int)->(co.lang.bool) = {
    a = 20;         // local rebinding only — caller unchanged
    a.value = 30;   // object mutation — caller sees 30
}
```

If `positive_int` is passed to `checkPositive(a)`:

```
a = 20         → local rebinding only
a.value = 30   → mutates the passed object
```

---

3. Literal Objects

All literals in FoLang create **Literal Objects**.

```
"Kumar"   → Literal Object — string
42        → Literal Object — int
true      → Literal Object — bool
```

A Literal Object is an **anonymous object created from a literal expression**.

Literal Objects participate in value equality just like every other object in FoLang:

```
10 == 10;   // true
```

In FoLang, `==` compares **values**, not object identity.

So two literal expressions with the same value compare equal, but that does not mean they are the same object.

Binding a Literal Object

When a literal is assigned to a named object, the name refers to the object created from that literal expression.

```
a co.lang.int = 10;
```

Here, `a` refers to the anonymous integer object created from literal `10`.

So:

```
a.value = 30;
```

mutates that same object, and `a` now has value `30`.

The important points are:

- literal expressions create anonymous objects
- literal-created objects are still ordinary objects unless another policy is applied
- `==` compares values, not identity
- once bound to a name, a literal-created object behaves like any other normal mutable object
- immutability applies only when explicitly requested, for example through `makeImmutable(...)`

Why Literal Objects Cannot Be Mutated Directly

Mutation can occur in two ways:

1. Through rebinding

   a co.lang.int = 10; a = 20;

   The above statement is valid because a is valid identifier and named handle to `co.lang.int` type.

   10 = 20; // ❌ invalid

   The above statement is invalid because 10 is not a valid identifier and if 10 in literal object type there is no named handle.

   The rebinding happens through named handles which are valid identifiers of `folang`

2. Through a property or method

   An object may also be mutated through one of its mutable properties or methods. a co.lang.int = 10; a.value = 20;

   A literal object cannot be mutated directly because it does not provide properties/methods that can be accessed.

   10.value = 20; // ❌ invalid

```
a co.lang.int = 10;
a.value = 30;
```

is valid after binding, a bare literal such as `10` cannot be mutated directly because there is no name or handle through which to perform mutation.

Not the Same as `toSnapshot(...)`

A literal object created by a literal expression is **not** the same thing as the internal snapshot representation produced by `co.utils.toSnapshot(...)`.

- literal expression → anonymous object
- `toSnapshot(x)` → internal snapshot representation used to reconstruct a fresh local object later

These are related concepts, but they are not the same mechanism.

Types of Literal objects:

```
 1. Simple types
 2. Complex or compound types (UDTS)
```

```
Simple literal forms include values such as `10`, `'A'`, and `"A"`.

k co.lang.int=10;

Compound types are in Json form


// Employee.fol
_ co.lang.class = {

     id co.lang.string;
     name co.lang.string;
}

k Employee = Employee{ id: "10", name: "ABC" };

> Even though compound literals are ended with brace block we need to end with semicolon as it is value not block.
```

Summary

- literal objects are real objects
- literal expressions create anonymous objects
- identical literals compare equal by value
- identical literals are not automatically the same object
- literal-created objects are mutable by default once bound to a handle
- only `makeImmutable(...)` makes an object immutable

---

4. Object Behaviour Policies

Any managed object can be given a behaviour policy using `co.utils.*`.

The policy operations are **in-place transformations of the object graph**:

- the object itself changes behaviour kind;
- there is no wrapper object;
- there is no alternate binding to capture;
- aliases continue to refer to the same transformed object graph.

Object policy belongs to the **object graph**, not to a variable name or alias. If two or more bindings refer to the same managed object, applying Immutable, Shared, or CopyOnWrite behaviour through one alias is observed through every alias that still refers to that object. Rebinding one alias later does not change the policy of the original object graph.

All policies are **deep by default**. The reachable object graph is the transitive closure obtained by starting at the policy root and recursively following managed-object references through members, nested objects, collection elements, and other managed references. Repeated references and cycles identify the same reachable object rather than creating additional logical objects. The applicable policy therefore governs the complete reachable graph, not only the root object.

Immutable, Shared, and CopyOnWrite are mutually exclusive object policies. Once an existing object graph enters one of these policy states, that policy is permanent for the lifetime of that graph; it cannot later be changed into either of the other policy states. `makeValueImmutable(x)` and `makeImmutable(x)` both make the current object graph Immutable; `makeImmutable(x)` additionally prevents rebinding of the binding supplied as `x`. Binding immutability is distinct from object policy and does not make other aliases non-rebindable.

4.1 Immutable

```
co.utils.makeImmutable(positive_int);
```

After this call the object itself is immutable.

This is an in-place transformation, not a wrapper.

Any attempt to mutate the object or any reachable object beneath it is a **compiler error** where detectable statically, and a **runtime error** otherwise.

```
positvie_int co.lang.int = 10;
positive_int.value = 30   // ❌ compiler error
positive_int = 30         // ❌ compiler error
```

For nested objects:

```
// Employee.fol
_ co.lang.struct = { address Address; }
Address co.lang.Address = {city co.lang.string; state co.lang.string; lane co.lang.string;pin co.lang.string;}


emp Employee = Employee{
    address: Address{
            city: "Pune"

    }
};
co.utils.makeImmutable(emp);

emp = Employee{
    address: Address{
            city: "ABC"

    }
}; // ❌ compiler error


emp.address = newAddress              // ❌ compiler error
emp.address.city.value = "Mumbai"     // ❌ compiler error
```

Immutability is deep and total.

4.2 Value Immutable

```
positive_int co.lang.int = 20;

co.utils.makeValueImmutable(positive_int);

positive_int.value = 30; // ❌ current object is immutable
positive_int = 30;       // ✅ binding may reference a new object

co.utils.makeValueImmutable(emp);

emp.address.city = "ABC";       // ❌
emp.address.city.value = "ABC"; // ❌

emp = Employee{
    address: Address{
        city: "ABC"
    }
}; // ✅
```

Difference between Immutable and Immutable Value

```
    makeValueImmutable(x)
        └── current object graph cannot change

    makeImmutable(x)
        ├── current object graph cannot change
        └── binding cannot be reassigned
```

Table

Operation	Binding	Current value/object graph
`makeValueImmutable(x)`	Mutable	Immutable
`makeImmutable(x)`	Immutable	Immutable

4.3 Shared

```
co.utils.makeShared(positive_int);
```

A Shared Object is safe for concurrent and multi-threaded use.

The programmer continues to use the same object model — same accessors, same syntax — while FoLang guarantees the required safety properties internally.

Shared behaviour is deep:

- all reachable objects within the shared object are also shared

Note on Analogies

Comparisons to things like Java's `AtomicInteger` or `ConcurrentHashMap` are **analogies only**.

They may help explain the concept, but they are **not part of the formal language contract**.

FoLang specifies the behavioural guarantee, not the exact runtime implementation.

A good explanatory statement is:

> A shared `co.lang.int` may be thought of similarly to an atomic integer, and a shared map may be thought of similarly to a concurrent map. These comparisons are explanatory only. FoLang does not require any particular internal runtime representation.

4.4 CopyOnWrite

```
co.utils.copyOnWrite(positive_int);
```

A CopyOnWrite object passes by reference like a normal managed object until mutation is attempted. Merely passing or reading the object does not change the source state.

Normative CopyOnWrite Semantics

CopyOnWrite in FoLang has **whole-reachable-object-graph isolation semantics**. If any member or reachable object beneath a CopyOnWrite root is mutated in a context that must not affect the source object, the mutating context must observe an independent modified state for the complete logical graph rooted at that CopyOnWrite object, while external aliases that continue to refer to the source graph must continue to observe the unchanged source state.

For example:

```
a Employee = Employee{
    dept: Dept{ id: 10 },
    address: Address{ city: "Pune" }
};

co.utils.copyOnWrite(a);

process(emp Employee)->() = {
    emp.dept.id = 20;
}

process(a);
```

After `process(a)`, the caller's `a` must still observe the original `Employee` graph with `dept.id == 10`, while the mutating context must observe an `Employee` state in which `dept.id == 20`. The isolation applies to the complete logical graph rooted at `a`, including `dept`, `address`, and all other reachable managed objects.

The specification defines this **observable isolation contract**, not a required physical copying algorithm. A conforming backend is not required to allocate a complete duplicate graph if it can preserve the same observable semantics by another implementation technique.

Internal alias topology and cycles are part of the logical graph semantics. If multiple members logically refer to the same object, that relationship must remain correct in the isolated COW state. Cyclic relationships must likewise remain semantically intact. An implementation must not allow a mutation in the isolated COW state to become visible through an external alias that still denotes the original source state.

Reference Backend Implementation

The default FoLang backend is the **reference implementation** for these semantics. Its straightforward reference strategy is whole-graph materialization: on the first mutation requiring isolation, it clones the entire reachable managed-object graph rooted at the CopyOnWrite object, preserves cycles and repeated internal references, redirects the mutating context to the cloned root, and applies the mutation to the clone. External aliases remain attached to the original graph.

Conceptually, the reference implementation behaves as follows:

```
caller before mutation

a ──> Employee A
      ├── Dept B      (id = 10)
      ├── Address C
      └── ... other reachable objects

reference-backend COW materialization

caller                                mutating context
a ──> Employee A                      emp ──> Employee A'
      ├── Dept B   (id = 10)                 ├── Dept B'   (id = 20)
      ├── Address C                          ├── Address C'
      └── ...                                └── ...
```

This reference algorithm is deliberately simple and is intended to make the required semantics easy to understand, implement, inspect, and test. It is not a requirement that production backends use the same physical representation or copying strategy.

Reference Backend Whole-Graph Cost

Because the reference backend materializes the complete reachable graph, the cost is determined by the size of the COW root's reachable graph, not by the size of the individual member being modified.

For example:

```
employees co.core.list(Employee) = ...;
co.utils.copyOnWrite(employees);

process(emps co.core.list(Employee))->() = {
    emps[500].dept.id = 20;
}

process(employees);
```

Although the logical change is only to `emps[500].dept.id`, the COW root is the employee list. The reference backend therefore materializes an isolated copy of the entire reachable `co.core.List->(Employee)` graph before applying the mutation. This includes the list, all reachable `Employee` objects, their departments, addresses, and other reachable managed objects. The caller's original list graph remains intact.

Conceptually:

```
original caller graph                reference-backend COW graph

List<Employee> A                    List<Employee> A'
 ├── Employee 0                      ├── Employee 0'
 ├── Employee 1                      ├── Employee 1'
 ├── ...                             ├── ...
 ├── Employee 500                    ├── Employee 500'
 │    ├── Dept (id = 10)             │    ├── Dept' (id = 20)
 │    ├── Address                    │    ├── Address'
 │    └── ...                        │    └── ...
 ├── ...                             ├── ...
 └── Employee N                      └── Employee N'
```

This simple reference strategy may be unsuitable for some production workloads. A tiny nested mutation can require work proportional to a very large reachable graph, may temporarily keep both source and isolated graphs live, and can increase allocation pressure, peak memory use, memory-bandwidth consumption, and allocator fragmentation. These are characteristics of the **reference backend implementation**, not additional FoLang language semantics.

Conforming Alternative Backends

A third-party or specialized backend may implement the same CopyOnWrite semantics using structural sharing, path copying, persistent data structures, lazy materialization, reference counting, arenas, page-based techniques, virtual-memory mechanisms, a garbage-collected runtime, a custom memory manager, or another strategy. Such implementation choices are permitted provided that all externally observable FoLang semantics are preserved.

In particular, a backend may physically share unchanged storage between the source and isolated COW states as long as later mutations cannot leak across those logical states. FoLang source code must not be able to distinguish the optimized backend from the reference semantics except through non-semantic characteristics such as performance, memory consumption, allocation behaviour, or diagnostics that do not alter program meaning.

Backend implementers should use the written specification as the normative contract and may test their implementation against the reference backend and the FoLang conformance suite. Matching the reference backend's internal whole-graph allocation algorithm is **not** a conformance requirement; matching the required observable behavior is.

4.5 toSnapshot

```
co.utils.toSnapshot(positive_int);
```

`toSnapshot` converts an object into a **snapshot representation** — a value descriptor, not a live object. The snapshot representation itself cannot be mutated.

When passed to a function, the compiler/runtime uses the snapshot representation to construct a fresh independent local variable bound to the parameter name. That local is a normal mutable object with no shared identity with the original. All of this happens automatically — the developer writes only `co.utils.toSnapshot(positive_int)`.

```
process(a co.lang.int)->() = {
    a.value = 99;   // mutates the fresh local — positive_int completely unaffected
}

process(co.utils.toSnapshot(positive_int))

or

k co.lang.value = co.utils.toSnapshot(positive_int);

// positive_int unchanged
```

The flow:

```
positive_int
    ↓  co.utils.toSnapshot(positive_int)
    snapshot representation         ←  value descriptor — immutable by nature, not by policy
    ↓  passed to function
    compiler constructs fresh local 'a' from the snapshot representation
    'a' is a new independent normal object — mutations never reach positive_int
```

`toSnapshot` is deep — the snapshot representation covers the entire reachable object graph.

---

5. Policy Summary

Object Kind	Mutation allowed	Caller sees mutation	Thread safe	Copy on write	Deep
Literal expression object	❌ directly in source — no handle	—	value-like use	—	✅
Normal (default)	✅ via accessor	✅	❌	❌	—
Immutable	❌	—	✅	—	✅
Shared	✅ via accessor	✅	✅	❌	✅
CopyOnWrite	✅ on own copy	❌	✅	✅	✅
toSnapshot result	✅ on reconstructed local object	❌	independent snapshot	—	✅

---

6. Literal vs Value/snapshot

`co.lang.value` vs `co.lang.literal`

Literal (`co.lang.literal`) is literal representation of objects literals are object.

Literal representations use `to` conversion methods to produce typed objects. When no suitable conversion exists for a custom type, the developer must provide the required overload through the supported extension mechanism. A literal representation by itself does not carry the complete reconstruction metadata stored by a snapshot value.

A `co.lang.value` snapshot carries more information than a literal representation: it records the type information, literal/value representation, and reconstruction information required to create the corresponding object according to its declaration kind.

---

7. No Type Fragmentation

FoLang deliberately avoids special types for mutability or concurrency concerns.

There is no need for separate public type families such as:

```
ImmutableInt
SharedMap
CopyOnWriteList
AtomicInt
ConcurrentMap
```

Instead, FoLang keeps the type model uniform and applies behaviour policies through `co.utils.*`.

In many ecosystems, a programmer must constantly choose between:

```
normal integer     vs atomic integer
normal map         vs concurrent map
mutable value      vs immutable wrapper
raw structure      vs copy-on-write structure
```

FoLang instead aims to provide:

- one object model
- one programming style
- one mental model

while still allowing the programmer to opt into immutability, sharing, copy-on-write, or literal conversion when needed.

---

8. Formal Object-Policy and Identity Rules

The following rules define the previously open object-model precision points.

8.1 Aliases and Policy Ownership

Managed-object assignment copies a reference. Therefore multiple bindings may refer to the same object. Immutable, Shared, and CopyOnWrite state belongs to that object graph rather than to an individual alias. Applying one of these policies through any alias changes the policy observed through every alias that still refers to the same graph.

```
a Employee = Employee{...};
b := a;

co.utils.makeShared(a);

// a and b still refer to the same object graph.
// The graph is Shared through both aliases.
```

8.2 Rebinding Is Independent of Object Policy

Rebinding an alias changes only which object that binding refers to. It does not remove, transfer, or alter the policy of the previously referenced object.

```
a Employee = Employee{...};
b := a;

co.utils.makeShared(a);
b = Employee{...};
```

After the rebinding, `a` still refers to the original Shared graph. `b` refers to a different newly created object graph. The original graph remains Shared for its lifetime.

`makeImmutable(x)` is special only in that it also freezes the supplied binding `x`; the Immutable object policy itself still belongs to the object graph. Another alias to the same immutable graph cannot mutate that graph, although that other alias may itself be rebound unless its own binding has separately been made non-rebindable.

8.3 Deep Policy Scope

A deep policy covers the entire managed-object graph reachable from its root. Reachability is transitive: members, nested objects, collection elements, and other managed-object references are followed recursively until no new object identity is encountered. Cycles terminate by identity rather than by traversal depth, and repeated references to the same object remain repeated references to one object.

Thus applying a deep policy to an `Employee` also applies that policy to its reachable `dept`, `address`, nested collections, collection elements, and other managed objects in that graph. The policy invariant is maintained for the graph while that policy is active.

8.4 Mutation Visibility Across Calls

Function arguments use the managed reference model. Mutation visibility therefore follows the object's policy:

- **Normal** — mutation through the parameter mutates the same object graph and is visible to the caller and other aliases.
- **Shared** — mutation is visible to aliases of the same graph, with the concurrency-safety guarantees of Shared behaviour.
- **Immutable** — mutation is prohibited.
- **CopyOnWrite** — a mutation requiring isolation gives the mutating context an independent logical state for the complete reachable graph while the caller's original graph remains unchanged. The reference backend realizes this by materializing a complete private graph copy; other conforming backends may use different internal techniques.

The policy of an existing graph is not temporary or call-local. Once that graph becomes Immutable, Shared, or CopyOnWrite, it remains in that state for its lifetime.

8.5 Value Equality and Reference Identity

`==` performs deep value equality. It answers whether two values/object graphs are equal by value; it does not answer whether two bindings refer to the same managed object.

Reference identity is exposed through `sameRef()` for managed reference-semantic objects:

```
a Employee = Employee{name: "Rao"};
b := a;
c Employee = Employee{name: "Rao"};

a == b;        // true
a == c;        // true

a.sameRef(b);  // true  — same managed object
a.sameRef(c);  // false — equal value, different managed object
```

`co.lang.cstruct` remains value-semantic and is outside the managed-object reference-identity model.

8.6 Policy Lifetime and Non-Stacking

Immutable, Shared, and CopyOnWrite are mutually exclusive permanent states of an existing managed object graph. A graph may transition from its normal mutable state into one of these states, but it cannot subsequently transition from one policy state into another.

```
                ┌── Immutable
Normal ─────────┼── Shared
                └── CopyOnWrite

Immutable   -X-> Shared
Immutable   -X-> CopyOnWrite
Shared      -X-> Immutable
Shared      -X-> CopyOnWrite
CopyOnWrite -X-> Immutable
CopyOnWrite -X-> Shared
```

`makeValueImmutable(x)` and `makeImmutable(x)` both place the current object graph into the Immutable state. Their difference concerns the supplied binding: `makeValueImmutable(x)` permits that binding to be rebound later, whereas `makeImmutable(x)` also prevents rebinding of that binding. Rebinding to a different object does not transfer the old object's policy to the new object.

---

9. Formal Philosophy Statement

> All managed FoLang objects use reference semantics by default. `co.lang.cstruct` is an explicitly value-semantic ABI representation and is an exception to managed-object reference semantics.
> In FoLang, everything is an object and managed objects are mutable by default.
> Assignment of managed objects copies references, `co.lang.cstruct` assignment copies values, `==` compares values deeply, and `sameRef()` exposes managed-object reference identity.
> Developers may opt into Immutable, Shared, or CopyOnWrite behaviour without changing the public type of the object. These policies belong to the object graph, are deep, are observed through all aliases to that graph, are mutually exclusive, and are permanent for that graph's lifetime.
> Rebinding an alias changes the object referenced by that binding but does not modify or transfer the policy of the previously referenced object.
> CopyOnWrite has whole-graph isolation semantics: a mutation anywhere in the reachable graph that requires isolation must leave the source logical graph intact while the mutating context observes an independent modified logical graph. The reference backend demonstrates this by cloning the complete reachable graph, but conforming backends may use any internal mechanism that preserves the same observable semantics.
> This policy model is uniform across managed types, so programmers do not need separate type families for ordinary, immutable, concurrent, or snapshot-oriented use.
> Familiar analogies such as atomic integers or concurrent maps may help explain the design, but they are not part of the formal implementation contract.

FoLang Definition and Documentation License

Unless otherwise stated, the copyrightable material contained in the FoLang language definition and documentation is licensed under the Creative Commons Attribution 4.0 International License.

The CC BY 4.0 licence applies to the copyrightable expression, organization, and presentation of the FoLang language definition and documentation, including:

- the FoLang language specification;
- original FoLang-specific syntax forms;
- grammar productions and grammar notation;
- rules governing how FoLang syntax forms may be combined;
- semantic rules and behavioural descriptions;
- name-resolution and scope-resolution rules;
- type-system definitions and constraints;
- execution-model and control-abstraction descriptions;
- compiler-behaviour descriptions;
- source-code examples demonstrating FoLang syntax and semantics;
- diagrams, tables, illustrations, and formal notation;
- reference documentation;
- explanatory and instructional material.

This includes the documented expression and presentation of FoLang-specific constructs such as:

```
(booleanExpression).then({
    ...
}).default({
    ...
});
```

and other original FoLang syntax, grammar, and semantic-rule descriptions contained in this document.

Under CC BY 4.0, this material may be:

- copied and redistributed in any medium or format;
- adapted, modified, translated, and extended;
- used for commercial or non-commercial purposes.

A person exercising these permissions must:

- give appropriate attribution;
- provide a link to the CC BY 4.0 licence;
- indicate whether modifications were made;
- retain notices supplied with the licensed material where required;
- not imply endorsement by the FoLang project or its contributors.

Independent Use and Implementation

This licence applies to the copyrightable expression contained in the FoLang language definition and documentation.

It does not restrict the independent use or implementation of programming-language ideas, concepts, features, procedures, systems, or methods of operation.

For example, this licence does not claim exclusive rights over general programming-language concepts such as:

- continuations;
- modules;
- structs and classes;
- dynamic, lexical, or mixed scope;
- functions and closures;
- pattern matching;
- algebraic data types;

- dependent types;
- concurrency and parallelism;
- coroutines and asynchronous execution. etc.,

Another project may independently implement such concepts without copying the copyrightable expression of the FoLang documentation.

The FoLang-specific documentation describing how these concepts are expressed through FoLang syntax, grammar, semantic rules, examples, diagrams, and explanatory text remains subject to this licence when that material is copied or adapted.

Software Licences

This licence does not change or replace the licences applied to:

- the FoLang frontend source code;
- the default backend source code;
- other FoLang software components;
- generated compiler binaries;
- third-party software;
- third-party documentation or assets.

The FoLang frontend and backend continue to be governed by their separately stated software licences.

Other Rights

CC BY 4.0 does not grant rights relating to:

- trademarks;
- project names and branding;
- logos;
- patents;
- third-party material;
- privacy, publicity, or personality rights.

Such rights, where applicable, remain governed separately.

Suggested Attribution

> FoLang Language Definition and Documentation, including its syntax, grammar, and semantic-rule descriptions, by Kemeswara Rao Mithipati and FoLang contributors, licensed under CC BY 4.0.

When modifications are made, the attribution should also indicate that the material was modified.

Example:

> Based on the FoLang Language Definition and Documentation by Kemeswara Rao Mithipati and FoLang contributors, licensed under CC BY 4.0. Modified from the original. ***

Appendix A - Complete FoLang EBNF Grammar

The following grammar is the normative lexical and syntactic grammar for FoLang.

```
(*
   FoLang EBNF Grammar — current alpha profile

   Generated from:
     language-ref.md
     SHA-256: 2f2e990a645af26e3bd2ec67f0e51a6d2112068ec2dab42037797f363c8614eb

   Authority:
     The language reference is normative. This grammar is a syntactic
     consolidation of the source forms defined there, including Appendix C.

   Notation:
     =       defines a production
     ;       ends a production
     |       alternative
     [ ... ] optional
     { ... } zero or more
     ( ... ) grouping
     "..."   terminal text
     ? ... ? lexical or context-sensitive condition described in prose

   Parser/lexer layering:
     - Source text is UTF-8.
     - U+FEFF is permitted only as the first source code point.
     - white-space/comments are discarded between tokens.
     - newlines never terminate statements.
     - the parser consumes the token stream produced by the lexer.

   Filesystem-selected parser roots:
     src/appl.fol
         -> application-entry-file

     src/library.fol
         -> standalone-library-surface-file

     src/export.fol
     srclib/exports/export.fol
         -> export-surface-file

     srclib/ffi/library.fol
     srclib/system/library.fol
     srclib/advanced/library.fol
     srclib/dynamicvmrt/library.fol
         -> source-library-surface-file

     srclib/operators/library.fol
         -> operator-source-file
            (dedicated lexer/parser; not ordinary compilation-unit syntax)

     ordinary package files
         <Name>.fol
         <Fragment>.unit.fol
         <StructName>.comp.unit.fol
         package.fol
         -> the matching package-source-file alternative selected from the
            externally validated filename class

   Current-alpha rules that are deliberately explicit here:
     - A named function block body requires "=" before the body.
     - An anonymous function literal does not use "=" between signature/body.
     - Direct declaration/function/block bodies end at "}" and cannot take ";".
     - Braced expressions/literals still require the enclosing statement ";".
     - Object construction is Type{field: value, ...}.
     - A type's arrow tail "T->( ... )" carries three unrelated meanings that
       only the tail's own shape and the base's meaning separate: a derivation
       applied to T, the result list of a function type, or a generic
       type-argument binding list. The binding list admits positional arguments
       and two named spellings, both of which the reference writes:
           co.lang.int->([5])                          derivation
           co.lang.int->(&, meta={type=out})           derivation, attributes
           (co.lang.int)->(co.lang.int)                function type
           co.core.List->(co.lang.string)              positional type args
           co.core.Map->(key co.lang.string, val co.lang.int)   named, space form
           co.core.Map->(key=co.lang.string, val=co.lang.int)   named, "=" form
           co.core.Array->(dims=2, type=co.lang.int, sizes=[2,4])  bound values
       The positional and space-named spellings are return-item-list's shape.
       The "name=value" spelling is derivation-attribute-list's shape, and a
       bound argument's value may be a type, an integer, or a bracketed list,
       which is what the Array and Matrix declarations bind. That makes
       "name=value" the ONE tail shape a derivation and a generic binding list
       share, so those two readings are separated by the base type and not by
       the tail's first token. See parenthesized-type-list.
     - A built-in collection value is a collection type followed directly by
       its literal body, which is typed-collection-literal:
           co.core.List["A","B","C"]      "[" elements  "]"
           co.core.Map{"A": 1, "B": 2}    "{" entries   "}"
           co.core.Set(1,2,3)             "(" arguments ")"
           co.core.Set->(co.lang.int)(1,2,3)  generic args, then arguments
```

```
       The body form is fixed by the collection, not chosen by the writer.
       builtin-collection-type-name records the reference's Builtin Collections
       registry, which is the closed set of built-in names admissible in that
       prefix position.
     - There is no untyped object literal, and a braced map body is an object
       literal representation, so a braced compound value ALWAYS names its type
       and map-literal is not a primary-expression alternative. An array literal
       is not an object literal: "[ ... ]" is a simple literal like a string or
       an integer and stays available untyped. See C.4 and C.4.1.
     - .match and .match() mean automatic matcher selection;
       .match(expr) selects an explicit matcher.
     - Control-chain vocabulary uses .then(X) for one-shot conditional
       selection, .loop(block) for single-condition repeated execution,
       .otherwise(condition) only for another selection condition, and
       .default(X) only for a terminal selection fallback. A .loop(block) form
       has exactly one condition and cannot be followed by .otherwise(condition)
       or .default(X). Element iteration is owned by .each(...) itself: its
       explicit-binding form takes index/key, value, and the per-element action
       directly, while its single-argument form takes a callable callback. An
       .each(...) call cannot be followed by .loop(...). There is no conditionless
       .otherwise form and no .do control verb in the current alpha profile.
     - Built-in precedence follows Appendix C.9.
     - The only current-alpha pre-declared operator glyphs are ∪ and ∩. They
       are active binary infix operators at precedence 500, left associative;
       missing overload implementations fail during resolution, not parsing.
     - reserved-future-operator-fixity is a diagnostic-recognition production:
       encountering an explicitly specification-reserved future fixity in the
       current alpha profile produces an unsupported-feature parser error.
     - Examples never enable, disable, or reserve syntax. Active syntax comes
       from this grammar; reserved-future treatment applies only to spellings or
       forms the language reference explicitly identifies as future/reserved.
     - All built-in @co.* directives, annotations, pragmas and decorators parse
       through the single generic annotation production and are collected in
       full. Unsupported metadata semantics may be ignored after collection;
       metadata parser acceptance does not depend on examples. See C.7.1.
     - `self` and `forall` are contextual spellings. The lexer leaves them
       available as identifiers; the parser reclassifies `self` in class-method
       context and `forall` when the complete polymorphic `forall(...).` form is
       recognized in its applicable contextual syntax.

   Context-sensitive/semantic requirements intentionally remain outside EBNF:
     filename canonicalization and package ownership; project layout and
     reachability; import field uniqueness/mutual exclusion; library-kind
     capability restrictions; interface/signature/typeclass conformance;
     instance placement/liveness; overload resolution; dependent-index name
     resolution; generic-argument binding, where a named argument must name a
     declared type parameter of the type being applied, names and positions
     must not be mixed in one list, and a `name=value` tail is a derivation
     unless the base type is a generic declaration; whether a type prefix on a
     collection body names a collection type; annotation target/type checks;
     operator metadata uniqueness, ranges and arity/fixity compatibility; and
     all type/runtime semantics.
*)


(* ==================================================================== *)

(* 1. Source roots and externally selected file forms *)

(* ==================================================================== *)

compilation-unit = package-source-file
                 | application-entry-file
                 | library-surface-file
                 | export-surface-file ;

source-filename = companion-unit-filename
                | ordinary-unit-filename
                | package-metadata-filename
                | application-entry-filename
                | library-surface-filename
                | export-surface-filename
                | ordinary-source-filename ;

companion-unit-filename = filename-identifier, ".comp.unit.fol" ;

ordinary-unit-filename = filename-identifier, ".unit.fol" ;

package-metadata-filename = "package.fol" ;

application-entry-filename = "appl.fol" ;

library-surface-filename = "library.fol" ;

export-surface-filename = "export.fol" ;

ordinary-source-filename = filename-identifier, ".fol" ;

filename-identifier = identifier-head, { "_", identifier-segment } ;
```

```
package-source-file = package-primary-source-file
                    | ordinary-unit-source-file
                    | companion-unit-source-file
                    | package-metadata-source-file ;

package-primary-source-file = file-preamble, primary-declaration ;

ordinary-unit-source-file = file-preamble, unit-declaration ;

companion-unit-source-file = file-preamble, unit-declaration ;

package-metadata-source-file = file-preamble, package-alias-declaration ;

application-entry-file = file-preamble, { entry-item } ;

library-surface-file = standalone-library-surface-file
                     | source-library-surface-file ;

standalone-library-surface-file =
    file-preamble, standalone-library-kind-annotation, library-declaration ;

source-library-surface-file =
    file-preamble, library-declaration ;

standalone-library-kind-annotation =
    "@co.dap.library", "(", "type", "=", library-kind-string, ")" ;

library-kind-string =
    ? string-literal whose decoded value is exactly one of:
      application, dynamicvmrt, advanced, system, ffi ? ;

(* Both src/export.fol and srclib/exports/export.fol use exactly this source
   shape. Their filesystem location determines standalone-export versus
   project-local-export semantics; neither location is a package. *)
export-surface-file = export-annotation, export-surface-guard ;

export-annotation = annotation ;

export-surface-guard =
    ? zero-width structural condition: the file contains exactly one metadata
      application; its qualified name is co.dap.export; it contains exactly the
      project-level export selector fields permitted by the current reference,
      including a packages map when required; package-selection keys are package
      qualified names; and no imports, declarations, statements, or additional
      metadata applications occur in the file ? ;

file-preamble = { file-directive } ;

file-directive = import-directive
               | alias-directive
               | use-directive
               | dynamic-runtime-directive ;

entry-item = file-directive
           | entry-type-declaration
           | bare-function-pattern-clause
           | capturing-function-pattern-clause
           | entry-statement ;

entry-statement = variable-declaration
                | inferred-variable-declaration
                | grouped-variable-declaration
                | multiple-assignment-statement
                | expression-statement
                | empty-statement ;

primary-declaration = struct-declaration
                    | cstruct-declaration
                    | enum-declaration
                    | union-declaration
                    | class-declaration
                    | interface-declaration
                    | signature-declaration
                    | module-declaration
                    | typeclass-declaration
                    | object-declaration
                    | instance-declaration
                    | matcher-instance-declaration ;


(* ==================================================================== *)

(* 2. Directives, annotations, and metadata *)

(* ==================================================================== *)

annotations = { annotation } ;
```

```
annotation = "@", qualified-name,
             [ "(", [ annotation-argument-list ], ")" ] ;

annotation-argument-list = annotation-argument,
                           { ",", annotation-argument }, [ "," ] ;

annotation-argument = [ annotation-key, annotation-binder ], annotation-value ;

annotation-binder = "=" | ":" ;

annotation-key = annotation-key-segment,
                 { "-", annotation-key-segment } ;

annotation-key-segment = identifier | "for" ;

annotation-value = literal
                 | type-expression
                 | qualified-name
                 | declaration-reference
                 | annotation-list
                 | annotation-map
                 | annotation-arrow-pair ;

annotation-list = "[", [ annotation-value,
                         { ",", annotation-value }, [ "," ] ], "]" ;

annotation-map = "{", [ annotation-map-entry,
                        { ",", annotation-map-entry }, [ "," ] ], "}" ;

annotation-map-entry = annotation-map-key, annotation-binder, annotation-value
                     | annotation-map-key ;

annotation-map-key = annotation-key | qualified-name ;

annotation-arrow-pair = string-literal, "=>", string-literal ;

import-directive = "@co.ddap.import", "(", import-field,
                   { ",", import-field }, ")" ;

import-field = "package", "=", string-literal
             | "library", "=", string-literal
             | "src-library", "=", "true"
             | "as", "=", string-literal ;

alias-directive = "@co.ddap.alias", "(", co-path, ",",
                  "as", "=", string-literal, ")" ;

use-directive = "@co.ddap.use", "(", use-field,
                { ",", use-field }, ")" ;

use-field = "from", "=", string-literal
          | "methods", "=", "[", [ identifier,
                                   { ",", identifier } ], "]" ;

dynamic-runtime-directive = "@co.ddap.dynamicruntime",
                            [ "(", [ annotation-argument-list ], ")" ] ;


(* ==================================================================== *)

(* 3. Names and references *)

(* ==================================================================== *)

filename-derived-name = "_" ;

qualified-name = ( identifier | "co" ), { ".", identifier } ;

co-path = "co", ".", identifier, { ".", identifier } ;

declaration-reference = qualified-function-reference | qualified-name ;

qualified-function-reference = qualified-name, "(", [ type-list ], ")",
                               return-type-clause ;

lifecycle-name = special-method ;

special-method = "@@new" | "@@init" ;

special-binding = result-binding | self-binding ;

self-expression = "self", self-context-guard ;

self-context-guard =
    ? zero-width contextual condition: this occurrence is inside a method
      declared by a co.lang.class, including @@new and @@init; otherwise self is
      an ordinary identifier spelling ? ;

refinement-candidate = "_", refinement-candidate-guard ;
```

```
refinement-candidate-guard =
    ? zero-width contextual condition: this occurrence is inside the predicate
      of the co.lang.refinementType declaration currently being parsed ? ;

self-binding = "$" ;

result-binding = "$", digit, { digit } ;

wildcard = "_" ;


(* ==================================================================== *)

(* 4. Type syntax *)

(* ==================================================================== *)

type-expression = forall-type | union-type-expression ;

forall-type = "forall", "(", type-parameter-list, ")", ".", type-expression,
              forall-context-guard ;

forall-context-guard =
    ? zero-width contextual condition: this occurrence begins the complete
      polymorphic forall(...). form in an applicable polymorphic type context;
      otherwise the spelling forall remains an ordinary identifier ? ;

type-parameter-list = identifier, { ",", identifier } ;

union-type-expression = arrow-type-expression,
                        { "|", arrow-type-expression } ;

arrow-type-expression = type-postfix-expression,
                        [ "->", arrow-type-tail ]
                      | "(", [ function-type-parameter,
                               { ",", function-type-parameter } ],
                        ")", "->", arrow-type-tail ;

arrow-type-tail = type-derivation
                | parenthesized-type-list
                | type-expression ;

type-postfix-expression = type-atom, { type-argument-list } ;

type-atom = qualified-name
          | "(", type-expression, ")" ;

type-argument-list = "(", [ type-or-value-argument,
                            { ",", type-or-value-argument } ], ")" ;

type-or-value-argument = type-expression | dependent-index ;

dependent-index = integer-literal | qualified-name ;

type-list = type-expression, { ",", type-expression } ;

(* This is one shape with two readings: the result list of a function type, or
   a generic type-argument binding list applied to the base type. Both admit an
   optional binder name per item, which is exactly return-item-list's shape, so
   the two are spelled with one production rather than two indistinguishable
   ones. Which reading applies is decided from the base type, not from the
   tail.

   The third arrow tail, type-derivation, is a separate alternative told apart
   by its first token in every case but one. The reference now also writes a
   named generic argument as `name=Type` and binds non-type values the same way:

       co.core.Map->(key=co.lang.string, val=co.lang.int)
       co.core.Array->(dims=2, type=co.lang.int, sizes=[2,4])
       LinkedList->(T=co.lang.int)

   An `identifier "="` opener is therefore a shape a derivation-attribute-list
   and a generic binding list share, and only the base type separates them: a
   tail on a generic declaration binds that declaration's type parameters, while
   a tail on any other type is the derivation-attribute-list of C.11. The two
   spellings coexist in the reference — C.11 and C.4.1 write `key co.lang.string`
   and `key=co.lang.string` for the same instantiation — so both are admitted
   here and the choice between them carries no meaning. *)
parenthesized-type-list = "(", [ return-item-list ], ")" ;

type-derivation = "(", derivation-specification, ")" ;

derivation-specification = pointer-specification
                         | array-specification
                         | reference-specification
                         | range-type-specification
                         | slice-type-specification
                         | thunk-type-specification
```

```
                         | address-type-specification
                         | derivation-attribute-list ;

pointer-specification = pointer-stars,
                        [ ",", derivation-attribute-list ] ;

pointer-stars = ? one contiguous symbolic run consisting only of one or
                  more "*" characters; its length is the pointer degree ? ;

reference-specification = ( "&" | "&&" | "~" ),
                          [ ",", derivation-attribute-list ] ;

address-type-specification = "@", [ ",", derivation-attribute-list ] ;

thunk-type-specification = "^", [ ",", derivation-attribute-list ] ;

slice-type-specification = "[:]", [ ",", derivation-attribute-list ] ;

range-type-specification = "..", [ ",", derivation-attribute-list ] ;

array-specification = array-dimension-group, { array-dimension-group },
                      [ ",", derivation-attribute-list ] ;

array-dimension-group = "[", array-dimension-content, "]" ;

array-dimension-content = [ array-dimension ], { ",", [ array-dimension ] } ;

array-dimension = "..." | "." | dependent-index ;

(* Attributes carried by a derivation, and — on a generic base type — the
   `name=value` spelling of a generic type-argument binding list. One shape,
   two readings, separated by the base type exactly as parenthesized-type-list
   describes. annotation-value is what admits a bound value that is not a type:
   the integer of `dims=2` and the bracketed list of `sizes=[2,4]`. *)
derivation-attribute-list = derivation-attribute,
                            { ",", derivation-attribute } ;

derivation-attribute = annotation-key, "=", annotation-value ;

return-type-clause = "->", "(", [ return-item-list ], ")" ;

return-item-list = return-item, { ",", return-item } ;

return-item = [ identifier ], type-expression ;


(* ====================================================================== *)

(* 5. Common declaration components *)

(* ====================================================================== *)

generic-parameter-clause = "(", generic-parameter,
                           { ",", generic-parameter }, ")" ;

generic-parameter = identifier, [ generic-arity-clause ] ;

generic-arity-clause = "(", generic-arity-slot,
                       { ",", generic-arity-slot }, ")" ;

generic-arity-slot = "_" | identifier ;

kind-options = "->", "(", [ annotation-argument-list ], ")" ;

field-declaration = annotations, identifier, type-expression,
                    [ "=", expression ], statement-end ;

embedded-field-declaration = annotations, type-expression, statement-end ;

value-specification = annotations, identifier, type-expression, statement-end ;

variable-declaration = annotations, typed-variable-declarator,
                       { ",", typed-variable-declarator }, statement-end ;

typed-variable-declarator = identifier, type-expression,
                            [ "=", expression ] ;

inferred-variable-declaration = annotations, inferred-variable-declarator,
                                { ",", inferred-variable-declarator },
                                statement-end ;

inferred-variable-declarator = identifier, definition-operator, expression ;

definition-operator = ( ":=" | "?=" ),
                      multi-symbol-infix-operator-boundary-guard ;


(* ====================================================================== *)
```

```
(* 6. Data and type declarations *)

(* ===================================================================== *)

struct-declaration = annotations, filename-derived-name,
                     "co.lang.struct", "=", struct-body ;

surface-struct-declaration = annotations, identifier,
                             "co.lang.struct", "=", struct-body ;

struct-body = "{", { struct-member }, body-close ;

pure-field-declaration = annotations, identifier, type-expression,
                         statement-end ;

struct-member = pure-field-declaration | embedded-field-declaration ;

cstruct-declaration = annotations, filename-derived-name,
                      "co.lang.cstruct", "=", cstruct-body ;

surface-cstruct-declaration = annotations, identifier,
                              "co.lang.cstruct", "=", cstruct-body ;

cstruct-body = "{", { pure-field-declaration }, body-close ;

enum-declaration = annotations, filename-derived-name,
                   "co.lang.enum", "=", enum-body ;

enum-body = "{", [ enum-variant,
                    { enum-separator, enum-variant }, [ enum-separator ] ],
            body-close ;

enum-separator = "," ;

enum-variant = annotations, identifier,
               [ "(", [ type-list ], ")" ],
               [ "=", constant-expression ] ;

union-declaration = annotations, filename-derived-name,
                    "co.lang.union", "=", union-body ;

union-body = "{", { pure-field-declaration }, body-close ;

data-declaration = annotations, identifier,
                   [ generic-parameter-clause ], "co.lang.data", "=",
                   data-variant, { "|", data-variant }, statement-end ;

data-variant = qualified-name,
               [ "(", [ type-list ], ")" ] ;

type-declaration = parameterized-type-declaration
                   | simple-type-declaration
                   | refinement-type-declaration ;

parameterized-type-declaration = annotations, identifier,
                                 generic-parameter-clause, "co.lang.type",
                                 [ kind-options ], [ "=", type-expression ],
                                 statement-end ;

simple-type-declaration = annotations, identifier, type-declaration-kind,
                          [ kind-options ], [ "=", type-expression ],
                          statement-end ;

type-declaration-kind = "co.lang.type"
                      | "co.lang.newtype"
                      | "co.lang.opaquetype"
                      | "co.lang.subtype"
                      | "co.lang.supertype"
                      | "co.lang.dependentType"
                      | "co.lang.kind" ;

refinement-type-declaration =
    annotations, identifier, "co.lang.refinementType", "=",
    refinement-type-expression, statement-end ;

refinement-type-expression =
    "(", type-expression, ")", ".where", "(", expression, ")" ;

entry-type-declaration = entry-parameterized-type-declaration
                       | entry-simple-type-declaration
                       | refinement-type-declaration ;

entry-parameterized-type-declaration = annotations, identifier,
                                       generic-parameter-clause,
                                       "co.lang.type",
                                       [ kind-options ],
                                       [ "=", type-expression ],
                                       statement-end ;
```

```
entry-simple-type-declaration = annotations, identifier,
                                ( "co.lang.type"
                                | "co.lang.newtype"
                                | "co.lang.opaquetype"
                                | "co.lang.subtype"
                                | "co.lang.supertype"
                                | "co.lang.dependentType" ),
                                [ kind-options ],
                                [ "=", type-expression ],
                                statement-end ;

package-alias-declaration = filename-derived-name, "co.lang.package", "=",
                            package-alias-body, statement-end ;

package-alias-body = "{", "name", ":", string-literal, "}" ;


(* ==================================================================== *)

(* 7. Containers, contracts, instances, and library surfaces *)

(* ==================================================================== *)

unit-declaration = annotations, filename-derived-name,
                   "co.lang.unit", "=", unit-body ;

unit-body = "{", { unit-member }, body-close ;

unit-member = function-declaration
            | closure-declaration
            | data-declaration
            | type-declaration
            | type-level-function-declaration
            | function-object-declaration
            | delegate-declaration
            | extern-variable-declaration ;

extern-variable-declaration =
    "@co.dap.declare", "(", "extern", ")", identifier, type-expression,
    statement-end ;

class-declaration = annotations, filename-derived-name,
                    "co.lang.class", [ kind-options ], "=", class-body ;

class-body = "{", { class-member }, body-close ;

class-member = field-declaration
             | function-declaration
             | lifecycle-method-declaration ;

lifecycle-method-declaration = annotations, lifecycle-name,
                               parameter-list, [ return-type-clause ],
                               function-definition ;

interface-declaration = annotations, filename-derived-name,
                        "co.lang.interface", "=", interface-body ;

interface-body = "{", { function-specification }, body-close ;

signature-declaration = annotations, filename-derived-name,
                        "co.lang.signature", "=", signature-body ;

signature-body = "{", { signature-member }, body-close ;

signature-member = value-specification
                 | function-specification
                 | signature-type-component
                 | associated-type-requirement ;

signature-type-component = annotations, identifier,
                           [ generic-parameter-clause ], "co.lang.type",
                           [ "=", type-expression ], statement-end ;

associated-type-requirement = annotations, identifier,
                              [ generic-parameter-clause ],
                              "co.lang.associatedType", statement-end ;

module-declaration = annotations, filename-derived-name,
                     "co.lang.module", [ kind-options ], "=", module-body ;

module-body = "{", { module-member }, body-close ;

module-member = variable-declaration
              | inferred-variable-declaration
              | function-declaration
              | signature-type-component
              | associated-type-binding ;

associated-type-binding = annotations, identifier,
```

```
                            [ generic-parameter-clause ],
                            "co.lang.associatedType", "=", type-expression,
                            statement-end ;

library-declaration = filename-derived-name,
                      "co.lang.library", "=", library-body ;

library-body = "{", { library-member }, body-close ;

library-member = import-directive
               | surface-struct-declaration
               | surface-cstruct-declaration
               | function-declaration ;

object-declaration = annotations, filename-derived-name,
                     "co.lang.object", [ kind-options ], "=", object-body ;

object-body = "{", { field-declaration | function-declaration }, body-close ;

instance-declaration = annotations, filename-derived-name,
                       "co.lang.instance", [ kind-options ], "=",
                       instance-body ;

instance-body = "{", { function-declaration | variable-declaration }, body-close ;

typeclass-declaration = annotations, filename-derived-name,
                        typeclass-parameter-clause, "co.lang.typeclass", "=",
                        contract-body ;

typeclass-parameter-clause = generic-parameter-clause ;

matcher-instance-declaration = annotations, filename-derived-name,
                               "co.lang.matcher", matcher-options, "=",
                               matcher-body ;

matcher-options = "->", "(", "type", annotation-binder, type-expression, ")" ;

matcher-body = "{", { function-declaration }, body-close ;

contract-body = "{", { function-specification | value-specification }, body-close ;

named-block-declaration = annotations, identifier,
                          "co.lang.block", "=", block, body-closure-guard ;


(* ====================================================================== *)

(* 8. Functions and callable declarations *)

(* ====================================================================== *)

delegate-declaration = annotations, identifier,
                       "co.lang.delegate", "=", function-type, statement-end ;

function-object-declaration = annotations, identifier,
                              "co.lang.function", "=",
                              function-object-binding ;

function-object-binding = expression, statement-end ;

type-level-function-declaration = annotations, function-name, parameter-list,
                           { parameter-list }, type-level-return-clause,
                           type-level-binding ;

type-level-return-clause = "->", "(", type-level-result-kind,
                                 { "|", type-level-result-kind }, ")" ;

type-level-result-kind = "co.lang.dependentType"
                       | "co.lang.type" ;

type-level-binding = function-definition
                         | function-delegation
                         | "=", type-expression, statement-end
                         | "=", non-block-expression, statement-end
                         | statement-end ;

function-declaration = annotations, [ receiver-clause ], function-name,
                       parameter-list, { parameter-list },
                       [ return-type-clause ], function-binding ;

function-name = identifier ;

receiver-clause = "(", ( type-expression
                        | identifier, type-expression ), ")" ;

parameter-list = "(", [ parameter,
                        { ",", parameter } ], ")" ;

parameter = [ "..." ], [ "~" ], identifier, [ "?" ],
```

```
           [ type-expression ], [ "=", expression ] ;

function-binding = function-definition
                 | function-delegation
                 | function-alias-binding
                 | statement-end ;

function-definition = "=", block, body-closure-guard ;

function-delegation = "=>>", expression,
                      { "=>>", expression }, statement-end ;

function-alias-binding = "=", non-block-expression, statement-end ;

function-specification = annotations, [ receiver-clause ], function-name,
                         parameter-list, { parameter-list },
                         [ return-type-clause ], statement-end ;

function-type = "(", [ function-type-parameter,
                       { ",", function-type-parameter } ], ")",
                return-type-clause ;

function-type-parameter = type-expression
                        | identifier, type-expression ;

anonymous-function-expression =
    [ "forall", "(", type-parameter-list, ")", "." ],
    parameter-list, return-type-clause, block ;

lambda-expression = "|", [ lambda-parameter,
                             { ",", lambda-parameter } ], "|", "=>",
                    ( expression | block ) ;

lambda-parameter = identifier, [ type-expression ] ;

closure-declaration = annotations, identifier, "=", parameter-list,
                      { parameter-list }, "==>>", expression,
                      statement-end ;

local-function-declaration = annotations, function-name, parameter-list,
                             { parameter-list }, return-type-clause,
                             function-definition ;


(* ====================================================================== *)

(* 9. Function-pattern clauses and patterns *)

(* ====================================================================== *)

bare-function-pattern-clause = annotations, identifier, pattern-parameter-list,
                               [ where-clause ], "=>", pattern-result ;

capturing-function-pattern-clause = annotations, "let", identifier,
                                    pattern-parameter-list,
                                    [ where-clause ], "=", pattern-result ;

pattern-parameter-list = "(", [ pattern,
                                { ",", pattern } ], ")" ;

where-clause = ".where", "(", expression, ")" ;

pattern-result = block, body-closure-guard
               | non-block-expression, statement-end ;

pattern = wildcard
        | literal-pattern
        | binding-pattern
        | constructor-pattern
        | record-pattern
        | tuple-pattern
        | qualified-name ;

literal-pattern = literal
                | ( "+" | "-" ),
                  ( integer-literal | floating-literal ) ;

binding-pattern = identifier ;

constructor-pattern = qualified-name, "(", [ pattern,
                                             { ",", pattern } ], ")" ;

record-pattern = qualified-name, "{", [ record-pattern-field,
                                        { ",", record-pattern-field } ], "}" ;

record-pattern-field = identifier, [ ":", pattern ] ;

tuple-pattern = "(", pattern, ",", pattern,
                { ",", pattern }, ")" ;
```

```
match-case = ".case", "(", match-case-body, ")" ;

match-case-body = pattern, [ ":", expression ], "=>",
                  ( expression | block ) ;

match-default = ".default", "(", ( expression | block ), ")" ;


(* ====================================================================== *)

(* 10. Statements, blocks, and termination *)

(* ====================================================================== *)

block = "{", { block-item }, [ block-tail-expression ], "}" ;

block-item = use-directive | statement ;

block-tail-expression = expression ;

statement = named-block-declaration
          | variable-declaration
          | inferred-variable-declaration
          | grouped-variable-declaration
          | let-value-declaration
          | local-function-declaration
          | closure-declaration
          | multiple-assignment-statement
          | return-statement
          | expression-statement
          | labeled-block
          | block-statement
          | empty-statement ;

grouped-variable-declaration = "(", typed-variable-declarator,
                               { ",", typed-variable-declarator }, ")",
                               statement-end ;

let-value-declaration = "let", identifier, [ type-expression ], "=",
                        expression, statement-end ;

multiple-assignment-statement = assignment-target, ",", assignment-target,
                                { ",", assignment-target }, "=",
                                expression-list, statement-end ;

assignment-target = postfix-expression
                  | tuple-assignment-target ;

tuple-assignment-target = "(", assignment-target, ",", assignment-target,
                          { ",", assignment-target }, ")" ;

return-statement = ( "this" | self-expression ), ".return",
                   [ expression-list ], statement-end ;

expression-statement = annotations, non-block-expression, statement-end ;

labeled-block = identifier, ":", block, body-closure-guard ;

empty-statement = ";" ;

expression-list = expression, { ",", expression } ;

statement-end = ";" ;

body-close = "}", body-closure-guard ;

body-closure-guard =
    ? zero-width condition: the next significant token is not ";", or there is no next token ? ;

block-statement = block, body-closure-guard ;

non-block-expression = expression, non-block-expression-guard ;

non-block-expression-guard =
    ? zero-width condition: the complete source span is not admissible as an
      unparenthesized block production; when both block and another braced
      expression are possible, the block reading has priority ? ;


(* ====================================================================== *)

(* 11. Expressions and built-in operator precedence *)

(* ====================================================================== *)

expression = assignment-expression
           | extended-operator-expression ;
```

```
assignment-expression = logical-or-expression,
                        [ runtime-assignment-operator,
                          assignment-expression ] ;

runtime-assignment-operator = "="
                            | compound-assignment-operator ;

compound-assignment-operator = ( "+=" | "-=" | "*=" | "/=" | "%="
                               | "**=" | "&=" | "^=" | "|=" ),
                               multi-symbol-infix-operator-boundary-guard ;

constant-expression = ( logical-or-expression
                      | extended-operator-expression ),
                      ? zero-width condition: no runtime-assignment-operator
                        occurs anywhere in this constant-expression subtree ? ;

logical-or-expression = logical-and-expression,
                        { logical-or-operator, logical-and-expression } ;

logical-or-operator = "||", multi-symbol-infix-operator-boundary-guard ;

logical-and-expression = bitwise-or-expression,
                         { logical-and-operator, bitwise-or-expression } ;

logical-and-operator = "&&", multi-symbol-infix-operator-boundary-guard ;

bitwise-or-expression = bitwise-xor-expression,
                        { "|", bitwise-xor-expression } ;

bitwise-xor-expression = bitwise-and-expression,
                         { "^", bitwise-and-expression } ;

bitwise-and-expression = equality-expression,
                         { "&", equality-expression } ;

equality-expression = relational-expression,
                      { equality-operator, relational-expression } ;

equality-operator = ( "==" | "!=" ),
                    multi-symbol-infix-operator-boundary-guard ;

relational-expression = range-expression,
                        { relational-operator, range-expression } ;

relational-operator = "<" | ">" | multi-symbol-relational-operator ;

multi-symbol-relational-operator = ( "<=" | ">=" ),
                                   multi-symbol-infix-operator-boundary-guard ;

range-expression = additive-expression,
                   [ range-operator, [ additive-expression ] ]
                 | range-operator, additive-expression ;

range-operator = ( ".." | "<.." | "..<" | "<..<" ),
                 multi-symbol-range-operator-boundary-guard ;

additive-expression = predeclared-glyph-expression,
                      { additive-operator, predeclared-glyph-expression } ;

additive-operator = "+" | "-" ;

predeclared-glyph-expression = multiplicative-expression,
                               { predeclared-operator-glyph,
                                 multiplicative-expression } ;

multiplicative-expression = unary-expression,
                            { multiplicative-operator, unary-expression } ;

multiplicative-operator = "*" | "/" | "%" ;

unary-expression = { prefix-operator }, power-expression ;

prefix-operator = "+" | "-" | "!" ;

power-expression = postfix-expression,
                   [ power-operator, unary-expression ] ;

power-operator = "**", multi-symbol-infix-operator-boundary-guard ;

postfix-expression = primary-expression,
                     { postfix-suffix | postfix-operator } ;

postfix-operator = "!" ;

postfix-suffix = call-suffix
               | index-suffix
               | member-suffix
               | match-suffix ;
```

```
call-suffix = "(", [ argument-list ], ")" ;

argument-list = argument, { ",", argument } ;

argument = ( [ identifier, "=" ], expression )
         | block
         | lambda-expression
         | wildcard ;

index-suffix = "[", [ expression-list ], "]" ;

member-suffix = ".", ( member-identifier | "for" | lifecycle-name ) ;

member-identifier =
    ? an identifier token other than "match"; "match" is routed through
      match-suffix so matcher-chain syntax has one parser path ? ;

(* map-literal is deliberately absent. C.4/C.4.1 admit no untyped object
   literal, and a braced `{ ... }` map body is an object literal representation,
   so it is a collection BODY and never a value in its own right — it is
   reachable only through typed-collection-literal, behind a type prefix.
   array-literal is present because an array literal is not an object literal:
   C.4.1 makes `[ ... ]` a simple literal in the same sense as a string,
   character, or integer literal, so it carries no type prefix and needs none.
   The asymmetry between the two is that rule, not an oversight. A bare braced
   group in expression position is therefore always the block alternative. *)
primary-expression = literal
                   | special-binding
                   | "this"
                   | self-expression
                   | refinement-candidate
                   | qualified-name
                   | grouped-expression
                   | tuple-expression
                   | array-literal
                   | typed-collection-literal
                   | object-construction
                   | anonymous-class-expression
                   | block
                   | anonymous-function-expression
                   | let-expression
                   | comprehension-expression ;

grouped-expression = "(", expression, ")" ;

tuple-expression = "(", expression, ",", expression,
                   { ",", expression }, ")" ;

array-literal = "[", [ expression,
                       { ",", expression }, [ "," ] ], "]" ;

map-literal = "{", [ map-entry,
                     { ",", map-entry }, [ "," ] ], "}" ;

map-entry = expression, ":", expression ;

(* A built-in collection value names its collection type and then supplies the
   literal body that type takes. The generic arguments may be written on the
   type prefix, in which case the body follows the completed arrow tail:

       co.core.List["A","B","C"]
       co.core.Map{"A": 1, "B": 2}
       co.core.Set(1,2,3)
       co.core.List->(co.lang.string)["A","B","C"]
       co.core.Map->(key co.lang.string, val co.lang.int){"A": 1}
       co.core.Set->(co.lang.int)(1,2,3)

   Without the arrow tail, `Type[ ... ]` and `Type( ... )` are also admissible
   as an index-suffix and a call-suffix on the same source span, and
   `Type{ ... }` is also admissible as object-construction. The overlap is
   resolved by typed-collection-literal-guard, not by a syntactic distinction.

   The no-arrow-tail `Type( ... )` collection value shares the same token span
   with postfix-expression's ordinary call-suffix. Naming call-suffix in the
   first alternative and applying the guard makes the contextual collection
   reading explicit: when the prefix names a supported collection type the body
   is the collection's arguments, and otherwise it remains an ordinary call.
   After an explicit arrow tail the body is always a collection body. *)
typed-collection-literal =
      type-postfix-expression,
      ( array-literal | map-literal | call-suffix ),
      typed-collection-literal-guard
    | type-postfix-expression, "->", parenthesized-type-list,
      ( array-literal | map-literal | call-suffix ) ;

typed-collection-literal-guard =
    ? zero-width condition: the prefix names a supported collection type rather
      than a
      value in scope — a builtin-collection-type-name, a file-local alias bound
```

```
      to one by an alias-directive, or a user-declared collection type — and a
      following body form is the one fixed for that collection; an empty braced
      body on a recognized supported braced collection is an empty typed
      collection; for
      every other type an empty braced body is object construction; and a
      non-empty braced body is object construction whenever every entry has the
      object-field-initializer shape ? ;

(* The reference's Builtin Collections registry: the built-in names that can
   stand in a typed-collection-literal prefix. It is a name registry consulted
   by typed-collection-literal-guard, not a parser entry path — the prefix is
   parsed as an ordinary type-postfix-expression, so no production references
   this one. Which body form each name takes, and every other collection
   semantic, stays outside this grammar. *)
builtin-collection-type-name = "co.core.List"
                             | "co.core.Set"
                             | "co.core.Map"
                             | "co.core.Tree"
                             | "co.core.Trie"
                             | "co.core.Array"
                             | "co.core.Tuple" ;

(* co.core.Tree, co.core.Trie, co.core.Array, and co.core.Tuple are reserved
   built-in collection names, but their constructor body forms are unsupported
   in the current alpha profile. The registry reserves their names, while
   typed-collection-literal-guard refuses a constructor reading until the
   language reference defines those forms. Their use as collection constructors
   requires the explicitly specified current-alpha unsupported-feature diagnostic. *)

object-construction = type-postfix-expression, "{",
                      [ object-field-initializer,
                        { ",", object-field-initializer }, [ "," ] ], "}" ;

object-field-initializer = identifier, ":", expression ;

anonymous-class-expression = "co.lang.class", "{",
                             { class-member }, "}" ;

let-expression = "let", "(", "{", let-binding,
                 { ",", let-binding }, "}", ")",
                 ".in", "(", "{", expression, "}", ")" ;

let-binding = ( identifier | special-binding ), "=", expression ;

comprehension-expression = "for", "(", comprehension-binding, ")",
                           ".yield", "(", expression-list, ")" ;

comprehension-binding = pattern, "<-", expression ;

match-suffix = ".match", [ "(", [ expression ], ")" ],
               { match-case }, [ match-default ] ;

multi-symbol-infix-operator-boundary-guard =
    ? zero-width condition: a multi-symbol infix operator has an explicit
      boundary on both operand-facing sides; a boundary is whitespace, a
      comment, or an applicable delimiter, checked before separators are
      discarded ? ;

multi-symbol-range-operator-boundary-guard =
    ? zero-width condition: a multi-symbol range operator has an explicit
      boundary on every side for which an operand is present ? ;

extended-operator-expression =
    ? expression containing a registered project-local custom operator,
      parsed by precedence climbing from its declared alpha-supported fixity,
      precedence, associativity, and arity; ∪ and ∩ are not handled by this
      extension path because they are language-owned operators in the ordinary
      precedence grammar; explicitly reserved future fixities remain
      unsupported; every registered multi-symbol operator satisfies the
      operand-facing boundary rule for its fixity ? ;


(* ==================================================================== *)

(* 12. Informative control-chain shapes (not parser entry paths) *)

(* ==================================================================== *)

(* Control vocabulary in the current alpha profile:
     - .then(X) conditionally evaluates X once; X may be a block or value.
     - .then(...) may participate in .otherwise(condition) selection chains.
     - .loop(X) is a single-condition repetition form and cannot participate in
       .otherwise(condition) chains or take a .default(X) fallback.
     - .otherwise(condition) always introduces another selection condition; there
       is no conditionless .otherwise form.
     - .default(X) is the optional terminal fallback of a selection chain; it is
       not part of loop syntax.
     - .each(...) is itself element iteration and cannot be followed by .loop.
       The explicit-binding form is .each(index-or-key, value, action), where
```

```
       action may be any expression (including a block or anonymous function) or
       a lambda-expression. The single-argument form accepts a callable callback;
       semantic analysis requires it to match the receiver's iteration tuple.
   These informative productions document recognizable shapes; ordinary parsing
   still proceeds through the general postfix/member/call grammar above. *)

informative-condition-chain =
    "(", expression, ")", ".then", "(", block, ")",
    { ".otherwise", "(", expression, ")", ".then", "(", block, ")" },
    [ ".default", "(", block, ")" ] ;

informative-loop-chain =
    "(", expression, ")", ".loop", "(", block, ")" ;

informative-ternary-chain =
    "(", expression, ")", ".then", "(", expression, ")",
    { ".otherwise", "(", expression, ")", ".then", "(", expression, ")" },
    ".default", "(", expression, ")" ;

informative-each-chain =
    postfix-expression, ".each", "(",
    ( ( identifier | "_" ), ",", identifier, ",",
      ( expression | lambda-expression )
    | ( expression | lambda-expression ) ), ")" ;

informative-contains-chain =
    postfix-expression, ".contains", "(", expression, ")",
    ".then", "(", block, ")",
    [ ".default", "(", block, ")" ] ;

informative-pipeline-chain =
    postfix-expression,
    { ( ".filter" | ".map" | ".reduce" | ".forEach"
      | ".sortBy" | ".groupBy" | ".fold" ),
      "(", argument-list, ")" } ;


(* ===================================================================== *)

(* 13. Literals and lexical grammar *)

(* ===================================================================== *)

literal = builtin-literal ;

builtin-literal = integer-literal
                | floating-literal
                | string-literal
                | character-literal
                | boolean-literal
                | none-literal ;

integer-literal = ( binary-integer-literal
                  | octal-integer-literal
                  | decimal-integer-literal
                  | hexadecimal-integer-literal ),
                  [ integer-suffix ] ;

binary-integer-literal = ( "0b" | "0B" ), binary-digit-sequence ;

octal-integer-literal = "0", [ octal-digit-sequence ] ;

decimal-integer-literal = nonzero-digit, { decimal-digit } ;

hexadecimal-integer-literal = hexadecimal-prefix,
                              hexadecimal-digit-sequence ;

hexadecimal-prefix = "0x" | "0X" ;

binary-digit-sequence = binary-digit, { binary-digit } ;

octal-digit-sequence = octal-digit, { octal-digit } ;

decimal-digit-sequence = decimal-digit, { decimal-digit } ;

hexadecimal-digit-sequence = hexadecimal-digit, { hexadecimal-digit } ;

integer-suffix = unsigned-suffix,
                 [ long-suffix | long-long-suffix | size-suffix ]
               | long-suffix, [ unsigned-suffix ]
               | long-long-suffix, [ unsigned-suffix ]
               | size-suffix, [ unsigned-suffix ] ;

unsigned-suffix = "u" | "U" ;

long-suffix = "l" | "L" ;

long-long-suffix = "ll" | "LL" ;
```

```
size-suffix = "z" | "Z" ;

floating-literal = decimal-floating-literal
                 | hexadecimal-floating-literal ;

decimal-floating-literal = fractional-constant,
                           [ exponent-part ],
                           [ floating-point-suffix ]
                         | decimal-digit-sequence,
                           exponent-part,
                           [ floating-point-suffix ] ;

fractional-constant = decimal-digit-sequence, ".",
                      decimal-digit-sequence ;

hexadecimal-floating-literal = hexadecimal-prefix,
                               ( hexadecimal-fractional-constant
                               | hexadecimal-digit-sequence ),
                               binary-exponent-part,
                               [ floating-point-suffix ] ;

hexadecimal-fractional-constant = hexadecimal-digit-sequence, ".",
                                  hexadecimal-digit-sequence ;

exponent-part = ( "e" | "E" ), [ sign ], decimal-digit-sequence ;

binary-exponent-part = ( "p" | "P" ), [ sign ], decimal-digit-sequence ;

sign = "+" | "-" ;

floating-point-suffix = "f" | "F" | "l" | "L"
                      | "f16" | "F16"
                      | "f32" | "F32"
                      | "f64" | "F64"
                      | "f128" | "F128"
                      | "bf16" | "BF16" ;

character-literal = single-quote, alpha-basic-c-character, single-quote ;

alpha-basic-c-character =
    ? any translation character except apostrophe, backslash,
      carriage return, or line feed ? ;

string-literal = double-quote, { alpha-basic-s-character }, double-quote ;

alpha-basic-s-character =
    ? any translation character except double quote, backslash,
      carriage return, or line feed ? ;

double-quote = ? Unicode scalar value U+0022 ? ;

single-quote = ? Unicode scalar value U+0027 ? ;

backslash = ? Unicode scalar value U+005C ? ;

boolean-literal = "co.const.true" | "co.const.false" ;

none-literal = "co.const.none" ;

byte-order-mark =
    ? U+FEFF, permitted only as the first code point of a source file ? ;

identifier = identifier-head, { "_", identifier-segment },
             identifier-trailing-guard ;

identifier-trailing-guard =
    ? a zero-width assertion that the next character is not "_" ? ;

identifier-head = ascii-letter, { ascii-alphanumeric } ;

identifier-segment = ascii-alphanumeric, { ascii-alphanumeric } ;

ascii-alphanumeric = ascii-letter | decimal-digit ;

ascii-letter = "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H"
             | "I" | "J" | "K" | "L" | "M" | "N" | "O" | "P"
             | "Q" | "R" | "S" | "T" | "U" | "V" | "W" | "X"
             | "Y" | "Z"
             | "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h"
             | "i" | "j" | "k" | "l" | "m" | "n" | "o" | "p"
             | "q" | "r" | "s" | "t" | "u" | "v" | "w" | "x"
             | "y" | "z" ;

binary-digit = "0" | "1" ;

octal-digit = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" ;

hexadecimal-digit = decimal-digit
                  | "a" | "b" | "c" | "d" | "e" | "f"
```

```
                  | "A" | "B" | "C" | "D" | "E" | "F" ;

digit = decimal-digit ;

decimal-digit = "0" | nonzero-digit ;

nonzero-digit = "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ;

hard-reserved-word = "co" | "let" | "this" | "for" | "fo" ;

contextual-keyword = "self" | "forall" ;

reserved-operator = "::=" | "->>" | "<->" | "`" | backslash | "#" ;

predeclared-operator-glyph = "∪" | "∩" ;

reserved-future-operator =
    ? a complete documented reserved/future symbolic spelling that is not a
      current built-in operator and not a predeclared-operator-glyph; the lexer
      recognizes the complete spelling and the parser reports unsupported ? ;

token = identifier
      | keyword-token
      | literal
      | delimiter-token
      | predeclared-operator-glyph
      | reserved-operator
      | symbolic-token
      | reserved-future-operator ;

keyword-token = hard-reserved-word ;

delimiter-token = "(" | ")" | "{" | "}" | "[" | "]"
                | "," | ";" | double-quote | single-quote ;

symbolic-token = ? the complete maximal contiguous run of one or more symbol
                   characters after comments, literals, and closed composite
                   spellings are recognized; the run is preserved whole for
                   contextual classification and is never split as a
                   fallback ? ;

token-separator = white-space ;

line-comment = "//", { ? any Unicode scalar value except CR or LF ? } ;

block-comment = "/*", { block-comment-character }, "*/" ;

block-comment-character = ? any Unicode scalar value that does not begin the
                            two-character sequence */ ? ;

line-break = "\r\n" | "\n" | "\r" ;

horizontal-white-space = " " | "\t" | "\f" ;

white-space = horizontal-white-space | line-break | line-comment | block-comment ;


(* ====================================================================== *)

(* 14. Dedicated project-local operator-source grammar *)

(* ====================================================================== *)

operator-source-file = operator-library-declaration ;

operator-library-declaration =
    "_", "co.lang.library", "=", operator-library-body ;

operator-library-body = "{", { operator-declaration }, body-close ;

operator-declaration = operator-symbol, "co.lang.operator", "=",
                       operator-body, statement-end ;

operator-body = "{", operator-property,
                { ",", operator-property }, "}" ;

operator-property = "fixity", ":", operator-fixity
                  | "precedence", ":", ( "0" | decimal-integer-literal )
                  | "associativity", ":", operator-associativity
                  | "arity", ":", operator-arity
                  | "commutative", ":", boolean-literal
                  | "idempotent", ":", boolean-literal
                  | "identity", ":", operator-identity-value
                  | "foldable", ":", boolean-literal
                  | "vectorizable", ":", boolean-literal
                  | "distributes_over", ":", operator-symbol-list
                  | "desugar", ":", string-literal ;

operator-fixity = "co.operator.fixity.infix"
```

```
               | "co.operator.fixity.prefix"
               | "co.operator.fixity.postfix"
               | reserved-future-operator-fixity ;

reserved-future-operator-fixity =
      "co.operator.fixity.circumfix"
    | "co.operator.fixity.postcircumfix"
    | "co.operator.fixity.precircumfix"
    | "co.operator.fixity.mixfix"
    | "co.operator.fixity.ternary"
    | "co.operator.fixity.distfix" ;

operator-associativity = "co.operator.associativity.left"
                       | "co.operator.associativity.right"
                       | "co.operator.associativity.none" ;

operator-arity = "co.operator.arity.unary"
               | "co.operator.arity.binary"
               | "co.operator.arity.ternary"
               | decimal-integer-literal ;

operator-identity-value = literal ;

operator-symbol-list = "[", [ operator-symbol-reference,
                       { ",", operator-symbol-reference } ], "]" ;

operator-symbol-reference = character-literal | string-literal ;

operator-symbol = ? a maximal run of one or more symbol characters, where a
                    symbol character is any character that is not an ASCII
                    letter, digit, underscore, whitespace, or one of the
                    delimiters ( ) { } [ ] , ; " ' ; the run must not be a
                    language-owned or hard-reserved symbol and must not contain
                    // or /* ? ;
```

Appendix B - Grammar Decisions and Rationale

Appendix C - Grammar Clarifications and Alpha Parser Policy

This appendix records grammar decisions that are normative for the current FoLang alpha language profile. It synchronizes lexical rules already present in the consolidated FoLang EBNF with language-reference decisions finalized after that grammar draft.

Where an older example or explanatory paragraph elsewhere in this document conflicts with a rule in this appendix, the rule in this appendix governs until the older material is corrected. Examples are illustrative and do not enable or disable grammar. A form is active only when the current profile's normative grammar admits it; a future form is reserved while unsupported only when this specification explicitly identifies it as such.

C.1 Source Encoding and Identifier Lexical Grammar

FoLang source text is UTF-8. A U+FEFF byte-order mark is permitted only as the first code point of a source file; U+FEFF anywhere else is an error.

Ordinary FoLang identifiers are intentionally ASCII-only even though the source encoding is UTF-8. No Unicode normalization rule is required for ordinary identifiers because non-ASCII characters are not admitted into an identifier token.

An ordinary identifier:

- begins with an ASCII letter `A-Z` or `a-z`;
- may continue with ASCII letters or decimal digits;
- may contain `_` only between two non-empty alphanumeric segments;
- cannot begin with `_`;
- cannot contain consecutive underscores;
- cannot end with `_`;
- cannot be the single spelling `_` because `_` is a dedicated contextual token used for wildcard/discard positions, filename-derived declaration positions, unnamed type-constructor parameter slots such as `F(_)`, and the candidate-value placeholder inside a `co.lang.refinementType` predicate;
- is checked against the reserved-word table after its character sequence has been recognized. A hard-reserved word is emitted as its reserved token, not as an ordinary identifier.

Contextual keywords are different: their spelling is first available as an identifier spelling, and the parser reclassifies the occurrence only when the grammar-defined contextual form is active. In particular, `self` is contextual in class-method contexts, while `forall` is contextual only for the polymorphic type-expression form `forall(...).<type-body>`.

Normative lexical grammar:

```
identifier = identifier-head, { "_", identifier-segment },
             identifier-trailing-guard ;

identifier-trailing-guard =
    ? a zero-width assertion that the next character is not "_" ? ;

identifier-head = ascii-letter, { ascii-alphanumeric } ;
identifier-segment = ascii-alphanumeric, { ascii-alphanumeric } ;

ascii-alphanumeric = ascii-letter | decimal-digit ;

ascii-letter = "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H"
             | "I" | "J" | "K" | "L" | "M" | "N" | "O" | "P"
             | "Q" | "R" | "S" | "T" | "U" | "V" | "W" | "X"
             | "Y" | "Z"
             | "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h"
             | "i" | "j" | "k" | "l" | "m" | "n" | "o" | "p"
             | "q" | "r" | "s" | "t" | "u" | "v" | "w" | "x"
             | "y" | "z" ;

decimal-digit = "0" | nonzero-digit ;
nonzero-digit = "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ;
```

Examples:

```
name        valid
myVar2      valid
v1_hr       valid
a_b_c       valid

123hr       invalid: identifier cannot begin with a digit
_x          invalid: identifier cannot begin with underscore
_1          invalid: identifier cannot begin with underscore
a_          invalid: trailing underscore
a__b        invalid: consecutive underscores
_           contextual token, not an identifier
```

The hard-reserved words in the current alpha grammar are `co`, `let`, `this`, `for`, and `fo`. `fo` is permanently reserved and cannot be used as an ordinary identifier. `self` and `forall` are contextual rather than globally hard-reserved. `self` has its class-method meaning in every method declared by a `co.lang.class`, including `@@new` and `@@init`. `forall` is recognized contextually only when it begins a polymorphic type expression of the form `forall(...).<type-body>` in a type-expression position; otherwise the spelling is tokenized and resolved as an ordinary identifier.

C.2 Numeric Literal Lexical Grammar

FoLang uses the numeric-literal subset already selected by the consolidated grammar. Numeric digit separators are not supported. In particular, forms such as `1'000`, `0x1'a`, and `0b1011'0010` are invalid.

A numeric sign is not part of the ordinary numeric literal token. In an ordinary expression, unary `+` or `-` is parsed as a prefix operator. In a pattern, the grammar explicitly admits a leading `+` or `-` before an integer or floating literal so signed numeric literal patterns remain valid.

C.2.1 Integer literals

```
integer-literal = ( binary-integer-literal
                  | octal-integer-literal
                  | decimal-integer-literal
                  | hexadecimal-integer-literal ),
                  [ integer-suffix ] ;

binary-integer-literal = ( "0b" | "0B" ), binary-digit-sequence ;
octal-integer-literal = "0", [ octal-digit-sequence ] ;
decimal-integer-literal = nonzero-digit, { decimal-digit } ;
hexadecimal-integer-literal = hexadecimal-prefix,
                              hexadecimal-digit-sequence ;

hexadecimal-prefix = "0x" | "0X" ;

binary-digit-sequence = binary-digit, { binary-digit } ;
octal-digit-sequence = octal-digit, { octal-digit } ;
decimal-digit-sequence = decimal-digit, { decimal-digit } ;
hexadecimal-digit-sequence = hexadecimal-digit, { hexadecimal-digit } ;

binary-digit = "0" | "1" ;
octal-digit = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" ;
hexadecimal-digit = decimal-digit
                  | "a" | "b" | "c" | "d" | "e" | "f"
                  | "A" | "B" | "C" | "D" | "E" | "F" ;

integer-suffix = unsigned-suffix,
                 [ long-suffix | long-long-suffix | size-suffix ]
               | long-suffix, [ unsigned-suffix ]
               | long-long-suffix, [ unsigned-suffix ]
               | size-suffix, [ unsigned-suffix ] ;

unsigned-suffix = "u" | "U" ;
long-suffix = "l" | "L" ;
long-long-suffix = "ll" | "LL" ;
size-suffix = "z" | "Z" ;
```

The literal `0` is valid through the octal production. A decimal integer with a nonzero value begins with `1-9`.

C.2.2 Floating literals

A decimal or hexadecimal floating literal that contains a radix point must contain at least one digit on both sides of the point. Therefore `1.0` and `0.10` are valid, while `1.` and `.10` are invalid. Scientific notation without a radix point, such as `1e5`, remains valid.

```
floating-literal = decimal-floating-literal
                 | hexadecimal-floating-literal ;

decimal-floating-literal = fractional-constant,
                           [ exponent-part ],
                           [ floating-point-suffix ]
                         | decimal-digit-sequence,
                           exponent-part,
                           [ floating-point-suffix ] ;

fractional-constant = decimal-digit-sequence, ".",
                      decimal-digit-sequence ;

hexadecimal-floating-literal = hexadecimal-prefix,
                               ( hexadecimal-fractional-constant
                               | hexadecimal-digit-sequence ),
                               binary-exponent-part,
                               [ floating-point-suffix ] ;

hexadecimal-fractional-constant = hexadecimal-digit-sequence, ".",
                                  hexadecimal-digit-sequence ;

exponent-part = ( "e" | "E" ), [ sign ], decimal-digit-sequence ;
binary-exponent-part = ( "p" | "P" ), [ sign ], decimal-digit-sequence ;
sign = "+" | "-" ;

floating-point-suffix = "f" | "F" | "l" | "L"
                      | "f16" | "F16"
                      | "f32" | "F32"
                      | "f64" | "F64"
                      | "f128" | "F128"
                      | "bf16" | "BF16" ;
```

Backend-conditionally-supported floating suffixes are accepted only when the configured backend/compiler contract supports the corresponding representation.

Signed numeric patterns use:

```
literal-pattern = literal
                | ( "+" | "-" ),
                  ( integer-literal | floating-literal ) ;
```

C.3 Comments, Whitespace, and Line Breaks

FoLang supports `//` line comments and non-nesting `/* ... */` block comments.

- A line comment starts with `//` and extends up to, but does not consume as part of its body, the next CR or LF line terminator.
- A block comment starts with `/*` and ends at the first following `*/`.
- Block comments do not nest. An inner `/*` is ordinary block-comment content; the first subsequent `*/` closes the block comment.
- Comments are recognized before ordinary symbolic-run scanning.
- Spaces, horizontal tabs, form-feed characters, line breaks, and comments are token separators and are discarded between tokens.
- A line break never terminates a FoLang statement. FoLang has no automatic semicolon insertion.

Normative lexical grammar:

```
line-comment = "//", { ? any Unicode scalar value except CR or LF ? } ;

block-comment = "/*", { block-comment-character }, "*/" ;
block-comment-character = ? any Unicode scalar value that does not begin the
                            two-character sequence */ ? ;

line-break = "\r\n" | "\n" | "\r" ;
horizontal-white-space = " " | "\t" | "\f" ;

white-space = horizontal-white-space
            | line-break
            | line-comment
            | block-comment ;

token-separator = white-space ;
```

C.4 Canonical Compound and Object Construction Syntax

A user-defined object/struct/class value is constructed with an explicit type followed immediately by a braced field initializer. The canonical form is Go-like:

```
b := B{age: 25.0};
emp := Employee{name: "Rao", id: 1};
point := Point{x: 10.0, y: 20.0};
```

Object field initializers use `:` between the field name and value and `,` between fields. `=` is not an object-field initializer binder.

Normative shape:

```
object-construction = type-postfix-expression, "{",
                      [ object-field-initializer,
                        { ",", object-field-initializer }, [ "," ] ], "}" ;

object-field-initializer = identifier, ":", expression ;
```

Therefore these are invalid object-construction spellings:

```
b := B{age = 25.0};          // invalid
emp := {name: "Rao"};       // invalid as a UDT construction: explicit type required
emp := Employee{name = "Rao"}; // invalid
```

There is no untyped object literal in FoLang. A braced value is a body, never a value in its own right, so every object value names its type. This is a general rule and not a restriction on user-defined types alone: it applies equally to the braced built-in collection value of C.4.1.

A typed map literal remains a distinct expression from object construction. Its entries also use `:`, but the left side of an entry is an expression rather than a field identifier.

A closing `}` belonging to object construction closes the expression only. It does **not** terminate the enclosing statement; the statement still requires `;` as specified in C.6.

C.4.1 Built-in collection values

There are exactly two current-alpha construction forms; no third collection-construction inference form exists. When the declaration itself is type-deduced, the constructor carries its generic arguments explicitly and the literal body follows the completed arrow tail described in C.11:

```
x := co.core.List->(co.lang.string)["A","B","C"];
map := co.core.Map->(key=co.lang.string, val=co.lang.int){"A": 1, "B": 2};
y := co.core.Set->(co.lang.int)(1,2,3);
```

When the surrounding declaration already supplies the generic collection type, the value constructor has no arrow tail and does not repeat those generic arguments:

```
x co.core.List->(co.lang.string) = co.core.List["A","B","C"];
y co.core.Set->(co.lang.int) = co.core.Set(1,2,3);
map co.core.Map->(key=co.lang.string, val=co.lang.int) = co.core.Map{"A": 1, "B": 2};
```

Normative shape:

```
typed-collection-literal =
      type-postfix-expression,
      ( array-literal | map-literal | call-suffix ),
      typed-collection-literal-guard
    | type-postfix-expression, "->", parenthesized-type-list,
      ( array-literal | map-literal | call-suffix ) ;
```

The type prefix on a braced collection value is required, for the reason C.4 gives: there is no untyped object literal, and a `{ ... }` map body is an object literal representation. An untyped `{ ... }` is therefore never a value.

An array literal is not an object literal. `[ ... ]` is a simple literal in the same sense as a string, character, or integer literal, so it carries no type prefix and needs none. C.6 governs its termination exactly as it governs any other simple literal:

```
xs  := [1, 2, 3];                     // simple literal, valid untyped
cfg := co.core.Map->(key=co.lang.string, val=co.lang.any){"host": "db", "port": 5432};

cfg := {"host": "db", "port": 5432};   // invalid: untyped map literal
```

C.4.2 Disambiguating a typed collection body from the same source shape

Without an arrow tail, a typed collection body is spelled identically to three existing forms. The readings are separated as follows, and the separation is contextual rather than syntactic:

```
Type{ ... }   -> for a recognized supported braced collection constructor,
                 an empty body is an empty typed collection and a non-empty
                 body is a typed collection unless every entry has the shape
                 identifier ":" expression; for every other type, an empty
                 body or identifier-field body is object construction
Type[ ... ]   -> typed list literal when Type names a supported collection type;
                 otherwise an index-suffix applied to the value Type
Type( ... )   -> typed collection value when Type names a supported collection type;
                 otherwise an ordinary call
```

`Employee{name: "Rao", id: 1}` and `Employee{}` are therefore object construction. A no-arrow collection constructor such as `co.core.Map{"A": 1}` is valid only as the right-hand side of a declaration whose declared collection type already supplies the generic arguments, for example `m co.core.Map->(key=co.lang.string, val=co.lang.int) = co.core.Map{"A": 1};`. Outside that typed-declaration context, a collection constructor must carry its generic arguments explicitly with the arrow tail, for example `m := co.core.Map->(key=co.lang.string, val=co.lang.int){"A": 1};`. There is no third inference form. An explicit arrow tail removes the overlap entirely: after `Type->( ... )` the following body is always the collection's literal body.

Only `co.core.List`, `co.core.Set`, and `co.core.Map` have current-alpha collection-constructor forms. `co.core.Tree`, `co.core.Trie`, `co.core.Array`, and `co.core.Tuple` remain reserved built-in collection names, but their collection-constructor body forms are intentionally unspecified and unsupported in the current profile. Using one of those names as a collection constructor therefore produces an unsupported-feature diagnostic until a later specification revision explicitly defines the applicable body grammar and semantics. Examples may illustrate such a later form but do not enable it. These names must not silently inherit the body syntax of `List`, `Set`, or `Map`.

A closing `}`, `]`, or `)` belonging to a typed collection value closes the expression only, exactly as in C.4 and C.6.3. The enclosing statement still requires its `;`.

C.5 Match Invocation and Matcher Selection

FoLang distinguishes automatic matcher selection from explicit matcher selection by whether a matcher argument is supplied.

C.5.1 No matcher argument

When `.match` has no matcher argument, the compiler selects the applicable built-in/default matcher from the subject type and case-pattern forms.

Both existing no-argument spellings are accepted and equivalent:

```
value.match.case(...).default(...);
value.match().case(...).default(...);
```

The empty parentheses do not select a different matcher.

C.5.2 Explicit matcher argument

When an argument is supplied to `.match(...)`, that expression explicitly identifies the matcher to use. It may identify a named built-in matcher or a user-defined matcher.

```
value.match(co.pattern.Type).case(...);
value.match(co.pattern.Value).case(...);
value.match(PositiveEvenMatcher).case(...).default(...);
```

A user-defined matcher such as `PositiveEvenMatcher` follows the normal matcher declaration and import/name-resolution rules.

The parser-level shape remains:

```
match-suffix = ".match", [ "(", [ expression ], ")" ],
               { match-case }, [ match-default ] ;
```

Semantic interpretation is:

```
.match            -> no matcher argument -> automatic built-in/default matcher selection
.match()          -> no matcher argument -> automatic built-in/default matcher selection
.match(matcher)   -> explicit matcher selection
```

C.6 Statement and Expression Termination

FoLang uses explicit termination. Newlines never terminate statements, and there is no automatic semicolon insertion.

The governing rule is:

```
simple statement or expression statement    -> terminated by ;
direct block/body                            -> terminated by its closing }
braced expression/literal                    -> } closes the expression, then ; closes its statement
```

C.6.1 Semicolon termination

A semicolon is mandatory after every simple statement and expression statement, including:

- variable declarations and initializations;
- assignments and compound assignments;
- function/method calls used as statements;
- return statements expressed through `this.return ...`;
- expression-bodied function-pattern clauses;
- object-construction expressions used in declarations, assignments, or standalone expression statements;
- typed collection values in either legal form: a declared collection type with a no-arrow constructor on the right-hand side (for example `x co.core.List->(co.lang.int) = co.core.List[1,2,3];`) or an explicitly parameterized constructor used with type deduction (for example `x := co.core.List->(co.lang.int)[1,2,3];`);
- generic instantiations written as a typed declaration, such as `k LinkedList->(T co.lang.int);`;
- array, tuple, or other simple literal expressions when they form a simple statement; a map is written as the typed collection value of the previous item, since there is no untyped map literal;
- forward declarations and other declaration forms whose syntax is a simple declaration rather than a block body.

Examples:

```
x co.lang.int = 10;
x = 20;
co.out.println(x);
emp := Employee{name: "Rao", id: 1};
cfg := co.core.Map->(key=co.lang.string, val=co.lang.any){"host": "db", "port": 5432};
xs  := [1, 2, 3];
```

C.6.2 Block/body termination

A direct block or declaration body terminates at its closing `}` and is not followed by a semicolon.

```
// Employee.fol
_ co.lang.struct = {
    id   co.lang.int;
    name co.lang.string;
}

// function body
calculate()->(co.lang.int) = {
    this.return 10;
}

// block-bodied function-pattern clause
classify(n) => {
    this.return "positive";
}
```

Writing `};` after one of these direct block/body forms is invalid.

C.6.3 A braced expression is not a block terminator

An object construction, typed map literal, typed collection value, anonymous value expression, or other braced **expression** is not a declaration/function/block body merely because it ends with `}`. The enclosing simple statement still requires its semicolon.

```
emp := Employee{id: 1, name: "Rao"};
this.return Employee{id: 1};
cfg := co.core.Map->(key=co.lang.string, val=co.lang.int){"a": 1, "b": 2};
ages := co.core.Map->(key=co.lang.string, val=co.lang.int){"A": 30, "B": 40};
```

Built-in directives and annotations are self-delimiting metadata forms rather than simple statements and therefore do not acquire a trailing semicolon merely because they appear on their own source line.

C.7 Current Alpha, Reserved, Future, and Unsupported Syntax

The current alpha parser distinguishes **active grammar** from **explicitly reserved future syntax**. This distinction is defined by the specification itself and is never inferred from the presence or absence of examples.

```
source matches active lexical/syntactic grammar
    -> lexer and parser accept it according to the applicable grammar rules

source uses a spelling/form explicitly identified as reserved future syntax
    -> lexer recognizes the complete language-owned spelling where defined
    -> parser recognizes the explicitly reserved unsupported construct
    -> parser reports an unsupported-feature diagnostic
    -> semantic analysis is not used to pretend the feature is active

source matches neither category
    -> ordinary lexical or syntax error
```

A design discussion, inventory entry, example, or descriptive table does not by itself reserve syntax. A proposed feature receives the reserved-future treatment only when this specification explicitly declares the relevant token, operator spelling, declaration form, or syntactic pattern to be reserved/future in the current profile.

The diagnostic should identify the explicitly reserved **syntactic** feature. Backend- or later-stage generic metadata is not an example of this parser category: a syntactically valid generic metadata field is collected and preserved even when the frontend does not implement its later-stage semantics.

This rule does not turn arbitrary unknown text into reserved syntax. A character sequence or symbolic run that is neither valid current syntax nor an explicitly documented reserved/future spelling follows the ordinary lexical or parse-error rules.

C.7.1 Built-in `@co.*` Metadata Forms

Built-in directives, annotations, pragmas, and decorators under `@co.*` use the generic metadata grammar and are part of the active grammar independently of whether a particular metadata name or field appears in an example.

```
annotation = "@", qualified-name,
             [ "(", [ annotation-argument-list ], ")" ] ;
```

The parser must preserve the complete application:

```
declared @co.* directive/annotation/pragma/decorator
    -> parse the qualified metadata name
    -> parse and collect every supplied positional/named argument, field,
       attribute, and argument expression
    -> attach the complete metadata node to its target
    -> if no semantic handler implements the metadata form, silently ignore the
       collected form and its arguments
    -> if a semantic handler exists, validate its field schema and target rules
       during semantic resolution
```

Collection is not activation. An ignored metadata form has no semantic effect and contributes nothing to liveness, but its syntax and supplied fields were still parsed and preserved. Examples document intended use; they do not control parser acceptance or attribute collection.

An annotation whose name is outside `co.*` is an ordinary user-defined annotation and is resolved by the normal rules in C.8. Nothing here exempts external metadata from name resolution.

C.7.2 `co.*` Package Declarations Are Package API, Not Grammar

Ordinary declarations provided by the language's standard `co.*` `.folenc` package artifact are available according to that artifact rather than according to examples in this document. The toolchain loads the standard artifact implicitly, reconstructs its exported package contexts, and resolves those declarations through the ordinary package/symbol/member model.

Therefore a type or unit-level function newly added to an existing standard package—for example a later `co.core.RBTree`—may be used through ordinary already-defined FoLang syntax as soon as the applicable standard package artifact provides that declaration. No grammar revision is needed merely to add an ordinary package member.

A package API declaration cannot manufacture new syntax. A special constructor body, new operator spelling, new declaration form, or other grammar-level facility requires an explicit core-language specification revision defining the syntax and semantics. An example alone is never sufficient.

C.8 User-Defined Annotation Application

A user-defined annotation is applied with the same annotation syntax as a built-in annotation. There is no separate application grammar for user annotations.

Normative annotation shape:

```
annotation = "@", qualified-name,
             [ "(", [ annotation-argument-list ], ")" ] ;
```

The difference is name resolution: built-in `co.*` annotations are intrinsically available, while a user-defined annotation is an ordinary user declaration and must be resolved through normal package/import rules. When the annotation is declared in another package, that package must be imported before the annotation is used.

Example declaration:

```
// my/annotations/Audited.fol
_ co.lang.object->(for=annotation) = {
    value   co.lang.string;
    enabled co.lang.bool;
}
```

Example use from another package:

```
@co.ddap.import(package="my.annotations", as="ann")

@ann.Audited(value="payroll", enabled=co.const.true)
_ co.lang.class = {
    ...
}
```

An import without `as=` may use the complete imported package path according to the normal import rules:

```
@co.ddap.import(package="my.annotations")

@my.annotations.Audited(value="payroll")
_ co.lang.class = {
    ...
}
```

The annotation's permitted targets, required fields, field types, defaults, and liveness remain semantic checks; application syntax is identical to built-in annotation syntax.

C.8.1 Refinement-Type Candidate Placeholder

The token `_` has one additional contextual meaning inside the predicate of a `co.lang.refinementType` declaration. In the canonical form:

```
positiveInt co.lang.refinementType = (co.lang.int).where(_ > 0);
```

`_` denotes the candidate value of the immediately preceding base type. It is not an ordinary identifier and does not create a binding in the surrounding scope. Multiple `_` occurrences in the same refinement predicate denote that same candidate value.

This interpretation is restricted to the refinement predicate. It does not change `_` into a general expression identifier and does not alter its wildcard, discard, filename-derived declaration-name, or type-constructor-placeholder meanings in their respective contexts.

The predicate must resolve to `co.lang.bool`. For the remaining validation rule for candidates that are not statically known, see Refinement Types.

C.9 Built-In Operator Parse Table

FoLang follows the conventional precedence ordering used by mainstream C-family languages for the ordinary arithmetic, relational, equality, bitwise, logical, and assignment families, with FoLang's exponentiation and range levels inserted explicitly. Larger precedence numbers bind more tightly.

Precedence	Operator/form	Fixity	Associativity	Arity / parse role
700	call `(...)`, index `[...]`, member `.`, postfix `!`	postfix	left	call/index/member syntax; postfix `!` unary
650	`**`	infix	right	binary
600	`+`, `-`, `!`	prefix	right	unary
550	`*`, `/`, `%`	infix	left	binary
500	`∪`, `∩`	infix	left	binary
450	`+`, `-`	infix	left	binary
400	`..`, `<..`, `..<`, `<..<`	infix/range	none	binary with one bound optionally omitted according to range grammar
350	`<`, `<=`, `>`, `>=`	infix	left	binary
300	`==`, `!=`	infix	left	binary
250	`&`	infix	left	binary
200	`^`	infix	left	binary
150	`|`	infix	left	binary
100	`&&`	infix	left	binary; short-circuit
50	`||`	infix	left	binary; short-circuit
10	`=`, `+=`, `-=`, `*=`, `/=`, `%=`, `**=`, `&=`, `^=`, `|=`	infix assignment	right	binary assignment

Examples of grouping:

```
a + b * c;       // a + (b * c)
a ** b ** c;     // a ** (b ** c)
a || b && c;     // a || (b && c)
a = b = c;       // a = (b = c)
```

FoLang's left-to-right operand **evaluation order** is independent from operator grouping. Precedence and associativity determine the parse tree; the evaluation-order rules determine when the operands of that tree are evaluated.

The definition spellings `:=` and `?=` are statement-level definition operators, not general expression operators. They therefore do not receive a general expression-precedence level and cannot be chained as ordinary binary expressions.

Structural spellings such as `=>`, `=>>`, `==>>`, `->`, `<-`, `::=`, `->>`, and `<->` are not ordinary expression operators merely because they contain symbol characters.

Multi-symbol expression operators continue to obey the explicit operand-facing boundary rule: whitespace, a comment, or an applicable delimiter must delimit each operand-facing side required by the operator's fixity.

C.10 Pre-Declared Operator Glyphs in the Current Alpha Profile

The current alpha profile defines the following language-owned pre-declared operator glyphs:

```
∪, ∩
```

Both glyphs are enabled expression operators. Their parser properties are fixed by C.9:

```
fixity        = infix
precedence    = 500
associativity = left
arity         = binary
```

A project must not redeclare either glyph with `co.lang.operator`. Implementations are supplied through ordinary `mode=overload` operator functions in the legal operator owner for the operand type. Multiple implementations may coexist when their normalized operand signatures are distinct.

The lexer recognizes `∪` and `∩` as registered language-owned operator tokens, and the parser constructs the corresponding infix expression according to the properties above. Absence of an applicable overload is a resolution error, not a lexical or parse-time unsupported-feature error.

C.11 Type Arrow Tail and Generic Type Arguments

`->` is a structural spelling, not an expression operator (C.9). In **type** position, a `->` tail attached to a type carries three unrelated meanings that only the tail's shape and the base type's meaning separate:

```
co.lang.int->([5])                  derivation applied to co.lang.int
co.lang.int->(&, meta={type=out})   derivation applied to co.lang.int
(co.lang.int)->(co.lang.int)        function type from int to int
co.core.List->(co.lang.string)      generic type arguments applied to List
LinkedList->(T=co.lang.int)         generic type arguments, named
```

A tail whose first token starts a derivation specification — `*`, `&`, `&&`, `~`, `@`, `^`, `[:]`, `..`, `[`, or an `attribute=` pair — is a type derivation. Any other parenthesized tail is a `parenthesized-type-list`, which is read as a function result list when the base is a parameter list and as a generic type-argument list when the base is a generic declaration.

Items in that list carry an optional binder name, so both spellings below are admitted and mean the same instantiation:

```
co.core.List->(co.lang.string)
co.core.Map->(key=co.lang.string, val=co.lang.int)
Employee->(T=co.lang.int, R=co.lang.string)
```

Normative shape:

```
arrow-type-expression = type-postfix-expression, [ "->", arrow-type-tail ]
                      | "(", [ function-type-parameter,
                               { ",", function-type-parameter } ],
                        ")", "->", arrow-type-tail ;

arrow-type-tail = type-derivation
                | parenthesized-type-list
                | type-expression ;

parenthesized-type-list = "(", [ return-item-list ], ")" ;

return-item-list = return-item, { ",", return-item } ;

return-item = [ identifier ], type-expression ;
```

A binder name in a generic argument list names the type parameter it binds. When names are used, they must be declared parameter names of the type being applied; when they are omitted, arguments bind positionally. Names and positions must not be mixed in one list. These are semantic checks, not parse-level ones.

C.11.1 Both generic application spellings are admitted

FoLang accepts two spellings for applying a type to type arguments, and they are not in conflict:

```
Option(co.lang.int)                 type-argument-list, positional only
co.core.List->(co.lang.string)      arrow tail, positional or named
```

`type-argument-list` is the direct application form used by `co.lang.type` constructors and admits a `dependent-index` argument. The arrow tail additionally admits binder names, which is what a multi-parameter built-in collection such as `co.core.Map` relies on. Where a declaration reads naturally in either spelling, both denote the same applied type.

C.11.2 Instantiating a generic declaration through a typed declaration

A typed declaration whose type is a generic declaration with its arrow tail supplied is a complete instantiation. It is the declarative equivalent of the explicit `new`/`init` pair:

```
k := LinkedList.new(co.lang.int);       // explicit two-step form
k LinkedList->(T co.lang.int);          // equivalent instantiation

c := Employee.new(co.lang.int, co.lang.string).init(1, "Rao");
c Employee->(T co.lang.int, R co.lang.string);
```

This introduces no new statement form. It is an ordinary `variable-declaration` whose `type-expression` carries an arrow tail, so C.6.1 applies unchanged and the declaration is terminated by `;`.

`@@new` and `@@init` remain reserved for the cases described in Generics, where the two steps must be separated or overridden explicitly.